

Unity®

PERFORMANCE TUNING TIPS 2026

Written By
CyberAgent Game & Entertainment Division



CyberAgent

Unityパフォーマンスチューニングティップス2026

株式会社サイバーエージェント | ゲーム・エンターテイメント事業部 著

Chapter 1

Unity Managed Code Stripping Practical Guide

Mukai Yuichiro

Applibot, Inc.

yucchiy X @yucchiy_ blog.yucchiy.com

As a means of reducing app size, this chapter introduces Managed Code Stripping, a Unity feature that removes unused code at build time. It can be enabled with a single setting, but increasing the level of reduction can cause problems in which a specific feature works in the Editor yet breaks only in the built app. This chapter covers how Managed Code Stripping works and how to use it, the typical patterns in which more code is removed than necessary, and how to deal with them.

The verification environment is as follows.

- Unity 6000.3.19f1
- Addressables 4.0.1
- macOS Sequoia 15.7.3

To state the conclusion first, Managed Code Stripping is not an optimization that you can finish by setting the level to High.

It is safe to use only when combined with an operation that makes dependencies invisible to static analysis explicit and keeps verifying the size and behavior of the built app.

1.1 What Is Managed Code Stripping

1.1.1 Overview

Managed Code Stripping is a standard Unity feature that detects unused managed code (C# code) at build time and removes it from the built app.

By removing classes and methods that are not used from scenes or scripts, it reduces the code size of the app.

The target is not limited to user code. It also covers UnityEngine modules, the .NET class library, and the assemblies of packages and plugins in the project, so it can reduce unused code inside libraries that are difficult to modify yourself.

In IL2CPP builds in particular, reducing managed code also reduces the C++ code that IL2CPP subsequently generates, which shortens app size and build time.

Note that the similarly named Strip Engine Code setting targets the native code of the Unity engine, and is a separate feature from Managed Code Stripping, which targets managed code.

Managed Stripping Level

How aggressively code is removed is controlled by the Managed Stripping Level setting.

You can set it from "Managed Stripping Level" in Edit > Project Settings > Player > Other Settings > Optimization, and the following levels are available.

Table 1.1: Managed Stripping Level options

Level	Overview
Disabled	Performs no stripping. Selectable only in Mono builds
Minimal	Reduces only UnityEngine and the .NET class library, and does not remove user code. The default for IL2CPP builds

Table 1.1: Managed Stripping Level options

Level	Overview
Low	Includes user code in the analysis, but reduces conservatively
Medium	Reduces all assemblies, balancing size and safety
High	Reduces all assemblies most aggressively

In Mono builds, the default is Disabled, so no stripping is performed unless you configure it.

In IL2CPP builds, on the other hand, the default is Minimal and Disabled cannot be selected, so stripping is always performed to some degree.

Note that the Unity 6.3 documentation announces the future deprecation of Low, and its use is not recommended.

References: Managed code stripping `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping.html` / Configure managed code stripping `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-configure.html`

1.1.2 Basic Principles

Managed Code Stripping is implemented by **UnityLinker**, a tool developed in-house by Unity.

UnityLinker runs in the app build pipeline after the C# code has been compiled, and removes unused code from the compiled assemblies (DLLs).

In IL2CPP builds, the assemblies that have undergone this stripping become the input for the subsequent C++ code conversion.

UnityLinker determines unused code in the following three steps.

1. Determine the code that serves as the starting point of the analysis (**root**)

2. Recursively follow the types and members statically referenced from the root, and mark them as in use (**mark**)
3. Regard code that remains unmarked at the end as unreachable and remove it from the assemblies

Typical items treated as roots are as follows.

- Types referenced from scenes and assets included in the build
- Types that inherit from MonoBehaviour or ScriptableObject
- Methods with attributes such as [RuntimeInitializeOnLoadMethod]
- Code explicitly preserved with link.xml or the [Preserve] attribute (covered in the countermeasures below)

What matters here is that whether code is kept is decided not by whether it is "actually used at runtime" but by whether it is "reachable from a root in build-time static analysis."

Static analysis is analysis that mechanically follows only the references appearing in compiled code, without executing the code.

For example, creating a type with new or calling a method directly leaves the information on the referenced type and member in the compiled code as it is, so Unity Linker can follow it.

On the other hand, code that is invoked in a way that leaves no reference in the code is regarded as unreachable, regardless of whether it is used at runtime.

Therefore, code that has only references invisible to static analysis is judged unused and removed, even if it is needed at runtime.

Also, marking is performed per member rather than per type, so even in a type that remains, methods that are not used are removed individually.

What code is actually removed is examined in the next section with concrete code and measurement results.

Checking What Was Stripped with Mono.Cecil

In the sections that follow, this chapter repeatedly checks which types and methods remain in the build.

For that check, it uses **Mono.Cecil**, a library for analyzing .NET assemblies.

It reads an assembly included in the build output (for a Mono build for macOS, "AppName.app/Contents/Resources/Data/Managed/Assembly-CSharp.dll") and writes the list of types and methods to a file.

▼List 1.1 Outputting the list of types and methods

```
// Read the assembly and write out the list of types and methods (including .ctor)
using var assembly = AssemblyDefinition.ReadAssembly(dllPath);
var lines = assembly.MainModule.Types
    .Where(type => type.Name != "<Module>")
    .SelectMany(type => new[] { type.FullName }
        .Concat(type.Methods.Select(method => $"  {method.Name}")));
File.WriteAllLines(outputPath, lines);
```

The method list also includes constructors (.ctor), so it can also be used to check the symptom, described later, in which "the type remains but only the constructor disappears."

Mono.Cecil can be installed and used through the Unity Package Manager. If you enable overrideReferences in an Editor Assembly Definition and add Mono.Cecil.dll to precompiledReferences, it can be referenced from Editor code.

By taking this list from two builds with different Levels and comparing the difference, you can identify which types disappeared and which methods were removed per member.

After a build, the stripped assemblies may remain under Library as intermediate products.

In this environment, DLLs with the same content as the build output remain in Library/Bee/artifacts/MacStandalonePlayerBuildProgram/ManagedStripped/, so you

can analyze them without extracting the build artifacts.

The official documentation does not state that stripped assemblies are saved at this path, so if you cannot find them, analyze the build output instead.

Wherever the following sections say "the types and methods were checked," the check was basically done with this method. Comparing the DLLs before and after stripping also reveals the breakdown of the reduction.

Reference: Unity linker marking rules `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-marking-rules.html`

1.1.3 Why Reduce Code

The main purpose of using Managed Code Stripping is to reduce app size.

As a secondary effect, unused code is not loaded into memory, so a reduction in runtime memory usage can also be expected.

To see how effective it is, we build a simple scene containing only a single script for quitting, and compare Minimal and High for Managed Stripping Level.

The measurement results are shown below (Table 1.2).

Table 1.2: Comparison of Minimal and High in a simple scene

Metric	Minimal	High	Reduction
Total size of managed assemblies	7,702,016 bytes	7,188,480 bytes	Approx. 6.7%
Number of assemblies (DLLs)	52	51	1
Total app size	89,069,307 bytes	88,548,299 bytes	Approx. 0.6%

Even in a small sample project, managed code decreased by 6.7%.

Looking at the breakdown, System.Runtime.Serialization.dll, which is not used anywhere, was removed entirely, mscorlib.dll shrank by about 272 KB, and

Assembly-CSharp.dll, which contains much code not referenced from the scene, also shrank by 22.5%.

The amount of managed code reduced is determined by the amount of libraries the project references and the proportion of code within them that is not actually used.

Large projects tend to introduce many libraries, and often many features within those libraries are not actually used, so such projects are likely to benefit more from code reduction.

In IL2CPP builds, the amount of code converted to C++ and compiled decreases by the amount of managed code reduced, so the effect extends to the size of the native binary and to build time.

1.2 Cautions When Applying Managed Code Stripping

As seen in the previous section, the decisions of Managed Code Stripping depend on build-time static analysis.

Therefore, code that has only references invisible to static analysis is removed even if it is needed at runtime.

What makes this problem troublesome is that the symptoms appear only in the built app.

It works normally in the Editor and in the default Mono build, where stripping is disabled, so it is hard to notice during development, and the defect surfaces in the form of only a specific feature breaking as soon as the Stripping Level is raised.

This section covers two typical patterns in which more code is stripped than necessary.

One is code invoked through reflection, and the other is code not referenced from

Unity entry points.

1.2.1 Code Invoked Through Reflection Disappears

We verify the decision "whether it is reachable from a root in static analysis," explained in the previous section, with concrete code.

Prepare three classes that are referenced in different ways (List 1.2).

▼List 1.2 Three classes referenced in different ways

```
// Created directly with new
public class DirectlyUsedService
{
    public string Describe() => "DirectlyUsedService is alive";
}

// Only the type is referenced with typeof
public class TypeofOnlyService
{
    public string Describe() => "TypeofOnlyService is alive";
}

// Created only through reflection
public class ReflectionOnlyService
{
    public string Describe() => "ReflectionOnlyService is alive";
}
```

From a MonoBehaviour placed in the scene, these three are used as follows (List 1.3).

▼List 1.3 MonoBehaviour that uses the three classes

```
public class StrippingProbe : MonoBehaviour
{
    private static readonly string ReflectionTypeName = "ReflectionOnlyService";
    private static readonly string TypeofTypeName = "TypeofOnlyService";
    private static readonly string TypeofMethodName = "Describe";

    private void Start()
    {
        // Static reference: the information on the referenced type and method remains
    in the compiled code
        var direct = new DirectlyUsedService();
```

```
        Debug.Log($"direct: {direct.Describe()}");

        // Static reference to the type only: typeof references the type, but there is
    no static reference to its members
        Debug.Log($"typeof: type is referenced: {typeof(TypeofOnlyService).Name}");

        // To check whether the members remain, retrieve the type again by a path
    invisible to static analysis
        var typeofType = Type.GetType(TypeofTypeName);
        var describeMethod = typeofType?.GetMethod(TypeofMethodName);
        if (describeMethod == null)
        {
            Debug.Log($"typeof: type {typeofType?.Name} exists but Describe is stripped
    ");
        }
        else
        {
            Debug.Log($"typeof: {describeMethod.Invoke(Activator.CreateInstance(
    typeofType), null)}");
        }

        // Reference invisible to static analysis: create through reflection
        var type = Type.GetType(ReflectionTypeName);
        if (type == null)
        {
            Debug.Log($"reflection: Type.GetType(\"{ReflectionTypeName}\") returned
    null");
        }
        else
        {
            var instance = Activator.CreateInstance(type);
            Debug.Log($"reflection: {type.GetMethod("Describe")?.Invoke(instance, null)
    }");
        }
    }
}
```

When this sample is built and run with Managed Stripping Level changed to Minimal and High, the results are as follows.

```
# Minimal
direct: DirectlyUsedService is alive
typeof: type is referenced: TypeofOnlyService
typeof: TypeofOnlyService is alive
reflection: ReflectionOnlyService is alive

# High
```

```
direct: DirectlyUsedService is alive
typeof: type is referenced: TypeofOnlyService
typeof: type TypeofOnlyService exists but Describe is stripped
reflection: Type.GetType("ReflectionOnlyService") returned null
```

With High, in the typeof case the type can be obtained but GetMethod returns null, and in the reflection case Type.GetType itself returns null.

In summary, the behavior is as follows (Table 1.3).

Table 1.3: What remains with Minimal and High

Target	Minimal	High
StrippingProbe	Remains	Remains
DirectlyUsedService (referenced with new)	Remains	Both type and method remain
TypeofOnlyService (only the type referenced with typeof)	Remains	Type remains, but Describe is removed
ReflectionOnlyService (reflection only)	Remains	Removed along with the type

Minimal does not remove user code, so all three remain as they are.

With High, the difference in how each is referenced appears directly in the result.

DirectlyUsedService references the type with new and also calls Describe directly, so both the type and the method remain.

TypeofOnlyService, which is referenced with typeof, remains as a type, but Describe, which is not called from anywhere, is removed per member.

And ReflectionOnlyService, which has no static reference at all, is removed along with its type.

Note that in this experiment, reading all type-name and method-name strings through static readonly fields is intentional.

UnityLinker understands some typical uses of reflection, and we confirmed that

when a constant string is passed directly, as in Type.GetType("ReflectionOnly Service"), the type is preserved even with High.

Also, when GetMethod was called directly on a Type obtained with typeof, all public methods of that type were preserved even if the method name was not a constant.

On the other hand, UnityLinker does not track strings passed through a field.

Reading from a field reproduces the real-world situation in which type names and member names come from paths not determined at build time, such as configuration files, serialized data, and server responses.

If you make the field const, it is embedded as a constant string at the use site, so it is treated the same as passing a constant directly.

Type Resolution in DI Containers Breaks

The examples so far are simplified for explanation, but the same structure of problem occurs in real libraries. A typical case is a DI container.

Many DI containers register the mapping between interfaces and implementation types, and at resolution time create an instance by invoking the constructor through reflection.

We verify this with a minimal container that imitates this behavior. As in real DI containers, it uses an interface that is registered with Register and resolved with Resolve (List 1.4).

▼List 1.4 Minimal DI container

```
public sealed class MiniContainer
{
    private readonly Dictionary<Type, Type> _registrations = new Dictionary<Type, Type
    >();

    // Register the implementation type with a type argument. TImplementation becomes a
    static reference equivalent to typeof
    public void Register<TInterface, TImplementation>() where TImplementation :
    TInterface
```

```
    {
        _registrations[typeof(TInterface)] = typeof(TImplementation);
    }

    // Instantiate the registered implementation type through reflection and return it
    public TInterface Resolve<TInterface>()
    {
        var implementationType = _registrations[typeof(TInterface)];
        return (TInterface)Activator.CreateInstance(implementationType);
    }
}
```

The registered implementation type is assumed to be used only through the container and is not statically referenced anywhere (List 1.5).

▼List 1.5 Registered interface and implementation

```
public interface INotificationService
{
    string Notify();
}

// An implementation used only through the container and not statically referenced
    anywhere
public sealed class NotificationService : INotificationService
{
    public string Notify() => "NotificationService.Notify succeeded";
}
```

A MonoBehaviour placed in the scene performs registration and resolution (List 1.6).

▼List 1.6 MonoBehaviour that registers and resolves

```
public sealed class DiContainerProbe : MonoBehaviour
{
    private void Start()
    {
        try
        {
            var container = new MiniContainer();
            container.Register<INotificationService, NotificationService>();
            Debug.Log($"[DiContainerProbe] explicit: {container.Resolve<
    INotificationService>().Notify()}");
        }
        catch (Exception exception)
```

```
        {
            Debug.Log($"[DiContainerProbe] explicit failed: {exception.GetType().Name}:
     {exception.Message}");
        }
    }
}
```

When this is built under the same conditions as before, the results are as follows.

```
# Minimal
[DiContainerProbe] explicit: NotificationService.Notify succeeded

# High
[DiContainerProbe] explicit failed: MissingMethodException: Default constructor not
    found for type NotificationService
```

With High, the type argument of Register<INotificationService, Notification Service>() becomes a static reference, so the NotificationService type itself remains.

Notify, which is called statically through the interface, also remains.

However, the default constructor, which is called only through reflection, has no static reference and is removed per member.

As a result, Activator.CreateInstance cannot find the constructor, and resolution fails with MissingMethodException.

This is the cause of the defect in which resolution fails only in the built app even though the type is registered.

The structure is the same in real DI containers.

As long as constructor resolution relies on reflection, the implicit default constructor is a candidate for removal even for a type that is explicitly registered.

With a method that scans assemblies for implementation types and registers them automatically, implementation types that have no static reference are removed along

with their types, and the registration itself fails.

1.2.2 Code Not Referenced from Unity Entry Points Disappears

Another pattern is code that is referenced from neither scenes nor scripts and is needed only through external content such as AssetBundles.

As seen in the previous section, the sources of roots are the scenes and assets included in the build.

Content built separately as an AssetBundle is not included in the build of the app itself, so the types that content references are not treated as roots.

Therefore, types needed only by the content are judged unused unless the app code has a static reference to them, regardless of whether they are loaded at runtime.

We reproduce this pattern with a Timeline asset.

The Timeline asset is included only in the AssetBundle, and the app does not statically reference Unity.Timeline; it loads the asset by name and plays it with Playable Director (List 1.7).

▼List 1.7 MonoBehaviour that plays a Timeline asset from an AssetBundle

```
public class TimelineAssetBundleProbe : MonoBehaviour
{
    private const string BundleDirectoryName = "TimelineBundles";
    private const string BundleName = "timeline.bundle";
    private const string TimelineAssetName = "TimelineOnly";

    private void Start()
    {
        var bundlePath = Path.Combine(Application.streamingAssetsPath,
    BundleDirectoryName, BundleName);
        var bundle = AssetBundle.LoadFromFile(bundlePath);
        if (bundle == null)
        {
            Debug.Log("timeline bundle: failed to load asset bundle");
            return;
```

```
        }

        var asset = bundle.LoadAsset(TimelineAssetName);
        if (asset == null)
        {
            Debug.Log($"timeline bundle: asset {TimelineAssetName} failed to load");
            bundle.Unload(true);
            return;
        }

        Debug.Log($"timeline bundle: loaded {asset.GetType().AssemblyQualifiedName}");
        var playableAsset = asset as PlayableAsset;
        if (playableAsset == null)
        {
            Debug.Log($"timeline bundle: asset {TimelineAssetName} is not playable");
            bundle.Unload(true);
            return;
        }

        var director = gameObject.AddComponent<PlayableDirector>();
        director.playableAsset = playableAsset;
        director.Play();
        Debug.Log($"timeline bundle: play state={director.state}, graph valid={director
    .playableGraph.IsValid()}");
        bundle.Unload(false);
    }
}
```

By deleting the original Timeline asset from the project after creating the Asset Bundle and then building the app, this imitates a setup in which content is updated separately from the app.

Build with Managed Stripping Level changed to Minimal and High. Both levels give the same result.

```
# Both Minimal and High
The referenced script (UnityEngine.Timeline.TimelineAsset) on this Behaviour is missing
    !
timeline bundle: asset TimelineOnly failed to load
```

Also, when we checked the Managed folder of the built app, Unity.Timeline.dll itself did not exist in either build.

Note that the result does not differ between Minimal and High.

The countermeasure for this pattern is not to adjust the Stripping Level but to explicitly specify the types and assemblies to preserve.

The means for this are covered in the next section, and the mechanism when using Addressables is covered in the advanced section.

References: How code stripping affects content `https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-content.html` / A case where a Timeline asset disappears (Qiita, 2021) `https://qiita.com/KyoheiOkawa/items/0cf9cc09679eababec75`

1.3 Countermeasures for Over-Stripping

1.3.1 Basic Countermeasures

The most basic way to prevent necessary code from being removed is to explicitly tell UnityLinker that "this code is used."

Unity provides the link.xml file and the [Preserve] attribute for this purpose.

Code specified by either is treated as a root of the analysis and is not removed even without a static reference.

link.xml

link.xml is an XML file that lists the assemblies, types, and members to preserve.

If you place it under the project's Assets, UnityLinker reads it at build time.

ReflectionOnlyService, which was removed along with its type in the previous sample, can be preserved with the following link.xml (List 1.8).

▼List 1.8　link.xml that preserves ReflectionOnlyService

```
<linker>
  <assembly fullname="Assembly-CSharp">
    <type fullname="ReflectionOnlyService" preserve="all" />
  </assembly>
```

```
</linker>
```

When this link.xml was placed directly under Assets and the app was rebuilt with High, the execution result changed as follows.

```
# High (without link.xml)
reflection: Type.GetType("ReflectionOnlyService") returned null

# High (with link.xml)
reflection: ReflectionOnlyService is alive
```

Checking the assemblies included in the build also confirms that ReflectionOnly Service is preserved along with Describe.

link.xml specifies what to preserve outside the code, so it can also be used for types in DLLs and packages that you cannot edit yourself.

The preserved scope can be specified per assembly, per type, or per member such as a method or field.

Note that preserving an entire assembly with preserve="all" is convenient, but it cancels out the effect of stripping on that assembly.

It can be used to isolate a problem, but for a permanent fix it is better to narrow down to the necessary types and members.

[Preserve] Attribute

For code you can edit yourself, you can attach the [Preserve] attribute of the Unity Engine.Scripting namespace directly to the target.

Note that the scope preserved by [Preserve] changes depending on where it is attached.

We verify this difference with two classes: one with the attribute on the type, and one with it only on the method (List 1.9, List 1.10).

▼List 1.9　When Preserve is attached to the type

```
using UnityEngine.Scripting;

// Attach [Preserve] to the type. Do not attach it to the method
[Preserve]
public sealed class TypePreservedService
{
    public string Describe() => "TypePreservedService.Describe is alive";
}
```

▼List 1.10　When Preserve is attached only to the method

```
using UnityEngine.Scripting;

// Do not attach it to the type; attach [Preserve] only to the method
public sealed class MethodPreservedService
{
    [Preserve]
    public string Describe() => "MethodPreservedService.Describe is alive";
}
```

When these two were created and invoked through reflection without any static reference, the results with High were as follows (Table 1.4).

Table 1.4: Position of Preserve and the scope preserved with High

Target	Type	Default constructor	Describe
TypePreservedService	Remains	Remains	Removed
MethodPreservedService	Remains	Removed	Remains

[Preserve] attached to a type preserves the type and its default constructor, but does not preserve Describe, which has no attribute.

[Preserve] attached to a method preserves that type and method, but this time the default constructor is not preserved, and creation with Activator.CreateInstance fails with MissingMethodException.

To make it work all the way through creating and invoking via reflection, you must attach [Preserve] to both the type and the method.

If "it does not work even though I attached [Preserve]," first suspect the correspondence between the position where the attribute is attached and the preserved scope.

Reference: Preserving code using annotations https://docs.unity3d.com/6000.3/Documentation/Manual/managed-code-stripping-preserving.html

1.3.2 Advanced Topics

Using Attributes Other Than Preserve

Whereas [Preserve] specifies a single point, "keep this element," the Unity Engine.Scripting namespace also has attributes that declare preservation by pattern, such as "keep all implementations of this interface."

In a structure common in frameworks, such as "find types implemented by the user through reflection and invoke them," you can handle this by attaching an attribute to the interface or base class, instead of specifying each implementation type with [Preserve].

- RequireImplementorsAttribute https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequireImplementorsAttribute.html: when attached to an interface, preserves the types that implement that interface
- RequireDerivedAttribute https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequireDerivedAttribute.html: when attached to a class, preserves the types that inherit from that type
- RequiredMemberAttribute https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequiredMemberAttribute.html: when attached to a member, also preserves that member if its type remains
- RequiredInterfaceAttribute https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Scripting.RequiredInterfaceAttribute.html: when attached to a type, preserves the implementation information of the specified interface

As a representative, we verify the behavior of [RequireImplementors].

The attribute is attached to the interface, and the implementation is not statically referenced anywhere (List 1.11).

List 1.11 Interface with RequireImplementors attached, and its implementation

```
// When attached to the interface, the implementation types become targets of
    preservation
[RequireImplementors]
public interface IRequiredContract
{
}

// An implementation not statically referenced anywhere
public sealed class RequiredContractImpl : IRequiredContract
{
}
```

When built with High, RequiredContractImpl was preserved along with its default constructor.

An implementation type with no static reference remains solely through the declaration on the interface side.

Preservation was also confirmed in the same way for derived types of a base class with [RequireDerived] attached.

Generating link.xml at Build Time with IUnityLinkerProcessor

When the code to preserve is known in advance, simply specifying the target classes and members in link.xml is sufficient.

However, in situations where the types to preserve differ by build target, or the necessary types change depending on the content included in the build, continually updating link.xml by hand is not realistic.

With **IUnityLinkerProcessor**, you can dynamically generate link.xml at build time and pass it to UnityLinker.

IUnityLinkerProcessor is an Editor interface, and if you place the implementation class in an Editor assembly, it is called automatically at build time.

The link.xml at the path returned by GenerateAdditionalLinkXmlFile is added to the input of UnityLinker.

The minimal implementation is as follows (List 1.12).

▼List 1.12 Minimal IUnityLinkerProcessor implementation that generates link.xml

```
using System.IO;
using UnityEditor.Build;
using UnityEditor.Build.Reporting;
using UnityEditor.UnityLinker;
using UnityEngine;

public sealed class PreserveListLinkerProcessor : IUnityLinkerProcessor
{
    private const string LinkXmlContent = @"<linker>
  <assembly fullname=""Assembly-CSharp"">
    <type fullname=""GeneratedPreserveService"" preserve=""all"" />
  </assembly>
</linker>
";

    public int callbackOrder => 0;

    // The link.xml at the returned path is added to the input of UnityLinker
    public string GenerateAdditionalLinkXmlFile(BuildReport report,
    UnityLinkerBuildPipelineData data)
    {
        var path = Path.Combine(Path.GetTempPath(), "managed-stripping-sample-generated
    -link.xml");
        File.WriteAllText(path, LinkXmlContent);
        Debug.Log("[PreserveListLinkerProcessor] generated 1 assembly / 1 type.");
        return path;
    }
}
```

In this example, the generated content is fixed for explanation, but in practice you would replace this part with processing that builds it from build-time input, such as reading a settings asset or analyzing content dependencies.

When we prepared GeneratedPreserveService, which has no static reference, and

built with High, we confirmed that the type, constructor, and method were preserved even though no link.xml was placed under Assets.

```
[GeneratedLinkXmlProbe] type=present
[GeneratedLinkXmlProbe] ctor=present, method=GeneratedPreserveService.Describe is alive
```

Reference: IUnityLinkerProcessor `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Build.IUnityLinkerProcessor.html`

link.xml Generated by Addressables

Addressables addresses the problem seen in the previous section, in which "types needed only by the content disappear," by automatically generating link.xml as an artifact of the content build.

We check what is actually generated. The content is kept minimal: we create one prefab that only displays a Cube mesh (a GameObject with MeshFilter and Mesh Renderer attached) and register it in an Addressables group.

Running the content build generates link.xml.

In Addressables 4.0.1, the output location is Library/com.unity.addressables/aa/<platform>/AddressablesLink/link.xml.

The content generated this time was as follows (List 1.13).

▼List 1.13 link.xml generated by Addressables

```
<linker>
  <assembly fullname="Unity.Addressables, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null" preserve="all">
    <type fullname="UnityEngine.AddressableAssets.Addressables" preserve="all" />
  </assembly>
  <assembly fullname="Unity.ResourceManager, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null" preserve="all">
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    AssetBundleProvider" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    BundledAssetProvider" preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.
    IAssetBundleResource" preserve="all" />
```

```
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.InstanceProvider"
    preserve="all" />
    <type fullname="UnityEngine.ResourceManagement.ResourceProviders.SceneProvider"
    preserve="all" />
  </assembly>
  <assembly fullname="UnityEngine.CoreModule, Version=0.0.0.0, Culture=neutral,
    PublicKeyToken=null">
    <type fullname="UnityEngine.GameObject" preserve="all" />
    <type fullname="UnityEngine.MeshFilter" preserve="all" />
    <type fullname="UnityEngine.MeshRenderer" preserve="all" />
    <type fullname="UnityEngine.Transform" preserve="all" />
  </assembly>
</linker>
```

The content falls into two kinds.

One is the provider types of the Addressables runtime (Unity.Addressables and Unity.ResourceManager), which become necessary depending on the group configuration.

The other is the types referenced by the registered content. This time, the Game Object, Transform, MeshFilter, and MeshRenderer that make up the prefab are listed as preservation targets as they are.

This shows that Addressables analyzes the dependencies on the content side and includes types in the preservation targets even when the app code does not reference them.

Reference: Addressables Build Artifacts `https://docs.unity3d.com/ja/Packages/com.unity.addressables@1.20/manual/BuildArtifacts.html` (the description of link.xml is in the 1.20 manual)

Replacing Code Invocation via Reflection with Static Calls by a Source Generator

All the countermeasures so far specify preservation so that necessary code is not removed.

In contrast, there is also a method of replacing the reflection itself, which creates

dependencies invisible to static analysis, with code generation by a **Source Generator**.

A Source Generator is a mechanism in which the C# compiler analyzes source code at compile time and generates additional source code.

If type discovery and creation are completed at compile time, the dependencies are expressed as static references from the start, and the preservation specification itself becomes unnecessary.

As an example, consider dispatch processing that receives a command name string and executes the corresponding handler.

Handlers declare their command names with an attribute. Creating them with reflection at runtime would mean searching assemblies for attributed types and creating them (List 1.14).

▼List 1.14　Handlers that declare command names with an attribute

```
[AttributeUsage(AttributeTargets.Class)]
public sealed class CommandHandlerAttribute : Attribute
{
    public CommandHandlerAttribute(string command)
    {
        Command = command;
    }

    public string Command { get; }
}

public interface ICommandHandler
{
    string Execute();
}

// The handlers are not statically referenced anywhere
[CommandHandler("greet")]
public sealed class GreetCommandHandler : ICommandHandler
{
    public string Execute() => "GreetCommandHandler.Execute succeeded";
}
```

```
[CommandHandler("farewell")]
public sealed class FarewellCommandHandler : ICommandHandler
{
    public string Execute() => "FarewellCommandHandler.Execute succeeded";
}
```

The two handlers are not statically referenced anywhere, so if the app is built with High while keeping the runtime-scan design, they are removed along with their types, as seen in the previous section.

We implement the resolution of handlers for these command names with a Source Generator.

We prepare a generator that collects classes with [CommandHandler] at compile time and generates a dispatcher that looks up a concrete type from a command string.

The generator works in two stages: the first (ForAttributeWithMetadataName) collects command names and type names from attributed classes, and the second (GenerateDispatcher) pours the collected pairs into a template.

The full code of the generator is shown below (List 1.15).

▼List 1.15　Source Generator that generates the dispatcher

```
#nullable enable
using System.Collections.Immutable;
using System.Linq;
using System.Text;
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.Text;
namespace StrippingSample.SourceGenerators
{
    [Generator]
    public sealed class CommandDispatcherGenerator : IIncrementalGenerator
    {
        public void Initialize(IncrementalGeneratorInitializationContext context)
        {
            // Collect pairs of command names and type names from classes with [
    CommandHandler]
            var handlers = context.SyntaxProvider
```

```
            .ForAttributeWithMetadataName(
                "CommandHandlerAttribute",
                static (_, _) => true,
                static (syntaxContext, _) => (
                    Command: (string)syntaxContext.Attributes[0].
ConstructorArguments[0].Value!,
                    TypeName: ((INamedTypeSymbol)syntaxContext.TargetSymbol)
                        .ToDisplayString(SymbolDisplayFormat.FullyQualifiedFormat))
)
            .Collect();
        // Generate the dispatcher from the collected pairs
        context.RegisterSourceOutput(handlers, static (sourceContext,
commandHandlers) =>
        {
            if (!commandHandlers.IsDefaultOrEmpty)
            {
                sourceContext.AddSource(
                    "CommandDispatcher.g.cs",
                    SourceText.From(GenerateDispatcher(commandHandlers), Encoding.
UTF8));
            }
        });
    }
    private static string GenerateDispatcher(
        ImmutableArray<(string Command, string TypeName)> handlers)
    {
        // To make the output deterministic, sort by command name before pouring
them in
        var cases = string.Join("\n", handlers
            .Distinct()
            .OrderBy(static handler => handler.Command, System.StringComparer.
Ordinal)
            .Select(static handler => $$"""
                        case {{SymbolDisplay.FormatLiteral(handler.Command, true)
}}:
                            return new {{handler.TypeName}}().Execute();
            """));
        return $$"""
            // <auto-generated/>
            public static class CommandDispatcher
            {
                public static string Execute(string command)
                {
                    switch (command)
                    {
            {{cases}}
                        default:
                            throw new global::System.ArgumentException($"Unknown
```

```
    command: {command}");
                        }
                    }
                }
                """;
        }
    }
}
```

The sorting by command name ensures that the same input generates the same code, so that the output does not fluctuate from build to build.

With this generator, code like the following is generated at compile time (List 1.16).

▼List 1.16　Generated CommandDispatcher

```
// <auto-generated/>
public static class CommandDispatcher
{
    public static string Execute(string command)
    {
        switch (command)
        {
            case "farewell":
                return new global::FarewellCommandHandler().Execute();
            case "greet":
                return new global::GreetCommandHandler().Execute();
            default:
                throw new global::System.ArgumentException($"Unknown command: {command}
    ");
        }
    }
}
```

The processing that looked up types from strings at runtime has become a table mapping strings to concrete types.

Both construction (new) and the Execute call appear in the generated code, so static references to the type, constructor, and method are self-contained within the generated code.

The calling side only passes a command string and never references the concrete types of the handlers.

As in the reflection experiment, the command string is passed through a static readonly field, keeping it an input that UnityLinker cannot track (List 1.17).

▼List 1.17 Calling side of the generated dispatcher

```
// Pass the command string through a field to make it an input that UnityLinker cannot
    track
private static readonly string[] Commands = { "farewell", "greet" };

private static void RunGeneratedDispatcher()
{
    foreach (var command in Commands)
    {
        var result = CommandDispatcher.Execute(command);
        Debug.Log($"[SourceGeneratorProbe] dispatch command={command}, result={result}"
    );
    }
}
```

When this was built with High and run, we confirmed that both commands executed successfully.

```
[SourceGeneratorProbe] dispatch command=farewell, result=FarewellCommandHandler.Execute
     succeeded
[SourceGeneratorProbe] dispatch command=greet, result=GreetCommandHandler.Execute
    succeeded
```

The string itself remains an input that cannot be tracked at build time, but the mapping from string to type is now a switch in generated code rather than a runtime search, so to UnityLinker it looks like an ordinary static reference, no different from handwritten code.

A check with Mono.Cecil also showed that the types, constructors, and Execute of the two handlers, as well as CommandDispatcher.Execute, were preserved.

The benefit of this replacement is not limited to resistance to stripping.

- Because the dependencies are visible to UnityLinker, maintaining preservation specifications with link.xml or attributes becomes unnecessary
- Runtime reflection searches are eliminated, which reduces startup cost and the possibility of runtime failures

- Because targets are collected at compile time, missing registrations and duplicates can be diagnosed at compile time

Some real libraries also adopt such an implementation.

Among serializers, MessagePack for C# has long provided code generation for IL2CPP environments and has now moved to Source Generator-based generation.

Among DI containers, VContainer offers an option to generate injection code with a Source Generator and reduce runtime reflection, and it resolves the symptom seen in the previous section, "the constructor disappears and resolution fails," with a Source Generator.

For how to introduce a Source Generator into a Unity project (a setup that places a netstandard2.0 DLL with the RoslynAnalyzer label), see the Unity manual.

References: Create and use a source generator `https://docs.unity3d.com/6000.3/Documentation/Manual/create-source-generator.html` / MessagePack for C# `https://github.com/MessagePack-CSharp/MessagePack-CSharp` / Memory Pack `https://github.com/Cysharp/MemoryPack` / VContainer `https://github.com/hadashiA/VContainer`

1.4 Summary

Managed Code Stripping is not an optimization that you can finish by raising the Stripping Level.

As this chapter has shown, dependencies through reflection or external content are invisible to static analysis, and code that is needed at runtime disappears from the build.

Moreover, because the symptoms appear only in the built app, you need to operate two practices together: making invisible dependencies explicit, and continuing to verify with the built app.

The means of specifying preservation are used selectively according to the target and situation (Table 1.5).

Table 1.5: Choosing among the countermeasures

Method	Suitable situation
[Preserve]	Preserve a small number of elements in code you can edit yourself
link.xml	Preserve fixed targets, including DLLs you cannot edit
Require-family attributes	Declare a contract on the framework side to "keep implementations or derived types"
IUnityLinkerProcessor	The preservation targets vary by build target or content
Source Generator	Replace the reflection-based search itself with static references

In addition, the following are policies for making stripping more effective.

- Except in cases where it is a problem, do not overuse [Preserve] and link.xml
- Reduce type discovery and creation through reflection, and consider switching to static references through explicit registration or a Source Generator
- Narrow code preservation specifications down to the type or member level wherever possible. Limit preserve="all" for an entire assembly to isolating problems
- For preserving types used only by content built separately from the app, use mechanisms such as Addressables' link.xml or generation by IUnityLinker Processor rather than doing it by hand
- Run and verify builds with a raised Stripping Level from the middle of development

Chapter 2

Turning the Tuning Bible into a Skill for AI Coding Agents

Teruhisa Nakamura

Applibot, Inc.

Name: Teruhisa Nakamura Affiliation: Applibot, Inc.

2.1 Introduction

Tuning knowledge such as that in this book has one structural weakness: it only helps people who have read it. Each chapter is full of findings backed by measurements, but whether they are actually raised in a project's code review depends on whether the reviewer has read the relevant chapter and can recall it at the moment of review. You cannot expect every team member to memorize every chapter, and when people rotate during the operation phase, the knowledge fades. Meanwhile, AI coding agents (Claude Code, OpenAI Codex, GitHub Copilot, and others) are now a permanent presence where code is written. If you build them into review requests or CI, the same standard can be applied to every diff. The only missing piece is a systematic set of evaluation criteria for Unity performance. This chapter therefore shows how to convert the knowledge in the Tuning Bible into "evaluation items that an AI agent can execute" and package it as a general-purpose skill that does not depend on a specific tool. Based on the author's actual case of combining 77 rules built on the knowledge published in the previous book with 86 rules extracted from the manuscripts of this book (2026 edition), for a total of 163 evaluation rules packaged as a skill, it summarizes design guidelines for the conversion and operational considerations.

2.2 Agent Skills as a Common Format

2.2.1 The SKILL.md Specification

As a way to give AI agents specialized knowledge, an open specification called Agent Skills[*1] was released in late 2025. A skill is a directory centered on a Markdown file named SKILL.md, with a structure like the following.

List 2.1 Skill directory structure

```
unity-perf-review/ ├──
 SKILL.md          # Metadata + execution steps (required) ├──
 references/       # Reference documents such as rule sets ├──
 scripts/          # Executable scripts (optional) └──
 assets/           # Templates, etc. (optional)
```

The top of SKILL.md contains metadata called front matter. Only two fields are required: name and description.

List 2.2 SKILL.md front matter (description is shortened for print, and optional fields such as metadata are omitted)

```
---
name: unity-perf-review
description: >
  Unity Reviews the project's code, assets, and settings from a performance perspective
    .
  Checks C# scripts, shaders, .meta files, and ProjectSettings against
  163 the rules and outputs Critical / Warning / Info findings.
  Unity Use when asked for a performance review or optimization investigation of
license: MIT
---
```

How you write the description is important. The agent matches the user's request against the description to trigger the skill automatically, so writing not only "what it does" but also "when to use it" improves triggering accuracy.

[*1] Agent Skills specification `https://agentskills.io/specification`

2.2.2 Support in Major Agents

The value of this specification is that multiple agents can read the same format. Table 2.1 shows the support status as of August 2026.

Table 2.1: Skill support in major coding agents (as of August 2026)

Agent	Support	Location
Claude Code	Native SKILL.md support[*2]	claude/skills/ or ~/.claude/skills/
OpenAI Codex	SKILL.md supported	agents/skills/ or ~/.agents/skills/
GitHub Copilot	SKILL.md supported[*3]	github/skills/
Gemini CLI	SKILL.md supported[*4]	gemini/skills/ or ~/.gemini/skills/

Codex, Copilot, and Gemini CLI also support the shared .agents/skills/ location. Copilot also reads the project's .claude/skills/, so a single placement in .claude/skills/ reaches both Claude Code and Copilot (Copilot's personal locations are ~/.copilot/skills/ and ~/.agents/skills/). Support status and locations change over time, so check each agent's official documentation for primary information when adopting the skill. Once tuning knowledge is captured in SKILL.md format, you can distribute the same review criteria (rules and procedures) to members who use any agent. Agent Skills standardizes the package format and procedures, but differences in models and tool permissions mean the execution results are not identical. Even so, a division of labor holds: "review criteria live in the repository, execution is done by the agent."

2.3 Converting Findings into Evaluation Items

The core of building a skill is not writing SKILL.md but breaking the findings of each chapter down to a granularity that an agent can apply mechanically. The author standardized on the following format.

▼List 2.3 Rule format (an example extracted from the ParticleSystem chapter; the exclusion conditions of grep_pattern and the details of the remedy are shortened for print)

```
### US11 Passing a large number of seconds to ParticleSystem.Simulate
- Level: Warning (Critical when the Noise module is enabled)
- Target: code review / asset settings
- Detection pattern: a large number of seconds is passed as the first argument of
    Simulate.
  Especially when used with a ParticleSystem that has NoiseModule enabled
- grep_pattern: \.Simulate\(
- Reason: Simulate simulates in one batch up to the specified time.
  In measurements, specifying 20 seconds called NoiseJob about 2000 times,
  causing a spike of about 50 ms
- Countermeasure: avoid using Simulate on particles with Noise enabled
```

This format embodies four design decisions.

2.3.1 Make the Feasibility of Mechanical Detection Explicit

Each rule has a grep_pattern, a machine-executable search specification for C# code and Unity YAML (.meta, Prefab, ProjectSettings). Most are a single regular expression, but some match across multiple lines or detect the absence of a required statement. The agent performs primary detection with this specification and reads only the hit locations closely to make a judgment. This makes review feasible at a realistic cost even for large repositories. It is equally important not to force rules into a regular expression when they cannot be expressed as one. For example, "a shared asset not registered in Addressables is referenced from multiple bundles" cannot be found with grep. Such rules are explicitly marked (manual check), and the remedy field lists a means of verification such as Addressables Analyze. If detectability is misrepresented, the agent either produces many false positives or silently skips rules it cannot detect.

2.3.2 Keep Measured Values in the Reason Field

A rule whose reason reads "Reason: performance degrades" is not persuasive. Quote the measured values from the manuscripts directly in the reason field.

- Uncompressed texture: at 2048x2048, Compressed is 2.7MB versus Uncompressed 16.0MB (about 6x difference)
- Manual Playable with one graph per character: 8.396ms for 600 characters versus 4.500ms for a single graph for all (1.87x difference)
- Combining Simulate(20f) with Noise: a spike of about 50ms from roughly 2000

NoiseJob calls

With numbers, the agent's finding itself explains "why this should be fixed." It also gives the developer who receives the review a path back to the relevant chapter of the manuscript. Note that all of these are values measured in specific environments as described in the manuscripts. The reason field quotes representative values, and the measurement conditions are left to the source chapter.

2.3.3 Make It Sustainable with Levels and Overrides

Rules are assigned one of three levels: Critical / Warning / Info (Critical = a definite cause of degradation, which must be fixed once its applicability conditions are confirmed; Warning = a latent risk that becomes a problem under certain conditions; Info = an optional improvement recommendation). A mechanism for projects to override them (rules-overrides) is also provided. For example, even if "LINQ usage is Critical," it can be relaxed to Warning during initialization (init context) and set to Skip in Editor tools (tooling). Tuning rules are inherently context-dependent. Without an override mechanism, complaints such as "many findings do not suit our project" lead to the whole skill being disabled. A two-layer structure, with common criteria and project-specific application, is the key to longevity.

2.3.4 Separate Findings That Cannot Be Detected Statically into Diagnostic Knowledge

Many findings in the manuscripts cannot be learned from reading the code. One example is knowledge about how to read the profiler: "If you reduced CPU processing but the frame time did not change and the time moved to a wait marker such as Semaphore.WaitForSignal, suspect that the render thread is on the critical path, and check what it is waiting on in the Timeline view." The limitation that a wait marker alone cannot establish the bottleneck is also retained as part of the finding. These are separated from the evaluation rules into a different file as a "symptom → cause → remedy" table (Table 2.2). In the author's repository, a profile analysis agent (for Claude Code) loads it only during diagnosis. The skill distributed in this chapter bundles it as reference material, and it serves as the basis for reporting suspicions that static review cannot judge as "profiling recommended."

Table 2.2: Example of diagnostic knowledge

Symptom	Cause	Remedy
CPU chart shows a comb pattern of valleys and spikes	Forgot to offset the phase of throttled (every-N-frame) updates	Distribute the phase with index % interval
MeshSkinning.CalcMatrices is large	Bone Transform hierarchy is materialized	Apply Optimize Game Objects
iOS only: no errors and no S hader.CreateGPUProgram, but only GpuProgramMetal. GetCachedPipeline occurs	Suspect duplicate packing of shaders in AssetBundles	Check for duplicate instance s and move the shaders to a dedicated bundle
iOS only: PSO compilation occurs in a SubPass shader even after warmup complete s	Warmup failure for shaders inside a SubPass (Native Render Pass)	Review the SubPass usage, or pre-render during loading

2.4 Assembling the Skill

2.4.1 Progressive Disclosure and the Checklist Approach

Writing all 163 rules in the body of SKILL.md would put pressure on the agent's context. The design philosophy of Agent Skills is progressive disclosure.

1. The agent always sees only the name and description in the front matter
2. When the skill is triggered, it reads the body of SKILL.md (the execution flow)
3. It reads only the reference documents it needs from references/

What is read initially (including whether paths are included) depends on the agent implementation, but the principle that "the body and reference materials are not read until triggering" is common to all. Completeness needs attention here. A naive approach would be to load entire category-specific rule files according to the target file type, but with this approach the rules the agent reads before the code can, in the worst case, exceed 20,000 tokens (an estimate from character count; the token count depends on the model). If you rely on the model's attention to "remember and apply all" of dozens of rules, some of them will be silently skipped. The author therefore automatically generated a checklist that compresses every rule into one line with "ID / level / applicability condition / grep_pattern / rule name / detail file name," and split

the review into two stages.

1. **Primary scan**: The bundled script scan.py mechanically runs the grep_pattern of every checklist line against the target files and outputs a list of hits (file:line) and Coverage (in environments without python3, the agent falls back to running every grep line directly)
2. **Secondary review**: Only for rules that hit, read the detection conditions, reason, and remedy from the detail file and exclude false positives

Some rules that grep cannot express are marked MANUAL on the checklist and are read closely only when they apply to the target type. Rules where the absence of a required statement is the condition for a finding are marked ABSENT, and their details are checked when grep produces no hit (this is explicit so that the correspondence between hits and finding conditions is not inverted). The primary scan was made a script because having an LLM run dozens of greps one by one can lead to skipped items, and differences in regular expression dialects (for example, PCRE lookahead does not work in the standard macOS grep) make results environment-dependent. This decouples the completeness of the primary scan from the model's memory. The correctness of findings (excluding false positives in the secondary review) remains the agent's judgment. Only the checklists are read in advance (about 16KB for code and about 8KB for assets; the token count depends on the model, but roughly 10,000 tokens in total), and details are read only for the hits. Because the checklists are generated automatically from the rule bodies, adding a rule requires no manual synchronization.

2.4.2 Excluding Rules by Environment

Some rules depend on the Unity version or configuration. The RenderGraph-related rules included in this skill (for URP) target Unity 6 and later, the LTO rules are for IL2CPP only, and iOS-specific PSO issues are irrelevant to titles that ship only on Android. Checking rules that do not apply only adds noise and effort. Environment-dependent rules therefore carry applicability condition tags (unity>=6000.3, urp, il2cpp, ios, and so on), which are matched against an environment profile generated from the project's configuration files, excluding "only the rules that clearly do not meet the conditions." The primary scan script handles tag matching deterministically, items that could not be detected (unknown) are treated conservatively (the rule

remains a target for checking), and the IDs of skipped rules are listed in the report so that it can be audited. The generation procedure is described in the section on the adoption steps.

2.4.3 Enforce Rule IDs in the Output

The output format requires every finding to carry a rule ID.

▼List 2.4 Example of review output

```
### Critical
- Assets/Scripts/EffectPlayer.cs:42 -
  Simulate(20f) call on a particle with Noise enabled [US11]
  - Evidence: NoiseJob is called about 2000 times, causing a spike of about 50 ms (
    measured)
  - Recommendation: remove Noise or minimize the number of seconds
```

With IDs, you can trace the basis of a finding back to the rule file and verify it, which distinguishes it from the agent "just sounding plausible." When a finding is a false positive, it is also clear which rule's detection pattern should be fixed.

2.5 Distribution and Placement for Each Agent

Publishing a skill requires only sharing the directory. A GitHub repository or a zip file both work. Users copy it to the locations shown in Table 2.1, and each agent discovers the skill. The operational key is to keep a single source for the rules. In the author's repository, the rule bodies are kept only in the rules/ directory, and the distribution package is assembled by a script (the user-side adoption steps are in the next section).

▼List 2.5 Assembling the distribution package (for maintainers)

```
./package-skill.sh
cp -r dist/unity-perf-review /path/to/repo/.claude/skills/   # Claude Code
cp -r dist/unity-perf-review /path/to/repo/.agents/skills/   # Codex / Copilot / Gemini
     CLI
cp -r dist/unity-perf-review /path/to/repo/.github/skills/   # Copilot-specific
    placement (optional, since either of the two lines above is sufficient)
```

Once you start managing rules separately for each agent, the criteria drift apart with every update. I recommend deciding at the outset on "one source, placement

by script."

2.6 Adopting It in Your Own Project

This section describes the steps to run the mechanism described so far in your own project. If python3 is installed, copying and initial generation take about 10 minutes. Only the skill directory and the environment profile (tuning-env.md) are added to the repository, and nothing in the Unity Assets / ProjectSettings is changed.

2.6.1 Obtaining and Placing the Skill

The complete skill (SKILL.md, references/ containing the 163 rules, three scripts for environment detection, primary scan, and checklist regeneration, and a README with the adoption steps) is published in skills/unity-perf-review/ of this book's repository[*5]. Simply obtain it and copy it to the skill directory of the agent you use. Note that a skill is a distributable containing "instructions for the agent and executable scripts." For skills from untrusted sources, check the contents of SKILL.md and scripts/ before adopting them, and pin the version.

▼List 2.6 Obtaining and placing the skill

```
git clone https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026
    .git
cd UnityPerformanceTuningTips2026/skills

# Copy to one of these, depending on the agent you use
cp -r unity-perf-review /path/to/your-repo/.claude/skills/   # Claude Code
cp -r unity-perf-review /path/to/your-repo/.agents/skills/   # Codex / Copilot / Gemini
     CLI (shared placement)
cp -r unity-perf-review /path/to/your-repo/.github/skills/   # Copilot-specific
    placement (optional, since either of the two lines above is sufficient)
```

If you place it at the repository root, the same review criteria are distributed to the whole team. If you are only trying it individually, it also works in the home directory (such as ~/.claude/skills/).

[*5] https://github.com/CyberAgentGameEntertainment/UnityPerformanceTuningTips2026/tree/main/skills/unity-perf-review

2.6.2 State After Placement

For Claude Code, once placement and the environment profile generation in the next section are done, the repository looks like this (List 2.7).

▼List 2.7 Repository structure after placement

```
your-repo/
|-- .claude/
|   \`-- skills/
|       \`-- unity-perf-review/
|           |-- SKILL.md                      # name/description + execution flow
|           |-- README.md                     # Setup instructions for humans
|           |-- references/
|           |   |-- checklist-code.md         # For primary scan (code)
|           |   |-- checklist-asset.md        # For primary scan (assets/settings)
|           |   |-- checklist-profile.md      # For profile analysis (supplementary
    material not used in static review)
|           |   |-- rules-csharp.md 13 files in total, etc.  # Detailed rules (163 in
    total)
|           |   |-- diagnosis-knowledge.md    # Diagnostic collection of symptom->cause
    ->remedy
|           |   \`-- rules-overrides.example.md
|           \`-- scripts/
|               |-- detect-env.py             # Environment profile generation
|               |-- scan.py                   # Automated execution of the primary scan
|               \`-- gen-checklists.py         # Regenerating the checklist when rules
    are modified
|-- tuning-env.md                             # Environment profile (generated in the
    next section)
\`-- (The Unity project itself is not modified)
```

The three checklists are by target (code / assets / profile), and one rule can appear on more than one of them, so the total number of lines does not match the 163 rules. At the time of placement, in principle only the name and description in the front matter are loaded into the agent (a few hundred tokens in total; as noted earlier, some implementations add paths and the like). When you request a review, the necessary parts are loaded in the following order according to the progressive disclosure described earlier: the body of SKILL.md, then the checklist for the target, then the details of the rules that hit. Normally you do not need to load the full text of all 163 rules at once (about 125KB, roughly over 40,000 tokens). rules-overrides.example.md has no effect as it is. When you start making project-specific adjustments, rename it to

rules-overrides.md and edit it (described later).

2.6.3 Generating the Environment Profile

As described earlier, the 163 rules include environment-dependent ones. To let the agent decide which rules to exclude because they do not apply to the project, generate an environment profile.

▼List 2.8 Generate the environment profile (run from skills/ in the cloned repository)

```
python3 unity-perf-review/scripts/detect-env.py \
    /path/to/your-repo /path/to/your-repo/tuning-env.md
```

If you run it after placement, replace the paths with the placement location (the skill directory of the agent you use, such as .claude/skills/ or .agents/skills/). Even in a monorepo where the Unity project is in a subdirectory rather than at the repository root, passing the repository root makes the script search the configuration files recursively and detect them. Running the scripts requires python3 (in environments without it, you can create tuning-env.md manually, or skip this step and operate by checking all rules. The primary scan script scan.py also uses python3, but without it the agent falls back to running grep directly). The script estimates the Unity version, render pipeline, Scripting Backend, and presence of Addressables from ProjectVersion.txt, Packages/manifest.json, GraphicsSettings.asset, and ProjectSettings.asset. These are only estimates from the configuration files, and the authoritative information is what you enter manually in tuning-env.md. Items that could not be detected (for example, when the Scripting Backend differs by platform) become unknown and are treated conservatively (the rule remains a target for checking). Open the generated tuning-env.md and check its contents; in particular, manually correct the shipping platforms and whether SideFX VAT is used, and confirm that render_pipeline has the expected value.

2.6.4 Project-Specific Adjustments (Optional)

If some places intentionally deviate from the rules, record level overrides in rules-overrides.md. Examples are allowing LINQ during initialization or disabling the logging rule in Editor tools. The division of roles is that the environment profile handles excluding rules that are "fundamentally inapplicable in that environment," while over-

rides handle "adjustments for project circumstances." To add or modify rules themselves, edit rules-*.md in references/ and regenerate the checklists with python3 scripts/gen-checklists.py references/. The checklists are generated files, so do not edit them directly. This cycle of adding and regenerating can be completed with the distributed package alone.

2.6.5 Running It

Simply ask the agent. Normally the skill is triggered automatically when the request matches the description (triggering is a model decision, so to ensure it is used, state the skill name unity-perf-review in the request or use each agent's explicit invocation).

▼List 2.9 Example review requests

```
Review this diff from a performance perspective
Assets/Scripts/Battle/ Check everything under this path against the Tuning Bible rules
```

The Coverage section at the end of the output shows the number of rules checked, the number of rules skipped because of the environment (with IDs), and the number of grep hits. These counts are transcribed from the output of the primary scan script and do not depend on the agent's memory (only the counts of findings adopted and false positives excluded in the secondary review are tallied by the agent). If you feel too many rules are excluded, adjust tuning-env.md; if there are too many findings, adjust rules-overrides.md.

2.7 Operational Considerations

These are points I noticed from actually adopting the skill in a project. **For the first run, start with a diff or a specific directory rather than the whole repository.** The primary scan finishes quickly even on a whole project, but in a codebase with a long operating history, dozens of rules can hit, and the load of the secondary review (excluding false positives) concentrates there. Reviewing a narrow scope, moving permanently accepted items into overrides, and then widening the target makes findings take hold without being buried in noise. **Static analysis also flags intended settings.** For example, a texture that is intentionally left uncompressed also hits AS02 (texture compression not set). Do not take the detection results at face value and fix everything in bulk; confirm the intent one item at a time before

fixing. Recording permanently accepted items in overrides with a reason reduces noise from the next run onward. The unit of adjustment in overrides is the whole rule or a context (in-game = hot path / init = initialization / tooling = Editor); for exceptions limited to specific assets, write the path and reason in the notes field as input for the agent's judgment (a dedicated scope specification is not supported in the current version). **Measured values come with a measurement environment.** The numbers quoted in the rules are measurements on a specific Unity version and device. A measurement on a Pixel 8a cannot be extrapolated to decisions about the latest high-end devices. Write the target version in the rule itself, and use updates to Unity or the render pipeline as triggers for reviewing rules. **The final judgment rests with people.** To borrow the words of another chapter of this book, "people decide and revisit the conventions, and machines enforce them." An AI agent is a new implementation of the "machine that enforces," and humans remain responsible for the validity of the conventions, that is, the evaluation items. If you disagree with an agent's finding, treat it as a signal to improve the rule's detection pattern or reason field.

2.8 Conclusion

What I realized from turning the findings of the Tuning Bible into rules is that "a manuscript that states measured values together with remedies becomes a machine-readable evaluation item almost as it is." Conversely, a rule of thumb without supporting numbers shows its weakness at the rule-writing stage. This is also feedback for the writing side. Findings that state the measurement environment and numbers reach not only the readers of the book but every code review through AI agents. The skill and rule set introduced in this chapter are a starting point rather than a single correct answer. I hope they grow into a living set of conventions, as each project's overrides mature and findings from new chapters are added as rules.

[*2] Claude Code Skills documentation `https://code.claude.com/docs/en/skills`

[*3] GitHub Docs "About agent skills" `https://docs.github.com/en/copilot/concepts/agents/about-agent-skills`

[*4] Gemini CLI Agent Skills documentation `https://geminicli.com/docs/cli/using-agent-skills/`

Chapter 3

Introduction

Yuki Shiotsuka

QualiArts

Title: Safely Pooling Animators with ResetControllerState Name：Yuki Shiotsuka Affiliation：QualiArts

This article covers Animator.ResetControllerState, added in Unity 6.3.

This article organizes ResetControllerState as an API for implementing initialization on return to a pool clearly and easily, and explains safe pool-return handling based on the behavior confirmed in Unity 6000.3.21f1.

3.1 Problems that occur with Animator pooling

ObjectPool is a standard technique that reduces the load of frequent creation and destruction by reusing instances that have already been created. Unity's ObjectPool also provides hooks that let you run code when an instance is retrieved, returned, or destroyed [*1].

If you deactivate a GameObject for pooling, the Animator performs a Rebind when the GameObject is activated again, which can cancel out the performance gained from pooling. For this reason, the official Unity description of the Animator class recommends disabling the component rather than the GameObject [*2]. This article assumes a pool that follows this recommendation: GameObjects are kept active,

[*1] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html

[*2] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.html

the Animator and Renderer are disabled on return, and they are enabled again on acquisition.

On the other hand, because a pool reuses instances, a GameObject or component can come back still holding the state from its previous use. The same applies to the Animator: disabling it stops only the evaluation, and the internal state remains as it is. If nothing is done, the next use starts with the State and the various Parameters carried over from the previous use, so playback can start from a State left midway in the previous use, or an unintended transition can occur because of the remaining Parameters.

ResetControllerState is an API that returns the State and Parameters of each Layer of an AnimatorController to their default state in one call. The release notes also introduce it as an addition that makes it easier to pool animated characters [*3].

3.2 What ResetControllerState Resets

If you specify true for the resetParameters argument of ResetControllerState, the Parameters are returned to their default values in addition to the State. If you specify false, only the State is returned [*4]. When returning a character with an Animator to the pool, normally use ResetControllerState(true) to return the State and Parameters together. Regardless of whether they are unused, used in regular transitions, Any State transitions, or BlendTrees, Float, Bool, and Int parameters are returned to their default values and Trigger parameters are reset. It also returns the current State of all Layers to the default State.

However, the timing at which the reset takes effect differs by item. Parameter values became the default values immediately after the call, but GetCurrentAnimator StateInfo returned the State from before the reset until the Animator was next evaluated.

AnimatorControllerPlayable also provides a ResetControllerState method that re-

[*3] https://unity.com/releases/editor/whats-new/6000.3.0f1

[*4] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.ResetControllerState.html

sets the State and Parameters of the Controller [*5].

3.3 What ResetControllerState Does Not Reset

On the other hand, the following items are not returned to their default values by ResetControllerState alone.

- Layer Weight changed with SetLayerWeight
- Animator.speed
- Custom runtime variables implemented in a StateMachineBehaviour

Layer Weight and Animator.speed are properties separate from the State and Parameters of the Controller. If you do not want changed values to remain when returning to the pool, restore them explicitly. Custom variables of a StateMachineBehaviour also require individual initialization.

In addition, the state of other components such as Rigidbody, ParticleSystem, and AudioSource, and the state of custom logic, are managed outside the Animator Controller and are therefore not covered by ResetControllerState.

3.4 Implementation Example

List 3.1 is an example that combines initialization and disabling on return to the pool. It is intended to be called from actionOnRelease of ObjectPool<T>. Depending on the project's specifications, it includes processing that is not needed.

▼List 3.1 Animator initialization example on return to the pool

```
// Combine the initialization and disabling required on return to the pool
public void ReturnToPool()
{
    // Return the State and Parameters of each Layer of the AnimatorController
    animator.ResetControllerState(true);
    // Set defaultLayerWeight separately as the value to restore on reuse
    animator.SetLayerWeight(layerIndex, defaultLayerWeight);
    // Reset the playback speed
    animator.speed = 1f;
    // Only when needed: complete the first evaluation after the reset within the return
```

[*5] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animations.AnimatorControllerPlayable.ResetControllerState.html

```
     processing
    animator.Update(0f);
    // Restore custom state outside the Animator
    ResetGameState();
    // Keep the GameObject active, stop evaluation and rendering, and store it
    animator.enabled = false;
    characterRenderer.enabled = false;
}
```

3.5 Summary

An ObjectPool is not complete just by reducing the number of creations and destructions. You need to decide what state a reused GameObject should start from on its next use, and design the processing that returns it to that state.

If you store the Animator disabled without deactivating the GameObject, the internal state of the Animator remains as it is. Using ResetControllerState(true) returns the State of each Layer of the AnimatorController and the Float, Bool, Int, and Trigger Parameters to their default values together. On the other hand, Layer Weight, Animator.speed, custom variables of a StateMachineBehaviour, and the state of other components and custom logic are not reset.

Separating what to initialize into items inside and outside the AnimatorController clarifies the responsibilities of the pool-return handling. It is advisable to build the return handling around ResetControllerState and supplement it only with project-specific state.

3.6 References

- Unity Scripting API - ObjectPool<T0>
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html`
- Unity Scripting API - Animator
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.html`
- Unity 6000.3.0f1 Release Notes

https://unity.com/releases/editor/whats-new/6000.3.0f1

- Unity Scripting API - Animator.ResetControllerState
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animator.ResetControllerState.html
- Unity Scripting API - AnimatorControllerPlayable.ResetControllerState
  https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Animations.AnimatorControllerPlayable.ResetControllerState.html

Chapter 4

Tuning Unity 3D Character Animation

Shunsuke Saito

Colorful Palette

- Name: Shunsuke Saito
- Affiliation: Colorful Palette

4.1 Introduction

This chapter introduces optimization techniques for 3D character animation in Unity from several perspectives, including asset settings and implementation.

4.1.1 Measurement environment

The implementation and measurements in this chapter were performed in the following environment.

Table 4.1: Measurement environment

Item	Value
Unity	6000.5.7f1
Platform	macOS Standalone (StandaloneOSX)
Scripting Backend	IL2CPP / IL2CPP Compiler Configuration = Release / Code Generation = OptimizeSpeed
Graphics API	Metal / GPU Skinning = Enabled
Quality Level	PC / Skin Weights = Four Bones

4.1.2 Assets used and their structure

The character used for measurement is based on unity-chan and was re-imported as a **Generic rig**. Before and after applying Optimize Game Objects, described later, it has the following specifications.

Table 4.2: Transform count and vertex count of the prefabs used for measurement

Prefab	Transform count	Vertex count
Base	166	13,391
Optimized（Optimize Game Objects）	24	13,391

4.2 Basic control of Unity Animation

First, let us outline the three methods for controlling animation in Unity.

4.2.1 Control using the Animator component

This is the most common method: assign `RuntimeAnimatorController` to `Animator`. It provides state machines, transitions, blend trees, and parameters, so the Mecanim workflow can be used as is.

▼List 4.1 Control with the Animator component

```
var animator = GetComponent<Animator>();
animator.runtimeAnimatorController = controller;   // Assign the state machine
animator.SetFloat("Blend", 0.5f);                  // Control with parameters
```

Note that the Animator internally **builds and runs a Playable Graph**. Both state machine evaluation and clip sampling are processed internally as graph nodes. The next two methods amount to building this internal graph yourself.

4.2.2 Build a Playable Graph with the Playable API and control it with automatic updates

In this method, you build `PlayableGraph` yourself. You can fully control the blend structure without a state machine, and you can insert an Animation Job (`Animation`

`ScriptPlayable`), described later, at any position in the graph.

▼List 4.2 Graph construction with the Playable API (automatic update)

```
var graph = PlayableGraph.Create("MyGraph");
var output = AnimationPlayableOutput.Create(graph, "Animation", animator);

var clipA = AnimationClipPlayable.Create(graph, waitClip);
var clipB = AnimationClipPlayable.Create(graph, walkClip);
var mixer = AnimationMixerPlayable.Create(graph, 2);
graph.Connect(clipA, 0, mixer, 0);
graph.Connect(clipB, 0, mixer, 1);
mixer.SetInputWeight(0, 0.5f);
mixer.SetInputWeight(1, 0.5f);

output.SetSourcePlayable(mixer);
graph.Play();
```

The update mode defaults to `DirectorUpdateMode.GameTime`. In this mode, the graph is **evaluated automatically** in the Unity lifecycle (the Animation phase between Update and LateUpdate), at the same timing as the Animator component. The caller does not need to do anything every frame.

4.2.3 Build a Playable Graph with the Playable API and control it with manual updates

If you set `DirectorUpdateMode.Manual` on the same graph, Unity stops evaluating it automatically, and the graph is evaluated only when you call `Evaluate()`.

▼List 4.3 Graph evaluation with manual updates

```
graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
graph.Play();

void Update()
{
    graph.Evaluate(Time.deltaTime);   // Evaluated only when called
}
```

Because you fully control the timing and frequency of evaluation, you can implement controls such as “stop updating only specific characters (graphs)” or “update only once every few frames” directly.

The relationship among the three methods is summarized below.

Table 4.3: Comparison of the three animation control methods

Method	Graph construction	Evaluation timing
Animator component	Unity (internal)	Unity's Animation phase
Playable API + automatic update	You	Unity's Animation phase
Playable API + manual update	You	When `Evaluate()` is called

4.3 Asset settings

Before introducing code-side optimizations, let us look at the optimizations that can be applied through asset settings. The theme of this section is **eliminating the bone Transform hierarchy**. There are two ways to achieve the same result.

4.3.1 Optimize Game Objects (at import)

Enabling `Optimize Game Objects` in the FBX import settings (Rig tab) stops the bone GameObjects / Transforms from being created at runtime, and the Animator holds only an internal skeleton. For this character, the number of **Transforms dropped from 166 to 24** (root + 23 Renderers).

Note: Because the bones no longer exist as GameObjects, code that traverses the hierarchy to access bones stops working. If you need only specific bones, such as weapon attachment points, keep them individually with `Extra Transforms to Expose`.

4.3.2 Optimize Hierarchy (runtime)

The API that does the same thing at runtime is `AnimatorUtility.OptimizeTransformHierarchy`.

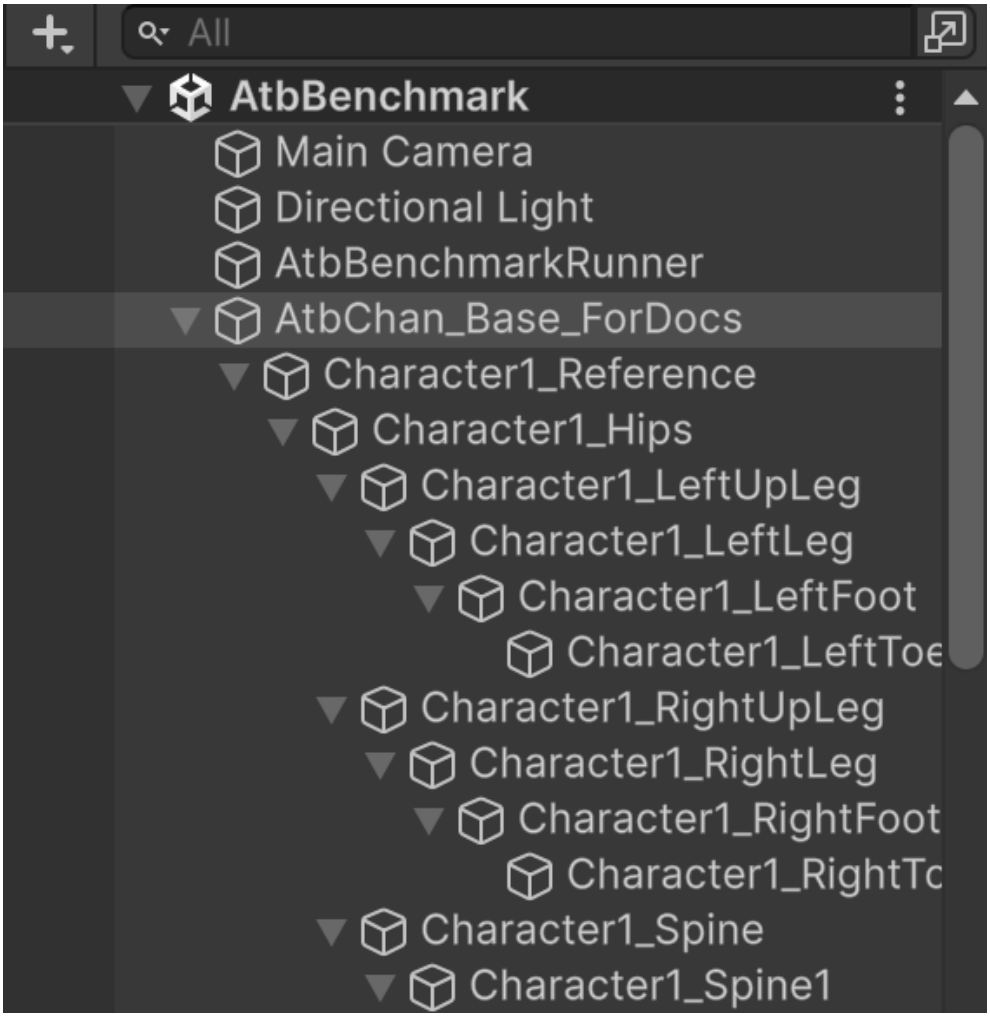
▼List 4.4 Hierarchy flattening at runtime

```
// go is the root that has an Animator (and a valid Avatar). The second argument is an
    array of the bone names to expose
```

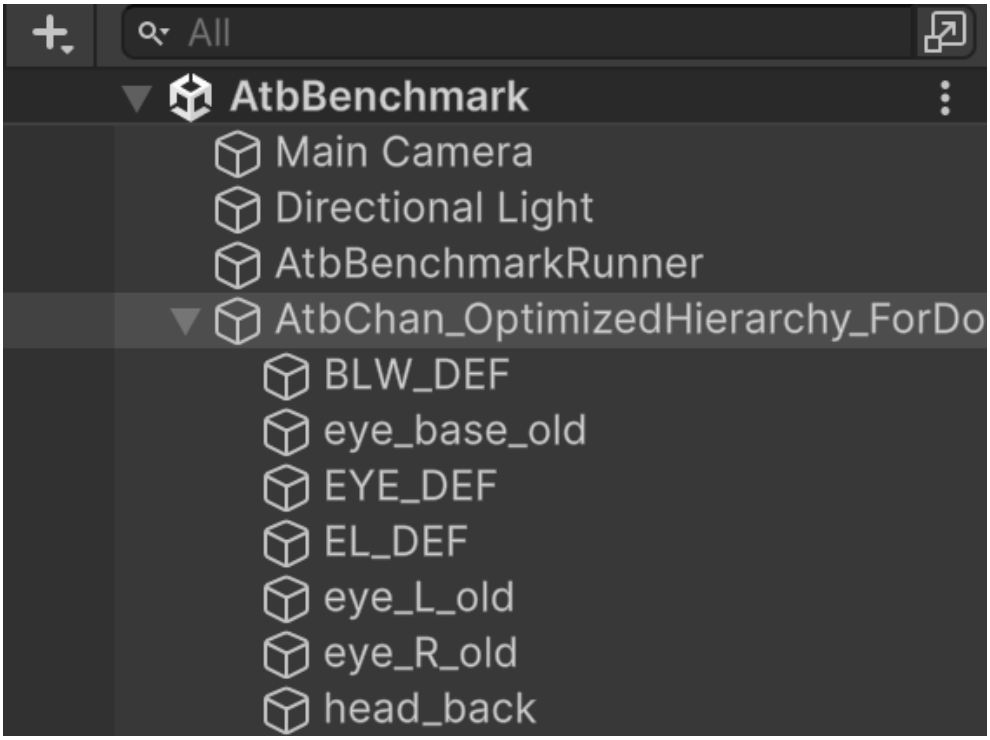
```
AnimatorUtility.OptimizeTransformHierarchy(go, null);

// There is also an inverse operation (for when you want to temporarily restore the
    hierarchy for Editor extensions or debugging)
AnimatorUtility.DeoptimizeTransformHierarchy(go);
```

Even for assets whose import settings you cannot change, running it at instantiation produces the same structure. In our check, applying it to the Base prefab (166 Transforms) resulted in **24 Transforms and** `hasTransformHierarchy == false`, which was exactly the same structure as the prefab with Optimize Game Objects applied at import.



▲Figure 4.1 Hierarchy before optimization. 166 bone Transforms continue under `Character1_Hips`.



▲Figure 4.2 After applying `AnimatorUtility.OptimizeTransformHierarchy`. The bone hierarchy disappears, leaving only 23 SkinnedMeshRenderers directly under the root (24 Transforms).

4.3.3 Measurement: effect of hierarchy flattening

We measured at two levels of rendering load. The first is a configuration with all Renderers enabled (300 characters × 23 Renderers = 6,900 Renderers). The second reduces the rendering load by enabling only the one SkinnedMeshRenderer with the most vertices for each character. Frame times are medians.

Table 4.4: Effect of hierarchy flattening (300 characters with all Renderers, and 600 characters with 1 Renderer)

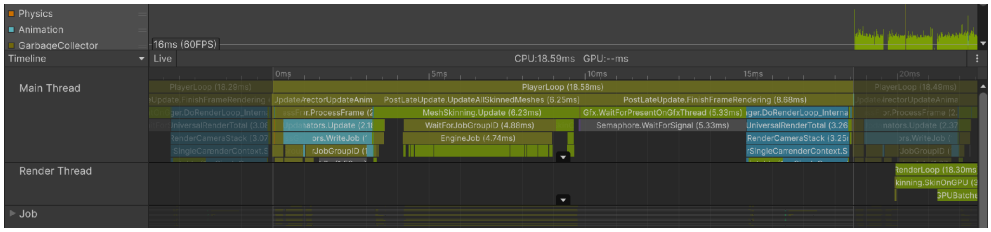
Configuration	Variant	Frame time	`MeshSkinning.Update` (main)	`MeshSkinning.CalcMatrices` (all)	`Semaphore.WaitForSignal`
300 characters, all Renderers	Base	18.59 ms	6.23 ms	36.633 ms	5.33 ms
300 characters, all Renderers	Optimize Game Objects	18.497 ms (±0)	**1.537 ms** (- 75.7 %)	**0 ms**	**10.255 ms**
300 characters, all Renderers	Optimize Hierarchy	18.307 ms (±0)	**1.504 ms** (- 76.2 %)	**0 ms**	**9.977 ms**
600 characters, 1 Renderer	Base	6.797 ms	0.735 ms	4.385 ms	5.741 ms
600 characters, 1 Renderer	Optimize Game Objects	**5.991 ms (- 11.9 %)**	0.207 ms	**0 ms**	4.901 ms
600 characters, 1 Renderer	Optimize Hierarchy	**6.009 ms (- 11.6 %)**	0.208 ms	**0 ms**	5.010 ms

CPU-side processing decreases by about the same proportion in both configurations. Main-thread skinning drops from 6.321 to about 1.5 ms (all Renderers) and from 0.735 to about 0.21 ms (1 Renderer), a reduction of more than 70 % in both cases. The skinning matrix calculation (`MeshSkinning.CalcMatrices`) **disappears completely**, from 36.6 ms / 4.4 ms total across all threads.

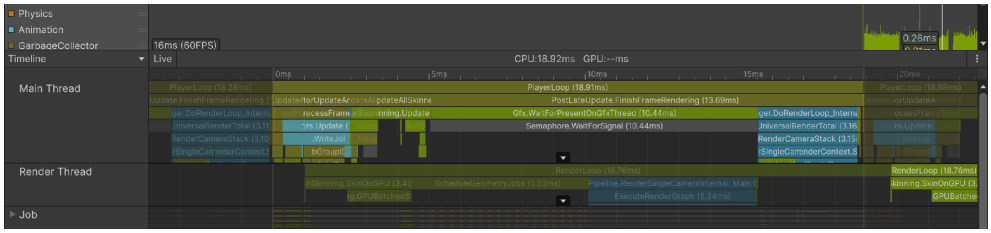
In the all-Renderers configuration, however, render thread processing is the critical path for frame time, so no reduction in frame time is seen.

In the 1-Renderer configuration with reduced rendering load, the same reduction appears directly as a **frame time reduction of around 12 %**. The trend is the same with 300 characters: 3.312 → 2.899 / 2.901 ms (- 12.5 % / - 12.4 %). **The difference between Optimize Game Objects at import and Optimize Hierarchy at runtime is less than 0.1 %**, which shows that when the structure is the same, the effect is the same.

The same thing appears in the Profiler Timeline. Both are frames near the median frame time, with 300 characters and all Renderers.



▲Figure 4.3 Base (166 Transforms). Of the 18.59 ms frame time, `MeshSkinning.Update` occupies 6.23 ms on the main thread, and `Gfx.WaitForPresentOnGfxThread` / `Semaphore.WaitForSignal` takes 5.33 ms.



▲Figure 4.4 After applying `AnimatorUtility.OptimizeTransformHierarchy`. The `MeshSkinning.Update` block shrinks until its label is unreadable, while `Semaphore.WaitForSignal` grows from 5.33 to 10.44 ms and `PlayerLoop` stays almost unchanged, from 18.58 to 18.91 ms. The CPU time that was removed is replaced by wait time.

4.3.4 Measurement: all Renderers disabled (animation evaluation only)

We remove rendering and skinning entirely and look at the contribution to animation evaluation itself, with 300 characters.

Table 4.5: Frame time with all Renderers disabled

Variant	Frame time
Base	2.702 ms

Table 4.5: Frame time with all Renderers disabled

Variant	Frame time
Optimize Game Objects	2.687 ms (- 0.6 %)
Optimize Hierarchy	2.496 ms (- 7.6 %)

Evaluation processing such as clip sampling and blending runs regardless of whether a Transform hierarchy exists, so the contribution here is almost nil. This shows that the purpose of this optimization is mainly to reduce the **skinning matrix calculation and the write-back to Transforms**, not the animation evaluation itself.

4.3.5 Conclusions of this chapter

- Hierarchy flattening reduces the **skinning matrix calculation (`MeshSkinning.CalcMatrices`)**
- The effect is equivalent between import time (Optimize Game Objects) and runtime (`AnimatorUtility.OptimizeTransformHierarchy`)
- Which to choose is an operational matter: use the import settings for assets you own, and use the runtime API when you cannot change the import settings
- **Appropriately setting the vertex and bone counts of the model itself during production** is the top priority

4.4 Using Job Threads

4.4.1 Unity's animation parallelization

For the Animator component and automatically updated Playable Graphs, Unity **schedules all graphs together and distributes them across the Animation Job Threads**.

The problem is manual updates. `PlayableGraph.Evaluate()` **parallelizes the evaluation inside that graph** and does not parallelize across graphs. Therefore, if you control N characters with the intuitive "1 character = 1 Playable Graph" configuration, you call `Evaluate()` N times, and **the N graphs are processed serially**.

This is an unavoidable specification of Unity's Playable Graph, but the countermea-

sure is simple: **combine the N characters into one graph**.

▼List 4.5 Consolidating all characters into one graph

```
// Attach each character's output and subtree to a single graph
_sharedGraph = PlayableGraph.Create("Shared");
_sharedGraph.SetTimeUpdateMode(DirectorUpdateMode.Manual);

foreach (var character in characters)
{
    var output = AnimationPlayableOutput.Create(_sharedGraph, "Animation", character.
    Animator);
    output.SetSourcePlayable(BuildClipSource(_sharedGraph));   // Mixer for each
    character
}

_sharedGraph.Play();

// Only one update
void Update() => _sharedGraph.Evaluate(Time.deltaTime);
```

4.4.2 Measurement 1: control method × number of characters

All Renderers are disabled to isolate the cost of animation evaluation. Frame times are medians.

Table 4.6: Frame time by control method and number of characters

Control method	100 characters	300 characters	600 characters
Animator component	0.899 ms	2.413 ms	**5.004 ms**
Playable API + automatic update	0.899 ms	2.498 ms	**5.093 ms**
Playable API + manual update (1 graph per character)	1.395 ms	3.925 ms	**8.396 ms**
Playable API + manual update (1 graph for all)	0.800 ms	2.115 ms	**4.500 ms**

The Animator and Playable automatic updates have almost the same cost. The claim that the Playable API is lighter does not hold, at least under the condition of simple animation control. The Animator's state machine evaluation (`AnimatorControllerPlayable.PrepareFrame`) takes only 2.624 ms in total across all threads even

with 600 characters, of which only 0.161 ms is on the main thread, so it is extremely unlikely to become a bottleneck. (Whether 2 ms is large or small, however, depends on the game.)

Manual updates vary by a factor of 1.87 depending on the graph structure. Even with the same 600 characters, depending on how the graphs are grouped, the result can be faster or slower than automatic updates.

4.4.3 Measurement 2: how threads are used

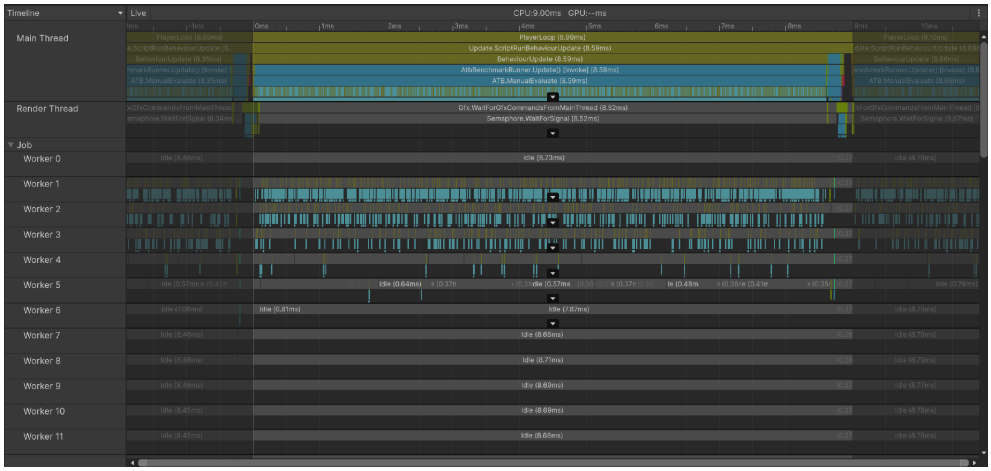
600 characters, Renderers disabled. These are results aggregated per thread from Unity Profiler captures.

Table 4.7: Difference in thread usage by graph granularity (600 characters)

1 graph per character (600 graphs)	1 graph for 600 characters	
Frame time	**9.490 ms**	**4.928 ms**
`Evaluate` on the main thread	9.028 ms	4.556 ms
Number of worker threads that handled animation processing	**5**	**15**
`Animator.ProcessGraph` total across all threads	0.920 ms	**19.034 ms**
`Animator.ProcessGraph` per worker	0.32 / 0.15 / 0.03 / 0.02 / 0.002 ms	**about 1.19 ms** each (15 workers, nearly even)

With 1 graph per character, almost all animation processing is on the main thread. The 600 `Evaluate()` calls simply run one after another, and the worker threads barely work.

With 1 graph for 600 characters, all 15 worker threads execute `Animator.ProcessGraph` almost evenly.



▲Figure 4.5 Timeline for 1 graph per character (600 graphs). `ATB.ManualEvaluate` on the main thread occupies about 8.6 ms, and most Job workers remain Idle.



▲Figure 4.6 Timeline for 1 graph for 600 characters. `ATB.ManualEvaluate` on the main thread is halved to about 4.3 ms, and all workers simultaneously execute `EngineJob` for about 1.2 ms each.

One note on how to read the numbers.

The numbers in this table come from captures taken with the Unity Profiler at-

tached, so they are slightly heavier than the measurements taken without the Profiler (4.500 ms / 8.396 ms). The summary table at the end of the chapter uses the numbers without the Profiler attached.

4.4.4 Conclusions of this chapter

Table 4.8: Summary of frame times with 600 characters

Configuration	Frame time with 600 characters
1 graph per character + manual update	8.396 ms
Animator (automatic)	5.004 ms
1 graph for 600 characters + manual update	**4.500 ms**

- For the Animator and automatically updated Playable Graphs, Unity distributes all graphs together across the Animation Job Threads
- Manual updates are parallelized only per graph
- If you choose manual updates, always include graph granularity as a design consideration
- Replacing with the Playable API has almost no optimization effect by itself. The real value of using the Playable API lies in the **flexibility of animation control**, such as being able to insert Animation Jobs and control the graph structure and update timing

4.5 Using Animation C# Jobs and SIMD support

The need to control animation procedurally always arises during game development. One example is making a character's feet follow the shape of the ground so that they stay in contact with it.

One technique for this case is Two-Bone IK.

When actually implementing procedural animation control such as Two-Bone IK, there are three options.

1. Rewrite Transforms directly with `LateUpdate`
2. Insert `IAnimationJob` into `AnimationScriptPlayable`

3. In addition to 2, compile with Burst

Here we focus on the performance of these methods.

All three were implemented to perform **exactly the same computation**. The solver is factored out into a single static method, and the LateUpdate, Job, and Burst versions share it, so the only difference is where it runs.

4.5.1 What are Animation C# Jobs

Animation C# Jobs is a mechanism that, through `AnimationScriptPlayable` and `IAnimationJob`, lets you **insert your own processing into the animation evaluation pipeline**. A job is called in the same flow as clip evaluation and blending, and accesses bones through `AnimationStream` and `TransformStreamHandle`.

The difference from `LateUpdate` is where the code runs. `LateUpdate` rewrites the Transforms directly on the main thread after animation evaluation has finished. The processing accumulates serially on the main thread in proportion to the number of characters. An Animation Job is called inside the evaluation pipeline (on the Animation Job Thread), so Unity can distribute it across worker threads just like the rest of the animation evaluation. Even with the same computation, whether it burdens the main thread changes.

The access restriction of going through `AnimationStream` is the price for this. A job can see only the handles bound in advance, and cannot touch an arbitrary `Transform` or `GameObject`.

4.5.2 Implementation

The job body simply calls the solver with `ProcessAnimation`.

▼List 4.6 Animation Job for Two-Bone IK (Burst-compatible)

```
[BurstCompile]
public struct TwoBoneIkJobBurst : IAnimationJob
{
    public TransformStreamHandle Root;
    public TransformStreamHandle Mid;
```

```
    public TransformStreamHandle Tip;
    public NativeArray<float3> Target;   // Length 1. Written from the main thread
    every frame
    public float Weight;
    public int Iterations;

    public void ProcessRootMotion(AnimationStream stream) { }

    [BurstCompile]
    public void ProcessAnimation(AnimationStream stream)
    {
        TwoBoneIkCore.SolveStreamIterations(
            ref stream, Root, Mid, Tip, Target[0], Weight, Iterations);
    }
}
```

Bones are read and written with the pair of `TransformStreamHandle` and `AnimationStream`.

▼List 4.7 Reading and writing bones from the AnimationStream

```
static void SolveStream(
    ref AnimationStream stream,
    TransformStreamHandle root, TransformStreamHandle mid, TransformStreamHandle tip,
    float3 target, float weight)
{
    var rootPosition = (float3)root.GetPosition(stream);
    var midPosition = (float3)mid.GetPosition(stream);
    var tipPosition = (float3)tip.GetPosition(stream);
    var rootRotation = (quaternion)root.GetRotation(stream);
    var midRotation = (quaternion)mid.GetRotation(stream);

    Solve(rootPosition, midPosition, tipPosition, target, rootRotation, midRotation,
    weight,
        out var newRootRotation, out var newMidRotation);

    mid.SetRotation(stream, newMidRotation);
    root.SetRotation(stream, newRootRotation);
}
```

Create the handles with `Animator.BindStreamTransform`, and insert the job into the graph after the clip.

▼List 4.8 Inserting an Animation Job into the graph

```
var job = new TwoBoneIkJobBurst
```

```
{
    Root = animator.BindStreamTransform(rootTransform),
    Mid = animator.BindStreamTransform(midTransform),
    Tip = animator.BindStreamTransform(tipTransform),
    Target = ikTarget,   // NativeArray<float3>
    Weight = 1f,
    Iterations = iterations,
};

var ikPlayable = AnimationScriptPlayable.Create(graph, job, 1);
graph.Connect(clipSource, 0, ikPlayable, 0);   // clipSource = mixer on the clip side
ikPlayable.SetInputWeight(0, 1f);
output.SetSourcePlayable(ikPlayable);
```

Inputs that change every frame, such as the IK target, are placed in `NativeArray` and written from the main thread.

Burst is applied by adding the `BurstCompile` attribute to the class.

You can check whether Burst Compiler optimization is actually taking effect with tools such as the Burst Inspector.

4.5.3 Measurement

300 characters, Renderers disabled. The number of IK solves per character is varied among 1 / 16 / 64.

Table 4.9: Frame time by number of IK solves (300 characters, Renderers disabled)

Implementation	1 solve	16 solves	64 solves
No IK (baseline)	2.501 ms	2.601 ms	2.596 ms
LateUpdate	2.699 ms	4.400 ms	**9.094 ms**
Animation Job	2.411 ms	2.496 ms	**2.795 ms**
Animation Job + Burst	2.494 ms	2.496 ms	**2.792 ms**

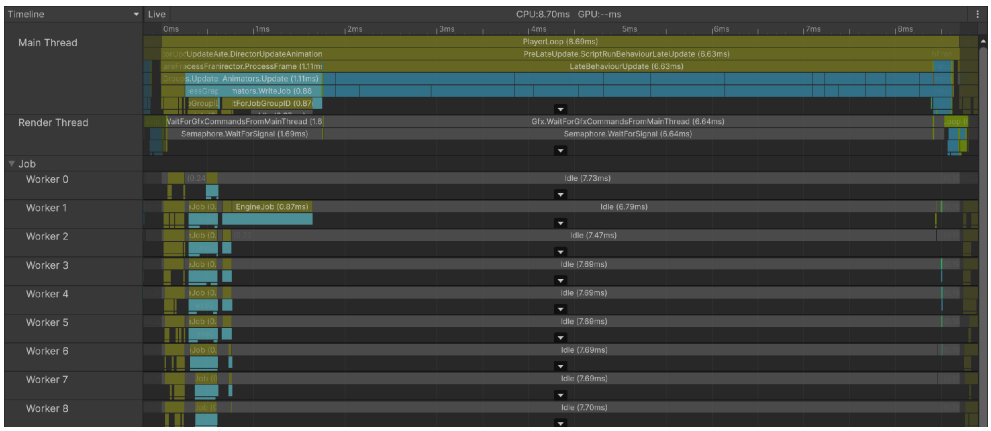
CPU time breakdown (at 64 solves):

Table 4.10: CPU time breakdown by IK implementation (64 solves)

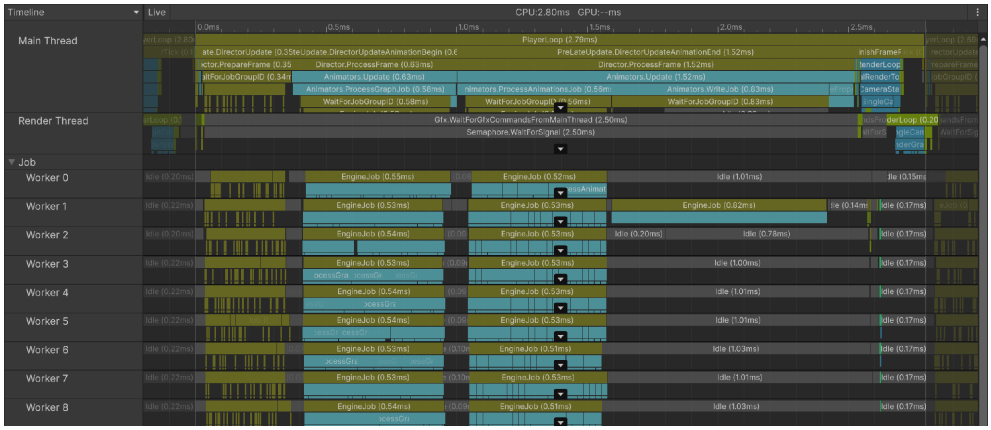
Implementation	IK CPU time (all threads)	Of which main thread	Added to frame time
LateUpdate	6.910 ms	**6.910 ms（100 %）**	+6.498 ms
Animation Job	7.790 ms	0.487 ms（6.3 %）	+0.199 ms
Animation Job + Burst	6.932 ms	0.436 ms（6.3 %）	+0.196 ms

4.5.4 Differences in execution threads seen in the Profiler Timeline

Which thread executes the same computation appears in the Unity Editor's Profiler. The following shows captures with 64 solves and 300 characters opened in the Profiler window, with a frame near the median frame time selected. Because these are captures taken with the Profiler attached, they do not match the numbers in the table above exactly.



▲Figure 4.7 LateUpdate implementation. Of the 8.698 ms frame time, `LateBehaviourUpdate` occupies 6.63 ms. The 15 worker threads stay `Idle` for most of the frame (about 6.8 to 7.7 ms).



▲Figure 4.8 Animation Job implementation. The same computation is distributed as `AnimationScriptPlayable.ProcessAnimation` across the 15 worker threads, each executing for about 0.5 ms. The frame time is 2.796 ms.

In the LateUpdate implementation, the `ATB.TwoBoneIk.LateUpdate` for 300 characters (6.577 ms in total) is entirely inside `LateBehaviourUpdate` on the main thread. The worker threads handle only the base animation evaluation (`Animator.ProcessGraph` at 0.12 to 0.29 ms per thread), and none of the IK computation appears on them.

In the Animation Job implementation, `AnimationScriptPlayable.ProcessAnimation` totals 7.891 ms across all threads, of which 0.498 ms is on the main thread. The remaining 7.393 ms is distributed almost evenly across the 15 worker threads (0.47 to 0.51 ms per thread). The total CPU time is higher than in the LateUpdate implementation, but because it is split across 16 threads, the frame time falls from 8.698 to 2.796 ms.

4.5.5 What can be read from this

In this case, the most effective optimization is not Burst but moving the execution thread. The LateUpdate implementation pays the entire IK CPU time on the main thread, so at 64 solves the frame time grows from 2.596 to 9.094 ms. Moving the same computation to an Animation Job almost eliminates the main-thread burden, and the amount added to frame time differs by a factor of **33**.

On the other hand, **the LateUpdate version has the lower total CPU time**. An Animation Job has a fixed cost per `AnimationScriptPlayable` node, which is about 2.16 ms for 300 characters. The gain from parallelization still far outweighs it.

Under the more realistic condition of one solve per character, the differences among the three implementations are within measurement noise. For Two-Bone IK with around 300 characters, no implementation makes a perceptible difference.

4.5.6 Burst does not help just by adding the attribute

The belief that adding `[BurstCompile]` makes code use SIMD and run several times faster is incorrect. In this measurement, Burst reduces the total CPU time by 11.0 %. Looking at the marginal cost per solve after subtracting the fixed node cost,

- Without Burst: (7.790 − 2.157) / 63 = **0.0894 ms**
- With Burst: (6.932 − 2.167) / 63 = **0.0756 ms** (− 15.4 %)

What Burst does is generate native code with LLVM, and whether SIMD instructions are emitted depends on the shape of the code as written. **Unless you implement code that can be properly vectorized with SIMD, you will not get a large benefit.**

Animation C# Jobs has two properties that make SIMD vectorization difficult.

Reads and writes of `AnimationStream` appear frequently in the processing. For each solve, bones are read with `GetPosition` and `GetRotation` and written back with `SetRotation`. Because the stream state changes with every iteration, dependencies remain between iterations, and multiple iterations cannot be bundled into a single vector instruction. Furthermore, the `AnimationStream` received by `IAnimationJob` covers only one character, so the job does not contain the data needed to compute four characters at once in the first place.

Transcendental functions are used heavily. These include `acos`, which obtains the bend angle from the law of cosines, the trigonometric functions that `quaternion.AxisAngle` calls internally, and the square root in `math.length`. These become cross-lane computations or scalar library calls, which stops vectorization.

Burst's LLVM also **explicitly reports that it cannot vectorize** the loop that repeatedly calls the solver (optimization remark `loop not vectorized: instruction return type cannot be vectorized`).

4.5.7 Strategy for fully using the benefits of Burst

To make full use of Burst, a candidate is a configuration that uses `C# Job System` to **read and write Transforms without going through** `AnimationStream`. Reading and writing Transforms can be consolidated on the job side with `TransformAccessArray` and `IJobParallelForTransform`, and you can also leave only bone gathering and write-back to the Animation Job and split the heavy computation into a batch job that handles N characters together. In both cases, **the data for N characters is gathered inside a single job** rather than per character, so the precondition for SIMD vectorization is met.

In addition, the contents of the batch job need to be arranged as follows.

- Use SoA (Structure of Arrays), with one array per component, so that characters are contiguous in memory
- Replace branches with `math.select` or `math.max` so that the loop body becomes a single straight line
- Move transcendental functions out of the loop (rewrite them with identities, use lookup tables, or approximate them)
- Write the code with `float4` rather than `float3`, one lane per character for four characters, so that it does not depend on the auto-vectorizer's decisions

The cost is design complexity. The greatest advantage of `IAnimationJob` is the convenience of being able to touch the latest bone state at the correct timing in the evaluation pipeline just by inserting one node into the graph, and this strategy gives that up. This chapter does not measure this configuration, so it does not show how much faster it would be. Treat it as a proposal: if you expect Burst to give several times the effect, a design change in this direction is a prerequisite.

4.5.8 Conclusions of this chapter

- The value of an Animation Job lies in being able to **use the Animation Job Thread**
- While the amount of computation is small, the fixed per-node cost of `AnimationScriptPlayable` is larger. Writing it plainly in LateUpdate gives a lower total CPU time
- Adding `[BurstCompile]` alone does not result in SIMD. You need to design the data and processing with SIMD application in mind

4.6 Implementing animation LOD

4.6.1 Mechanism and where it is effective

Depending on the game specification, there is basically no need to update even distant characters every frame. If you update only once every N frames, the average load (cost) of that character becomes 1/N. This is “animation LOD”. It applies the same idea as mesh LOD, which controls rendering cost in stages by distance, to the **animation update frequency**.

It is effective in situations such as the following.

- There are hundreds of characters on screen, and most are in the background or are crowd characters
- Motion equivalent to 30 fps / 15 fps is perceptually acceptable for distant characters (the farther the camera, the less noticeable the thinning)
- Animation evaluation accounts for a non-negligible share of the frame time

Conversely, if it is applied to a character in close-up, the stutter is immediately noticeable. In practice, the expected approach is to switch the thinning level according to distance or screen occupancy.

4.6.2 Simple implementation: call `Animator.Update()` yourself

Set `animator.enabled = false` to stop Unity's automatic updates, and call `Animator.Update()` explicitly only on the frames that need an update. The time for the skipped frames is accumulated and passed in.

This implementation has the side effect of losing the parallelism on the Animation Job Thread described so far.

▼List 4.9 Thinning implementation that calls Animator.Update() itself

```
animator.enabled = false;   // At initialization. Stop Unity's automatic updates

void Update()
{
    foreach (var character in _characters)
    {
        character.PendingDeltaTime += Time.deltaTime;
        if (_frameIndex % interval != character.Slot) continue;   // Slot = index %
    interval

        character.Animator.Update(character.PendingDeltaTime);    // Explicit update
    with the accumulated time
        character.PendingDeltaTime = 0f;
    }
    _frameIndex++;
}
```

4.6.3 Implementation example using the Playable API directly

For a manually updated (`DirectorUpdateMode.Manual`) Playable Graph, you simply thin out the calls to `Evaluate()` itself.

▼List 4.10 Implementation that thins out Evaluate() on a manually updated graph

```
graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
graph.Play();

void Update()
{
    foreach (var character in _characters)
    {
```

```
        character.PendingDeltaTime += Time.deltaTime;
        if (_frameIndex % interval != character.Slot) continue;

        character.Graph.Evaluate(character.PendingDeltaTime);
        character.PendingDeltaTime = 0f;
    }
    _frameIndex++;
}
```

One note: in the configuration recommended in the previous chapter, “consolidate all characters into one graph”, the whole graph is evaluated with a single `Evaluate()` call, so **per-character thinning cannot be expressed**. With one graph per character, as in the code above, you can thin updates, but you lose the parallelization gained from consolidation.

In both implementations, `Slot` (phase) is important. If all characters are updated on the same frame, the load concentrates on that frame. If you offset the phase with `index % interval`, 1/N of the characters are updated every frame.

4.6.4 Measurement

600 characters, Renderers disabled. First, the implementation using the Animator.

The **mean** represents the total load over 12 seconds, and the **gap between the median and p95** represents how erratic the frame time is.

Table 4.11: Frame time statistics for thinning and phase offset (Animator, 600 characters)

Method	Mean	Median	p95	p99	Max	Standard deviation
Update every frame (Animator automatic)	5.037 ms	5.000 ms	5.303 ms	5.707 ms	16.901 ms	0.433
Once every 2 frames, no phase offset	4.781 ms	0.648 ms	9.607 ms	9.816 ms	12.194 ms	4.544

Table 4.11: Frame time statistics for thinning and phase offset (Animator, 600 characters)

Method	Mean	Median	p95	p99	Max	Standard deviation
Once every 2 frames, phase offset	4.831 ms	4.803 ms	5.099 ms	5.212 ms	8.804 ms	0.171
Once every 4 frames, no phase offset	2.613 ms	**0.382 ms**	**9.470 ms**	9.658 ms	11.171 ms	3.863
Once every 4 frames, phase offset	2.578 ms	**2.593 ms**	**2.707 ms**	2.908 ms	11.722 ms	0.185

The same trend appears with the Playable API (1 graph per character, manual update).

Table 4.12: Frame time statistics for thinning and phase offset (Playable API, 1 graph per character)

Method	Mean	Median	p95	Standard deviation
Update every frame	8.496 ms	8.493 ms	9.001 ms	0.337
Once every 4 frames, no phase offset	2.438 ms	0.369 ms	8.855 ms	3.570
Once every 4 frames, phase offset	2.486 ms	**2.495 ms**	**2.613 ms**	0.173

4.6.5 Offsetting the phase does not reduce the load; it flattens it

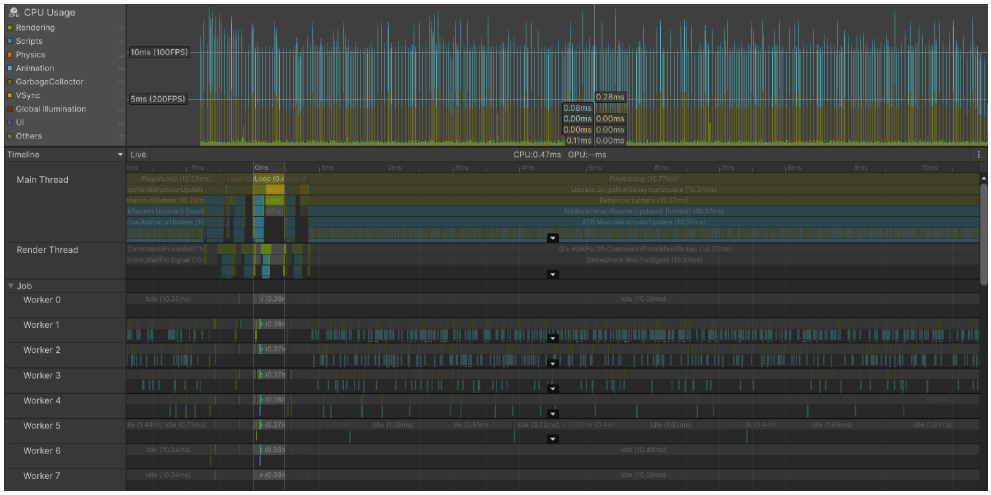
When updates are thinned to one quarter, the mean is almost the same with or without a phase offset, and **the total load is almost the same**. Offsetting the phase does not reduce the work.

The frame time distributions, however, differ as follows.

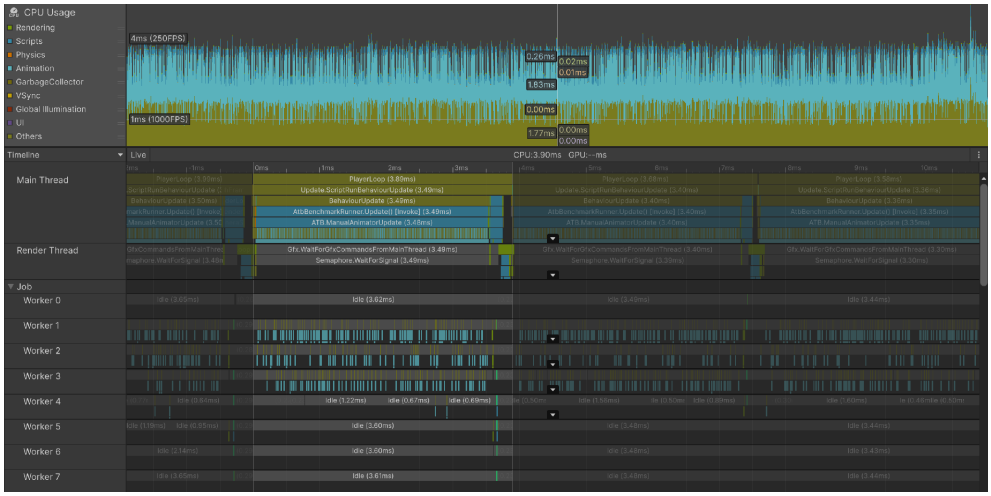
- No phase offset: out of every 4 frames, 3 frames do nothing, and 1 frame processes all 600 characters at once. The median drops, but p95 and the standard deviation jump
- Phase offset: 150 characters are processed every frame. The median and p95 nearly coincide, and the variation is small

If you look only at the median of 0.382 ms and celebrate a 1/13 reduction, you end up with a game that has a 9.5 ms spike once every 4 frames. **Phase offset is a measure for perceived quality**, and the standard deviation of 0.185 is more stable than the 0.433 of updating every frame.

This difference in distribution appears directly in the Profiler's CPU chart (both are captures with the Profiler attached, so they are heavier than the non-attached numbers in the table above. The vertical axis scale of the chart is auto-adjusted, so it differs between the two images).



▲Figure 4.9 Once every 4 frames, no phase offset. The CPU chart forms a comb shape of "3 valley frames near 0 ms + 1 spike frame of about 10 ms". In the Timeline below, on both sides of the selected empty frame (0.47 ms) are frames where `ATB.ManualAnimatorUpdate` processes all 600 characters at once (10.37 ms).



▲Figure 4.10 Once every 4 frames, phase offset. Because 150 characters are processed every frame, the waveform is flat (about 3.5 ms) and the spikes are gone.

4.6.6 The naive implementation loses parallelism

Thinning to once every 2 frames removes almost nothing. Although the number of updates is halved, the reduction in mean frame time is only **4.1 %**.

The reason can be seen in the breakdown of the Profiler markers.

Table 4.13: Parallelism lost in the naive thinning implementation

Method	Characters updated per frame	Animation update time on the main thread	`Animator.ProcessGraph` total across all threads	Of which main thread
Update every frame (Unity's batch update)	600	`Animators.Update` 4.046 ms	20.104 ms	1.257 ms
Once every 2 frames, phase offset	300	`ATB.ManualAnimatorUpdate` 4.532 ms	0.549 ms	0.317 ms

Table 4.13: Parallelism lost in the naive thinning implementation

Method	Characters updated per frame	Animation update time on the main thread	`Animator.ProcessGraph` total across all threads	Of which main thread
Once every 4 frames, phase offset	150	`ATB.ManualAnimatorUpdate` 2.316 ms	0.278 ms	0.180 ms

With an update every frame, only 6 % of the work remains on the main thread, and the rest is spread across the worker threads. When you call `Animator.Update()` explicitly, however, it is **evaluated synchronously on the spot**, so the worker threads are barely used. You give up the parallelization of Unity's batch update in exchange for thinning.

Converted to the main-thread cost per character,

- Update every frame: 4.046 ms / 600 characters = **6.7 µs per character** (parallel)
- Once every 2 frames: 4.532 ms / 300 characters = **15.1 µs per character** (serial)
- Once every 4 frames: 2.316 ms / 150 characters = **15.4 µs per character** (serial)

The moment it becomes serial, the cost per character rises by about 2.3 times, and after that it is constant regardless of the thinning ratio. In other words, **the 2x reduction obtained from 2x thinning is almost canceled out by the 2.3x degradation from serialization**. The measured 4.1 % is merely the residual.

With the naive implementation, a substantial effect appears only at 4x thinning (**−48.8 %**). This is, however, an avoidable loss.

4.6.7 Thinning while preserving parallelism

It is unavoidable that `Animator.Update()` is synchronous. However, if you **choose every frame which characters take part in Unity's batch update**, you can thin updates while preserving parallelism. There are two implementations, and both keep

using the Animator (and its state machine).

Method 1: toggle `Animator.enabled`

Unity's animation update collects the Animators that are enabled at that moment into one batch and distributes it across the Animation Job Threads. Therefore, instead of calling `Animator.Update()`, **you only need to set `enabled = true` on just the characters you want updated on that frame**. Unity performs the evaluation, so the parallelization remains.

▼List 4.11 Thinning implementation that toggles Animator.enabled

```
// At initialization. Make up for skipped frames with speed
foreach (var character in _characters)
{
    var animator = character.Animator;
    animator.keepAnimatorStateOnDisable = true;   // Do not rewind the state on
    disabling (required)
    animator.speed = interval;                    // Compensate for being updated only
    once every interval frames
}

void Update()
{
    foreach (var character in _characters)
    {
        var scheduled = _frameIndex % interval == character.Slot;
        if (character.Animator.enabled != scheduled)
        {
            character.Animator.enabled = scheduled;   // Only choose which characters
    take part in Unity's batch update
        }
    }
    _frameIndex++;
}
```

Do not forget `keepAnimatorStateOnDisable = true`. With the default `false`, the controller state is rewound every time the Animator is disabled, and the animation does not advance.

The constraint is that `Animator.speed` is a **rate**. You cannot pass the accumulated Δt exactly, as with `Animator.Update(pendingDeltaTime)`, and it advances only by the `Time.deltaTime` × `interval` of the update frame. In an environment where the

frame time fluctuates, the progress also fluctuates, but the average playback speed is preserved.

Method 2: split graphs by phase

The conclusion of the previous chapter was that “combining N characters into one manual graph enables parallelization”. We combine this with thinning. **Create as many graphs as there are phases, and call `Evaluate()` on only one graph per frame.** Each graph holds N/interval characters, so parallelization works within that single `Evaluate()` call.

If you have the graph hold the state machine as `AnimatorControllerPlayable`, the same evaluation runs as when it is assigned to an Animator component.

▼List 4.12 Thinning implementation that splits graphs by phase

```
// At initialization: one manual graph per phase bucket
for (var bucket = 0; bucket < interval; bucket++)
{
    var graph = PlayableGraph.Create($"Bucket_{bucket}");
    graph.SetTimeUpdateMode(DirectorUpdateMode.Manual);
    _bucketGraphs[bucket] = graph;
}

foreach (var character in _characters)
{
    var graph = _bucketGraphs[character.Slot];
    character.Animator.runtimeAnimatorController = null;   // Leave the evaluation to
    the graph
    var output = AnimationPlayableOutput.Create(graph, "Animation", character.Animator)
    ;
    output.SetSourcePlayable(AnimatorControllerPlayable.Create(graph, controller));
}

foreach (var graph in _bucketGraphs) graph.Play();

void Update()
{
    for (var bucket = 0; bucket < interval; bucket++) _pendingDeltaTime[bucket] += Time
    .deltaTime;

    var scheduled = _frameIndex % interval;
    _bucketGraphs[scheduled].Evaluate(_pendingDeltaTime[scheduled]);   // Only one
    bucket per frame
    _pendingDeltaTime[scheduled] = 0f;
```

```
    _frameIndex++;
}
```

With this method, you can **pass the accumulated Δt as is** to `Evaluate()`, so the progress is accurate. In exchange, you hold as many graphs as there are thinning levels, and to change the LOD level dynamically you must move characters between graphs (recreate their outputs).

Measurement: comparison of three implementations

600 characters, Renderers disabled. Mean frame time and the reduction rate relative to updating every frame (5.037 ms).

Table 4.14: Frame time comparison of three thinning implementations (600 characters)

Implementation	1/2 thinning	1/4 thinning
Explicit `Animator.Update()` call (phase offset)	4.831 ms (− 4.1 %)	2.578 ms (− 48.8 %)
`Animator.enabled` toggling	**3.350 ms (− 33.5 %)**	**1.835 ms (− 63.6 %)**
Manual graph per phase (`AnimatorControllerPlayable`)	**3.082 ms (− 38.8 %)**	**1.541 ms (− 69.4 %)**

The naive implementation is the least worthwhile. The difference is especially large at 1/2 thinning, where − 4.1 % became − 33.5 % / − 38.8 %. The conclusion of the previous section, “2x thinning is eaten up by the loss of parallelization”, **does not hold if you change the implementation**.

You can confirm that parallelism is preserved with the Profiler markers.

Table 4.15: Per-thread breakdown of three thinning implementations

Method	`Animator.ProcessGraph` total across all threads	Of which main thread	Update time on the main thread
Update every frame (600 characters per frame)	20.104 ms	1.257 ms	`Animators.Update` 4.046 ms
`Animator.Update()`, 1/4 (150 characters per frame)	0.278 ms	0.180 ms	`ATB.ManualAnimatorUpdate` 2.316 ms

Table 4.15: Per-thread breakdown of three thinning implementations

Method	`Animator.ProcessGraph` total across all threads	Of which main thread	Update time on the main thread
`Animator.enabled`, 1/4 (150 characters per frame)	**4.424 ms**	0.288 ms	`Animators.Update` 1.104 ms + toggling 0.125 ms
Manual graph per phase, 1/4 (150 characters per frame)	**4.253 ms**	0.279 ms	`ATB.BucketEvaluate` 1.278 ms

In the two new implementations, **the amount of work in `Animator.ProcessGraph` is exactly 1/4 of updating every frame** (naturally, since only 150 characters are evaluated per frame), and that work is still spread across the worker threads. In the `Animator.Update()` version, by contrast, almost nothing is counted in this marker, and the bulk of the work is inside `ATB.ManualAnimatorUpdate` on the main thread.

Listing the main-thread cost per character makes the difference clear.

- Update every frame: 4.046 ms / 600 characters = **6.7 µs per character**
- `Animator.enabled` toggling, 1/4: 1.229 ms / 150 characters = **8.2 µs per character**
- Manual graph per phase, 1/4: 1.278 ms / 150 characters = **8.5 µs per character**
- `Animator.Update()`, 1/4: 2.316 ms / 150 characters = **15.4 µs per character**

In the two new implementations, the cost per character stays almost the same as updating every frame (the difference is the overhead of toggling and calling `Evaluate`). As a result, the reduction rate approaches the thinning ratio.

We also confirmed that the playback speed is preserved. Measuring the distance per second traveled by the fingertip bone of the last character spawned (the one with the largest phase slot), we get 0.509 m/s for updating every frame, versus 0.496 m/s for `Animator.enabled` toggling and 0.517 m/s for the manual graph per phase. Within the error from thinning approximating the motion linearly, the speed is unchanged in both.

One note on the distribution: at 1/2 thinning, the manual graph version has a heavier tail, with a p99 of 9.102 ms and a standard deviation of 1.086, than the `enabled` toggling version (p99 4.100 ms, standard deviation 0.562). At 1/4, conversely, the manual graph version is more stable (standard deviation 0.306 versus 0.340). The behavior changes with the balance between the number of graphs and the number of characters per graph, so if you adopt it, look at p99 as well as the mean.

Which to choose depends on the requirements. If you want to keep using the Animator's state machine and the Inspector workflow as is, toggling `Animator.enabled` is overwhelmingly easier (it takes a few lines and has almost no effect on existing code). If you need accurate progress or graph-side control such as inserting an Animation Job, use a manual graph per phase. **In either case, there is no reason to choose an implementation that calls `Animator.Update()` yourself.**

4.6.8 Conclusions of this chapter

- Design animation LOD as a set of "thinning the update frequency + phase offset"
- Phase offset does not reduce the total load (mean 2.613 vs. 2.578 ms). What it reduces is the **variation in frame time** (standard deviation 3.863 → 0.185), which is a measure for perceived quality
- **Do not choose an implementation that calls `Animator.Update()` yourself.** Because it evaluates synchronously on the spot, it loses job parallelization, and the main-thread cost per character rises about 2.3 times, from 6.7 to 15.4 µs
- Instead, **choose every frame which characters take part in Unity's batch update**. Either toggle `Animator.enabled` (with compensation through `keepAnimatorStateOnDisable = true` and `Animator.speed`), or split manual graphs by phase and attach `AnimatorControllerPlayable`. The cost is 8.2 to 8.5 µs per character, and parallelization is preserved
- Changing the implementation makes **even 2x thinning effective**: − 4.1 % became − 33.5 % / − 38.8 %. With 4x it is − 63.6 % / − 69.4 %
- Per-character thinning cannot be expressed with a single consolidated graph. However, "splitting graphs by the number of phases" achieves both parallelization and update frequency control

- Do not judge by the median alone. Look at the mean for the total load, and at p95, p99, and the standard deviation for perceived quality

4.7 Summary

These are the measurement results of each chapter.

Table 4.16: Summary of measurement results by chapter

Chapter	Measure	Condition	Result
Asset settings	Optimize Game Objects	600 characters, 1 Renderer	6.797 → 5.991 ms (- **11.9 %**)
Asset settings	Optimize Hierarchy (runtime)	600 characters, 1 Renderer	6.797 → 6.009 ms (- **11.6 %**)
Asset settings	The two methods above	300 characters, all Renderers	18.205 → 18.3 to 18.5 ms (no improvement, renderin g-bound)
Job Thread	Consolidate manual update graphs into one	600 characters	8.396 → 4.500 ms (- **46.4 %**)
Animation C# Jobs	Convert Two-Bone IK to an Animation Job	300 characters, 64 solves	9.094 → 2.795 ms (- **69.3 %**)
Animation C# Jobs	Additionally compile with Burst	300 characters, 64 solves	2.795 → 2.792 ms (no difference in frame time)
Animation LOD	1/4 thinning, explicit `Animator.Update()` call	600 characters	Mean 5.037 → 2.578 ms (- **48.8 %**)
Animation LOD	1/4 thinning, manual graph per phase	600 characters	Mean 5.037 → 1.541 ms (- **69.4 %**)
Animation LOD	1/2 thinning, explicit `Animator.Update()` call	600 characters	Mean 5.037 → 4.831 ms (- 4.1 %)
Animation LOD	1/2 thinning, manual graph per phase	600 characters	Mean 5.037 → 3.082 ms (- **38.8 %**)

4.7.1 Final optimization approach

This is how to proceed with animation optimization for large numbers of characters, based on the measurements in this chapter.

1. Optimize the asset settings

First, set appropriate values for the vertex and bone counts in the model specification itself. Ideally, you first measure benchmarks on the baseline supported devices and establish an appropriate model regulation. Then consider applying Optimize Game Objects on the Unity side. In a 3D title, it is difficult to make major changes to upstream model production late in the project, once mass production is well under way, so it is best done early in the project.

2. Increase the parallelism of processing on the Animation Job Thread

Unity parallelizes the Animator and automatically updated Playable Graphs automatically, but manual updates are parallelized only per graph. When designing character control, keep in mind whether you can make effective use of the Animation Job Thread.

3. Optimize, including applying Burst to Animation Jobs

First move post-processing such as IK from the main thread to an Animation Job.

If further performance is needed, apply a data and processing design that allows SIMD vectorization.

4. Apply application-level techniques such as animation LOD

After exhausting asset, engine setting, and code optimizations, the last resort is not processing the animation in the first place.

This depends on the project requirements, but consider a mechanism that reduces processing while exploring the line that is acceptable for the game experience.

4.8 Footnotes

4.8.1 Unity official documentation

- Playable API
  `https://docs.unity3d.com/Manual/Playables.html` – Building a Playable Graph and `DirectorUpdateMode`
- AnimatorUtility.OptimizeTransformHierarchy
  `https://docs.unity3d.com/ScriptReference/AnimatorUtility.OptimizeTransformHierarchy.html` – Flattening the hierarchy at runtime
- Model Import Settings: Rig（Optimize Game Objects）
  `https://docs.unity3d.com/Manual/FBXImporter-Rig.html`
- IAnimationJob
  `https://docs.unity3d.com/ScriptReference/Animations.IAnimationJob.html` – Jobs inserted into `AnimationScriptPlayable`
- AnimationScriptPlayable
  `https://docs.unity3d.com/ScriptReference/Animations.AnimationScriptPlayable.html`
- C# Job System
  `https://docs.unity3d.com/Manual/JobSystem.html` – A mechanism for writing parallel processing without going through `AnimationStream`

4.8.2 Assets used

- Unity-chan 3D model data
  `https://unity-chan.com/` – © Unity Technologies Japan/UCL

Chapter 5

Asset Bloat in the Live Operation Phase and Investigation with Project Auditor

Zaitsu Hiromu

Sumzap, Inc.

X @RomGameclient

Name：Zaitsu Hiromu Affiliation：Sumzap, Inc. X account / Website / GitHub account (optional): @RomGameclient

In a project that stays in live operation, the app size and memory usage grow little by little. This chapter covers why bloat recurs even on teams that know the countermeasures, how to stop it, and how to find the assets that keep growing. It does not cover reducing the size of individual assets themselves.

5.1 Why Bloat Progresses in the Live Operation Phase

Bloat accumulates gradually through day-to-day operation: high-resolution banners added for a seasonal event, voice lines and cut-ins for limited-time characters, externally produced materials accepted for a collaboration, and campaign assets that remain after the announcement has ended. Each item may be acceptable on its own, but over successive releases the total becomes significant. Moreover, the

more it accumulates, the higher the cost of reviewing it, so cleanup keeps dropping in priority. The cause lies less in technology than in operational rules becoming lax. In the live operation phase, the following four things happen repeatedly. ・Standards become outdated: even when devices and requirements change and the standards no longer match reality, nobody rewrites them. Exceptions approved with the claim that this is just for this time pile up. ・Checks become lax: as deadlines approach, verification of delivered materials is skipped, and handovers are left incomplete when the person in charge changes. ・Temporary workarounds remain: uncompressed textures or individually exported sprites added in a hurry stay in place, and the work to revert them is never done. ・Decision criteria are not recorded: because the rationale for settings is not documented, the same criteria cannot be reproduced when the person in charge changes. None of these can be solved by introducing a tool alone.

5.2 Keep Operating the Standards

Decide the target metrics, select the supported devices, and verify in the scene with the highest load. How to create such a convention is covered in Chapter 1 of the previous book, "Unity Performance Tuning Bible". The problem in the operation phase comes after the convention is decided. Standards drift from reality as soon as they are set. Therefore, decide in advance the events that trigger a review, such as a change in supported devices, an update of Unity or the rendering pipeline, or a change in the measurement method. In particular, a change in the measurement method shifts the numbers even for the same asset, so record how to measure together with the standards. Make the numeric values in the standards checked at export, at import, or in the automated build. Decide per item whether to fail or pass the build, and even when it passes, leave the measured values in a place visible to the whole team, such as the build log or a report. A warning shown only on the author's own screen gets overlooked. Fixing an asset that exceeds the standards does not end with correcting a value. If you change compression settings, you must check under the same conditions as production whether the appearance remains acceptable; if you rearrange bundles, you must check that no references have broken. As the previous book warns, redoing work after mass production is a process that includes ordering, quality checks, and debugging. Rather than fixing everything at once, the ideal is to measure at every release and fix the increase each time.

5.3 Classification of Bloat Patterns

Summarizing countermeasures by material type tells you which materials are heavy, but not why they became heavy. If the cause remains, the same bloat recurs. Therefore, this chapter classifies bloat by cause into six patterns that cut across material types.

Table 5.1: Six bloat patterns

Pattern	What is happening
Incorrect compression settings	Import settings are left uncompressed or at defaults, occupying several times the necess ary size
Excessive resolution	Assets are imported without specifying a maximum resolution, so size grows in propor tion to area
Unnecessary accompanying data	Unused auxiliary data is left enabled, shipped with the asset, and also held in memory
Wrong format choice	A format that does not suit the purpose or pl atform is chosen, using extra size or being d ecompressed at runtime and consuming memory
Mismatched grouping	The way Sprite Atlases or bundles are grouped does not match the actual unit of use, so unused content is loaded together, such as loading an entire atlas to display a single Sprite
Duplication across bundles	Shared assets are referenced without being registered, and are included once for every r eferencing bundle

The first four can be determined at asset import time, but the last two are determined by relationships with other assets and so cannot be determined at that point. This difference is why the next section splits the countermeasures into two stages.

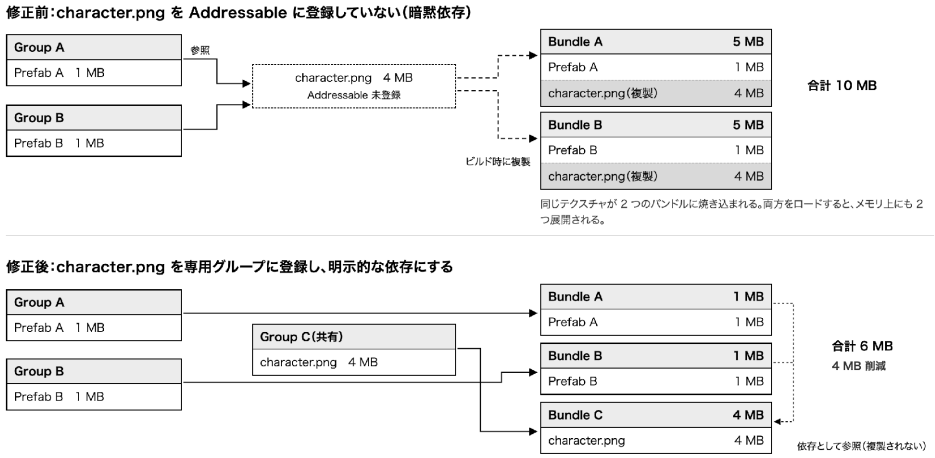
5.4 Automatic Application at Import and Pre-Build Inspection

To stop recurrence, apply the correct settings when assets are imported, and then find what slipped through before the build. There are two means of automatic application. If values must vary by condition, use AssetPostprocessor, which lets you insert processing before and after import. If you only need to fix default values per type, a Preset is sufficient. However, applying settings at import alone lets some things slip through. Assets brought in together with their .meta files keep their settings as they are, and the last two patterns in the previous section cannot be determined at this stage. Therefore, at the pre-build or automated build stage, inspect whether any assets that violate the standards have slipped in. If you write limit standards such as the maximum texture resolution or mesh polygon count as tests, the same criteria can be used in both the Editor and the automated build. OSS for this purpose is also available, such as AssetRegulationManager [*1]. An operation that does not turn such inspection into a mechanism and only writes the standards in a document and announces them leads directly to the two conditions mentioned at the beginning: outdated standards and unrecorded decision criteria. Deciding the rules is a human task, but leave the enforcement to machines.

5.5 How Duplication Across Bundles Occurs

Of the six patterns, duplication across bundles is the hardest to see. This is because the cause lies not in the settings of individual assets but in the reference relationships between assets. Addressables manages assets in groups and packs them into one bundle per group at build time. At this point, assets not registered as Addressable are treated as implicit dependencies and are duplicated into each bundle that references them.

[*1] `https://github.com/CyberAgentGameEntertainment/AssetRegulationManager/blob/main/README_JA.md`



▲Figure 5.1 Duplication of an unregistered shared asset (top) and the case where it is registered in a dedicated group (bottom)

In the example in the figure, simply having two Prefabs reference a 4MB texture makes the bundle total 10MB. If this texture is registered in a dedicated group and made an explicit dependency, the duplication is eliminated and the total drops to 6MB. As the number of referencing assets grows, the duplicates grow by the same number, and if they are loaded at the same time, the same amount is also expanded in memory, so the impact increases the longer it is left. Note that actual bundles are compressed, so the numbers are a rough guide to illustrate the mechanism. Such duplication cannot be traced by eye, so detect it with a tool. With Addressables, you can use Addressables Analyze [*2] (it was temporarily removed in the 2.0 series and restored in 2.3.1). There is also a feature that automatically resolves detected duplicates, but it rewrites the group configuration, so check the build result after applying it.

5.6 Investigating Bloated Assets

Standards and inspection can stop new bloat, but assets that have already accumulated must be found separately. For asset profiling, there are several tools with

[*2] `https://docs.unity3d.com/Packages/com.unity.addressables@latest`

different purposes.

Table 5.2: Tools for investigating size and memory

Tool	What it shows
Build Report / Editor Log	Breakdown of build size by category
Addressables Analyze	Detection and fixing of duplicated bundle dependencies in Addressables
Project Auditor	Import settings and static configuration issues across the whole project
Memory Profiler	Runtime memory usage and object reference relationships

Of these, Memory Profiler is covered in Chapter 3, "Profiling Tools," of the previous book. This chapter covers Project Auditor, which the previous book did not introduce.

5.7 Finding Issues with Project Auditor

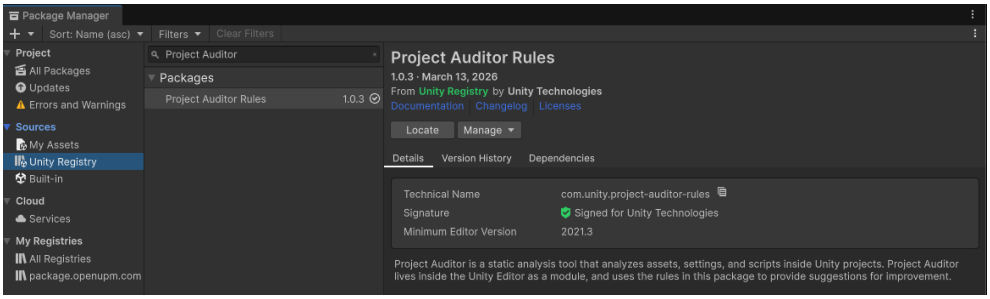
Project Auditor is a tool that statically analyzes the assets, code, and settings in a project and outputs a list of problem areas. Even at a stage where there has been no time to formalize standards, it lets you grasp what could become a problem. It was an experimental package for a long time, but version 1.0 was released in February 2025 and it became an official release. The screens and items below were confirmed with Unity 6.4 and Rules package 1.0.3.

5.7.1 Installation Differs by Unity Version

Before Unity 6.4, add com.unity.project-auditor from the Package Manager [*3]. From 6.4 onward it is bundled with the Editor, so adding it is unnecessary [*4], but the rules used for judgment have been separated into com.unity.project-auditor-rules, so analysis is not possible unless you install that package (Figure 5.2).

[*3] https://docs.unity3d.com/Packages/com.unity.project-auditor@1.0/manual/index.html

[*4] https://docs.unity3d.com/6000.4/Documentation/Manual/project-auditor/project-auditor-introduction.html

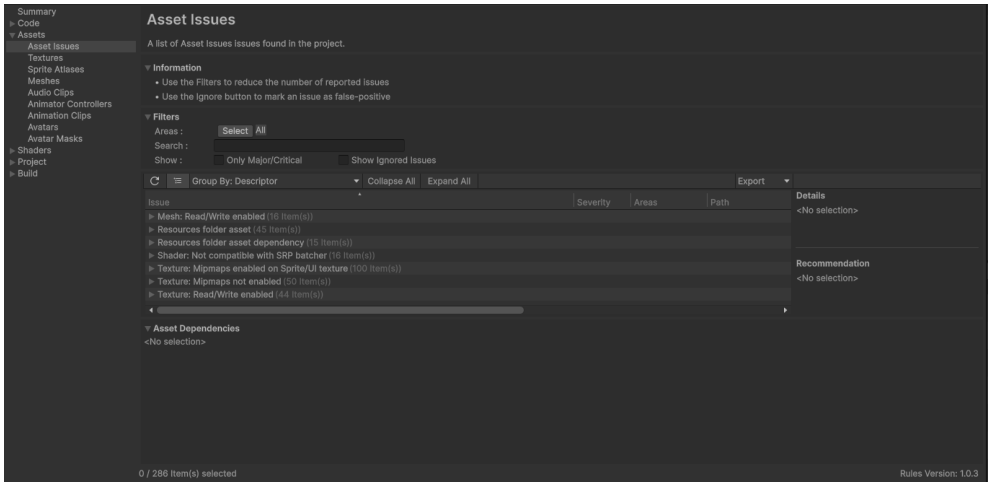


▲Figure 5.2　The Rules package. The analysis core exists as an Editor module

Because the rules are separated from the core, you can update only the rules without raising the Editor version.

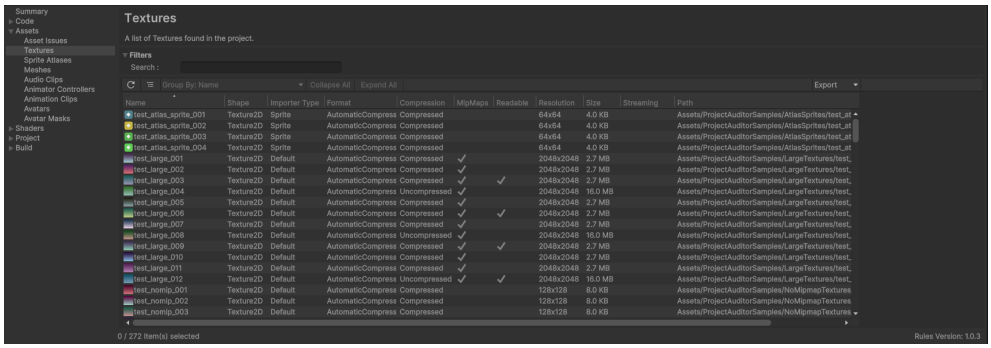
5.7.2　Using the Assets View

When you open Window > Analysis > Project Auditor and run the analysis, the results are divided into the Summary, Code, Assets, Shaders, Project, and Build views (more views may be added depending on the Unity version). The view used for investigating size and memory is the Assets view, which is divided into nine, including Asset Issues, Textures, and Sprite Atlases. The first one to open is Asset Issues (Figure 5.3). Detected problems are grouped by type, and the number found for each is displayed.



▲Figure 5.3　The Asset Issues view. Detection example in a project prepared for verification

Expanding a type shows the individual assets, and selecting one shows the details and the recommended action on the right. Once you have the overall picture, look at the breakdown in an individual view such as Textures (Figure 5.4).



▲Figure 5.4　The Textures view. Settings and resulting size are displayed side by side

The advantage of this view is that settings and results appear on the same row. Even for the same 2048×2048 texture, the Compressed row occupies 2.7MB and the Uncompressed row occupies 16.0MB. You can see without opening the Inspector that a difference in a single setting produces a nearly sixfold difference. If you sort the Size column in descending order and check Compression, Resolution, and Format

from the top, you can find incorrect compression settings, excessive resolution, and wrong format choices.

5.7.3 Reducing the Detection Results

When you run the analysis on a project in live operation, the detection results can exceed several thousand. Trying to work through them from the top in this state does not get anywhere. Narrow the display with Filters in Figure 5.3 shown earlier. Checking Only Major/Critical leaves only high-severity items, and selecting an area in Areas lets you display only the findings related to size. Detected problems are assigned a severity. The documentation describes four levels, Major, Moderate, Minor, and Ignored, but Critical also exists by definition. Ignored is not a severity level but refers to the state that you have specified to ignore. Address issues in order starting from Major, and if Critical is detected, handle it with top priority. Static analysis also flags intended settings, so it is safer to check each item before fixing it. Apply Ignore to findings you judge to be false positives. Ignore is a state recorded in the report that is open, so it is not written to a settings file and is cleared when you rerun the analysis. Judgment parameters such as detection thresholds are saved in Project Settings/ProjectAuditorSettings.asset, so committing this file lets everyone share the same values. If you want to suppress a finding permanently, pass a disabling rule to AnalysisParams.WithAdditionalDiagnosticRules in a script on the automated build side.

5.7.4 Integrating into the Automated Build

Project Auditor can also be run from the command line [*5]. If you integrate it into the automated build and keep the number of detections for each release, you can track which release increased it. Preferences has a setting that logs the number of detections after the build and a setting that treats detections as errors. Before writing your own failure criteria, check whether these are sufficient. On the other hand, no API is provided for comparing two saved reports, so obtaining the difference from the previous run requires your own processing. There are also reports that the automatic analysis after the build never completes and stays stuck, or that the analysis exhausts

[*5] https://docs.unity3d.com/6000.4/Documentation/Manual/project-auditor/run-from-command-line.html

memory [*6]. Set a timeout so that an analysis failure does not stop the entire build.

5.8 Conclusion

Countering bloat is a topic where building a mechanism that stops recurrence as a team weighs more than techniques for making individual assets lighter. Standards loosening during operation happens in every project. That is why I feel that the approach that lasts longest is to spend human time on deciding and reviewing the criteria and leave the enforcement to machines. Project Auditor has only just become an official release, but it is sufficiently practical as an entry point for a project that has not had time to establish standards to understand its current state. As a first step, run the analysis once and look at the trends in your own project.

[*6] https://github.com/Unity-Technologies/ProjectAuditor/issues

Chapter 6

Choosing Between UI Systems: IMGUI / uGUI / UI Toolkit

Zaitsu Hiromu

Sumzap, Inc.

X @RomGameclient

Name：Zaitsu Hiromu Affiliation：Sumzap, Inc. X account / Website / GitHub account (optional): @RomGameclient

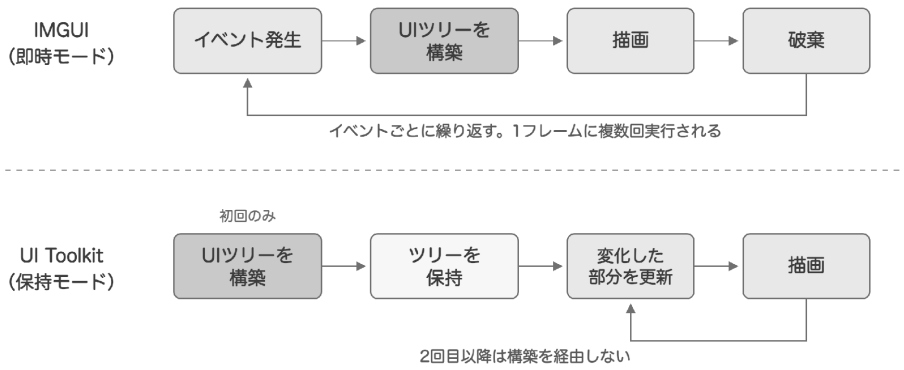
The previous book, "Unity Performance Tuning Bible" [*1], covered how to handle uGUI load, such as splitting Canvases and adjusting Raycast Target. This chapter goes one step further back and covers the choice of UI system itself, because that choice determines the load structure of the UI and the kinds of countermeasures available. Unity has three UI systems: IMGUI, uGUI, and UI Toolkit. This chapter takes up these three in turn and organizes how each is positioned today for runtime and for Editor extensions. The descriptions in this chapter are based on information as of Unity 6.5.

6.1 IMGUI

Runtime IMGUI is no longer an option for in-game UI. The official manual states explicitly that it is not intended for regular in-game UI that players interact with, and

[*1] https://cyberagentgameentertainment.github.io/UnityPerformanceTuningBible/ja/preface.html

lists in-game debug displays and tools, custom inspectors, and Editor extensions as its intended uses. It is still used today for debug overlays and runtime log viewers. For Editor extensions, on the other hand, IMGUI is officially current. The official UI system comparison page [*2] lists IMGUI as an alternative and gives two advantages: unrestricted access to Editor extension features, and a lightweight API that can quickly draw UI on screen. The official IMGUI migration guide [*3] explicitly lists the APIs that have no equivalent in UI Toolkit. Those with the greatest practical impact are InspectorTitlebar(), Toolbar(), IntPopup(), and LinkButton(). All of them are APIs for making an Editor window look like Unity itself, so where they are needed, you currently have to use IMGUI. However, there are load characteristics you should know about when using it for Editor extensions. IMGUI is an immediate mode system in which the UI is written every time inside OnGUI(); it does not retain UI tree information and repeats construction and destruction on every event. The difference from UI Toolkit, which is a retained mode system, is shown in Figure 6.1. OnGUI() runs multiple times per frame, and each run makes heap allocations likely. In an Editor window that is redrawn continuously, this becomes a load on the GC.



▲Figure 6.1 Drawing cycles of immediate mode and retained mode

[*2] https://docs.unity3d.com/6000.5/Documentation/Manual/UI-system-compare.html (accessed August 6, 2026)

[*3] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-IMGUI-migration.html

6.2 uGUI

It is a system based on Canvas and GameObjects. Each element is a Game Object and has a Transform hierarchy. It is runtime-only and cannot be used for Editor extensions. It is the de facto standard at runtime. It has three advantages. ・ You can place elements directly in the Scene view and preview them on the spot ・ You can animate the UI with Animator and Timeline ・ The Asset Store has a large number of packages built on the assumption of uGUI In terms of load, a Canvas that has a changed element rebuilds its mesh, and the cost of updating the Transform hierarchy and scanning Raycast Targets increases in proportion to the number of elements. Splitting Canvases and adjusting Raycast Target, covered in the previous book, are both typical countermeasures for the load that comes from this structure.

6.3 UI Toolkit

It is a retained mode system that builds a VisualElement tree once and afterward updates only the parts that changed. Logic is written in C#, UI structure in UXML, and styles in USS. It can be used for both runtime and Editor extensions, and it is the official first recommendation for Editor extensions. However, the official wording is "if you are building a complex Editor tool." For a tool that merely lays out a few fields, building it with C# alone, without bringing in UXML and USS, causes no trouble. In terms of load, the differences from uGUI translate directly into differences in countermeasures. Unlike uGUI, where changing a single element rebuilds the mesh of the entire Canvas, UI Toolkit updates only the parts that changed, so a countermeasure equivalent to splitting Canvases is unnecessary. However, layout calculation cost still grows with the number of elements, so reducing elements remains necessary. In the Unity 6 series, performance improvements are also continuing, such as faster event dispatching, jobified geometry generation, and better caching of layout calculations.

6.3.1 What Is Lacking Compared with uGUI

As of Unity 6.5, the following are things it cannot do. All of them are items that Unity lists as in development [*4], but if you are deciding on adoption now, you must assume they are unavailable. ・A feature for handling VisualElement keyframe animation in the Animation window ・UI editing in the Scene view ・A mechanism for referencing a VisualElement defined in UXML from a MonoBehaviour script ・Vertex data support for custom shaders The ecosystem also remains thin. Many Asset Store assets assume a GameObject hierarchy and do not work as they are. TextMeshPro also cannot be used from UI Toolkit, which uses its own text system (Advanced Text Generator, which became the default in 6.5). DOTween has also added shortcuts for VisualElement, but only for Unity 6.3 and later [*5]. When migrating from uGUI, there is no direct equivalent of anchors and pivots. UI Toolkit layout is a method that flows child elements in order inside a parent element. This changes the way UI is assembled, compared with uGUI, where position and size were specified for each element.

6.4 Changes in the Official Recommendation

The official UI system comparison page is published for each manual version, and comparing them shows that the reasons for recommending runtime UI have been swapped [*6]. The breakdown is shown in Table 6.1.

Table 6.1: Change in the reasons for recommending runtime UI

Requirement / use	Unity 2022.3 to 6.1	Unity 6.5
Handling a large amount of UI	UI Toolkit	UI Toolkit
Needing an authoring workflow for artists and designers	UI Toolkit	UI Toolkit
Textureless UI rendering	UI Toolkit	UI Toolkit

[*4] https://discussions.unity.com/t/ui-toolkit-development-status-and-next-milestones-november-2025/1698009

[*5] https://dotween.demigiant.com/download.php

[*6] https://docs.unity3d.com/6000.0/Documentation/Manual/UI-system-compare.html (accessed August 6, 2026)

Table 6.1: Change in the reasons for recommending runtime UI

Requirement / use	Unity 2022.3 to 6.1	Unity 6.5
UI placed and lit in 3D space	uGUI	UI Toolkit
Expression with custom shaders and materials	uGUI	UI Toolkit
Easy reference from a Mono Behaviour	uGUI	uGUI

Two items that stayed on the uGUI side consistently from 2022.3 to 6.1 [*7] moved to the UI Toolkit side when world-space UI was implemented in 6.2 and custom shaders were implemented for URP in 6.3. For the remaining item, "easy reference from a MonoBehaviour," a mechanism for referencing elements defined in UXML is also among the items in development. However, the structure of the comparison page itself has not changed even in Unity 6.5. It still presents uGUI as the default for runtime UI and lists UI Toolkit as its alternative. Only the reasons for the recommendation have been swapped; which one is placed as the default has not moved. The use-case list on the same page also changed its heading from "Recommendation" in 2022.3 through 6.2 to "Often used for" in 6.3 and later. Unity itself no longer calls it a recommendation.

6.5 Constraints When Using UI Systems Together

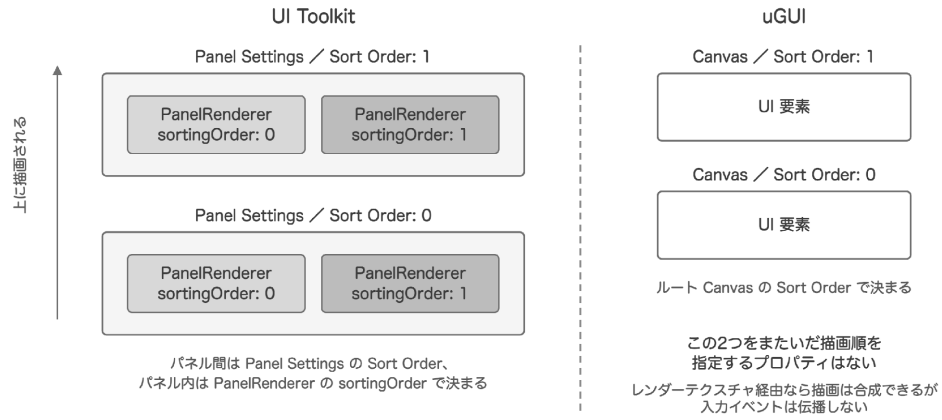
Using multiple UI systems in one project is possible in itself, but combining them causes several problems. This section looks at them separately for runtime and Editor extensions.

6.5.1 Runtime

uGUI and UI Toolkit can be used in the same project, but one cannot be embedded inside the other. The official documentation also lists inconsistencies in navigation and styling as constraints. Draw order can be controlled only within each system (Figure 6.2). To display UI Toolkit UI on screen, attach a Panel Renderer component to a GameObject. In Unity 6.4 and earlier this role is played by UIDocument, which

[*7] `https://docs.unity3d.com/2022.3/Documentation/Manual/UI-system-compare.html` (accessed August 6, 2026)

was replaced in 6.5 [*8]. If you use LTS 6.3, it is UIDocument. This Panel Renderer is the unit of ordering: panels are ordered by Sort Order in Panel Settings, and Panel Renderers within the same panel are ordered by PanelRenderer.sortingOrder. uGUI is ordered by the Sort Order of the root Canvas. There is no property that specifies an order across these two. The migration guide from uGUI [*9] also mentions compositing the rendering with a render texture, but states explicitly that input events do not reach it. Because it cannot be used for interactive UI, avoid designs that overlay uGUI and UI Toolkit.



▲Figure 6.2　How draw order is determined for UI Toolkit and uGUI

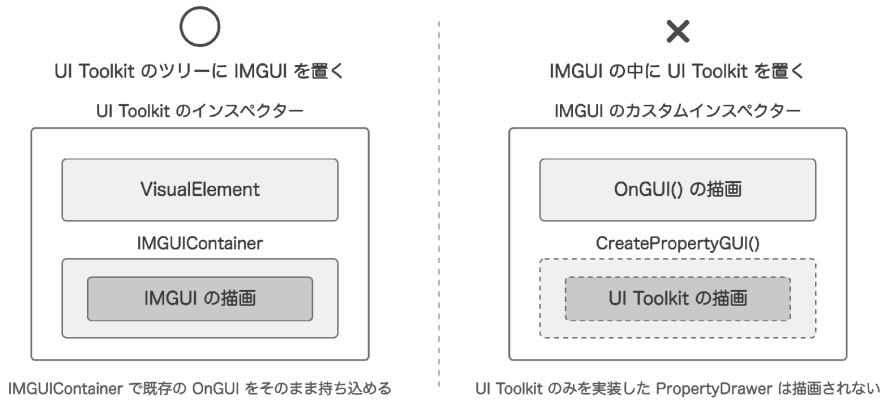
6.5.2 Editor Extensions

In Editor extensions, the two systems can be nested in only one direction (Figure 6.3). You can place IMGUI drawing inside a UI Toolkit tree with IMGUIContainer, but the reverse, placing a UI Toolkit tree inside IMGUI drawing, is not possible. This becomes a problem when you build your own inspector display. Inspector customization has two levels: a custom inspector that handles the whole component, and a

[*8] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-create-ui-document-component.html

[*9] https://docs.unity3d.com/6000.5/Documentation/Manual/UIE-Transitioning-From-UGUI.html

PropertyDrawer that replaces the appearance per field. The inner PropertyDrawer cannot choose which system's inspector calls it. If a PropertyDrawer implemented only in UI Toolkit is called from a custom inspector written in IMGUI or from within an IMGUI PropertyDrawer, the value field of that field is replaced with the display "No GUI Implemented." Since Unity 2022.2 the default for inspectors is UI Toolkit, so this does not occur in many cases, but in projects that still have IMGUI custom inspectors or PropertyDrawers, you need to provide an IMGUI version of the drawing in the PropertyDrawer as well.



▲Figure 6.3　Whether PropertyDrawers can be nested

6.6 Conclusion

What stood out in this investigation is that in Unity 6.5, the official reasons for recommendation have almost entirely moved to the UI Toolkit side. For runtime, uGUI remains the standard, and there is no reason to migrate something that already works, but the reasons to avoid UI Toolkit in new development have clearly decreased. When in doubt, it seems best to decide based on how much you depend on Animator and Timeline effects and on Asset Store assets. For Editor extensions, the following division should work for now: UI Toolkit for complex tools, and IMGUI where the look should match the Unity Editor itself. The choice of UI system is the most upstream tuning, because it determines whether the countermeasures covered

in the previous book are needed at all.

Chapter 7

Unity 6 Render Graph Tile Memory Optimization

Kiyohara Takayuki

CyberAgent, Inc. SGE Core Technology Division

X @nezumimusume

Name: Kiyohara Takayuki Affiliation：CyberAgent, Inc. SGE Core Technology Division X account: nezumimusume

7.0.1 Introduction

This chapter explains how to use Unity 6's Render Graph to take advantage of the tile memory of mobile GPUs (tile-based rendering architecture) and reduce GPU time, memory bandwidth, and memory usage. In mobile game development, memory bandwidth often becomes the bottleneck before GPU compute performance does. Memory bandwidth consumption is also directly tied to power consumption and heat, so it affects device temperature and battery drain as well as frame rate. This chapter covers the following four themes. Each has a section of the same name.

1. Tile memory load/store
2. Render Pass/Subpass
3. Frame buffer fetch
4. Memoryless

7.1 Fundamentals of Tile-Based Rendering

This section explains why mobile GPUs have tile memory.

7.1.1 Mobile GPUs and Memory Bandwidth

On modern GPUs, the gap between compute speed and memory read/write speed keeps widening, and memory access has become a major bottleneck. GPU core compute performance (FLOPS) grows substantially with each generation, but memory bandwidth growth has not kept pace. Discrete GPUs on PCs mitigate this problem by providing dedicated high-bandwidth graphics memory such as GDDR. Mobile SoCs cannot do the same.

Item	PC (discrete GPU)	Mobile SoC
Graphics memory	Dedicated (GDDR6, etc., hundreds of GB/s)	None (LPDDR shared by CPU/GPU/ISP, etc.)
Bandwidth scale	Hundreds of GB/s to 1 TB/s	Tens of GB/s (and shared)
Power constraints	Hundreds of watts of power and cooling	A few watts, passive cooling only

Mobile GPUs have no dedicated graphics memory and share the same LPDDR memory as the CPU. The memory is shared not only with the CPU but also with every other block on the SoC, such as the camera ISP and machine learning accelerators. The bandwidth itself is narrow, and the blocks compete for it. Furthermore, access to main memory (DRAM) is one of the most power-hungry operations on the SoC. The more bandwidth is used, the more power is consumed and the more heat is generated, thermal throttling lowers the CPU/GPU clocks, and the frame rate drops. In other words, on mobile the following chain occurs:

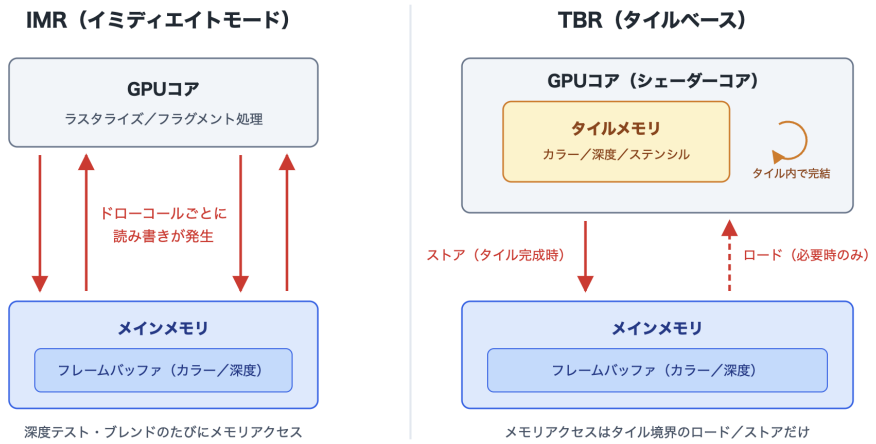
```
Memory bandwidth consumption → increased power consumption → heat → throttling →
     performance degradation
```

If a game runs at 60 FPS for the first few minutes but the frame rate drops after playing for a while, this bandwidth and heat issue is often the cause. GPUs with the tile-based rendering (TBR) architecture were created to render within this constraint.

7.1.2 IMR and TBR

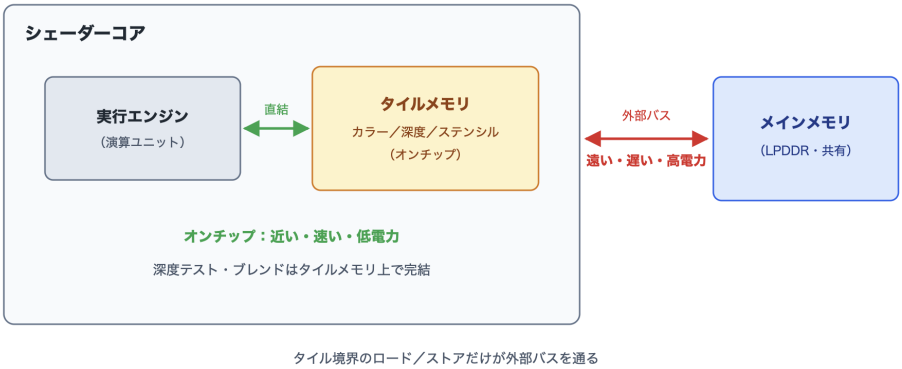
To understand TBR, let us first compare it with immediate mode rendering (IMR), the architecture that is mainstream in PC GPUs. IMR rasterizes the primitives submitted by a draw call immediately, runs the fragment shader, and writes the result to

the render target in memory. Reads and writes to the render target occur for every depth test and blend, but PC GPUs absorb this with high-bandwidth graphics memory and large caches. TBR, on the other hand, divides the screen into small tiles and completes rendering one tile at a time. Each shader core has a small on-chip memory called tile memory, and all drawing within a tile (reading and writing color, depth, and stencil, and blending) is performed in this tile memory. As a rule, main memory is accessed only twice: once when drawing of the tile starts (load) and once when it ends (store).



▲Figure 7.1 Comparison of IMR and TBR

The following figure is a conceptual diagram of a shader core in a TBR GPU. Tile memory is directly connected to the shader core. Because it is on-chip and physically close, access to tile memory is orders of magnitude faster and uses far less power than access to main memory. This structure can also be seen in real GPUs. For example, the shader core design diagram in the official ARM Mali-G71 documentation shows tile memory directly connected to the shader core.



▲Figure 7.2　Shader core and tile memory (conceptual diagram)

In this way, depth testing and blending, which involve the heaviest memory access in IMR, are completed within tile memory in TBR.

7.1.3　The Two Phases of TBR

A TBR GPU executes the rendering of one frame in two major phases.

Binning Phase (Geometry Phase)

First, the GPU runs the vertex shaders (the position calculation part) for all draw calls in the frame and sorts which tiles each primitive overlaps (binning). The result is written to main memory as a "per-tile primitive list." Note that the binning phase itself also consumes main memory bandwidth. The more vertex data there is, the more reads and writes of the primitive list occur. This is why high-polygon scenes tend to be disadvantageous on TBR GPUs.

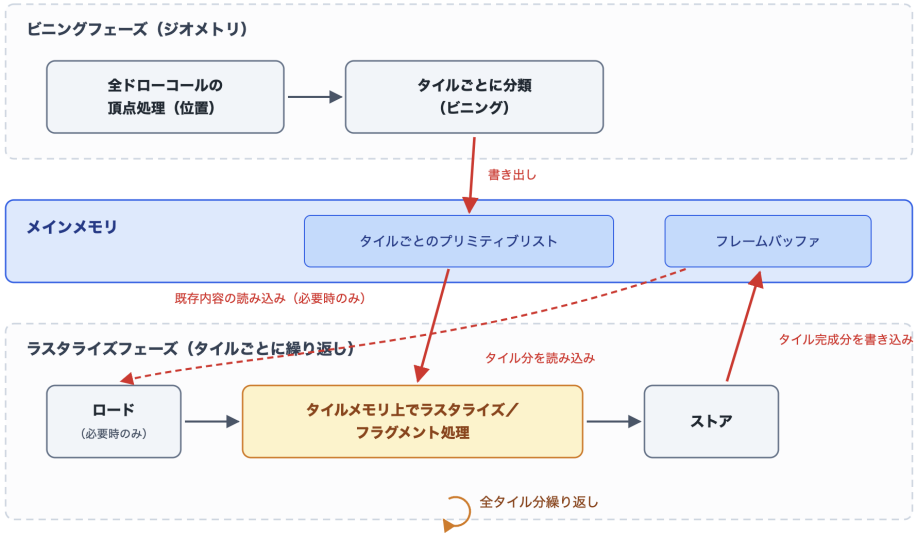
Rasterization Phase (Rendering Phase)

Next, the tiles are processed one at a time. For each tile, drawing proceeds as follows:

1. If necessary, load the existing contents of the render target into tile memory
2. Rasterize the primitives that overlap the tile and run the fragment shader (all reads and writes occur in tile memory)

3. Store the completed tile contents to main memory

Steps 1 and 3 are the load/store that is the theme of this chapter. The next section, "Tile Memory Load/Store," covers them in detail, but the cost structure of TBR is that the exchange with main memory at tile boundaries is more expensive than the drawing of the tile itself.



▲Figure 7.3　Binning phase and rasterization phase

Vendor-Specific Terminology and Characteristics

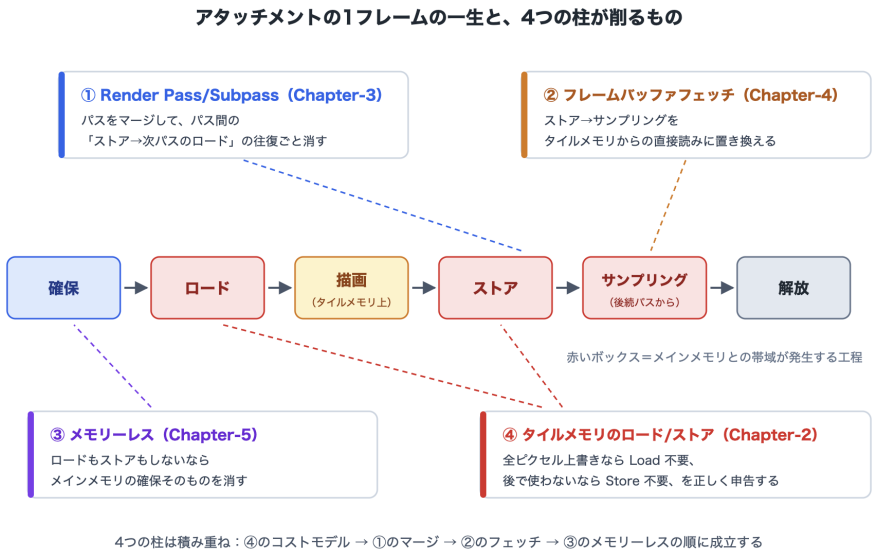
The basic structure of TBR is common, but terminology and implementation details differ by GPU vendor. In the rest of this chapter, the terms tile memory and binning are used as generic names.

Vendor (GPU)	Phase/mechanism name	Notes
Qualcomm Adreno	Binning / GMEM	Uses on-chip memory called GMEM, divided as needed. Has a mode that switches to IMR-like behavior depending on the rendering content

Vendor (GPU)	Phase/mechanism name	Notes
ARM Mali	Tiler / Tile Memory	Tiles are basically 16×16 pixels
Imagination (PowerVR/IMG)	TBDR	A TBDR that performs hidden surface removal (HSR) in hardware. The IMG DXT in Pixel 10 follows this lineage
Apple GPU	Tile Memory	A TBDR in the PowerVR lineage. Tile memory can be handled explicitly from Metal

7.1.4 The Four Themes of This Chapter

We have seen that load/store at tile boundaries is the cost. So how can that cost be reduced? The four themes of this chapter are all means of answering this question.



▲Figure 7.4　The lifetime of an attachment and the four themes

Consider the one-frame lifecycle of a render target (attachment).

```
Allocate → load → draw → store → sample (as a texture in subsequent passes) →
     release
```

Depending on which part of this lifecycle is reduced, the four themes can be organized as follows.

Theme	What is reduced	Approach	Section
Load/store	Unnecessary loads and unnecessary stores	State explicitly that "no load is needed if every pixel is overwritten" and "no store is needed if the result is not used later"	"Tile Memory Load/Store"
Render Pass/Subpass	Store→load round trips between passes	Merge multiple passes into one Render Pass and keep intermediate results in tile memory	"Render Pass / Subpass"
Frame buffer fetch	Store→sample round trips	Read the result of the previous subpass directly from tile memory	"Frame Buffer Fetch"
Memoryless	The main memory allocation itself	If there is neither load nor store, no entity in main memory is needed	"Memoryless"

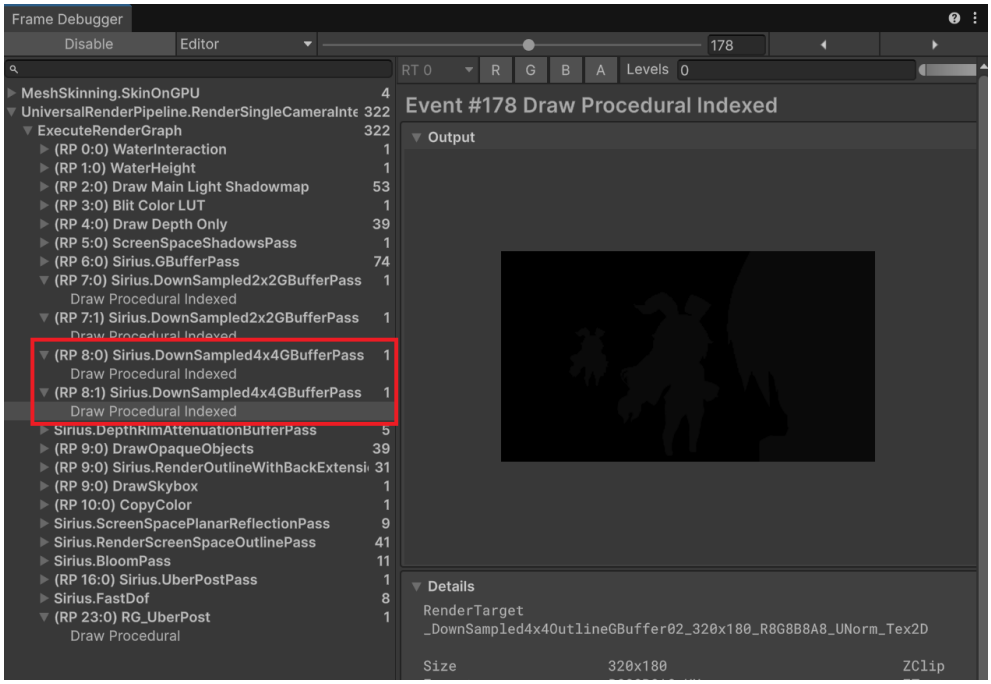
These four are not independent techniques. Frame buffer fetch can be used only after passes are merged, and memoryless is possible only after both load and store are gone. The section structure of this chapter follows this dependency: cost model (load/store) → mechanism (subpass) → application (fetch) → finishing touch (memoryless). Render Graph is also a system that collects per-pass resource usage information (what is read and what is written) and then builds an execution plan for rendering. It can therefore automatically perform optimizations such as pass merging, load/store action determination, and making resources memoryless. What this chapter explains is, in effect, how to tell Render Graph correctly how resources are used so that these optimizations take effect.

7.1.5 How to Use the Verification Tools

In this chapter, we repeatedly check the state of the pipeline with tools before measuring. The following three are mainly used.

7.1.6 Frame Debugger

Unity's Frame Debugger (Window > Analysis > Frame Debugger) is the basic tool for checking the structure of passes. What we look at in particular in this chapter is the `(RP n:m)` notation in event names. If the same Render Pass number (n) is followed by consecutive subpass numbers (m), as in `RP 4:0` and `RP 4:1`, those passes have been merged into a single Native Render Pass. Note that pass merging (Native Render Pass) works in environments such as Vulkan/Metal. If the Editor's graphics API is D3D or similar, merging cannot be observed. When checking in the Editor, change the Graphics API to Vulkan, or attach the Frame Debugger to a device build.



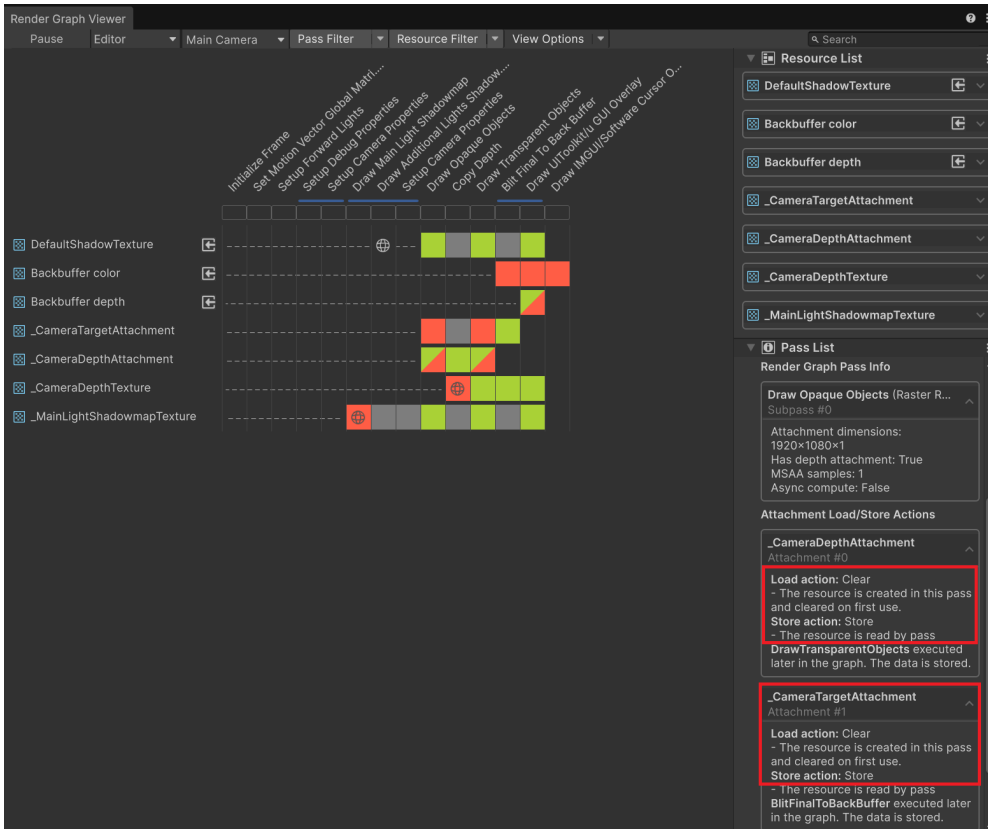
▲Figure 7.5　Merge state in the Frame Debugger

7.1.7　Render Graph Viewer

Render Graph Viewer (Window > Analysis > Render Graph Viewer) is a tool that visualizes the execution plan built by Render Graph itself.

- The list of passes and a timeline of the resources each pass accesses (which resource is created when and destroyed when)
- The Load Action / Store Action of each attachment and the Memoryless status. In this chapter, we use this display to check the results of optimization
- Whether passes are merged
- Why passes were not merged (Pass Break Reasoning)

The fourth item, Pass Break Reasoning, is the first information to check when passes that should be merged are not merged. It is used repeatedly from the "Render Pass / Subpass" section onward.



▲Figure 7.6　Checking merge state and load/store actions in Render Graph Viewer

7.1.8 RenderDoc

When you capture Vulkan with RenderDoc, you can check the `loadOp`/`storeOp` specified for each Render Pass attachment at the API level. Use it when you want to dig one level deeper than the Render Graph Viewer display, or when you want to compare Unity's display with the values actually passed to the driver.

7.2 Tile Memory Load/Store

This section explains load/store actions, which are the foundation of this entire chapter.

7.2.1 What Are Load/Store Actions

A TBR GPU reads the existing contents of the render target into tile memory when drawing of a tile starts (load), and writes the contents of tile memory to main memory when drawing ends (store). However, load and store are not always necessary. Graphics APIs provide actions for specifying, per attachment, what to do at the start and what to do at the end.

Load Actions

Action	Behavior	Bandwidth cost
Load	Reads the existing contents of main memory into tile memory	High (a read of resolution × format size)
Clear	Clears tile memory to a speci fied color	Almost zero (only an operati on within tile memory)
DontCare	Does nothing (the initial cont ents of tile memory are undef ined)	Zero

Store Actions

Action	Behavior	Bandwidth cost
Store	Writes the contents of tile memory to main memory	High (a write of resolution × format size)
Resolve	Writes out only the result of r esolving the MSAA samples	Medium (only the single reso lved sample)
DontCare	Writes nothing (the contents are discarded)	Zero

In other words, only Load and Store (and Resolve) consume bandwidth. Conversely, an attachment that can start with Clear/DontCare and end with DontCare has no exchange with main memory at all. This state is a prerequisite for memoryless, which is covered later. To optimize load/store, check the following two points for each attachment.

1. At the start, is the previous content needed? It is not needed if the pass

overwrites every pixel (DontCare). Use Clear if it starts from a cleared state, and Load only when drawing over the previous result.

2. At the end, will this content be used later? Use Store if it will be used by a subsequent pass or in the next frame, and DontCare if it is finished being used within this Render Pass.

7.2.2 Estimating Bandwidth Cost

The cost of one load/store can be estimated with simple multiplication.

```
Data size of one load (or store) = horizontal resolution × vertical resolution ×
    bytes per pixel
```

The following shows values for representative formats at 1080p (1920×1080).

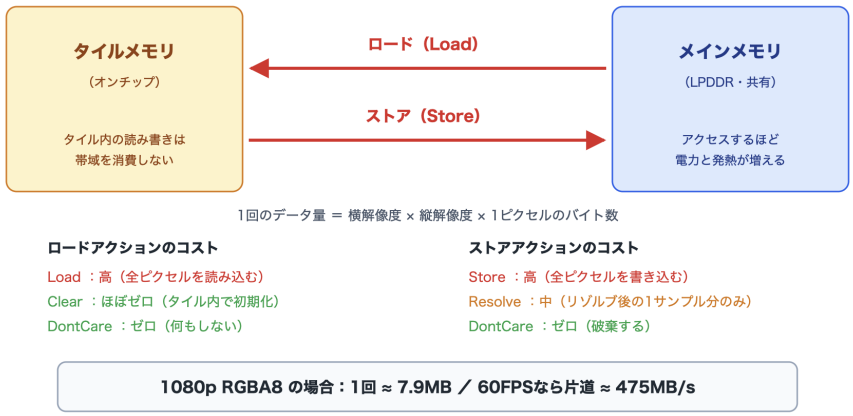
Format	Per pixel	One load/store	Bandwidth at 60 FPS
R8G8B8A8 (RGBA8)	4 bytes	About 7.9 MB	About 475 MB/s
B10G11R11_Float	4 bytes	About 7.9 MB	About 475 MB/s
R16G16B16A16_Float	8 bytes	About 15.9 MB	About 950 MB/s
D24S8 / D32 (depth)	4 bytes	About 7.9 MB	About 475 MB/s

For example, just "storing the camera color (RGBA8) and loading it in the next pass" consumes about 16 MB per round trip, or about 950 MB/s at 60 FPS. Considering that the effective bandwidth of a mobile SoC is tens of GB/s and is shared with the CPU, this is not a negligible amount. Also note the following points.

- MSAA applies a multiplier. If you Store a 4x MSAA color buffer without Resolve, the data size becomes four times larger (the relationship between MSAA and store is covered in the "Memoryless" section).
- Count both load and store. Every time a pass boundary is crossed, a pair of "Store of the previous pass + Load of the next pass" occurs.
- If the stored result is sampled in a subsequent pass, additional reads occur (for the portion that does not fit in the cache).

We recommend investigating once how many MB per frame travel between tile memory and main memory in your game's rendering pipeline. Simply checking each pass's attachments and actions in Render Graph Viewer and multiplying using the

table above reveals which passes to reduce first.



▲Figure 7.7　Tile memory ⇔ main memory load/store and bandwidth cost

7.2.3　Declaring Resource Usage with AccessFlags

So how are these load/store actions specified in Render Graph? In fact, Render Graph does not let you specify load/store actions directly. When adding a pass, you declare how resources are used to the builder, and Render Graph determines the actions from that information. What is used for this declaration is `AccessFlags`.

▼List 7.1　Declaring resources with AccessFlags

```
public override void RecordRenderGraph(RenderGraph renderGraph, ContextContainer
    frameData)
{
    using (var builder = renderGraph.AddRasterRenderPass<PassData>("Example", out var
    passData))
    {
        // Declare the output attachment.
        // AccessFlags.WriteAll = this pass writes all pixels (the previous content is
    not needed)
        builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.WriteAll);
        ...
```

```
    }
}
```

The main values of `AccessFlags` and their meanings are as follows.

AccessFlags	Meaning	Effect on actions
Read	Reads the previous content	The load action becomes Load
Write	Writes (possibly only some pixels)	The previous content must be preserved, so the load may become Load
ReadWrite	Both reads and writes	The load becomes Load
Discard	The previous content may be discarded	The load can be DontCare
WriteAll	Writes all pixels (Write + Discard)	The load can be DontCare

Pay particular attention to the difference between Write and WriteAll. A plain `Write` declares that "only some pixels may be written," so Render Graph has no choice but to set the load action to Load in order to preserve the contents of the pixels that were not written. For passes that always overwrite every pixel, such as a full-screen Blit, specify `WriteAll`. For 1080p RGBA8, this difference in specification alone determines whether a load of about 8 MB per frame occurs. On the other hand, the store action is determined by whether the resource is used in a subsequent pass. This is not specified by the developer; Render Graph decides it by looking at the entire graph. The next subsection looks at those rules.

7.2.4 Rules by Which Render Graph Determines Actions

When Render Graph builds Native Render Passes from the resource usage information of all recorded passes, it determines the load/store action of each attachment. The decision rules are summarized as follows.

Determining the Load Action

Condition	Determined action
Declared with AccessFlags.WriteAll (or Discard)	DontCare

Condition	Determined action
First use of the resource, and a clear is requested	Clear
The previous content is read or partially written	Load

Determining the Store Action

Condition	Determined action
Not used by any subsequent pass	DontCare
Used by a subsequent pass via UseTexture (as a texture)	Store
Used by subsequent passes only as SetInputAttachment (input attachment) and merged into the same Native Render Pass	DontCare (passed through tile memory)
Set as a global texture	Store (conservatively stored because subsequent use is not visible to Render Graph)
MSAA with a resolve target	Resolve

Two practical guidelines follow from these rules.

- Make global textures a last resort. A texture made global with `SetGlobalTextureAfterPass` or similar can no longer have its usage tracked by Render Graph, so it is always stored and is also excluded from memoryless conversion. Writing code that makes things global "just in case" prevents the optimization from taking effect.
- `UseTexture` and `SetInputAttachment` are entirely different in terms of bandwidth. Even though both "read the result of the previous pass," the former incurs a store→sample round trip, while the latter can be completed within tile memory (covered in detail in the "Frame Buffer Fetch" section).

Handling Imported Textures

When a texture managed on the application side, such as `RTHandle`, is imported with `renderGraph.ImportTexture`, Render Graph does not know how that texture is used outside the graph. It is therefore treated conservatively by default (it is loaded even on first use and stored even after the last use). Passing `ImportResourceParams` at import time relaxes this conservative behavior.

▼List 7.2 Parameter settings at import

```
var importParams = new ImportResourceParams
{
    clearOnFirstUse = true,   // Clear on first use (the load can be Clear)
    discardOnLastUse = true,  // Discard after the last use (the store can be DontCare)
};
var handle = renderGraph.ImportTexture(_rtHandle, importParams);
```

If an imported texture is "recreated every frame, so the previous content is not needed," set `clearOnFirstUse`/`discardOnLastUse`. For textures managed by Render Graph (those created with `renderGraph.CreateTexture`), all of this is determined automatically. As a rule, leave everything except textures that truly require lifecycle management to Render Graph.

Reading Load/Store Actions

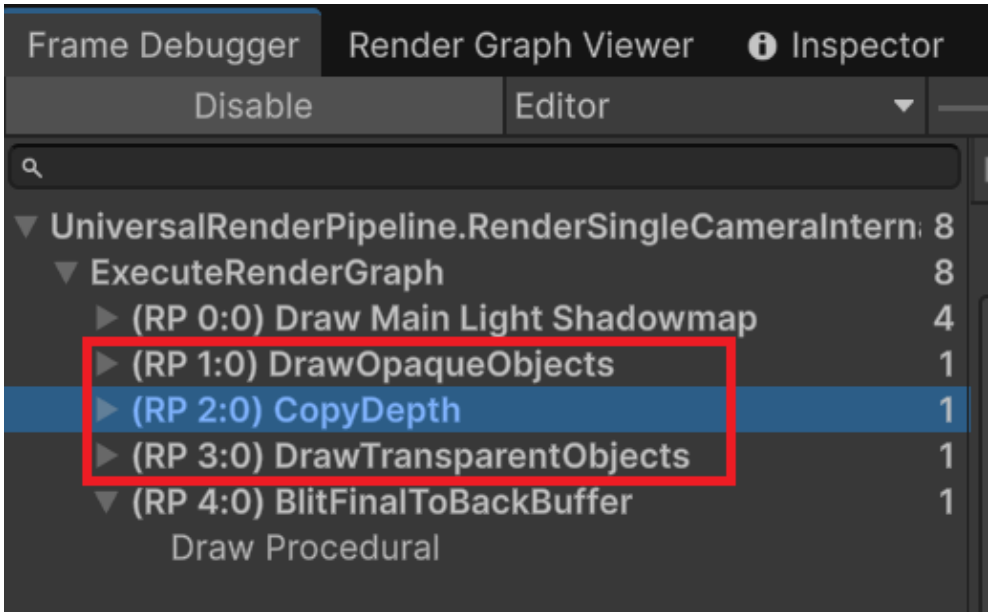
Now that we know the rules, let us read an actual frame. Looking at a standard URP forward rendering frame in Render Graph Viewer, you can observe a structure like the following, for example.

Pass	Attachment	Load	Store
MainLightShadow	Shadow map (depth)	Clear	Store
DrawOpaqueObjects	Camera color	Clear	(depends on subseq uent passes)
Camera depth	Clear	(depends on subseq uent passes)	
DrawTransparentObj ects	Camera color	(continues when merged)	Store
FinalBlit	Back buffer	DontCare	Store

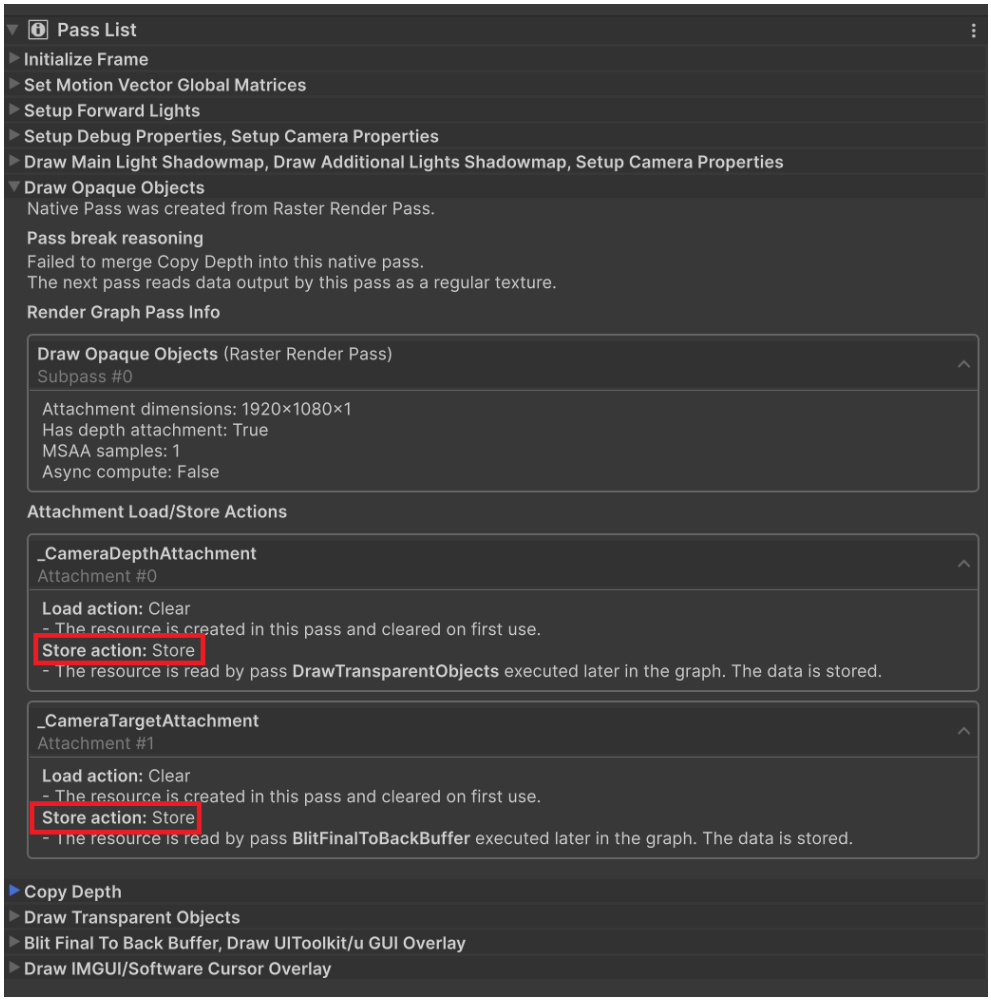
The key points for reading this are as follows.

- A shadow map is something that is "drawn and then sampled later," so it cannot be anything other than Store. This is a store that cannot be removed.
- If opaque and transparent passes are merged into a single Native Render Pass, no store/load of the camera color occurs between them. Conversely, if something is inserted between them and the merge breaks, a round trip appears there.
- Also pay attention to the store action of the camera depth. Turning on Depth Texture causes the camera depth to be stored. It is a common waste in real

projects that the setting is on even though no shader or post-processing requires `_CameraDepthTexture`.



▲Figure 7.8 Copy Depth splits Draw Opaque Objects and Draw Transparent Objects



▲Figure 7.9　Because they are split, Store occurs at the end of Draw Opaque Objects

In this way, the basic flow of tile memory optimization is to check, one by one, whether each attachment's Store has a legitimate reason (subsequent use).

7.2.5 Reduction Example with AccessFlags

Finally, let us confirm with a concrete example that the actions change with the `AccessFlags` specification alone. For a pass that performs a full-screen Blit, we compare the case where the output attachment is declared with `Write` and the case where it

is declared with `WriteAll`.

▼List 7.3 Comparison of AccessFlags Write and WriteAll

```
// Pattern A: Write (does not declare writing all pixels)
builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.Write);

// Pattern B: WriteAll (declares writing all pixels)
builder.SetRenderAttachment(_targetHandle, 0, AccessFlags.WriteAll);
```

The rendering result is the same in both cases. However, when checked in Render Graph Viewer, the load action is `Load` in pattern A and `DontCare` in pattern B. Because this is a full-screen Blit, the loaded content is entirely overwritten and thrown away. In other words, the load in pattern A is wasted.

7.3 Render Pass / Subpass

We have seen that store→load round trips between passes are a cost. This section explains Render Pass/Subpass, the mechanism for eliminating that cost, and pass merging by Render Graph.

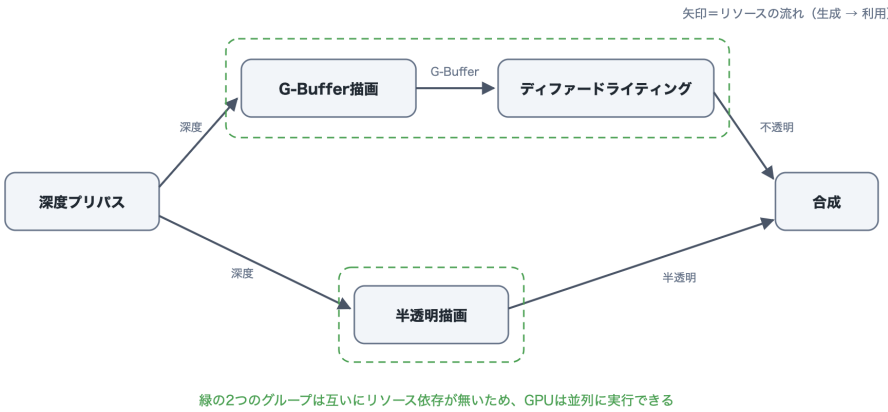
7.3.1 Resource Dependencies and the DAG

The process of completing the image of a game in one frame is called the rendering pipeline, and each individual process that makes it up is called a rendering pass. Each pass has resource dependencies: which resources it reads and which resources it writes. For example, typical geometry rendering that includes deferred rendering has the following dependencies.

1. Create the depth buffer with a depth pre-pass
2. Draw opaque objects to the G-Buffer (using the depth buffer)
3. Perform deferred lighting into the opaque buffer (using the G-Buffer and depth buffer)
4. Draw transparent objects to the transparent buffer (using the depth buffer)
5. Composite the opaque buffer and transparent buffer and draw to the frame buffer

If passes are drawn as nodes and resource dependencies as edges, this pipeline becomes a directed acyclic graph (DAG). Pass 4 does not use the outputs of passes

2 and 3, so it can run in parallel as long as the dependencies are communicated to the GPU.



▲Figure 7.10　DAG of rendering passes and parallelization

The "Graph" in the name Render Graph refers to this DAG. Render Graph is a system that collects the resource dependencies of passes, builds a DAG, and creates an execution plan based on it.

7.3.2 Vulkan RenderPass/Subpass

As a mechanism for communicating resource dependencies to the GPU, Vulkan has the concepts of RenderPass and Subpass. What Render Graph ultimately builds is this Vulkan RenderPass, so we briefly discuss the API level. A Vulkan RenderPass consists of the following elements.

- Attachments: declarations of the render targets used in the RenderPass. For each attachment, `loadOp` (load action) and `storeOp` (store action) are specified. These are the actual form of the actions seen in the previous section, "Tile Memory Load/Store."
- Subpasses: units of drawing within a RenderPass. Each subpass declares its output attachments (color/depth) and input attachments.
- Subpass dependencies: declarations of the execution order and memory dependencies between subpasses.

There are two properties to keep in mind here.

1. Load/store occurs only at RenderPass boundaries, not between subpasses. In other words, if multiple processes can be configured as subpasses of a single RenderPass, the intermediate results between them stay in tile memory.
2. Input attachments let you read the result of a previous subpass, but only at the same pixel position. This is the API-level form of the frame buffer fetch covered in the next section.

Expressing the geometry rendering above with RenderPass/Subpass gives the following.

- Geometry rendering (RenderPass)
  - Depth pre-pass (Subpass) — Output: depth
  - G-Buffer rendering (Subpass) — Input: depth / Output: G-Buffer
  - Deferred lighting (Subpass) — Input: G-Buffer / Output: opaque buffer
  - Transparent rendering (Subpass) — Input: depth / Output: transparent buffer
  - Composite (Subpass) — Input: opaque and transparent buffers / Output: frame buffer

With this configuration, depth, the G-Buffer, the opaque buffer, and the transparent buffer are all passed between subpasses, and only the final output frame buffer crosses the RenderPass boundary. A configuration that could have incurred five stores now incurs one. Incidentally, Metal has an equivalent mechanism. A Metal Render Pass has load/store actions and allows fragment functions to read color attachments in tile memory directly (as well as tile shaders). Metal has no explicit subpass declaration like Vulkan, but the optimizations performed by Render Graph work on Metal in the same way.

7.3.3 Unity's Native Render Pass

So how does Render Graph handle this Vulkan RenderPass/Subpass? The first thing to understand is that a pass added with `AddRasterRenderPass` does not become a subpass as is. A pass added with `RecordRenderGraph` is only a logical pass of Render Graph. After all passes in the frame have been recorded, Render Graph does the following in the compile step.

1. Build the DAG from the resource dependencies and prune unnecessary passes (culling)
2. Merge adjacent passes that satisfy the conditions into one Native Render Pass (equivalent to a Vulkan RenderPass), and make each merged pass a subpass
3. Determine the load/store action of each attachment (see "Rules by Which Render Graph Determines Actions" in the previous section)
4. Make eligible resources memoryless (the "Memoryless" section)

In other words, the developer's job is not to assemble subpasses by hand, but to design passes in a mergeable form and declare resource usage correctly. Whether merging succeeded is checked with the `(RP n:m)` notation in the Frame Debugger and with Render Graph Viewer.

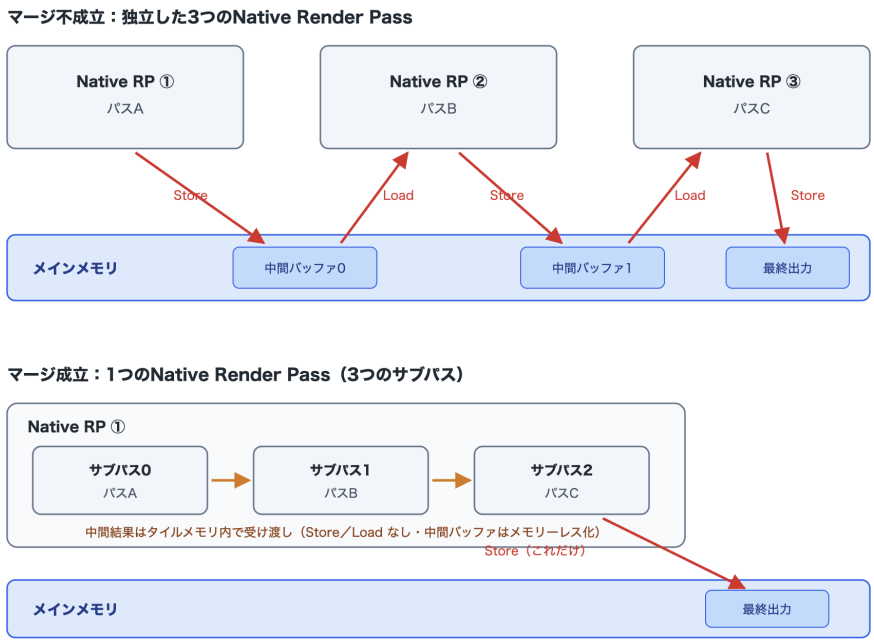
7.3.4 Pass Merge Conditions

The conditions under which Render Graph merges passes are summarized below.

1. Both are Raster Render Passes. `AddComputePass` and `AddUnsafePass` are not merged. If a Compute pass is inserted in a sequence of Raster passes, the Native Render Pass is broken there.
2. The passes are recorded consecutively. Merging is performed between passes that are adjacent in the graph.
3. The output attachment resolutions match. If the resolution of an intermediate buffer is changed (downsampled), merging is not possible at that boundary.
4. The MSAA sample counts match.
5. The later pass does not use the output of the earlier pass as a texture. Use via `UseTexture` requires store→sample and therefore prevents merging. Use as `SetInputAttachment` (input attachment) does not prevent merging. Input attachments and textures are different things.
6. The total number of attachments does not exceed the limit (8). There is an upper limit on the total number of attachments used by the merged Native Render Pass.
7. There is no use that escapes Render Graph's tracking, such as making a texture global.

If even one condition is not met, that pass becomes a standalone Native Render

Pass. The reason a pass was not merged is shown in Pass Break Reasoning in Render Graph Viewer, so check it first when passes are not merged.



▲Figure 7.11 Comparison of Native Render Pass structure before and after merging

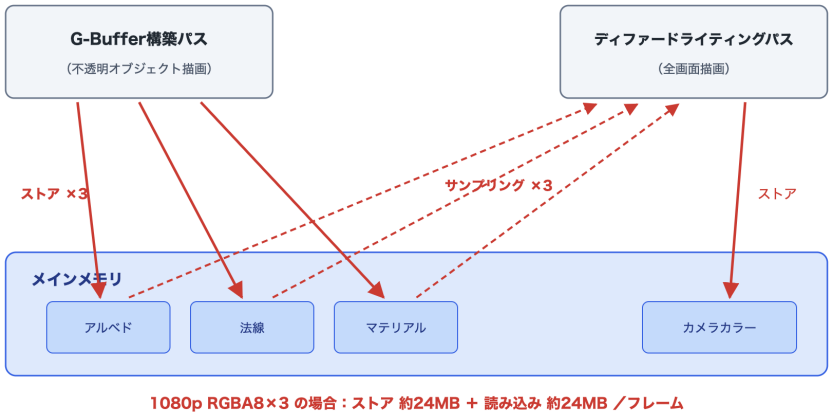
7.4 Frame Buffer Fetch

We have seen that passes can be merged into one Native Render Pass when the conditions are met. This section explains Frame Buffer Fetch (hereafter FBF), a technique for reading the result of a previous subpass directly from tile memory between merged subpasses.

7.4.1 What Is Frame Buffer Fetch

Consider deferred rendering, which creates a G-Buffer and performs lighting. A simple implementation proceeds as follows.

1. Create the G-Buffer (write albedo, normals, and material information to tile memory)
2. Store the albedo, normals, and material information to main memory
3. Sample the stored G-Buffer as textures and perform deferred lighting



▲Figure 7.12　Data flow of simple deferred rendering

A question arises here. What the lighting pass wants to read is the G-Buffer value "of the pixel currently being processed." That value has just been written to the tile memory of the same tile by the preceding pass. Can it not be read directly from tile memory without a round trip to main memory? FBF makes this possible. With FBF, the processing becomes as follows.

1. Create the G-Buffer (write to tile memory)
2. Read the G-Buffer directly from tile memory and perform deferred lighting



▲Figure 7.13　Data flow of deferred rendering using frame buffer fetch

The stores of the three G-Buffer targets and the reads for sampling them are eliminated. For three 1080p RGBA8 targets, this reduces bandwidth by about 24 MB per frame for the stores alone, or about 1.4 GB/s at 60 FPS. At the API level, FBF is realized as the Vulkan input attachment mentioned in the previous section, "Vulkan RenderPass/Subpass" (direct reading of the color attachment in Metal). FBF can therefore be used only between subpasses within the same Render Pass. Passes being merged is a prerequisite for FBF.

7.4.2 Understanding the Constraints

FBF is not a replacement for texture sampling. It has the following constraints.

1. It can read only the same pixel position as the fragment being processed. Tile memory holds only the contents of "the tile currently being processed," so offset sampling (reading neighboring pixels) is impossible in principle.
2. Filtering and mipmaps cannot be used. Only a single-value load is available.
3. The input-side and output-side resolutions must match (this is also a merge condition).

Let us look at an example where constraint 1 applies. Consider the process of "applying UV-distorting noise processing to the result of monochrome/sepia com-

positing." The noise shader samples with shifted UVs as follows.

▼List 7.4 Noise processing shader

```
// Distort the UVs using noise
float2 distortedUV = input.texcoord + noise * 0.02;
// Sample the texture with the distorted UVs.
// Frame buffer fetch cannot be used because the load position differs from the fragment
     position.
half4 color = SAMPLE_TEXTURE2D_X(_BlitTexture, sampler_LinearClamp, distortedUV);
```

Because the position to read differs from the position of the fragment being processed, this pass cannot use FBF. The composite result must be declared with `UseTexture`, and a store→sample round trip occurs between the composite pass and the noise pass. For this reason, checking at the effect design stage whether "this pass reads only the same pixel" lets you judge early whether FBF can be used. Processes that need only the same pixel (tone mapping, color grading, compositing, deferred lighting) are FBF candidates, while processes that read neighboring pixels (blur, distortion, Bloom) use textures.

7.4.3 Inside the Shader API

To use FBF from a Unity shader, use the macros provided by SRP Core.

▼List 7.5 How to use frame buffer fetch

```
// Declare input attachment 0
FRAMEBUFFER_INPUT_X_HALF(0);

half4 frag(Varyings input) : SV_Target
{
    // Load the value at the fragment position being processed from input attachment 0
    half4 color = LOAD_FRAMEBUFFER_X_INPUT(0, input.positionCS.xy);
    ...
}
```

The macros have variations depending on precision and purpose.

Declaration macro	Load macro	Purpose
FRAMEBUFFER_INPUT_HALF(n)	LOAD_FRAMEBUFFER_INPUT(n, pos)	half-precision color

Declaration macro	Load macro	Purpose
FRAMEBUFFER_INPUT_FLOAT(n)	LOAD_FRAMEBUFFER_INPUT(n, pos)	float precision
FRAMEBUFFER_INPUT_X_HALF(n)	LOAD_FRAMEBUFFER_X_INPUT(n, pos)	XR (single-pass instancing) compatible version

Variations with `_X`, like the `TEXTURE2D_X` macros, support XR render target arrays. In projects that may support XR, it is safe to standardize on the variants with `_X`. Knowing what these macros expand to on each platform makes it easier to understand the behavior and the fallback.

Platform	Expands to	What it is
Vulkan	subpassInput + subpassLoad	Render Pass input attachment
Metal	Color attachment read via the [[color(n)]] attribute	Direct read of tile memory
OpenGL ES	Extensions such as GL_EXT_shader_framebuffer_fetch (supported GPUs only)	Falls back as below when the extension is unsupported
Unsupported environments	Regular texture (_UnityFBInput0, etc.) + load instruction	Not FBF; becomes a regular texture read

Note the last row. The macros fall back so that they still work in unsupported environments, but in that case there is no bandwidth reduction. In other words, just because it works does not mean FBF is taking effect. To check whether it is effective, use the action display in Render Graph Viewer (whether the store of the input-side buffer is DontCare) and measurement of bandwidth on a physical device.

7.4.4 The Difference Between SetInputAttachment and Use Texture

On the C# side, declare the resource to be read with FBF using `SetInputAttachment`.

▼List 7.6 Declaring an input attachment

```
using (var builder = renderGraph.AddRasterRenderPass<PassData>("Lighting", out var
    passData))
```

```
{
    builder.SetRenderAttachment(cameraColor, 0, AccessFlags.Write);
    // Corresponds to FRAMEBUFFER_INPUT_X_HALF(0) in the shader
    builder.SetInputAttachment(gBufferAlbedo, 0);
    // Corresponds to FRAMEBUFFER_INPUT_X_HALF(1)
    builder.SetInputAttachment(gBufferNormal, 1);
    ...
}
```

The index in the second argument of `SetInputAttachment` corresponds to the `n` of `FRAMEBUFFER_INPUT_X_HALF(n)` on the shader side. Here, the following table summarizes the differences from `UseTexture`, which has appeared repeatedly since the "Tile Memory Load/Store" section.

Item	UseTexture	SetInputAttachment
Readable position	Any UV (offset, filtering, and mips allowed)	Only the fragment position being processed
Merging with the previous pass	Prevents	Does not prevent
Store of the previous pass's output	Store is forced	Can be DontCare when the merge holds
Bandwidth cost	Store + sampling read	Zero (completed within tile memory)
How to read in the shader	SAMPLE_TEXTURE2D family	LOAD_FRAMEBUFFER family

7.4.5 Fetching Depth

Not only color but also depth resides in tile memory. If depth can be fetched, it may be possible to eliminate the `_CameraDepthTexture` store→sample round trip even for processes that "reference the scene depth," such as soft particles, decals, and fog. SRP Core provides a set of macros for handling depth input attachments, but turning a depth attachment into an input is an area where constraints depending on the platform and URP version tend to arise. Until verification results are available, this chapter recommends the following alternative, which is certain to work. As an alternative, you can write depth to a color attachment and fetch it. In the subpass that builds the G-Buffer, write the linearized depth value to one channel of a color attachment (such as R16_Float), and read it with FBF in subsequent subpasses. This adds one attachment, but if it is made memoryless, the real memory cost is

zero, and the store round trip of the depth texture is reliably eliminated.

7.4.6 Platform Differences and Pitfalls

Finally, here are the platform differences to check when using FBF.

Environment	FBF support	Notes
Vulkan (Adreno)	○ Input attachment	Widely supported. Driver quality depends on the device
Vulkan (Mali)	○ Input attachment	Widely supported
Vulkan (IMG / Pixel 10)	○ Input attachment	TBDR. Watch for driver-specific behavior differences ("GPU Vendor Differences" in the final section)
Metal (Apple)	○ Color attachment read	Works reliably
OpenGL ES	△ Depends on extension	Only GPUs that support GL_EXT_shader_framebuffer_fetch. Others fall back
Editor (D3D, etc.)	× Fallback behavior	Works as a regular texture. You can verify operation but cannot measure the effect

The following three points deserve particular attention.

1. The effect is not visible in the Editor. Because the fallback renders correctly, build your workflow on the assumption that the difference becomes apparent only when measured on a physical device.
2. When MSAA is used together, the merge condition (matching sample counts) is more likely to be violated. Camera color with MSAA enabled and an intermediate buffer created without MSAA cannot be merged. Duplicate the TextureDesc of the intermediate buffer from the original Desc, as in `GetTextureDesc(cameraColor)`, so that the sample count is inherited.
3. There is a limit on the number of input attachments that one subpass can read. The limit of 8 attachments in total ("Pass Merge Conditions" in the previous section) includes the input side. When designing the G-Buffer, save attachments through channel packing.

7.5 Memoryless

So far, we have been able to eliminate both the load and the store of intermediate buffers. This raises a question. Does a buffer that is neither loaded nor stored need an entity in main memory?

7.5.1 What Is Memoryless

In fact, it does not. A resource that is used only in tile memory and discarded in tile memory does not need a main memory allocation at all. This state is called memoryless. Vulkan/Metal have a mode for creating a resource that "exists only in tile memory (on-chip memory) and has no backing store in main memory" (`LAZILY_ALLOCATED` memory in Vulkan, `MTLStorageMode.memoryless` in Metal). A resource specified as memoryless has the following properties.

- It does not consume main memory (the app's memory footprint decreases)
- Load cannot be specified as the load action and Store cannot be specified as the store action (because there is no entity to read or write)

In other words, memoryless is something that holds as a result of eliminating load/store in the previous sections. If load/store is gone, the resource can be made memoryless, and if it cannot be made memoryless, a load or store still remains somewhere.

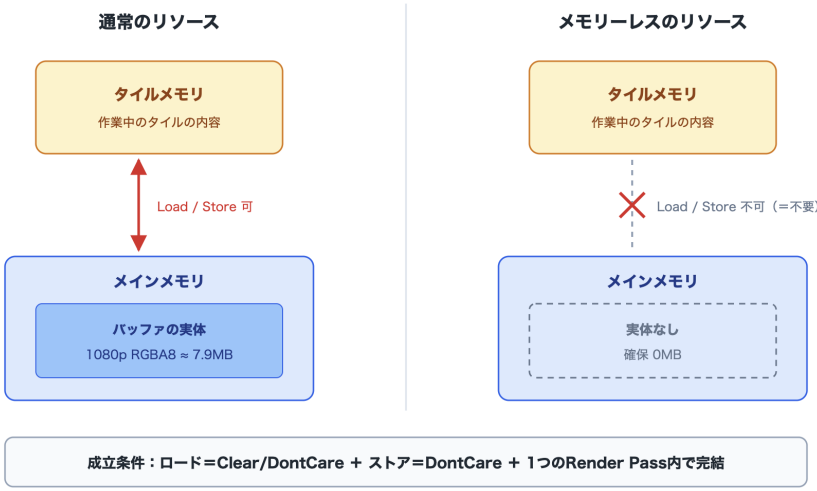
7.5.2 Conditions

Using the terms so far, the conditions under which a resource can be made memoryless are as follows.

1. The load action is Clear or DontCare (the previous content is not read from main memory. This is the WriteAll/Discard declaration seen in the "Declaring Resource Usage with AccessFlags" subsection, or a Clear on first use)
2. The store action is DontCare (it is not used as a texture by subsequent passes and is not carried over to the next frame. See the "Rules by Which Render Graph Determines Actions" subsection)
3. Its use is completed within one Native Render Pass (passing between sub-

passes, that is, input attachments/FBF, is allowed. See merging in the "Render Pass / Subpass" section and FBF in the "Frame Buffer Fetch" section)

As you can see, these three conditions are exactly what the previous sections have explained.



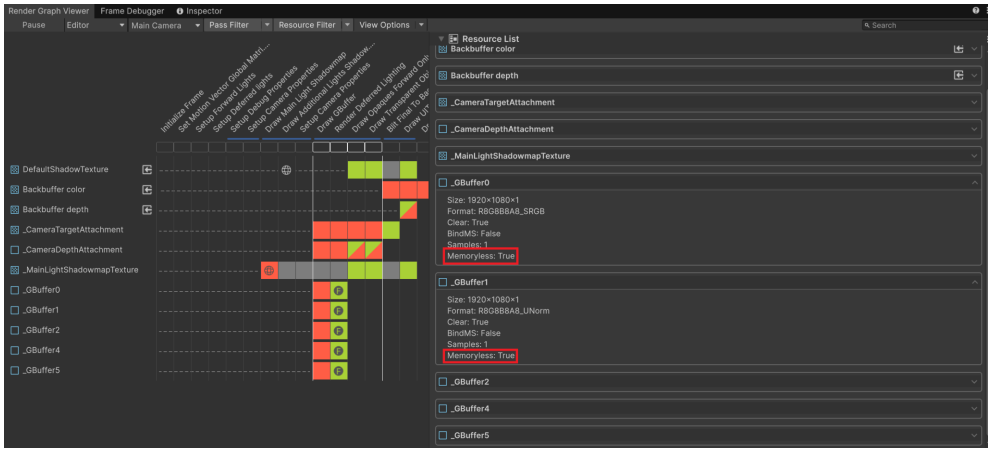
▲Figure 7.14　Memory map comparison before and after making resources memoryless

Automatic Memoryless Conversion and Explicit Specification

In Unity's Render Graph, memoryless is realized through two routes.

Automatic memoryless conversion

When a texture managed by Render Graph (created with `renderGraph.CreateTexture`) satisfies the three conditions in the "Conditions" subsection, Render Graph automatically makes it memoryless. The developer does not need to specify anything. Whether a resource has been made memoryless can be checked in the Resource List of Render Graph Viewer.



▲Figure 7.15　Memoryless display in Render Graph Viewer

Explicit specification

You can also specify it explicitly with the `memoryless` field of `TextureDesc`.

▼List 7.7　Explicitly specifying memoryless

```
var desc = renderGraph.GetTextureDesc(universalResourceData.cameraColor);
desc.memoryless = RenderTextureMemoryless.Color; // Specify memoryless
desc.name = "Intermediate Texture";
var handle = renderGraph.CreateTexture(desc);
```

Explicit specification has the advantage of leaving the design intent in the code. If you carelessly try to `UseTexture` a resource specified as memoryless in a subsequent pass, the inconsistency in the configuration surfaces at that point. If you leave it to the automatic mode, you cannot notice when a refactoring slips a `UseTexture` in somewhere and memoryless is lost (that is, memory usage and bandwidth increase). If the design intent is to keep a resource within tile memory, we recommend recording that in the code with an explicit specification.

7.5.3　Quantifying the Reduction

The reduction from memoryless can be calculated. The following are representative examples at 1080p.

Resource configuration	Calculation	Reduction
One RGBA8 intermediate buffer	1920×1080×4B	About 7.9 MB
One RGBA16F intermediate buffer	1920×1080×8B	About 15.9 MB
Three G-Buffer targets (RGBA8) + depth	7.9MB×4	About 31.7 MB
4x MSAA RGBA8 color	1920×1080×4B×4	About 31.7 MB
4x MSAA depth (D32)	1920×1080×4B×4	About 31.7 MB

As the table shows, MSAA and the G-Buffer yield the largest reductions. In particular, 4x MSAA color and depth together exceed 60 MB, so whether they are eliminated has a considerable effect on the memory budget of the whole app. Note that the reduction from memoryless is in "memory usage," while the reduction in bandwidth comes from the load/store reduction done in the previous sections. Keep in mind that these are separate metrics (memoryless holds as a result of load/store reduction, so in practice the two are achieved together).

7.5.4 Making MSAA Render Targets Memoryless

The effect of memoryless is greatest with MSAA. Let us start with the mechanism. With 4x MSAA, tile memory holds four samples of color and depth per pixel. However, what is finally displayed on screen is only the single sample obtained by averaging (resolving) the four samples. This is where the choice of store action matters.

- If you choose Store, the data for all four samples is written to main memory as is (four times the bandwidth, four times the memory).
- If you choose Resolve, the resolve is performed in tile memory, and only the single resolved sample is written out.

On a TBR GPU, the resolve can be performed in tile memory at almost no cost. And when Resolve is chosen, the four-sample MSAA buffer itself is not stored, so it satisfies the memoryless conditions. In other words, a correctly configured MSAA setup looks like this:

- MSAA color buffer (4 samples) → memoryless (no main memory allocation, no store)
- Resolve target buffer (1 sample) → regular store

- MSAA depth buffer (4 samples) → memoryless + no store (when depth is not used later)

This is why MSAA is said to be low-cost on TBR GPUs (compared with IMR). In URP, enabling the MSAA setting in the Universal Render Pipeline asset causes Render Graph to build this configuration automatically. What you should check is whether it has been broken. In Render Graph Viewer, check that the store action of the MSAA color buffer is Resolve and that Memoryless is yes. A typical case in which it breaks is when a subsequent pass requests the MSAA buffer itself as a texture (an incorrect declaration in a custom pass, a misconfigured Opaque Texture, and so on).

7.5.5 GPU Vendor Differences

The techniques covered in this chapter are all built on standard Vulkan/Metal features and work on the major vendors. Even so, understanding the characteristic differences between vendors makes it easier to interpret measurement results and troubleshoot.

Vendor	Architectural characteristics	Notes
Qualcomm Adreno	GMEM-based binning. Depending on the rendering content, it may switch to direct mode (IMR-like behavior)	When it switches to direct mode, the effect of tile-memory-based optimizations changes. Check with a profiler which mode is in use
ARM Mali	TBR based on 16×16 tiles	Rich external bandwidth counters, making it the reference vendor for the bandwidth measurements in this chapter
Imagination (IMG)	TBDR with hardware hidden surface removal (HSR). The IMG DXT in Pixel 10 falls into this category	HSR inherently reduces opaque overdraw, so where optimization pays off differs from other vendors. Because the drivers are new, verifying behavior differences is recommended

Vendor	Architectural characteristics	Notes
Apple	TBDR in the PowerVR lineage. Tile memory can be handled explicitly in Metal	FBF/memoryless work reliably. It is also effective to confirm the effect on iOS before moving on to Android verification

7.5.6 Conclusion

Let us summarize the four themes covered in this chapter once more. When using Render Graph, the developer does two things: declare resource usage correctly, and check the results with the Frame Debugger and Render Graph Viewer. This is unglamorous work, but its accumulation shows up as differences in device heat and battery consumption. We hope this chapter helps you optimize your projects.

7.5.7 References

- ARM, "Arm GPU Best Practices Developer Guide", https://developer.arm.com/documentation/101897/latest/
- Qualcomm, "Adreno GPU Best Practices", https://docs.qualcomm.com/doc/80-78185-2/topic/mobile_best_practices.html
- Apple, "Metal Best Practices Guide", https://developer.apple.com/library/archive/documentation/3DDrawing/Conceptual/MTLBestPracticesGuide/
- Unity, "Render graph system" (URP Documentation), https://docs.unity3d.com/Manual/urp/render-graph.html
- Rendering "Lighting and Reflections" to the Fullest on Mobile in Gakuen Idolmaster [CEDEC2024], https://gamemakers.jp/article/2025_01_29_90729/
- How to Achieve Blazing-Fast Deferred Rendering on Mobile, CEDiL, https://cedil.cesa.or.jp/cedil_sessions/view/2457
- Mastering RenderGraph from Unity 6: Basic Features, https://blog.sge-coretech.com/entry/2024/06/04/171757
- Mastering RenderGraph from Unity 6! Advanced Implementation 1: "Building a RenderPass", https://blog.sge-coretech.com/entry/2024/09/11/105032

- Introduction to Shader Optimization, Part 2: "What Is a Tile-Based Rendering GPU?", https://blog.sge-coretech.com/entry/2025/01/27/145242

Chapter 8

Graphics Optimization with Renderer Shader User Value

Noboru Seto

CyberAgent, Inc. SGE Core Technology Division

ruccho

8.1 Introduction

Unity 6.3 added a feature called Renderer Shader User Value (RSUV). RSUV is a 32-bit integer value that can be set per Renderer component and can also be referenced from shaders. In Unity 6000.3.0f1 and later, RSUV can be used with MeshRenderer and SkinnedMeshRenderer, and in Unity 6000.5.0f1, 6000.4.4f1, and 6000.3.13f1 and later, it is also supported on SpriteRenderer [*1], SpriteShape Renderer, and TilemapRenderer. Usage is simple: set the RSUV on a renderer with Renderer.SetShaderUserValue(), and reference it in the shader with unity_RendererUserValue.

```
uint value = /* ... */;
meshRenderer.SetShaderUserValue(value);
value = meshRenderer.GetShaderUserValue();
```

```
uint value = unity_RendererUserValue;
```

The same thing as in this example can also be achieved with material parameters. However, using RSUV **may improve memory allocation and frame rate compared**

[*1] Getting and setting RSUV on SpriteRenderer requires the 2D Module.

with materials, especially in situations that draw large numbers of objects. This chapter explains the background of RSUV, how to use it effectively, and also presents performance measurement results.

8.1.1 Unity's Mass-Rendering Features and the Background of RSUV

First, let us review the background that made RSUV necessary. Performance when displaying a large number of objects at once has always been a headache in Unity. The performance problems in mass rendering are mainly these three: CPU load from material switching, an increase in DrawCalls, and memory allocation from material duplication.

CPU Load from Material Switching

When drawing multiple objects with different materials, the material data had to be collected for each object and sent to the GPU every frame, and the CPU processing cost of this was large. The SRP Batcher solves this problem. By persisting material data in GPU memory, it can draw multiple objects with different materials at relatively low cost as long as they use the same shader.

Increase in DrawCalls

GPU instancing is a technique that draws multiple objects with the same shader, mesh, and material in a single DrawCall. Its conditions for application are stricter than the SRP Batcher's, but in exchange it is more effective than the SRP Batcher when drawing a large number of objects. With the introduction of the GPU Resident Drawer in Unity 6, GPU instancing is now applied automatically even to rendering with regular MeshRenderers when the conditions are met.

Memory Allocation from Material Duplication

The last problem is the situation where you want to vary the material while using the SRP Batcher or GPU instancing. Unity has long had MaterialPropertyBlock, which overrides only some parameters without duplicating the material instance, but it was not compatible with the SRP Batcher or the GPU Resident Drawer. RSUV solves this problem: while using the SRP Batcher or the GPU Resident Drawer, you can pass a different 32-bit parameter to the shader for each object **without using**

materials.

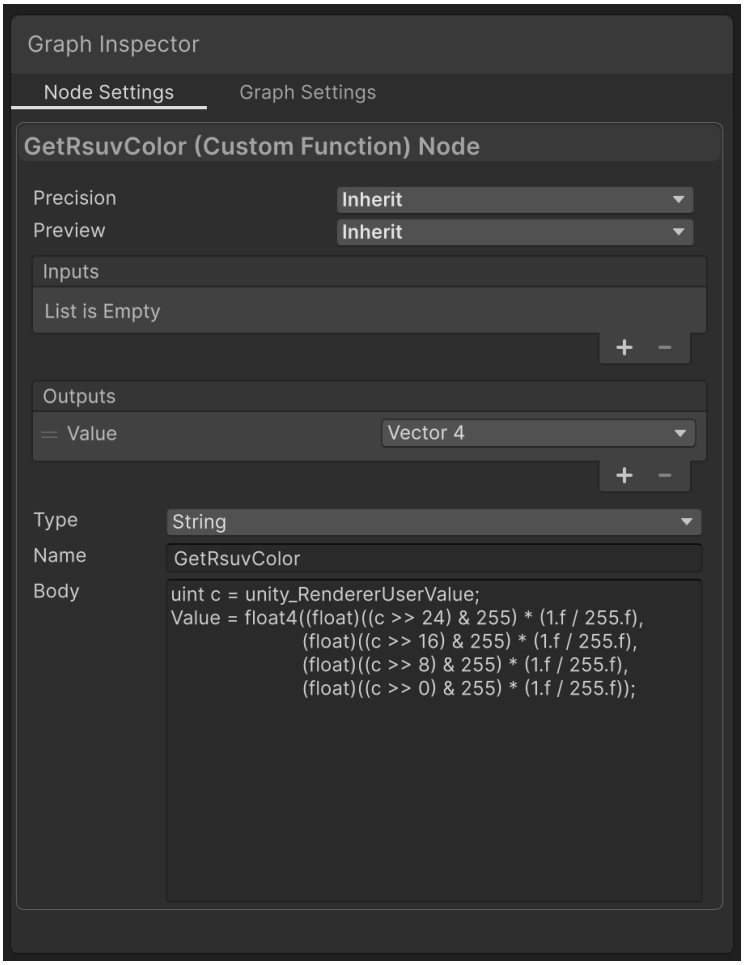
RSUV was the last piece needed to solve the three problems above at the same time. You can keep the cost of material switching low with the SRP Batcher, perform efficient GPU instancing with the GPU Resident Drawer, and still render with per-instance variation. Even in situations where automatic instancing by the GPU Resident Drawer does not apply, you benefit from avoiding duplication of material instances and thus reducing memory consumption.

8.2 Simple Usage

Let us look at how to use RSUV in practice. RSUV is represented as a 32-bit integer (uint) in C# / HLSL, so to pass, for example, a 32-bit color (four RGBA channels of 8 bits each), you pack it with bit operations as follows.

```
Color32 color = /* ... */;
var colorEncoded = (uint)(color.r << 24 | color.g << 16 | color.b << 8 | color.a);
meshRenderer.SetShaderUserValue(colorEncoded);
```

Next is the shader-side implementation, for which we assume ShaderGraph. ShaderGraph has no dedicated node for getting the RSUV, so use a Custom Function node, configure it as shown (Figure 8.1), and enter the code.



▲Figure 8.1　Node settings

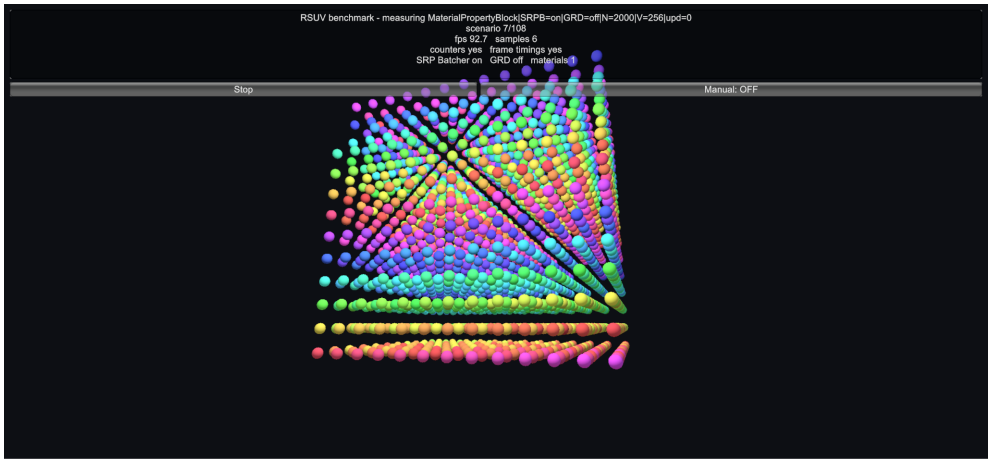
```
uint rsuv = unity_RendererUserValue;
Value = float4((float)((rsuv >> 24) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 16) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 8) & 255) * (1.f / 255.f),
                        (float)((rsuv >> 0) & 255) * (1.f / 255.f));
```

Connect the node and confirm that the RSUV is being received.

8.3 Performance Measurement

To confirm that RSUV is effective in mass-rendering situations, we measured memory allocation and frame rate under the following conditions.

- Unity 6000.3.15f1
- URP 17.3
- iPhone 12
- Scene: 10000 Spheres
- SRP Batcher enabled
- Compared with and without per-frame parameter changes
- Compared with the GPU Resident Drawer enabled and disabled
- A different shader parameter per Sphere, set in three ways
  - Per-Renderer material: create and assign 10000 Materials
  - RSUV: one Material, with an RSUV set per Renderer
  - MaterialPropertyBlock: one Material and 10000 MaterialPropertyBlocks (excluded from the SRP Batcher / GPU Resident Drawer)



▲Figure 8.2 Benchmark screen

First is a comparison of memory allocation for each parameter-setting method (Table 8.1).

Table 8.1: Memory increase

Condition	Memory increase
Per-Renderer material	15.79 MB
MaterialPropertyBlock	3.32 MB
RSUV	1.66 KB

Because materials are not duplicated, memory allocation is naturally reduced significantly as well. Next, let us look at the frame rate. First is the case where parameters do not change per frame (Table 8.2).

Table 8.2: Frame rate (FPS) when parameters do not change per frame

Condition	Without GPU Resident Drawer	With GPU Resident Drawer
Per-Renderer material	33.8	38.8
RSUV	49.9	59.8
MaterialPropertyBlock	28.9	28.9

As shown, RSUV performs better than per-Renderer materials, and the effect increases further when combined with the GPU Resident Drawer. RSUV alone reduces the cost of material switching, and GPU instancing with the GPU Resident Drawer additionally reduces the load of issuing draw commands on the render thread. Next, let us look at the case where parameters change per frame (Table 8.3).

Table 8.3: Frame rate (FPS) when parameters change per frame

Condition	Without GPU Resident Drawer	With GPU Resident Drawer
Per-Renderer material	11.1	8.1
RSUV	48.0	60.0
MaterialPropertyBlock	26.1	25.2

Compared with the case without parameter updates, performance drops significantly with per-Renderer materials, whereas RSUV shows almost no degradation. Changing one material parameter requires re-uploading the entire material to the

GPU, whereas RSUV is sent as part of a buffer that is always sent to the GPU for every frame and every Renderer [*2], so the additional cost incurred by updating RSUV stays low. Interestingly, mass updates of material parameters and the GPU Resident Drawer do not combine well, and performance appears to be lower than when the GPU Resident Drawer is disabled. As above, RSUV is advantageous in both "parameter updates" and "draw command issuing." The effect of RSUV is especially large in situations where RSUV makes GPU instancing possible, such as when a large number of objects share the same shader and mesh.

8.4 Passing Data Larger Than 32 Bits

RSUV is useful for improving performance in this way, but its major weakness is that it can pass only values that fit in 32 bits. This section introduces a technique to overcome that. To pass data that does not fit in 32 bits with RSUV, an effective method is to store the target data in a separate GraphicsBuffer and pass RSUV an index into that GraphicsBuffer. This method incurs an additional cost for uploading the GraphicsBuffer, so some care may be needed regarding the update frequency and size of the GraphicsBuffer.

```
[StructLayout(LayoutKind.Sequential)]
struct VariantGpuData
{
    public Vector4 baseColor;
    public Vector4 surface;
}
private static readonly int VariantBufferId = Shader.PropertyToID("_RsuvVariantBuffer")
    ;

// Upload and assign the GraphicsBuffer
var buffer =  new GraphicsBuffer(GraphicsBuffer.Target.Structured, count, UnsafeUtility
    .SizeOf<VariantGpuData>());
var cpuBuffer = new VariantGpuData[count];
FillCpuBufferValues(cpuBuffer);
buffer.SetData(cpuBuffer, 0, 0, count);
Shader.SetGlobalBuffer(VariantBufferId, buffer);

// Set the RSUV
```

[*2] Only when not instanced. When instanced, it is passed through a GraphicsBuffer together with the other instance parameters, and is uploaded only when the overall parameters change and the GraphicsBuffer becomes dirty.

```
renderer.SetShaderUserValue(index);

// ...

// Release it when no longer needed
buffer.Dispose();
```

```
struct RsuvVariantRecord
{
    float4 baseColor;
    float4 surface;
};

StructuredBuffer<RsuvVariantRecord> _RsuvVariantBuffer;

// Access
RsuvVariantRecord record = _RsuvVariantBuffer[unity_RendererUserValue];
```

8.4.1 Summary

This chapter introduced the background of RSUV, performance measurements, and basic and advanced usage. Although RSUV is limited to 32 bits compared with materials, it can bring significant performance improvements depending on the situation. Identify the situations where RSUV is effective and use it well.

Chapter 9

When to Use UnsafePass in Render Graph

ZHANG YUBING

CyberAgent, Inc. SGE Core Technology Division

Name：ZHANG YUBING Affiliation：CyberAgent, Inc. SGE Core Technology Division

9.1 Introduction

In Unity 6, when you build a rendering pipeline with Render Graph, you choose among RasterRenderPass, ComputePass, and UnsafePass depending on the work. This chapter describes the situations in which UnsafePass should be used.

Render Graph provides optimizations such as dependency analysis and subpass merging. Therefore, the basic rule is to prefer RasterRenderPass for regular rendering and ComputePass for work that uses a compute shader.

UnsafePass should mainly be chosen in the following four situations.

- When the work cannot be done with RasterRenderPass or ComputePass
- When the CommandBuffer itself is required
- When you want to migrate quickly from Compatibility Mode to Render Graph
- When a single rendering operation consists of many passes that cannot be merged

The first three are cases where the work is impossible without UnsafePass or the

implementation cost is high. The last one is not mandatory, but combining the passes into a single UnsafePass can be expected to reduce CPU load and make the Render Graph Viewer easier to read.

The following sections explain each of these situations in turn.

9.2 Situations Where UnsafePass Should Be Chosen

UnsafePass does not refer to the C# unsafe feature. It is a pass that suppresses part of the automatic management and validation done by Render Graph and exposes an API close to CommandBuffer.

9.2.1 When the work cannot be done with RasterRenderPass or ComputePass

RasterCommandBuffer and ComputeCommandBuffer, used by RasterRender Pass and ComputePass, are wrappers that expose only the CommandBuffer APIs needed for the role of each pass. The following APIs are not available.

- SetRenderTarget: specifies render targets, MRTs, mips, and array slices in detail while setting Load/Store Actions
- CopyTexture: copies a texture by specifying mips, array elements, and rectangular regions
- GenerateMips: generates mipmaps of a render target
- SetRandomWriteTarget/ClearRandomWriteTargets: sets or clears random write targets for pixel shaders
- RequestAsyncReadback: records an asynchronous readback request for a GPU resource

These APIs are available in the UnsafeCommandBuffer of UnsafePass. In addition, if you obtain the native CommandBuffer with CommandBufferHelpers.GetNative CommandBuffer, more advanced APIs also become available.

9.2.2 When the CommandBuffer itself is required

In engine callbacks, IMGUI, SDK rendering, and similar cases, external code receives a CommandBuffer and records arbitrary commands, so what is recorded is unknown until runtime. RasterRenderPass and ComputePass declare their work at recording time, so they cannot incorporate such black-box processing. By obtaining the native CommandBuffer in UnsafePass and passing it to the external code, you can incorporate it into Render Graph.

9.2.3 When you want to migrate quickly from Compatibility Mode to Render Graph

The CommandBuffer code written in the Execute method in Compatibility Mode can be moved almost as is, as a command sequence, into the SetRenderFunc of UnsafePass. This is effective when you want to migrate to Render Graph and verify its behavior with little effort.

However, this is only a provisional means for quick migration. If time allows, it is recommended to classify the work and port it to RasterRenderPass or ComputePass as appropriate, and to limit UnsafePass to the parts that cannot be expressed with the declarative API.

9.2.4 When the work consists of many passes that cannot be merged

Combining consecutive passes that cannot be merged into a single UnsafePass reduces CPU load and makes the Render Graph Viewer easier to read.

Typical Cases

- Ping-pong blur
- Bloom mip pyramid generation
- Multiple downsamples and upsamples
- Filtering in which the output of the previous operation is sampled by the next operation

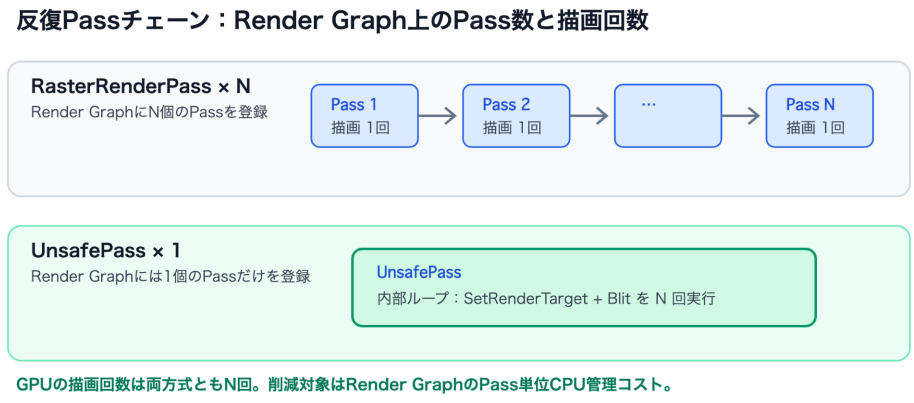
In these operations, the render target is switched for each pass, or the output of the preceding pass is sampled from neighboring pixels, so subpass merging does not apply.

Combining into UnsafePass Reduces CPU Load

Each time a RasterRenderPass is registered, the following processing occurs.

- Pass registration and PassData preparation
- Registration of resource dependencies with UseTexture and SetRenderAttachment
- Pass dependency analysis and checking of merge candidates
- Start and end of the profiling scope
- Render target setup and invocation of the execution callback

The more RasterRenderPasses there are, the higher the CPU cost becomes. If a chain of passes that cannot be merged is registered as RasterRenderPasses, the registration and management costs accumulate without any merging benefit. In contrast, if they are combined into a single UnsafePass, they are registered as one pass in Render Graph. The cost of draw commands and render target switches does not change, but the per-pass registration and management cost incurred by Render Graph can be reduced (Figure 9.1).



▲Figure 9.1 Number of passes in Render Graph and number of draws for a chain of passes that cannot be merged

Measurement Example of the CPU Load Reduction

In a measurement in the Editor performed in this project, for a 4-tap ping-pong blur on a 256×256 Render Texture, a method that registers a RasterRenderPass for each blur operation was compared with a method that executes the whole series in a single UnsafePass. The measurement conditions were Apple M4 Max (macOS 26.5), Unity 6000.3.19f1, and URP 17.3. 500 frames were captured for each combination, and the median was used after excluding the 60 frames following the switch. The measured value is the sum of the CPU markers for Render Graph recording, hash calculation, compilation, and execution.

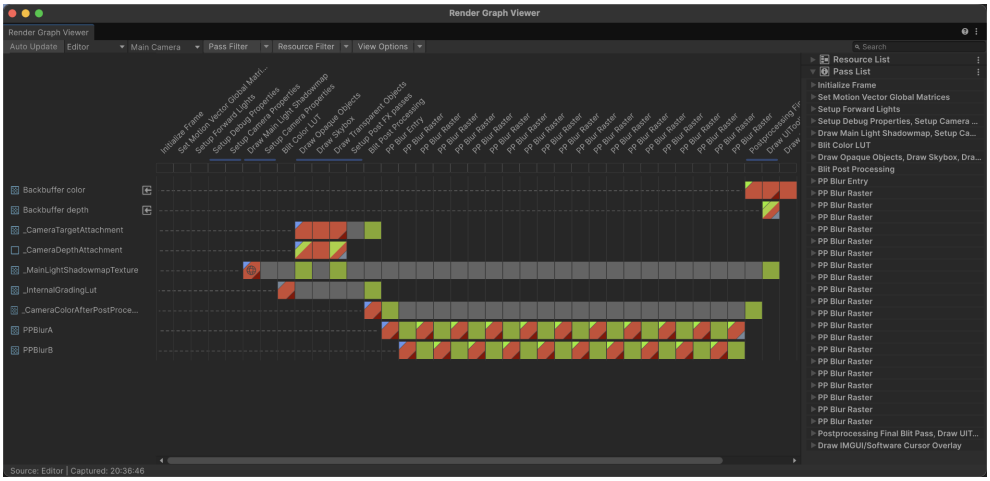
Table 9.1: Comparison of Render Graph-related CPU time in ping-pong blur

Blur count	RasterRenderPass× N (μ s)	UnsafePass×1 (μ s)	Difference (μ s)
5	286.4	280.0	6.4
10	334.8	313.6	21.2
20	427.8	389.1	38.7
50	675.9	592.9	83.0
100	1080.7	898.5	182.2

The measured values are for checking trends in the Editor environment. As the blur count increases, the difference between the RasterRenderPass method and the UnsafePass method widens. In a linear fit, the difference between the slopes of the two methods, that is, the additional fixed cost of a RasterRenderPass, was about 1.8μ s per pass. This value was measured in the Editor and does not indicate an absolute value on a physical device, but it confirms that the CPU load difference grows with the blur count. This effect comes from the reduction in per-pass registration and management costs, such as Render Graph recording, execution, profiling scopes, and render target setup. Because there is no difference in draw commands, converting to UnsafePass does not reduce rendering load.

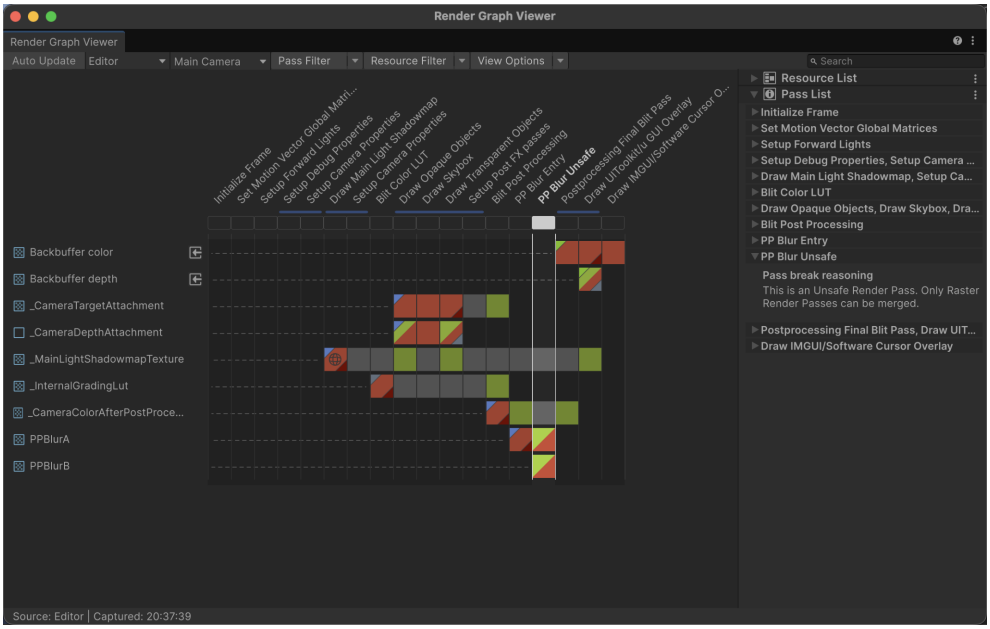
It Also Makes the Render Graph Viewer Easier to Read

When many RasterRenderPasses are registered, the Render Graph Viewer displays as many passes as were registered. The graph becomes long horizontally, so the boundaries between features are hard to see and resource dependencies are harder to grasp (Figure 9.2).



▲Figure 9.2 Render Graph Viewer display when a RasterRenderPass is registered for each blur operation

If the pass chain inside a feature is combined into a single UnsafePass, the Viewer shows it as one pass representing the whole feature (Figure 9.3). This makes the boundaries between features clear and makes resource dependencies easier to grasp.



▲Figure 9.3 Render Graph Viewer display when the series of blur operations is combined into a single UnsafePass

In addition to the reduced CPU load, another significant advantage is that noise during profiling is reduced.

However, it is not recommended to force operations of different features, or operations for which subpass merging applies, into an UnsafePass solely to make the Render Graph Viewer easier to read.

9.3 Notes on Using UnsafePass

9.3.1 Setting Render Targets

In RasterRenderPass, render targets are declared at recording time with Set RenderAttachment, whereas in UnsafePass they are set by calling SetRenderTarget at execution time.

9.3.2 Declare Dependencies

Using UnsafePass does not completely detach you from Render Graph. You must always declare read/write dependencies for the textures you use with UseTexture. If a declaration is missing, resource lifetime management and synchronization are not performed correctly.

In RasterRenderPass, the use of a render target is declared with SetRender Attachment, but in UnsafePass, calling SetRenderTarget alone does not register a dependency. You must also declare read/write dependencies for render targets with UseTexture.

9.3.3 Setting Load/Store Actions

RenderBufferLoadAction specifies how the existing contents are handled when the render target is enabled, and RenderBufferStoreAction specifies how the result is handled after rendering completes. On tile-based GPUs, unnecessary Load and Store operations increase transfers between tile memory and main memory, so appropriate settings are important.

In RasterRenderPass, Render Graph automatically determines the Load/Store Actions based on the declared AccessFlags. In UnsafePass, however, if you omit the actions, as in SetRenderTarget(target), the default behavior of Load/Store is applied, and unnecessary memory transfers can degrade performance.

Therefore, in UnsafePass, always specify the Load/Store Actions explicitly when calling SetRenderTarget.

- Load side: use DontCare if the whole screen is overwritten, and Load only if the existing contents are read
- Store side: use Store if the result is read by later processing, and DontCare only if it is not used anywhere

9.4 Code Example: A Ping-pong Blur Chain of Passes That Cannot Be Merged

This section presents actual code examples. Note that the code shown is an excerpt to illustrate the key points and is not complete code that works as is.

This is an example that applies blur repeatedly while alternately reading from and writing to two textures. It first shows the same processing implemented with Raster RenderPass, and then the version combined into a single UnsafePass. desc is the render texture descriptor for the blur, and blurMaterial is the material for the blur.

9.4.1 RasterRenderPass Method: Register a Pass for Each Blur Operation

In the RasterRenderPass method, as in List 9.1, AddRasterRenderPass is called for each blur operation in a loop. The input and output of each pass can be declared clearly, but as the blur count increases, the number of passes in Render Graph also increases.

▼List 9.1 Implementing ping-pong blur with RasterRenderPass

```
class BlurRasterPassData
{
    internal TextureHandle source;
    internal TextureHandle target;
    internal Material material;
}

var ping = renderGraph.CreateTexture(desc);
var pong = renderGraph.CreateTexture(desc);
var current = resourceData.activeColorTexture;
var writePing = true;

for (var i = 0; i < blurIterations; i++)
{
    var target = writePing ? ping : pong;

    using (var builder = renderGraph.AddRasterRenderPass<BlurRasterPassData>(
        "Ping-pong Blur Raster", out var passData))
    {
```

```
        passData.source = current;
        passData.target = target;
        passData.material = blurMaterial;

        builder.UseTexture(passData.source, AccessFlags.Read);
        builder.SetRenderAttachment(passData.target, 0, AccessFlags.Write);
        builder.SetRenderFunc((BlurRasterPassData data, RasterGraphContext context) =>
        {
            Blitter.BlitTexture(
                context.cmd, data.source, new Vector4(1, 1, 0, 0),
                data.material, 0);
        });
    }

    current = target;
    writePing = !writePing;
}
```

9.4.2 UnsafePass Method: Execute the Whole Series in One Pass

In the UnsafePass method, as in List 9.2, registration with Render Graph is done once, and render targets are switched inside SetRenderFunc. The draw commands for each blur operation remain, but per-pass management is consolidated into one.

▼List 9.2 Combining ping-pong blur into a single UnsafePass

```
class PingPongBlurPassData
{
    internal TextureHandle source;
    internal TextureHandle ping;
    internal TextureHandle pong;
    internal int iterations;
    internal Material material;
}

var ping = renderGraph.CreateTexture(desc);
var pong = renderGraph.CreateTexture(desc);
var output = blurIterations == 0
    ? resourceData.activeColorTexture
    : blurIterations % 2 == 0 ? ping : pong;

using (var builder = renderGraph.AddUnsafePass<PingPongBlurPassData>(
    "Ping-pong Blur", out var passData))
{
```

```
    passData.source = resourceData.activeColorTexture;
    passData.ping = ping;
    passData.pong = pong;
    passData.iterations = blurIterations;
    passData.material = blurMaterial;

    builder.UseTexture(passData.source, AccessFlags.Read);
    builder.UseTexture(passData.ping, AccessFlags.ReadWrite);
    builder.UseTexture(passData.pong, AccessFlags.ReadWrite);
    builder.SetRenderFunc((PingPongBlurPassData data, UnsafeGraphContext context) =>
    {
        var cmd = CommandBufferHelpers.GetNativeCommandBuffer(context.cmd);
        var current = data.source;
        var next = data.pong;
        var alternate = data.ping;
        for (var i = 0; i < data.iterations; i++)
        {
            cmd.SetRenderTarget(
                next,
                RenderBufferLoadAction.DontCare,
                RenderBufferStoreAction.Store);
            Blitter.BlitTexture(
                cmd, current, new Vector4(1, 1, 0, 0),
                data.material, 0);

            current = next;
            (next, alternate) = (alternate, next);
        }
    });
}
```

9.5 Summary

This chapter described when to use UnsafePass in Render Graph and the points to note when using it.

UnsafePass should be chosen in the following four situations.

- When you need an API that is not exposed in RasterCommandBuffer/Compute CommandBuffer
- When you need the CommandBuffer itself to pass to external code
- When you want to migrate quickly from Compatibility Mode to Render Graph
- When you want to combine a chain of passes that cannot be merged into a

single pass

Combining a chain of passes that cannot be merged into a single UnsafePass reduces the per-pass registration and management cost and also makes the Render Graph Viewer easier to read. In the Editor measurement in this project (Table 9.1), a fixed cost of about 1.8μ s per RasterRenderPass was eliminated. Note, however, that the draw commands themselves do not change, so this is not a technique aimed at reducing GPU load.

On the other hand, UnsafePass is also a pass that gives up part of the automatic management and validation done by Render Graph. When using it, be sure to declare dependencies with UseTexture and to specify Load/Store Actions explicitly in SetRenderTarget.

Prefer RasterRenderPass and ComputePass as a rule, and limit UnsafePass to work that cannot be expressed with the declarative API and to tidying up chains of passes that cannot be merged. This lets you combine Render Graph optimization with the flexibility of CommandBuffer.

9.6 References

- Unity Manual - Use the CommandBuffer interface in a render graph (Unsafe Pass)
  `https://docs.unity3d.com/6000.3/Documentation/Manual/urp/render-graph-unsafe-pass.html`
- Unity Manual - Optimize a render graph
  `https://docs.unity3d.com/6000.3/Documentation/Manual/urp/render-graph-optimize.html`
- Unity Scripting API - CommandBuffer
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.CommandBuffer.html`
- Unity Scripting API - RenderBufferLoadAction
  `https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.RenderBufferLoadAction.html`
- Unity Scripting API - RenderBufferStoreAction

https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.RenderBufferStoreAction.html

Chapter 10

How to Omit URP's FinalBlitPass

ZHANG YUBING

CyberAgent, Inc. SGE Core Technology Division

Name：ZHANG YUBING Affiliation：CyberAgent, Inc. SGE Core Technology Division

10.1 What Is FinalBlitPass

In the URP rendering pipeline, what is actually displayed on the screen is the Back Buffer. However, the BackBuffer is write-only and cannot be read. Therefore, when there is processing that reads intermediate results as input, such as post-processing, the camera first renders to an intermediate texture, which is finally blitted to the Back Buffer. The URP pass responsible for blitting from this intermediate texture to the BackBuffer is FinalBlitPass. It is not a mere copy; it also handles the final processing required immediately before output to the screen.

- Transfer that takes the camera Viewport into account
- Linear to sRGB conversion
- Encoding for HDR Output and compositing of the overlay UI
- Y flip when the UV origins differ
- Filter switching when scaling

10.1.1 Three Kinds of FinalBlitPass with the FinalBlit Function

URP actually has three kinds of passes whose role is to output the intermediate texture to the BackBuffer, and only one of them is selected depending on conditions.

UberPost

This pass is added when post-processing is enabled, and it also serves as the FinalBlit when the following conditions are met.

- It is the last camera in the camera stack
- FinalPost, described below, is not required
- There is no custom pass that runs at or after AfterRenderingPostProcessing

FinalPost

This pass is added when post-processing is enabled and there is a final step that UberPost cannot handle.

1. FXAA
2. Upscaling other than Linear: Point / FSR / STP, and so on.
3. TAA + CAS: RCAS sharpening must be run in a separate pass

FinalBlitPass

This is the last resort when neither of the above two writes to the BackBuffer. It actually appears mainly in the following cases.

- Post-processing is disabled and an intermediate texture exists
- Post-processing is enabled, but there is a custom pass after it
- Post-processing is enabled, but there is capture processing such as Camera.AddCommandBufferAsync

Relationship Among the Three

Table 10.1: Passes responsible for the final output and their conditions

Pass responsible for the final output	Condition
UberPost	Post-processing enabled, FinalPost not required, and no subsequent custom pass or capture
FinalPost	Post-processing enabled, and any of FXAA / upscaling other than Linear / TAA + CAS
FinalBlitPass	The above two do not write to the BackBuffer, and an intermediate texture exists
None of them	There is no intermediate texture, or the camera is not the last one in the camera stack

10.2 How to Omit FinalBlitPass

You do not need to be concerned about FinalBlitPass when using URP post-processing or creating a regular custom post-process. However, if you add a custom pass that corresponds to the "final step," URP's FinalBlitPass becomes a wasteful pass that only inserts one extra blit. Therefore, starting with Unity 6.3, a custom pass can declare that "the active target is now the BackBuffer," which allows URP's FinalBlitPass to be omitted.

▼List 10.1 Switching to the BackBuffer

```
var resourceData = frameData.Get<UniversalResourceData>();
resourceData.SwitchActiveTexturesToBackbuffer();
```

The implementation is simple (Runtime/FrameData/UniversalResourceData.cs).

▼List 10.2 Implementation of SwitchActiveTexturesToBackbuffer

```
public void SwitchActiveTexturesToBackbuffer()
{
    activeColorID = UniversalResourceData.ActiveID.BackBuffer;
    activeDepthID = UniversalResourceData.ActiveID.BackBuffer;
}
```

It only rewrites two IDs in frameData, and performs no blit, no copy, and no addition of a RenderPass. This API is not a "transfer operation" but a declaration to URP.

activeColorTexture / activeDepthTexture are simply a switch that changes the Texture Handle they return based on these IDs.

▼List 10.3 Switching activeColorTexture

```
public TextureHandle activeColorTexture
{
    get
    {
        switch (activeColorID)
        {
            case ActiveID.Camera:     return cameraColor;
            case ActiveID.BackBuffer: return backBufferColor;
            ...
        }
    }
}
```

After SwitchActiveTexturesToBackbuffer() is executed, URP's FinalBlitPass is no longer generated.

10.2.1 Usage Example: Custom FXAA

Taking a custom FXAA post-process as an example, we compare the flows before and after. The typical flow in the past required three blits.

▼List 10.4 Conventional FXAA flow

```
CameraColor → [FXAA] → FXAABuffer → [Blit] → CameraColor → [URP FinalBlit] →
     BackBuffer
```

From Unity 6.0 onward, CameraColor can be swapped directly, which reduces the number of blits to two, but an intermediate texture called FXAABuffer is still required, and FinalBlitPass is still created.

▼List 10.5 FXAA flow in Unity 6.0

```
CameraColor → [FXAA] → FXAABuffer → [URP FinalBlit] → BackBuffer
```

With the BackBuffer switch in Unity 6.3, one blit is enough. No intermediate texture is needed either.

▼List 10.6 FXAA flow in Unity 6.3

```
CameraColor → [FXAA] → BackBuffer
```

The code looks as follows. Note that the code that follows is partly excerpted to show the key points and does not work as is.

▼List 10.7 Custom FXAA that outputs directly to the BackBuffer

```
private class PassData
{
    public TextureHandle Source;
    public TextureHandle Target;
    public Material FXAAMaterial;
}

public override void RecordRenderGraph(RenderGraph renderGraph, ContextContainer
    frameData)
{
    var cameraData = frameData.Get<UniversalCameraData>();
    var resourceData = frameData.Get<UniversalResourceData>();

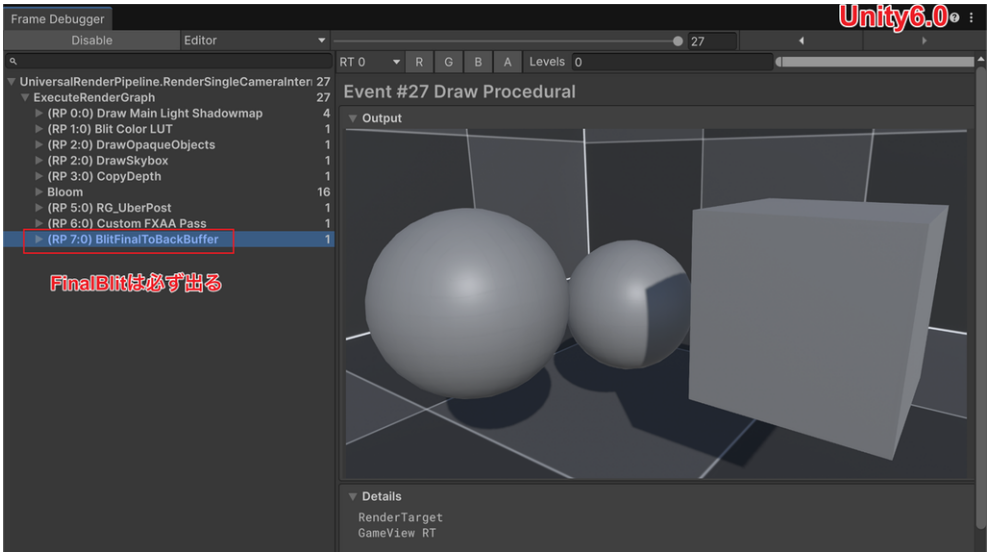
    // Obtain CameraColor before switching
    var cameraColor = resourceData.activeColorTexture;

    // From here on, activeColorTexture returns BackBufferColor
    // This stops URP's FinalBlitPass from being generated
    resourceData.SwitchActiveTexturesToBackbuffer();
    var backBufferColor = resourceData.activeColorTexture;

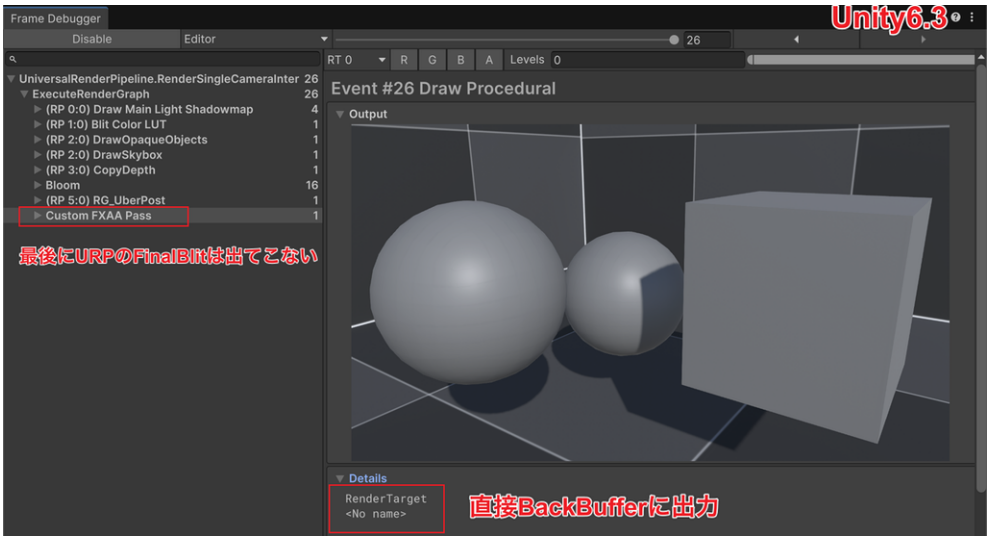
    // Determine whether LinearToSRGBConversion is needed
    var requireSrgbConversion = cameraData.targetTexture == null
                                && Display.main.requiresSrgbBlitToBackbuffer;
    CoreUtils.SetKeyword(_fxaaMaterial, ShaderKeywordStrings.LinearToSRGBConversion,
    requireSrgbConversion);

    using (var builder = renderGraph.AddRasterRenderPass<PassData>("Custom FXAA Pass",
    out var passData))
    {
        passData.Source = cameraColor;
        passData.Target = backBufferColor;
        passData.FXAAMaterial = _fxaaMaterial;
        builder.SetRenderAttachment(backBufferColor, 0, AccessFlags.WriteAll);
        builder.UseTexture(passData.Source, AccessFlags.Read);
        builder.SetRenderFunc(static (PassData data, RasterGraphContext context) =>
        {
            // A Y flip is needed only when the UV origins of Source and Target differ
            var yFlip = context.GetTextureUVOrigin(data.Source) != context.
    GetTextureUVOrigin(data.Target);
            var scaleBias = yFlip ? new Vector4(1, -1, 0, 1) : new Vector4(1, 1, 0, 0);
            Blitter.BlitTexture(context.cmd, data.Source, scaleBias, data.FXAAMaterial,
     0);
        });
    }
}
```

In Unity 6.0, FinalBlitPass was always created.



With the BackBuffer switch in Unity 6.3, the FXAA processing completes in a single blit and is output directly to the BackBuffer.



10.3 Notes on Omitting URP's FinalBlit

SwitchActiveTexturesToBackbuffer() is an API that only rewrites internal texture I Ds. Conversely, the caller is entirely responsible for keeping everything consistent.

10.3.1 Call SwitchActiveTexturesToBackbuffer() Only at the End of the Camera

From the moment this API is called, activeColorTexture / activeDepthTexture in all subsequent passes point to the BackBuffer. However, because the BackBuffer cannot be read, any subsequent processing that reads activeColorTexture / active DepthTexture breaks. Also, do not switch when cameraData.resolveFinalTarget == false (there are subsequent cameras). Overlay cameras are designed to render on top of CameraColor, so if an intermediate camera writes directly to the BackBuffer, its output is mixed with or overwritten by the rendering results of subsequent cameras and disappears. Therefore, limit the call to the point immediately before the custom pass that performs the final output in the last camera.

10.3.2 Implement the FinalBlit Function in the Custom Pass That Runs Last

Omitting FinalBlitPass means that the final processing it handled also disappears. The custom pass must take over that work.

Y Flip

If the UV origins of Source and Target do not match, the screen is upside down unless flipped. Compare the two with GetTextureUVOrigin(), added in Unity 6.3, to make this determination. This API is available only on execution-time GraphContexts such as RasterGraphContext, so it must be called inside SetRenderFunc, not at RecordRenderGraph recording time.

▼List 10.8 Determining the Y flip

```
var yFlip = context.GetTextureUVOrigin(data.Source) != context.GetTextureUVOrigin(data.
    Target);
var scaleBias = yFlip ? new Vector4(1, -1, 0, 1) : new Vector4(1, 1, 0, 0);
```

Note that SystemInfo.graphicsUVStartsAtTop only indicates the UV convention of the graphics API and cannot be used to determine whether a flip is needed for the current blit.

sRGB Conversion

You need to set the LinearToSRGBConversion keyword by checking cameraData.requireSrgbConversion, and implement the conversion in the shader according to that keyword.

▼List 10.9 Setting the sRGB conversion keyword

```
var requireSrgbConversion = cameraData.targetTexture == null && Display.main.
    requiresSrgbBlitToBackbuffer;
cmd.SetKeyword(ShaderGlobalKeywords.LinearToSRGBConversion, data.requireSrgbConversion)
    ;
```

▼List 10.10 sRGB conversion

```
#ifdef _LINEAR_TO_SRGB_CONVERSION
    color = LinearToSRGB(color);
#endif
```

Viewport Setup

The Viewport is normally set to the full screen, but if you want to set it to something other than the full screen, you need to set the Camera's Viewport.

▼List 10.11 Setting the Viewport

```
var targetViewPort = new Rect(0, 0, targetWidth, targetHeight);
cmd.SetViewport(targetViewPort);
```

HDR Output and Overlay UI Compositing

Encoding for HDR display output and compositing of the overlay UI when camera Data.rendersOverlayUI is true were also the job of FinalBlitPass. A title that needs HDR output support must implement equivalent processing by using HDROutput Utils.

There may seem to be many requirements, but it is sufficient to implement only the features the project actually uses. For a title with fixed SDR, a full-screen Viewport, and no overlay UI, only the Y flip and sRGB conversion are effectively required.

10.4 Summary

- In URP, there are three kinds of passes responsible for the final output to the BackBuffer: UberPost / FinalPost / FinalBlitPass, and only one of them is selected depending on conditions
- If a custom pass is injected at or after AfterRenderingPostProcessing, has PassesAfterPostProcessing becomes true and FinalBlitPass is almost always added
- With SwitchActiveTexturesToBackbuffer() in Unity 6.3, a custom pass can output directly to the BackBuffer, and this FinalBlitPass can be omitted
- However, this API is merely a declaration that rewrites the IDs of the camera's Active Color/Depth Texture. It must be called only in the final pass of the last camera, and the custom pass must itself handle the processing that FinalBlit Pass carried out, such as the Y flip, sRGB conversion, and Viewport

A single full-screen blit is a cost that cannot be ignored at high resolutions and on mobile. For a project with custom post-processing, it is well worth considering.

10.5 References

- Unity Manual: Get data from the current frame in URP
  `https://docs.unity3d.com/6000.3/Documentation/Manual/urp/accessing-frame-data.html`
- Class UniversalResourceData | Universal Render Pipeline | 17.3
  `https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.3/api/UnityEngine.Rendering.Universal.UniversalResourceData.html`
- Struct RasterGraphContext | Scriptable Render Pipeline Core | 17.3
  `https://docs.unity3d.com/Packages/com.unity.render-pipelines.core@17.3/api/UnityEngine.Rendering.RenderGraphModule.RasterGraphContext.html`

Chapter 11

Article Title

Nagatome Ryouhei

Colorful Palette

rngtm X @rn49rn49

Mobile Shader Optimization: Measuring and Reducing the Cost of Transcendental Functions

11.1 Name and Affiliation

Nagatome Ryouhei / Colorful Palette

11.2 GitHub Account

`https://github.com/rngtm`

11.3 X Account

`https://x.com/rn49rn49`

11.4 Introduction

Using transcendental functions such as sin and exp in a shader tends to increase the GPU processing load. This article uses Profiler measurements to explain how much these functions affect actual performance. It also covers an optimization technique that replaces transcendental functions with approximations, and presents measured values showing how much load reduction it actually achieves.

11.5 Environment

The Unity version, render pipeline, and devices used in this article are as follows.

Unity version	Unity6000.3.20f1
Render pipeline	Universal Render Pipeline 17.3.0
iOS devices	iPhoneX iPhone14 iPhone15 Pro
Android device	Galaxy S9

11.6 Transcendental Functions

A transcendental function is a function that cannot be expressed with a finite number of arithmetic operations or radicals. Representative examples are trigonometric, exponential, and logarithmic functions.

- Trigonometric functions such as sin and cos
- Exponential functions such as pow and pow2
- Logarithmic functions such as log and log10

Transcendental functions are costly, so it is considered desirable to avoid using them heavily.

11.6.1 Why Transcendental Functions Are Heavy

Transcendental functions are expensive for the following three reasons.

- They take many clock cycles
- There are few units (SFUs) that perform transcendental function operations
- Overhead from register pressure

11.6.2 Reason 1: Many Clock Cycles

GPUs in smartphones and similar devices have many arithmetic logic units (ALUs) for performing arithmetic operations. An ALU is designed to execute an addition or multiplication in one clock cycle.

However, a transcendental function is not simple enough to be computed with a single instruction. When a GPU computes a transcendental function internally, it decomposes it into a combination of many additions and multiplications by using polynomial approximation algorithms such as the Taylor expansion and the minimax method.

Whereas a simple arithmetic operation finishes in one instruction, a transcendental function is expanded by the compiler into dozens of arithmetic instructions, so the processing time is longer.

11.6.3 Reason 2: Few SFUs

Many GPUs have dedicated hardware called the SFU (Special Function Unit) for processing transcendental functions and similar operations quickly, and transcendental functions are sometimes executed on the SFU.

The number of SFUs is very small compared with the number of ALUs, so even when a transcendental function is executed on the SFU, the processing takes a long time.

11.6.4 Reason 3: Overhead from Register Pressure

A transcendental function such as sin() is more complex to compute than simple addition or multiplication, and it requires more temporary registers for the computation inside the GPU.

The total amount of registers in each GPU compute unit (SM: Streaming Multiprocessor) is physically fixed, so as the number of registers consumed per thread increases (register pressure rises), the number of threads that can be packed into one SM decreases.

Because fewer threads can run at once, the processing takes a long time.

11.7 Load of Various Functions

For each loop count from 0 to 10, we measure the GPU time when executing various transcendental functions, to find out which transcendental function is the heaviest. FrameTimingManager is used to measure the GPU time.

11.8 Shader Used for Load Measurement

The following shader was used for the measurement. The transcendental function to execute can be selected with a shader keyword, and the loop count can be changed with the variable _LoopCount.

```
Shader "Benchmark/Transcendental"
{
    Properties
    {
        // uGUI (CanvasRenderer) binds the texture to "_MainTex", so the name is fixed
        [PerRendererData][MainTexture] _MainTex("Sprite Texture", 2D) = "white" {}
        [MainColor] _BaseColor("Tint", Color) = (1, 1, 1, 1)

        // Stencil settings for Mask (UI/Mask). The Mask component swaps the material
    and writes to it
        _StencilComp("Stencil Comparison", Float) = 8
        _Stencil("Stencil ID", Float) = 0
        _StencilOp("Stencil Operation", Float) = 0
        _StencilWriteMask("Stencil Write Mask", Float) = 255
        _StencilReadMask("Stencil Read Mask", Float) = 255

        _ColorMask("Color Mask", Float) = 15

        [Toggle(UNITY_UI_ALPHACLIP)] _UseUIAlphaClip("Use Alpha Clip", Float) = 0
    }

    SubShader
    {
        Tags
        {
            "Queue" = "Transparent"
            "RenderType" = "Transparent"
            "RenderPipeline" = "UniversalPipeline"
            "IgnoreProjector" = "True"
            "PreviewType" = "Plane"
```

```
            "CanUseSpriteAtlas" = "True"
        }

        Stencil
        {
            Ref [_Stencil]
            Comp [_StencilComp]
            Pass [_StencilOp]
            ReadMask [_StencilReadMask]
            WriteMask [_StencilWriteMask]
        }

        Cull Off
        ZWrite Off
        ZTest [unity_GUIZTestMode]
        Blend SrcAlpha OneMinusSrcAlpha
        ColorMask [_ColorMask]

        Pass
        {
            HLSLPROGRAM

            #pragma vertex vert
            #pragma fragment frag
            // The loop count is a uniform (dynamic loop), so 3.0 is required instead
    of 2.0
            #pragma target 3.0

            // Alpha Clip support for RectMask2D / Mask
            #pragma multi_compile_local _ UNITY_UI_CLIP_RECT
            #pragma multi_compile_local _ UNITY_UI_ALPHACLIP

            // Switching the transcendental function to measure. Since we want to
    switch it at runtime with Shader.EnableKeyword,
            // use multi_compile instead of shader_feature (which strips unused
    variants).
            // Each keyword produces a separate variant, so only the selected function
    appears in the generated code (no branching overhead).
            // Because it is a global keyword, it can be controlled collectively from
    TranscendentalBenchmark.
            // If no keyword is enabled, the first variant, _FUNC_NONE, is used.
            #pragma multi_compile_fragment _FUNC_NONE _FUNC_SIN _FUNC_SIN_APPROX
    _FUNC_COS _FUNC_TAN _FUNC_TAN_TAYLOR _FUNC_TAN_PADE _FUNC_ASIN _FUNC_ACOS
    _FUNC_ATAN _FUNC_EXP _FUNC_EXP2 _FUNC_POW _FUNC_LOG _FUNC_LOG2 _FUNC_LOG10

            #include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.
    hlsl"
```

```
        struct Attributes
        {
            float4 positionOS : POSITION;
            float4 color : COLOR;
            float2 uv : TEXCOORD0;
            UNITY_VERTEX_INPUT_INSTANCE_ID
        };

        struct Varyings
        {
            float4 positionHCS : SV_POSITION;
            float2 uv : TEXCOORD0;
            float4 positionOS : TEXCOORD1; // Kept in the same space as _ClipRect (
Canvas local)
        };

        TEXTURE2D(_MainTex);
        SAMPLER(sampler_MainTex);

        CBUFFER_START(UnityPerMaterial)
            half4 _BaseColor;
        CBUFFER_END

        // A global value set from TranscendentalBenchmark.
        // It is not a per-material value, so it is not put in UnityPerMaterial (
doing so would break the SRP Batcher)
        int _TranscendentalLoopCount;
        float _PowExponent;

        // Equivalent of UnityGet2DClipping in UnityUI.cginc (defined separately in
 HLSL)
        float Get2DClipping(float2 position, float4 clipRect)
        {
            float2 inside = step(clipRect.xy, position.xy) * step(position.xy,
clipRect.zw);
            return inside.x * inside.y;
        }

        // Keep the value passed to the transcendental function within (0.05, 0.95)
.
        // This collectively avoids the asin/acos domain |x| <= 1, the log-family
domain x > 0, and exp-family overflow.
        // It is preprocessing common to all variants, so its cost cancels out in
the difference.
        float PrepareInput(float seed)
        {
            return seed;
            // return frac(seed) * 0.9 + 0.05;
```

```
        }

        float sinApprox(float x)
        {
            // Parabolic approximation of sin. It needs only multiplications and
abs, so we look at the cost difference from the real sin instruction.
            // The domain is |x| <= PI (the output of PrepareInput falls within
this range).
            // The coefficients are constant-folded, so there is no division cost
at runtime.
            return (4.0 / PI) * x - (4.0 / (PI * PI)) * x * abs(x);
        }

        float tanTaylor(float x)
        {
            // Taylor expansion of tan up to the third order (multiplications only)
. We look at the cost difference from the real tan instruction.
            // The domain is |x| < PI/2, and accuracy drops rapidly beyond π/4.
            float x2 = x * x;
            return x * (1.0 + x2 * 0.3333333); // 1.0/3.0
        }

        float tanPade(float x)
        {
            // Pade approximation of tan (multiplications plus one division). The
domain is |x| < PI/2.
            // The zero of the denominator, x ≈ ±
1.581, is close to the poles of tan at ±
PI/2, so the approximation follows the function up to near the poles.
            float x2 = x * x;
            return x * (15.0 - x2) / (15.0 - 6.0 * x2);
        }

        // The transcendental function to measure. Only one remains for each
keyword,
        // so only the expanded result of the selected function appears in the
generated code.
        float Transcendental(float x)
        {
            #if defined(_FUNC_SIN)
                return sin(x);
            #elif defined(_FUNC_SIN_APPROX)
                return sinApprox(x);
            #elif defined(_FUNC_COS)
                return cos(x);
            #elif defined(_FUNC_TAN)
                return tan(x);
            #elif defined(_FUNC_TAN_TAYLOR)
                return tanTaylor(x);
```

```
            #elif defined(_FUNC_TAN_PADE)
                return tanPade(x);
            #elif defined(_FUNC_ASIN)
                return asin(x);
            #elif defined(_FUNC_ACOS)
                return acos(x);
            #elif defined(_FUNC_ATAN)
                return atan(x);
            #elif defined(_FUNC_EXP)
                return exp(x);
            #elif defined(_FUNC_EXP2)
                return exp2(x);
            #elif defined(_FUNC_POW)
                // If the exponent is a constant, it is folded as in pow(x, 2) -> x
 * x,
                // so a uniform is passed to go through the general-purpose pow
path
                return pow(x, _PowExponent);
            #elif defined(_FUNC_LOG)
                return log(x);
            #elif defined(_FUNC_LOG2)
                return log2(x);
            #elif defined(_FUNC_LOG10)
                return log10(x);
            #else // _FUNC_NONE : baseline with no transcendental function
                return x;
            #endif
        }

        Varyings vert(Attributes IN)
        {
            Varyings OUT;
            OUT.positionOS = IN.positionOS;
            OUT.positionHCS = TransformObjectToHClip(IN.positionOS.xyz);
            OUT.uv = IN.uv * 300.0;
            return OUT;
        }

        half4 frag(Varyings IN) : SV_Target
        {
            // This is the measurement target (it has no visual meaning).
            // UNITY_LOOP suppresses loop unrolling so that the cost for
_TranscendentalLoopCount iterations is applied as is.
            // Except for the transcendental function itself, every variant has
exactly the same form,
            // so the difference from _FUNC_NONE is the cost of that function.
            float accum = 0.0;
```

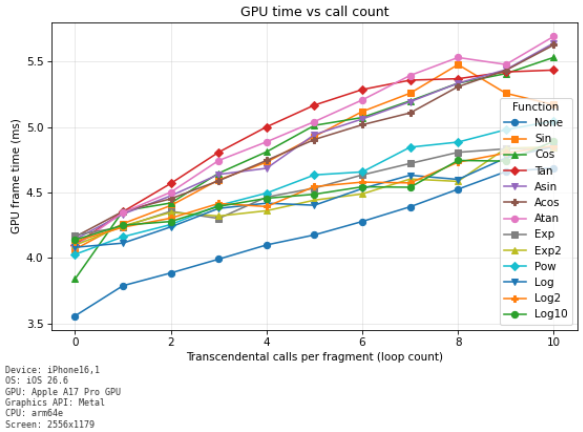
```
                UNITY_LOOP
                for (int i = 0; i < _TranscendentalLoopCount; i++)
                {
                    // If it were a loop invariant, it would be hoisted out of the loop
    , so i is mixed into the argument
                    float x = IN.uv.x + i * 0.017;

                    accum += Transcendental(x);
                }
                return half4(saturate(accum.xxx) * 0.02, 1);
            }
            ENDHLSL
        }
    }
}
```

11.9 Measurement Results

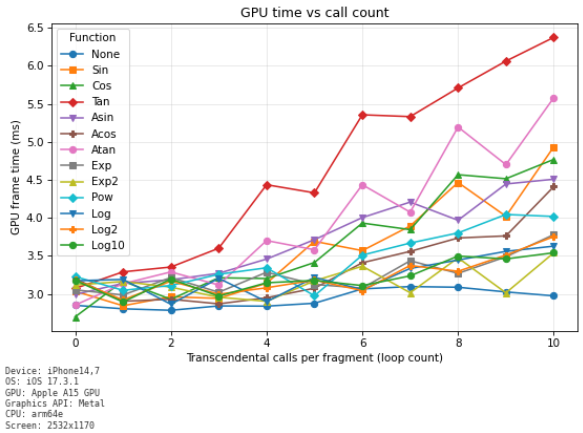
11.9.1 iPhone15 Pro

When the execution count is less than 7, tan is the heaviest, When the number of executions is 7 or more, Atan is the most expensive.

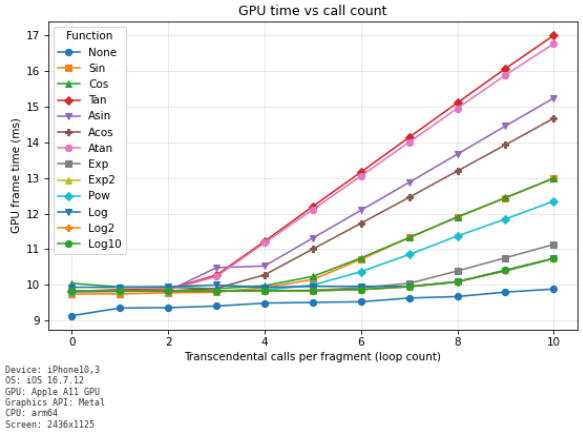


11.9.2 iPhone14

tan has the highest cost, and atan has the second highest.

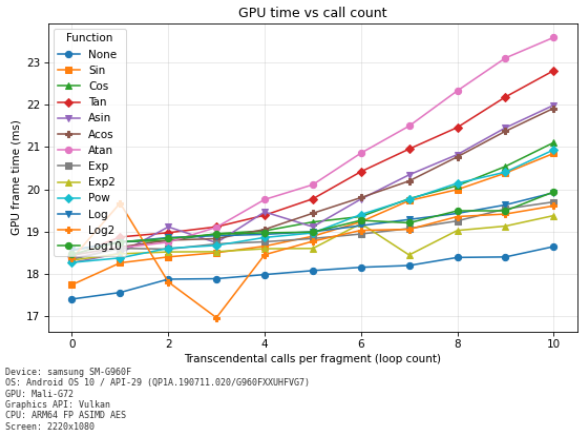


11.9.3 iPhoneX



11.9.4 Galaxy S9

Atan appears to have the highest cost.



11.9.5 Summary

Device	Notes
iPhone 15 Pro	tan is the heaviest when the execution count is less than 7; atan is the heaviest when it is 7 or more
iPhone 14	tan is the heaviest
iPhone X	tan is the heaviest
Galaxy S9	atan is the heaviest

11.10 Load of the sin Function

We measure the cost of the sin function, a representative transcendental function.

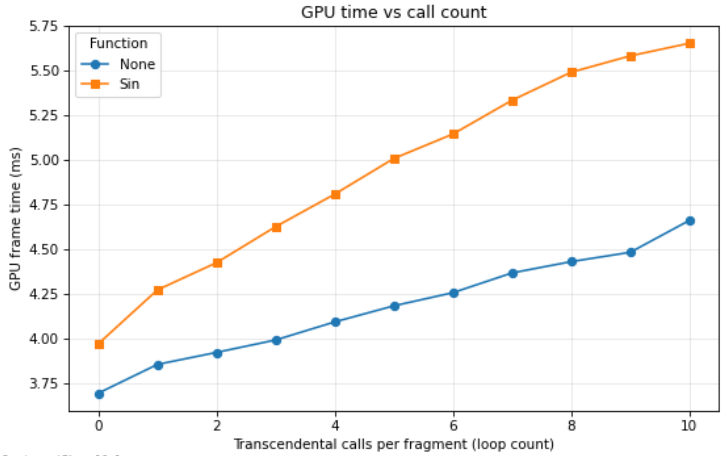
For each loop count from 0 to 10, we measure the GPU time required for the processing within a frame. At each measurement point, we measured for 10 seconds and summarized the GPU time in graphs.

First graph: GPU time for each loop count Second graph: the difference between the GPU time for each loop count and None.

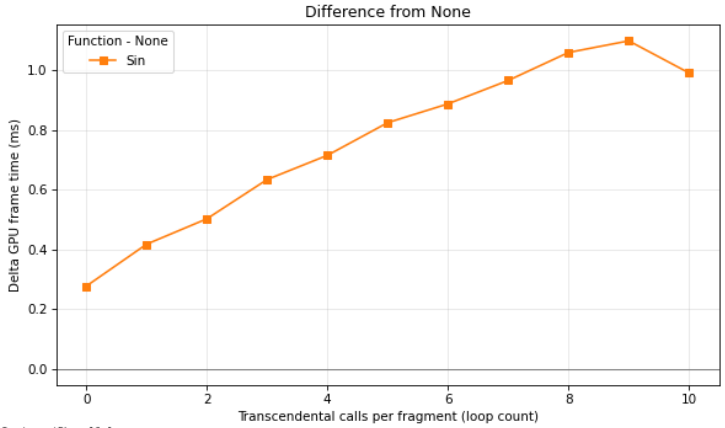
FrameTimingManager was used for the measurement.

11.10.1 iPhone15 Pro

The presence of a sin instruction in the shader increased the time by about 0.25ms. Each additional sin call increases the processing load by about 0.1ms.



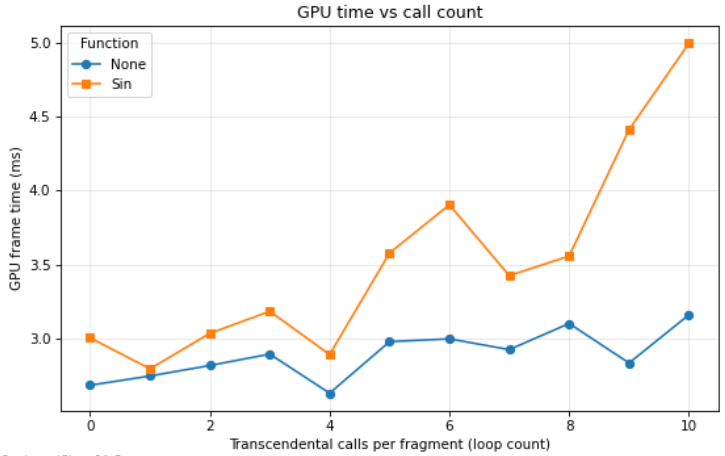

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```



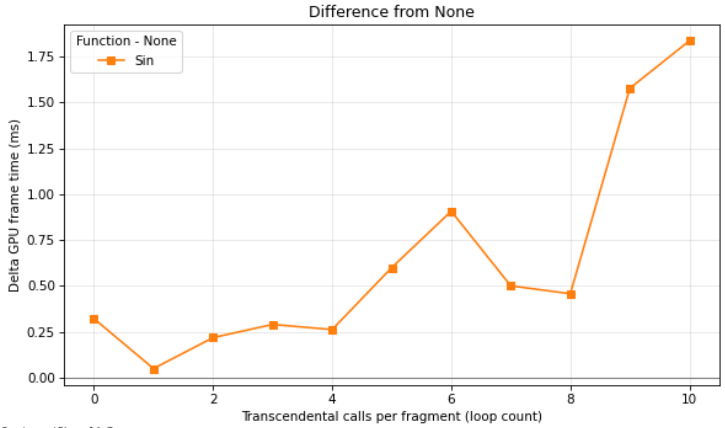

```
Device: iPhone16,1
OS: iOS 26.6
GPU: Apple A17 Pro GPU
Graphics API: Metal
CPU: arm64e
Screen: 2556x1179
```

11.10.2 iPhone14

The presence of a sin instruction in the shader increased the GPU time by about 0.25ms. Each additional sin call increases the processing load by about 0.1 to 0.2ms.



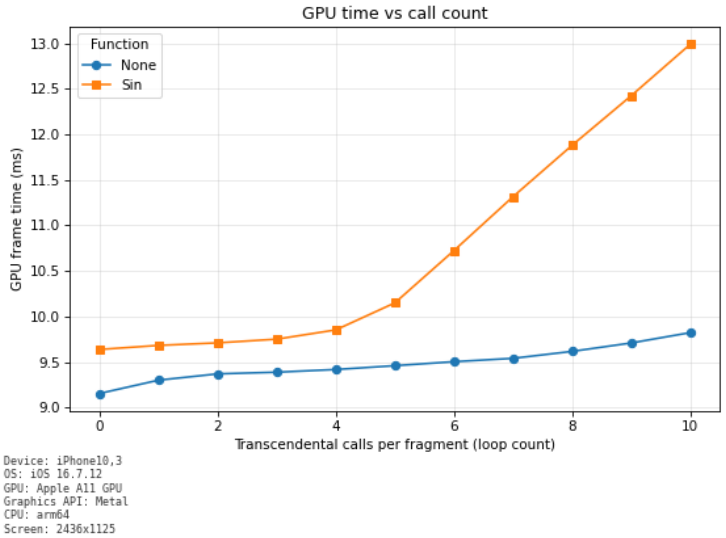

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

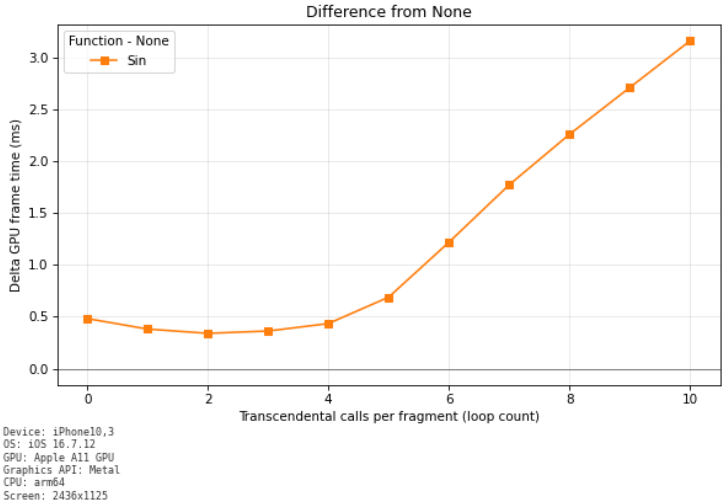



```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

11.10.3 iPhoneX

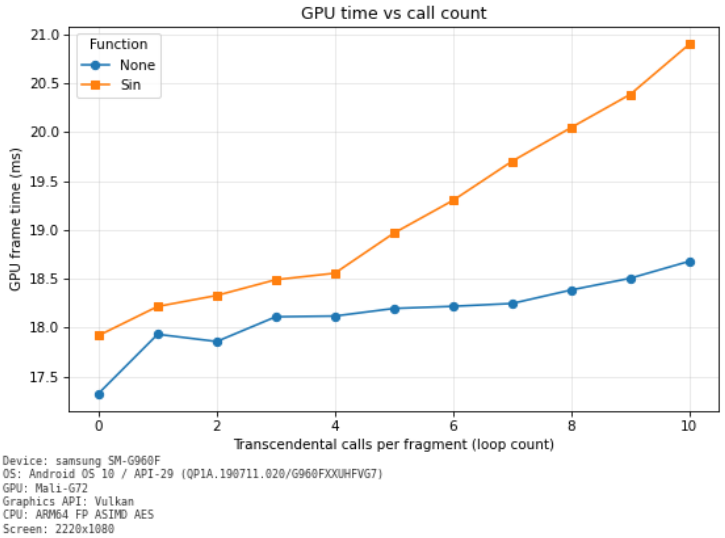
The presence of a sin instruction in the shader increased the time by about 0.5ms. With up to 4 sin calls, almost no increase in GPU time is observed, but beyond 5 calls, each additional sin call increases the processing load by about 0.6ms.

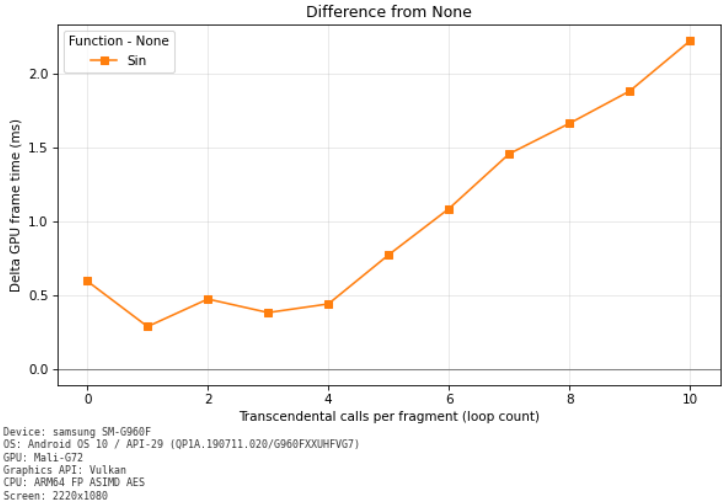




11.10.4 Galaxy S9

The presence of a sin instruction in the shader increased the time by about 0.5ms. With up to 4 sin calls, almost no increase in GPU time is observed, Beyond 5 calls, each additional sin call increases the processing load by about 0.2 to 0.3ms.





11.10.5 Summary of the sin Function Load

Device	GPU load increase from the sin function	Notes
iPhone 15 Pro	+0.25ms (baseline)	About 0.1ms increase per call
iPhone 14	+0.25ms (baseline)	About 0.1 to 0.2ms increase per call
iPhone X	+0.5ms (baseline)	About 0.6ms increase per call beyond 5 calls
Galaxy S9	+0.5ms (baseline)	About 0.2 to 0.3ms increase per call beyond 5 calls

Note: The "baseline" in the table refers to the initial load increase when a sin function exists in the shader.

When the number of sin functions in the shader increases, an increase in GPU time was observed even without executing the sin functions. This is considered to be an increase in load caused by register pressure.

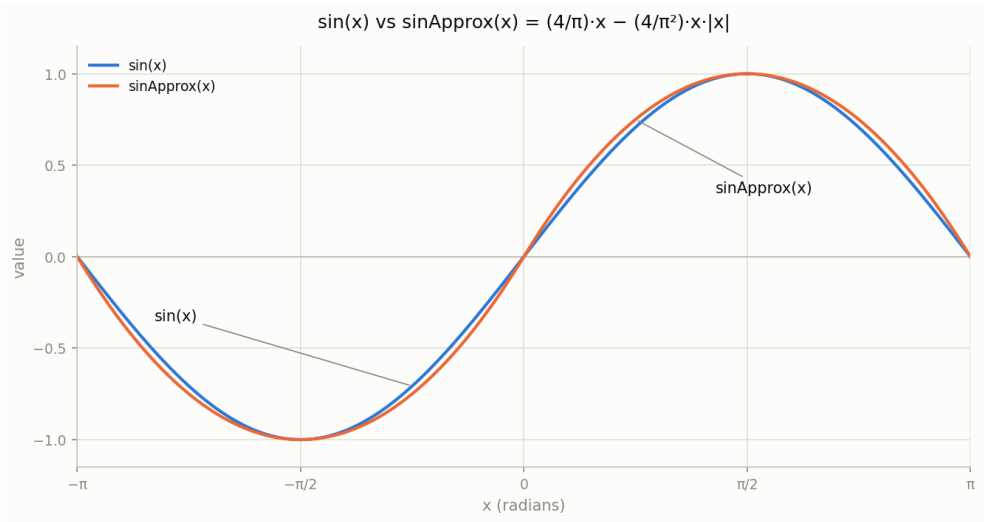
11.11 Parabolic Approximation of the sin Function

A known technique approximates sin by using the following polynomial.

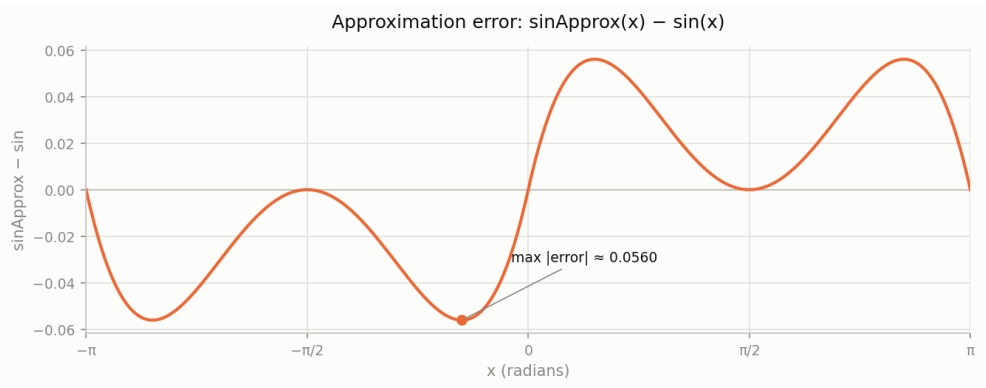
```
float sinApprox = (4.0 / PI) * x - (4.0 / (PI * PI)) * x * abs(x);
```

It can be computed with only multiplications and the absolute value abs(), so it is very lightweight.

The parabolic approximation of sin outputs values very close to sin (-PI<=x<=PI)



The maximum error is about 0.056.

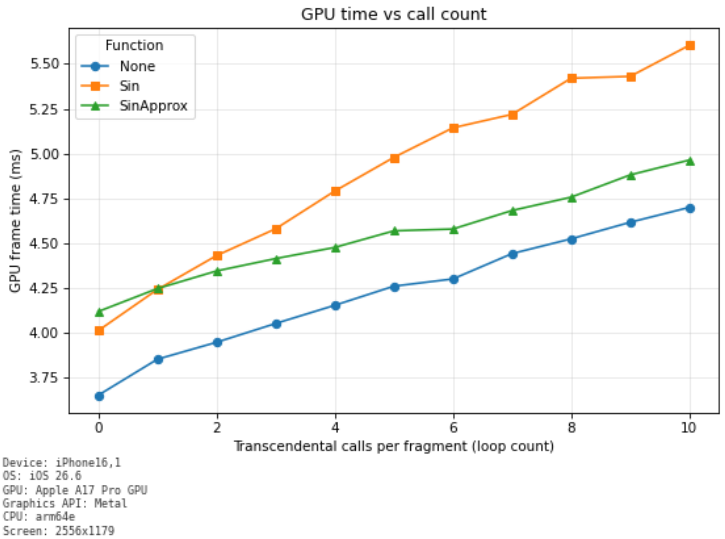


11.12 Measuring the Load of the sin Approximation

We measured the load of sinApprox, the parabolic approximation of sin, and summarized it in graphs.

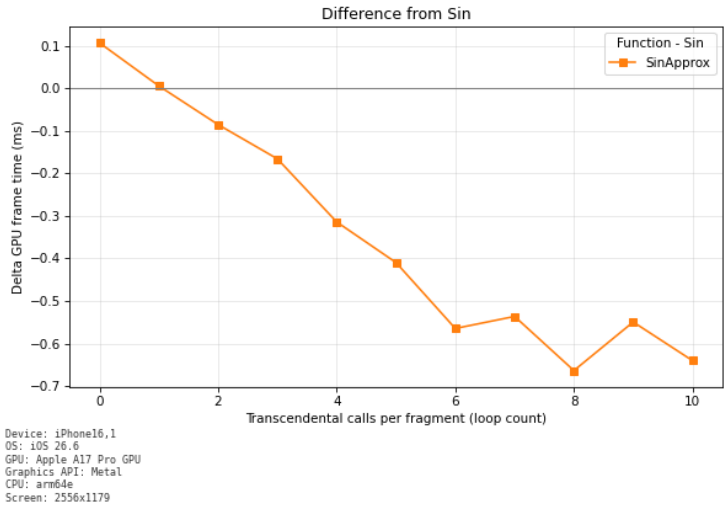
11.12.1 iPhone15 Pro

It is confirmed that the cost is lower than that of the sin function.

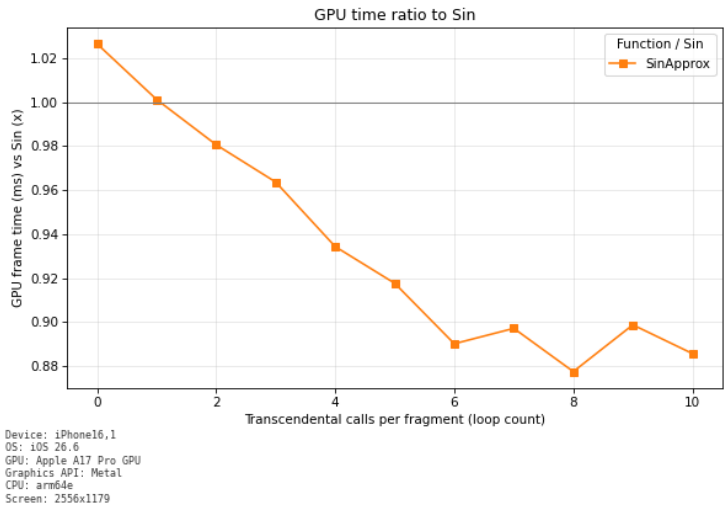


Comparison of the sin Function and the Approximation

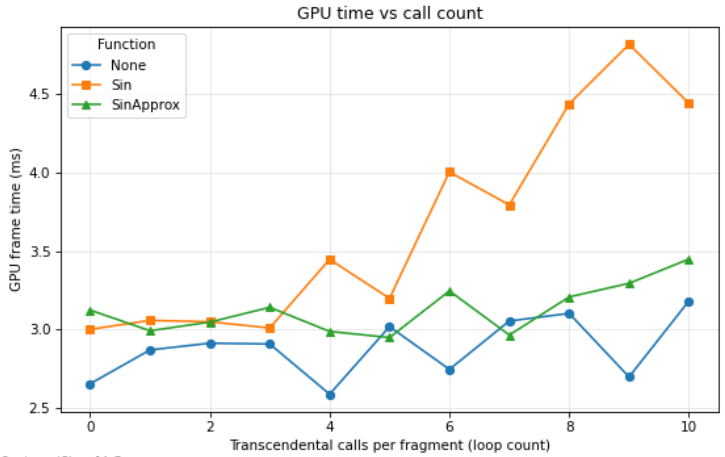
When the function is executed 10 times, the cost is about 0.6 ms lower.



When the function was executed 10 times, the load of sinApprox was about 90% of that of sin.



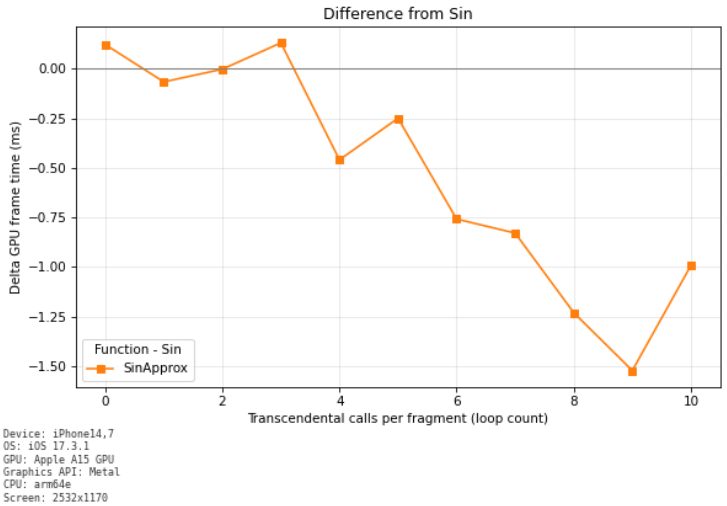
11.12.2 iPhone14



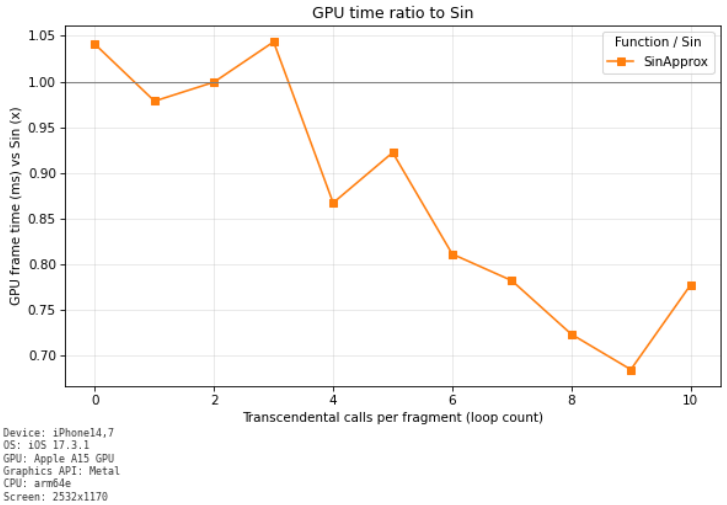

```
Device: iPhone14,7
OS: iOS 17.3.1
GPU: Apple A15 GPU
Graphics API: Metal
CPU: arm64e
Screen: 2532x1170
```

Comparison of the sin Function and the Approximation

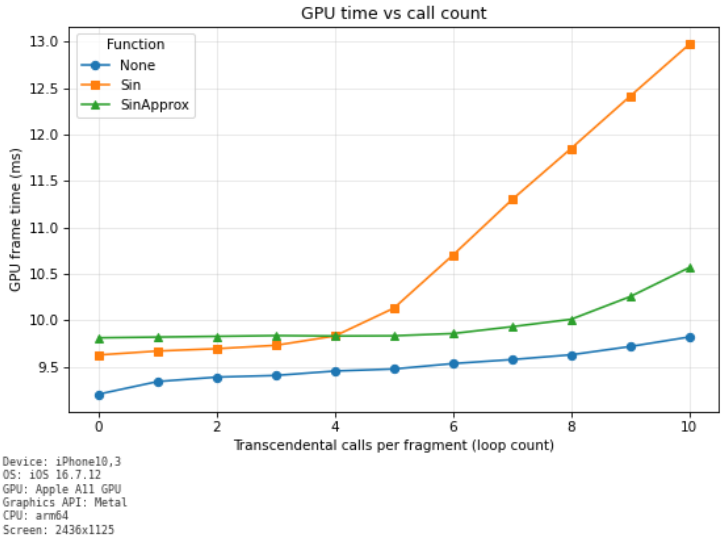
When the function is executed 10 times, the GPU time is about 1 ms shorter.



The cost of executing sinApprox 10 times was about 80% of that of sin.

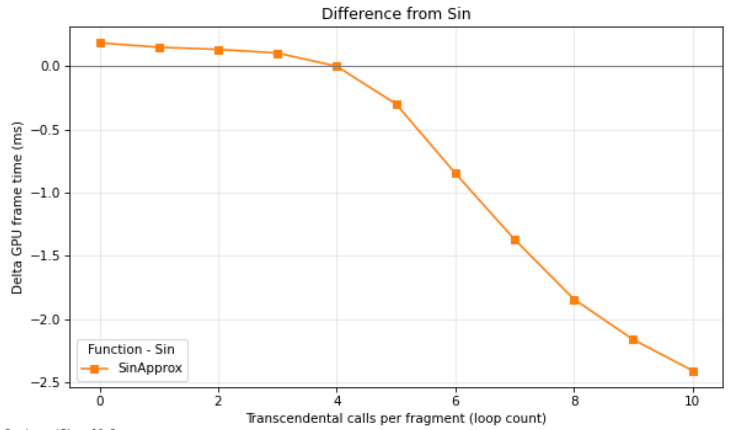


11.12.3 iPhoneX

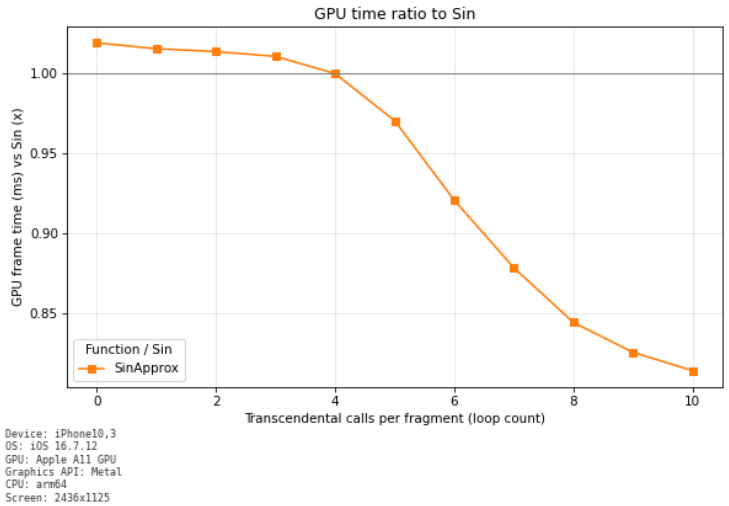


Comparison of the sin Function and the Approximation

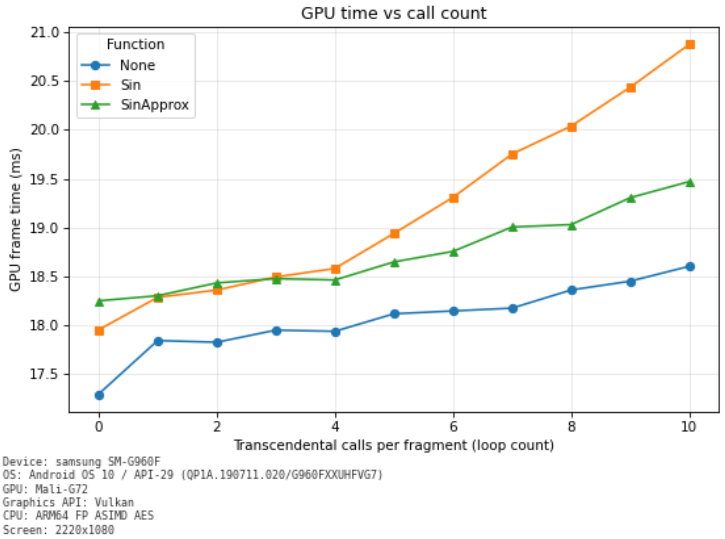
The cost of executing sinApprox 10 times was about 2.5 ms shorter than that of executing sin.



The GPU time of executing sinApprox 10 times was about 80% of that of sin.

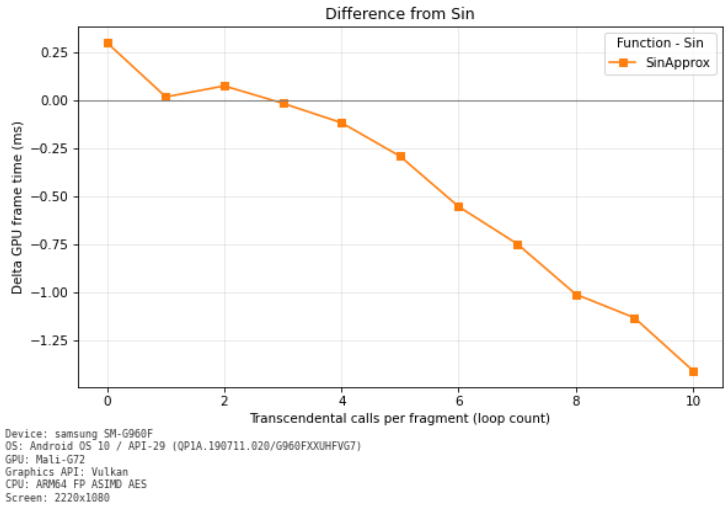


11.12.4 Galaxy S9

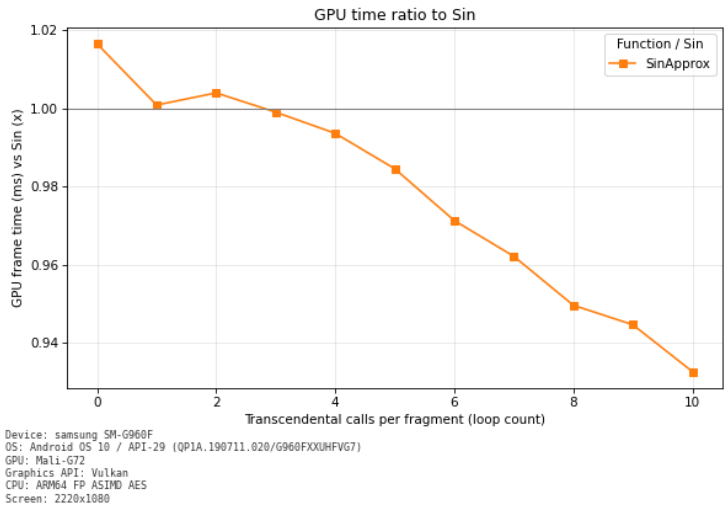


Comparison of the sin Function and the Approximation

When the function is executed 10 times, the GPU time is about 1.3 ms shorter.



The cost of executing sinApprox 10 times was about 94% of that of sin.



11.12.5 Summary of the sin Approximation Load

The GPU reduction achieved by sinApprox is summarized in the following table.

Device	Load of the sin function	Reduction from sinApprox	Notes
iPhone 15 Pro	Baseline	About 0.6ms reduction	The load of sinApprox is about 90% of sin
iPhone 14	Baseline	About 1.0ms reduction	The load of sinApprox is about 80% of sin
iPhone X	Baseline	About 2.5ms reduction	The load of sinApprox is about 80% of sin
Galaxy S9	Baseline	About 1.3ms reduction	The load of sinApprox is about 94% of sin

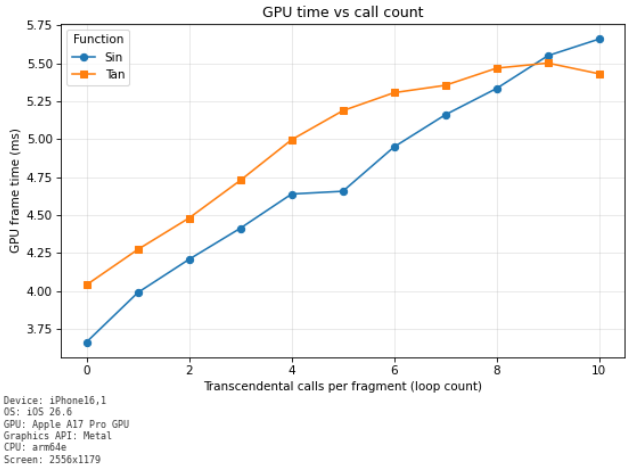
Note: The reduction is the difference in GPU time at a loop count of 10.

11.13 Load of the tan Function

We also measured the load of the tan function. The tan function calls division internally, so its cost is higher than that of the sin and cos functions.

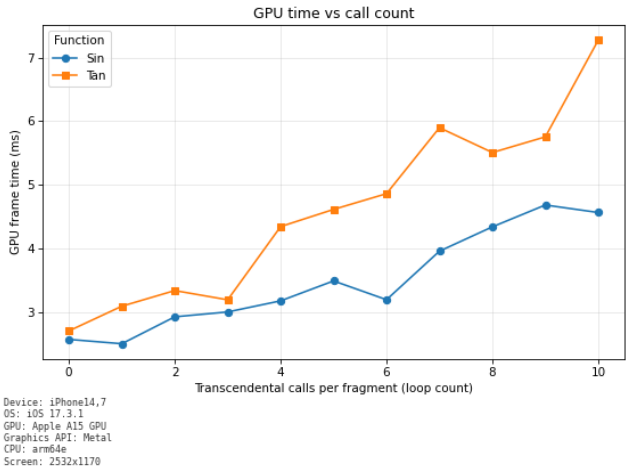
11.13.1 iPhone15Pro

It is more expensive than the sin function, but when the number of executions exceeds 9, the sin function became more expensive.



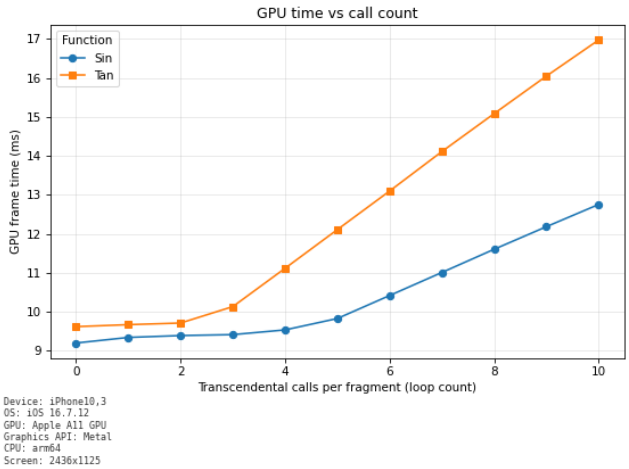
11.13.2 iPhone14

The tan function had a higher load than the sin function.



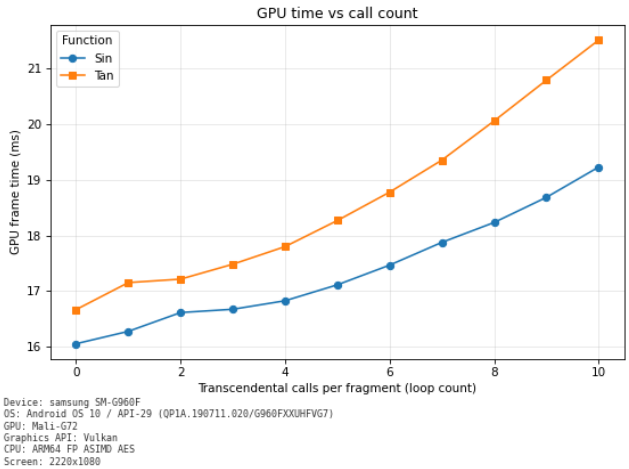
11.13.3 iPhoneX

The tan function had a higher load than the sin function.



11.13.4 Galaxy S9

The tan function had a higher load than the sin function.



11.14 Approximating the tan Function

Techniques for approximating the tan function include using the Taylor expansion as is and using the Padé approximation. The Taylor expansion has a lower load but also lower accuracy. The Padé approximation is more accurate but also has a higher load.

11.14.1 Technique 1. Approximation with the Taylor Expansion

This is an approximation that uses the Taylor expansion of tan. The closer |x| gets to PI/2, the larger the error becomes.

```
float tanTaylor(float x) {
    float x2 = x * x;
    return x * (1.0 + x2 * 0.3333333);
}
```

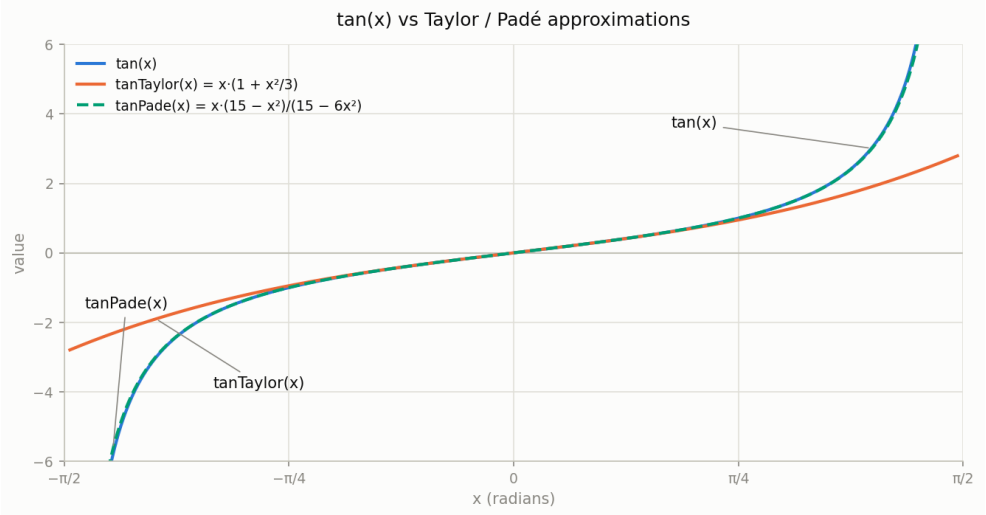
11.14.2 Technique 2. Padé Approximation

This is a method that uses the Padé approximation of tan. In the range |x| < PI/4, it outputs values very close to tan(x).

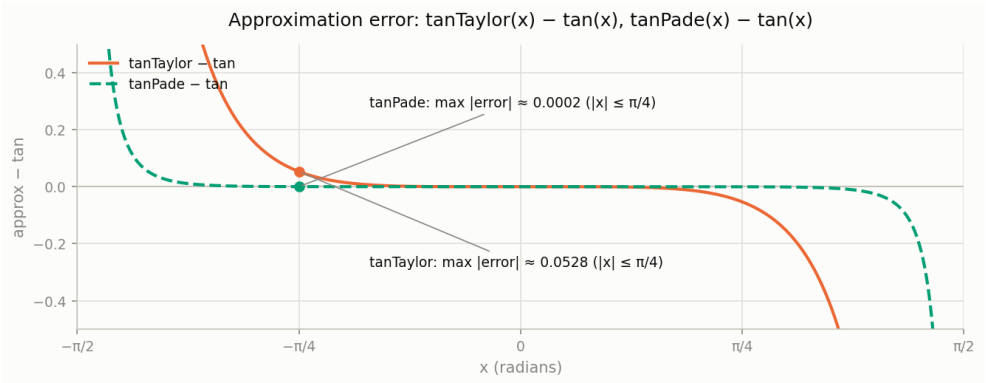
```
float tanPade(float x) {
    float x2 = x * x;
    return x * (15.0 - x2) / (15.0 - 6.0 * x2);
}
```

11.15 Graph of the tan Function Approximations

We plotted the introduced tanTaylor and tanPade on a graph.



Error Relative to the tan Function

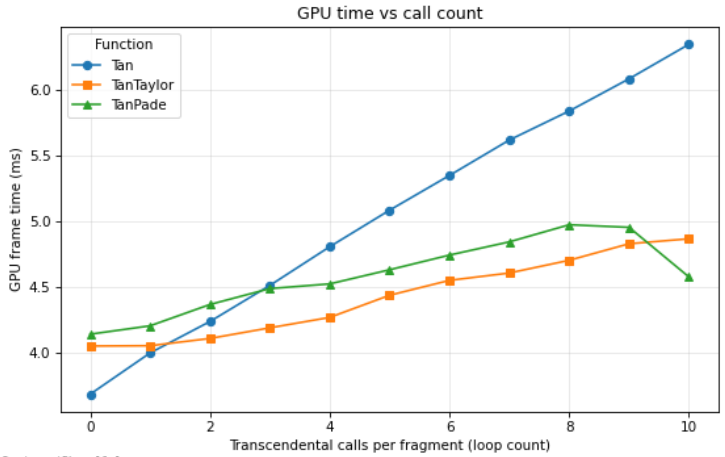


11.16 Measuring the Load of the tan Function Approximations

We measured the load of the approximations of the tan function.

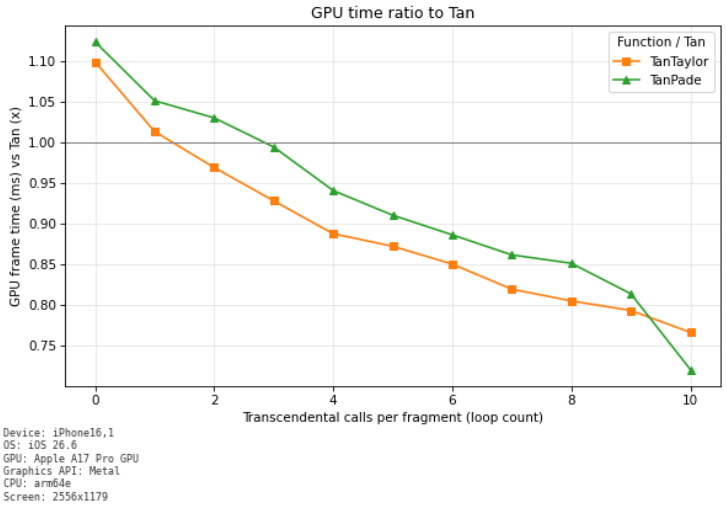
11.16.1 iPhone15 Pro

When the execution count is small, the overhead from register pressure stands out, but as the execution count grows, the approximations keep the load lower.

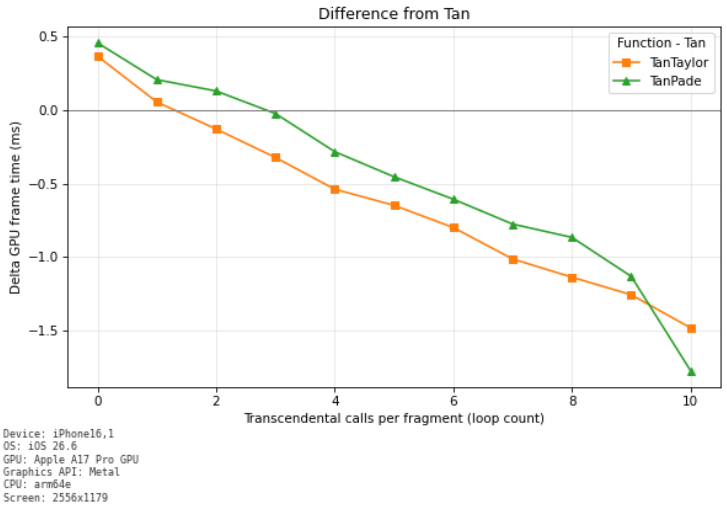


Comparison of the tan Function and the Approximations

When the function is executed 10 times, the GPU time of the approximation was about 80% of that of the tan function.

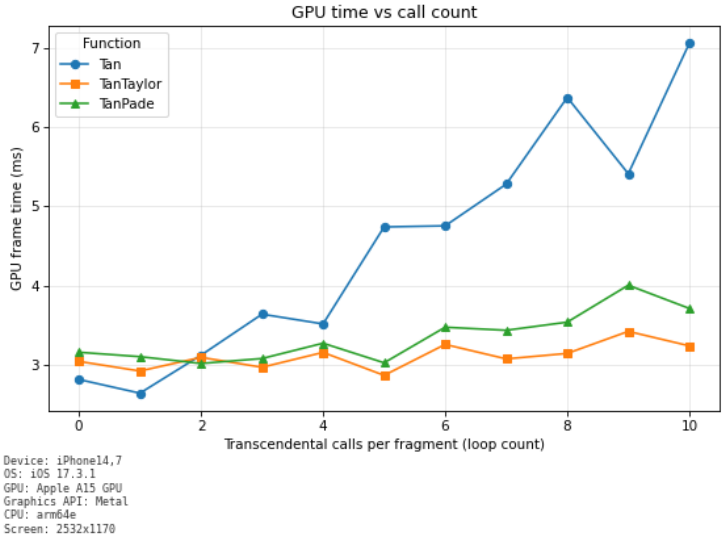


When executed 10 times, the approximation's processing time was about 1 ms shorter.



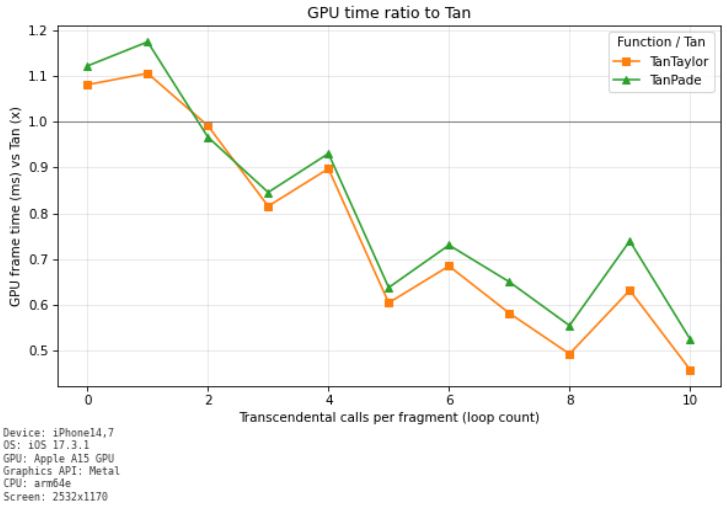
11.16.2 iPhone14

When the execution count is small, a slight overhead from register pressure is observed. but as the execution count grows, the approximations keep the load lower.

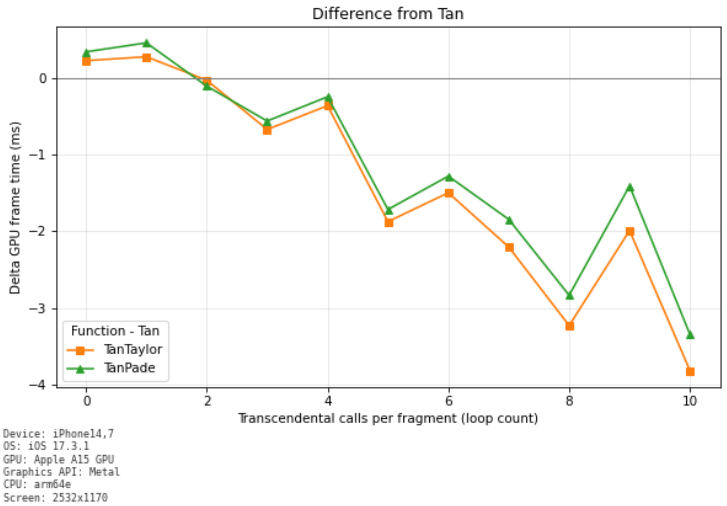


Comparison of the tan Function and the Approximations (iPhone14)

When the function is executed 10 times, the GPU time of the approximation was about 50% of that of the tan function.

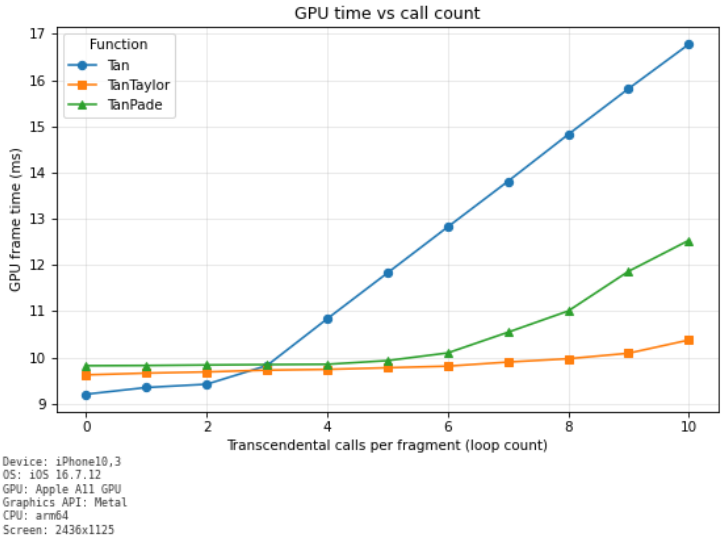


When executed 10 times, the approximation's processing time was about 3 ms shorter.



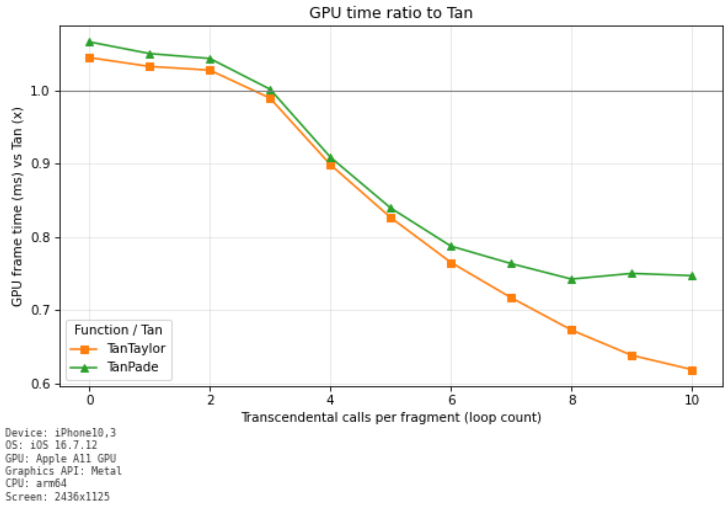
11.16.3 iPhoneX

When the execution count is small, a slight overhead from register pressure is observed. but as the execution count grows, the approximations keep the load lower. With the Taylor expansion, there is almost no increase in processing load, The Padé approximation slightly increases the processing cost.

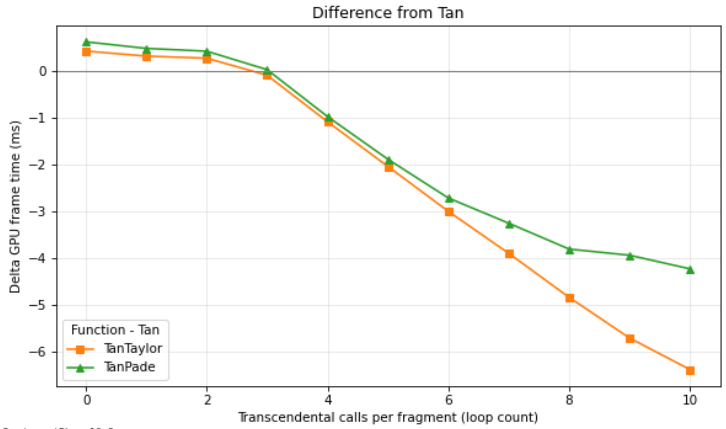


Comparison of the tan Function and the Approximations

When the function is executed 10 times, the GPU time of the Taylor expansion was about 60% of that of the tan function. The Padé approximation was about 75% of the GPU time.



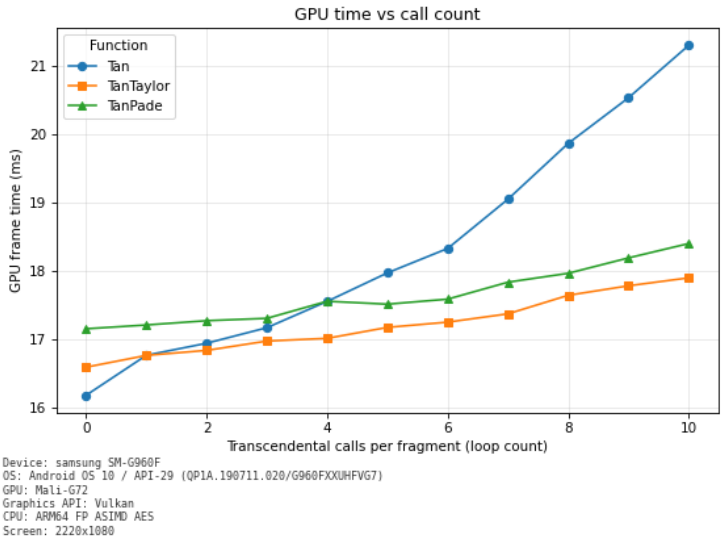
When the function was executed 10 times, the Taylor expansion was about 6ms shorter and the Padé approximation was about 4ms shorter.




```
Device: iPhone10,3
OS: iOS 16.7.12
GPU: Apple A11 GPU
Graphics API: Metal
CPU: arm64
Screen: 2436x1125
```

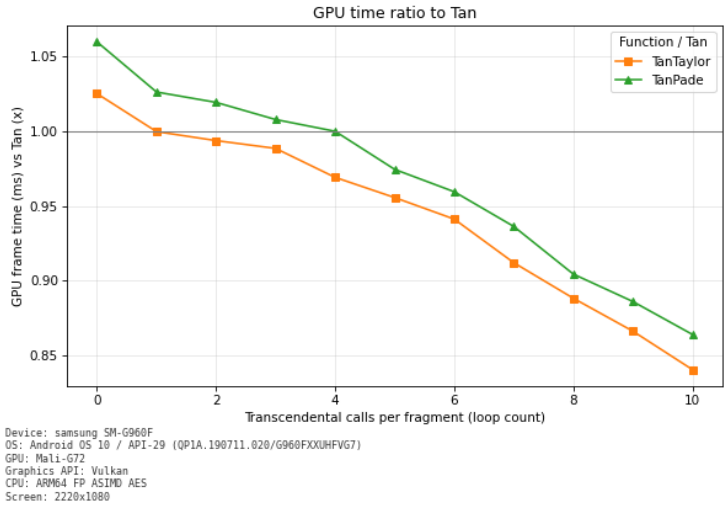
11.16.4 Galaxy S9

When the execution count is small, a slight overhead from register pressure is observed. but as the execution count grows, the approximations keep the load lower.

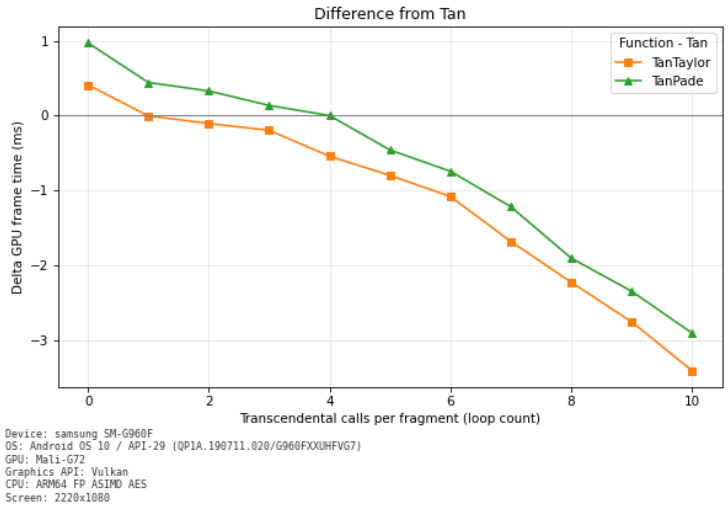


Comparison of the tan Function and the Approximations

When the function is executed 10 times, the GPU time of the approximation was about 85% of that of the tan function.



When the function was executed 10 times, the Taylor expansion was about 3ms shorter.



11.16.5 Summary of the tan Function Approximation Load

Device	GPU load trend	Notes
iPhone 15 Pro	The approximation is **about 20% lower**	The approximation reduces p rocessing time by about 1ms
iPhone 14	The approximation is **about 50% lower**	The approximation reduces p rocessing time by about 3ms
iPhone X	The approximation is **about 25 to 40% lower**	The Taylor expansion reduces it by about 6ms and the Padé approximation by about 4ms
Galaxy S9	The approximation is **about 15% lower**	The approximation reduces p rocessing time by about 3ms

Note: "GPU load trend" is the approximate reduction rate of the GPU time when running an approximation (the Taylor expansion or the Padé approximation), compared with the standard tan function.

Chapter 12

Unity LTO Settings and Performance Comparison on Android and iOS

Minhyuk Kim

CyberAgent, Inc. SGE Core Technology Division

Name：Minhyuk Kim Affiliation：CyberAgent, Inc. SGE Core Technology Division

The link-time optimization (LTO) setting for Android added in Unity 6000.5 applies only to engine code. A separate setting is required to apply LTO to the code generated from your own C#. We tried both settings and measured performance, binary size, and build time with benchmarks and real titles.

12.1 IL2CPP Compilation and Translation Units

In a Unity Android build (IL2CPP), C# is converted to machine code through the following flow: List 12.1.

▼List 12.1 IL2CPP build processing flow

```
C# Source
  ↓ C# Compile
IL Assembly (.dll)
  ↓ IL2CPP Convert
A large amount of C++ (.cpp)          <- "IL2CPP generated code"
  ↓ clang Compile with (each .cpp is optimized individually)
  ↓ Link
libil2cpp.so               ← C# Final native binary generated from the code
```

clang compiles C++ per translation unit (commonly called a TU) and produces object files. A TU is the unit that the compiler reads and optimizes individually.

In a normal C++ build, a function body in another TU cannot be referenced, so even short functions may not be inlined and a call instruction may remain. IL2CPP generated code contains many small functions and safety checks, so it is particularly affected by this constraint. For example, the single C# line `buf[idx]` becomes roughly the following C++: List 12.2.

▼List 12.2 Example of array access code generated by IL2CPP

```
if (buf == NULL) ThrowNullReferenceException();               // null check
if ((uint32_t)idx >= buf->max_length) ThrowIndexOutOfRange(); // bounds check
val = buf->m_Items[idx];
```

As the generated code grows, calls across TUs also increase, and so does the total time spent on calls.

12.2 What Are LTO and ThinLTO

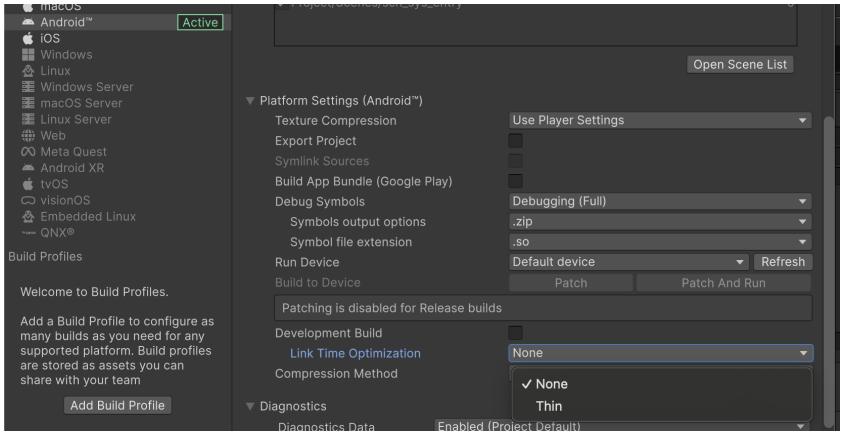
LTO (Link Time Optimization) is a clang/LLVM mechanism that performs inlining, constant propagation, and dead code elimination across TUs at link time. At compile time, object files are emitted as LLVM bitcode (an intermediate representation before optimization). At link time, they are optimized together and machine code is generated.

Full LTO merges all intermediate representations into one and optimizes them. It has the widest optimization scope, but because merging and optimizing the intermediate representations are concentrated in a single process, it uses a large amount of memory in large builds. In this case as well, memory usage reached 20 GB during the build of a real title, and the OS forcibly terminated the process.

ThinLTO creates a summary of each TU at compile time. At link time, it imports the required functions from other TUs and optimizes in parallel. It keeps memory usage and processing time lower than full LTO and is widely used in large builds.

12.3 ThinLTO Setting for Engine Code

Unity 6000.5 added a Link Time Optimization toggle.

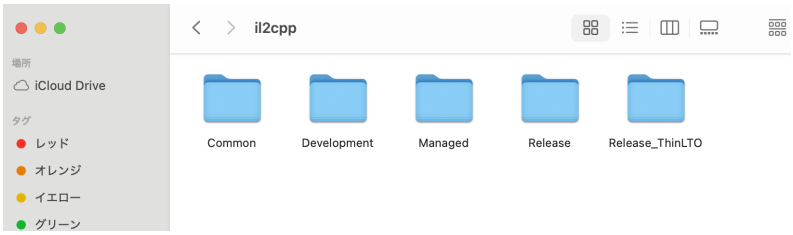


▲Figure 12.1 Build Profiles > Player Settings > Android

Enabling this toggle selects the engine binary provided by Unity with ThinLTO already applied. It targets Unity engine code and does not include IL2CPP generated code (code derived from C#). The Editor installation folder ships prebuilt .so files for both the regular and ThinLTO variants.

▼List 12.3 Variants of libunity.so bundled with the Android Player

```
PlaybackEngines/AndroidPlayer/Variations/il2cpp/ ├──
 Release/Libs/arm64-v8a/libunity.so          29,032,960 bytes └──
 Release_ThinLTO/Libs/arm64-v8a/libunity.so  30,042,536 bytes (+3.5%)
```



▲Figure 12.2 The Editor installation folder. Both `Release` and `Release_ThinLTO` are included, and one of them is selected at build time

12.3.1 Conditions for Enabling and Behavior When They Are Not Met

The toggle takes effect when the following three conditions are met. If any one is not met, no error or warning is shown, and `LinkTimeOptimization` is built as `None`.

1. IL2CPP scripting backend
2. Strip Engine Code is disabled (because the unstripped prebuilt is used as is)
3. Non-Development build

When Development Build is enabled, the toggle itself is not displayed. Whether it is applied can be confirmed with the following two checks: List 12.4.

▼List 12.4 Command to verify that the engine code LTO toggle is applied

```
# (1) Check whether ThinLTO is specified in the build input
grep -o 'LinkTimeOptimization"[: ]*"[A-Za-z]*"' Library/Bee/Player*-inputdata.json
# (2) Check whether libunity.so in the APK matches the Release_ThinLTO variant
unzip -p out.apk lib/arm64-v8a/libunity.so | md5
```

12.4 Effect of the engine code LTO toggle

I compared the bundled regular and ThinLTO binaries and measured performance on a physical device, focusing on the subsystems with the most changes.

12.4.1 Comparing Function Sizes in the Bundled Binaries

Unity bundles `.so` with symbols for both variants in the Editor installation folder.

▼List 12.5 libunity.so with symbols bundled with the Android Player

```
PlaybackEngines/AndroidPlayer/Variations/il2cpp/ ├──
 Release/Symbols/arm64-v8a/libunity.sym.so └──
 Release_ThinLTO/Symbols/arm64-v8a/libunity.sym.so
```

We matched the function names obtained from `llvm-nm --defined-only --print-size --demangle` and aggregated them by subsystem using namespaces and class names (Table 12.1).

Table 12.1: Function differences by subsystem due to the engine code LTO toggle

Subsystem	Number of functions (Release → ThinLTO)	Number of functions with changed size	Size change rate	`.text` difference
Particle	1,419 → 1,290	1,043	80.9%	+49.66%
Serialization/Asset	2,890 → 1,387	1,040	75.0%	-16.61%
Mesh/Geometry	1,055 → 765	483	63.1%	+6.79%
Scripting	8,924 → 7,680	4,302	56.1%	+10.27%
Gfx/Shader/Texture	5,062 → 3,935	2,044	52.1%	+24.21%
Physics	2,680 → 2,439	1,229	51.1%	+15.92%
NavMesh	739 → 582	296	51.1%	+11.87%
XR/VFX/Video/Other	3,828 → 2,483	1,230	49.6%	+13.94%
Transform/Scene/Obj	2,027 → 1,630	781	47.9%	+10.95%
Animation/Mecanim	3,226 → 2,428	1,134	46.8%	+13.18%
Box2D	2,169 → 1,562	712	46.4%	-0.43%
JobSystem/Threading	244 → 181	78	43.6%	+3.16%
UI/UIToolkit	883 → 701	293	42.0%	+1.38%
TextCore/Font/ICU	3,500 → 2,101	861	41.9%	-5.73%
(Unclassified)	29,193 → 21,077	7,511	37.6%	+3.37%
Renderer/Culling	2,789 → 2,199	785	35.7%	+27.58%
C++ standard/Compression/Infrastructure	13,158 → 9,484	3,071	32.6%	-13.31%
curl/TLS/HTTP	1,809 → 1,558	318	21.0%	+1.17%
Audio/FMOD	2,996 → 2,680	496	18.5%	+2.77%
Android/JNI	1,445 → 1,186	193	16.3%	-5.52%

Table 12.1: Function differences by subsystem due to the engine code LTO toggle

Subsystem	Number of functions (Release → ThinLTO)	Number of functions with changed size	Size change rate	`.text` difference
PhysX	6,000 → 5,997	0	0.0%	-0.05%
Total	96,036 → 73,345	27,900	38.8%	+4.41%

Note: "Functions with changed size" is the number of functions. "Size change rate" is the proportion of functions whose size changed among those present in both Release and ThinLTO.

The total function count dropped from 96,036 to 73,345, but the size of `.text` increased by 4.41%.

12.4.2 Benchmark with 13 Workloads

Based on Table 12.1, I built a benchmark scene that packs 13 workloads into a single APK.

Table 12.2: Benchmark workloads for the engine code LTO toggle

Workload	Target subsystem	Description
`particle`	Particle	20,000 particles + noise/size/color modules, `Simulate`
`gfx`	Gfx/Shader	Write 16 properties × float/vector + matrix to `MaterialPropertyBlock` and `Clear`
`cull`	Renderer/Culling	`CalculateFrustumPlanes` + 4,096 AABBs `TestPlanesAABB`
`physbind`	Physics bindings	`Rigidbody` / `Collider` property round trips
`anim`	Animation	Legacy `Animation` with 48 rigs × 8 bones × 10 curves, `Sample()`
`json`	Serialization	Round trip of `JsonUtility`
`instantiate`	Serialization	`Instantiate` → `DestroyImmediate` (native clone/transfer path)
`xform`	Transform/Scene	Rewriting Transforms in a hierarchy
`bind`	Scripting bindings	A large number of tiny binding calls
`mesh`	Mesh/Geometry	Rebuilding a 96×96 procedural mesh
`phys2d`	Box2D	2D rigid body `Simulate`
`text`	TextCore/Font	`TextGenerator.Populate`
`physics`	PhysX	3D rigid body `Simulate`

Note: Among the Table 12.1 subsystems, those for which it is difficult to create a stable synthetic load of about 100ms per sample from C# were excluded from measurement. Their performance impact was not evaluated.

12.4.3 Measurement Results by Workload

We measured the workloads introduced above on a Pixel 8a (Tensor G3) using APKs that differ only in `libunity.so`. The ABBA order of Release / ThinLTO / Thin LTO / Release was repeated three times, for a total of 12 sessions. The app was reinstalled for each session, with a 90-second cooldown between sessions.

Values are the time per operation (ns/op). The unit of an operation differs by workload, so compare absolute values only within the same row.

Table 12.3: Benchmark results by workload for the engine code LTO toggle

Workload	Release (ns/op)	ThinLTO (ns/op)	Difference
`gfx`	990.46	852.49	-13.93%
`cull`	96,520	87,499	-9.35%
`text`	275,043	265,123	-3.61%
`instantiate`	14,620	14,206	-2.83%
`xform`	6,071.8	6,008.1	-1.05%
`phys2d`	222,269	220,327	-0.87%
`mesh`	2,340,210	2,334,143	-0.26%
`json`	15,864.8	15,934.4	+0.44%
`bind`	140.80	141.47	+0.47%
`physbind`	607.28	610.74	+0.57%
`particle`	265,635	267,335	+0.64%
`anim`	28,100	28,563	+1.65%
`physics`	484,641	482,796	-0.38%
[Composite: geometric mean]	21,918	21,380	-2.46%

Note: Because the unit differs per row, the composite value is meaningful only as a ratio.

`gfx` and `cull`, which improved the most, call small functions from C# millions of times per sample. `TextGenerator.Populate` in `text` also runs a per-glyph loop internally. The cross-TU inlining had a large effect on these paths, where each operation is small and the number of calls is high.

12.4.4 Performance, Size, and Memory Comparison at Adoption

In this measurement, CPU time was reduced on some paths with many binding calls. However, the improvement in a real title depends on the proportion of such calls in a frame.

The costs that must be paid on adoption are summarized in Table 12.4.

Table 12.4: Costs of adopting the engine code LTO toggle

Category	Measured difference
Size increase from Strip Engine Code OFF	APK +5.67MB / Installation +20.3MB
Memory increase from Strip Engine Code OFF	RSS +18.0MB / of which hard-to-reclaim area +8.0MB
engine ThinLTO itself	APK size +398KB (30,621,595 → 31,019,967) / memory not measured

Note: The installation size increase is the sum of the APK increase and the increase of `.so` extracted on the device (+14.65MB in total for arm64 `.so`).

Unity has published results of 10 cold-start measurements on each of five projects for the same toggle. They report that CPU time decreased from 13.88ms to 13.36ms (-3.7%), time to first display decreased from 247.96ms to 229.90ms (-7.3%), APK size increased by +3.3%, and the impact on build time was minor [*1].

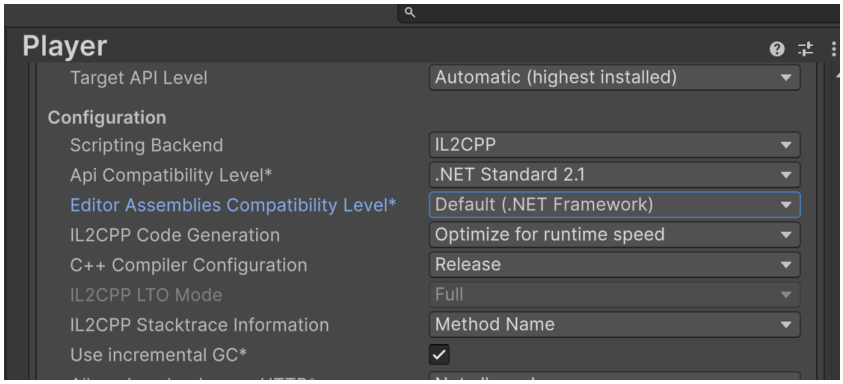
The CPU improvement is at a level close to this measurement. However, the published APK size increase of +3.3% is a comparison between configurations that both disable Strip Engine Code, so it does not include the installation size increase of +20.3MB and the memory increase of +8.0MB that accompany the disabling.

[*1] Unity Android platform team, "Boosting Android performance with LTO optimizations in Unity 6.5" `https://discussions.unity.com/t/boosting-android-performance-with-lto-optimizations-in-unity-6-5/1720008`. Hereafter, "Unity's published values" in this chapter refers to the numbers in this article.

12.5 How to Apply LTO to libil2cpp.so

To apply LTO to libil2cpp.so that includes code converted from C#, you can use Unity's Master configuration or pass clang flags directly.

12.5.1 Master Configuration and the IL2CPP LTO Mode Setting



▲Figure 12.3 Settings screen for IL2CPP LTO Mode and C++ Compiler Configuration

Setting C++ Compiler Configuration in Player Settings > Configuration to Master enables LTO for libil2cpp.so. IL2CPP LTO Mode is the item that selects the Thin / Full mode used in that case.

According to Unity's official article, IL2CPP LTO Mode was added in 6000.5. On Android before 6000.5, there is no difference between Release and Master regarding the presence of LTO. However, in 6000.5 the toggle cannot be selected on the Android platform and is fixed to Thin; from 6000.6, Thin / Full can be selected in the Master configuration. Even as of 6000.7, there is no option that enables only LTO while keeping the Release configuration.

The flags corresponding to C++ Compiler Configuration in Player Settings can be checked in IL2CPP compiler configuration in the Bee build graph (Il2CppCompiler Configuration).

Table 12.5: Compiler flags by Unity C++ Compiler Configuration

Version	Release	Master
Android 6000.3 (arm64)	-Os (size-oriented), no LTO	-Os, no LTO (identical)
Android 6000.5 (arm64)	-Os	-O2 (optimization balancing speed and code size) + ThinLTO
iOS 6000.3 / 6000.5	-O3 (optimization that pursues speed more aggress ively at the cost of possible code size growth), LTO OFF	-O3, LTO OFF (identical)

12.5.2 Adding `-flto=thin` to the Release Configuration

As described above, switching to the Master configuration is the fastest way to obtain the effect of LTO, but the optimization level also changes from `-Os` to `-O2` at the same time as LTO (Table 12.5), and the drawbacks are increased size and build time. If you keep the Release configuration and apply only ThinLTO, you can measure the effect of LTO separately and also limit the increase in size and build time. To apply ThinLTO to `libil2cpp.so` generated from your own C#, you can pass clang flags directly, as in List 12.6.

▼List 12.6 Build preprocessing that enables ThinLTO in the Release configuration

```
// Save the existing additional arguments → append → always restore
var saved = PlayerSettings.GetAdditionalIl2CppArgs();
try
{
    var extra = "--compiler-flags=-flto=thin --linker-flags=-flto=thin";
    PlayerSettings.SetAdditionalIl2CppArgs(
        string.IsNullOrEmpty(saved) ? extra : saved + " " + extra);
    BuildPipeline.BuildPlayer(options);
}
finally
{
    PlayerSettings.SetAdditionalIl2CppArgs(saved);
}
```

Verify whether it has been applied in two steps (List 12.7).

The first is whether the flag was passed to the build commands. Unity's build

system (Bee) records the compile and link commands it executed in Library/Bee/Player*.dag.json. Search this file with -flto=thin and check that the flag is included in both the compilation of the IL2CPP-generated code and the link of libil2cpp.so.

The second is whether the generated object files are LLVM bitcode. When LTO is enabled, the compile output .o is bitcode instead of machine code. Select one output file of a compile command found in the first step and display its first 4 bytes.

▼List 12.7 Commands to confirm IL2CPP ThinLTO is applied

```
# 1. Check whether the compile and link commands contain -flto=thin
rg -n -- '-flto=thin' Library/Bee/Player*.dag.json
# 2. First 4 bytes of the generated object (specify the .o found in 1.)
xxd -l 4 Library/Bee/artifacts/Android/.../GeneratedCode.o
```

For bitcode, 4243 c0de (BC..) is displayed; for an ordinary machine-code object, 7f45 4c46 (.ELF) is displayed. If you pick an arbitrary .o in the folder, you may pick up an object of a third-party library that is not subject to LTO. For the second command, always specify a file that appeared in the search results of the first.

12.5.3 Benchmark Design

Generating Synthetic Code

This benchmark mainly measures calls across TUs, which normal compilation cannot optimize. An asmdef boundary and a C++ TU boundary are not the same concept, so I checked the generated C++ after IL2CPP conversion and also confirmed in the Bee compile graph that the caller and callee fall into different TUs. The code generator produces the following.

- 16 asmdef assemblies × 24 classes × 8 methods = 3,072 methods
- One round of the call chain passes through 16 assembly boundaries, and every method additionally calls helpers in a shared assembly (Core.Mix / Core.Idx / struct operations)
- The benchmark visits all classes, so managed stripping does not remove them
- A different constant per method prevents folding by ICF

An example of the 3,072 generated methods is shown in List 12.8.

▼List 12.8 Example of a method generated by the synthetic benchmark

```
public static int M2(int sel, int x, int[] buf)
{
    x = Core.Mix(x, 655663086);      // Cross-boundary call to the shared assembly
    x += buf[Core.Idx(x, 7)];        // Array access with bounds check (Idx is masked
    with & 1023)
    return M3(sel, x, buf);          // Small function chain
}
```

Two Workloads

Table 12.6: Synthetic benchmark workloads for IL2CPP ThinLTO

Workload	What it measures
direct	Cross-assembly static call chain (the form where LTO should be most effective)
iface	The same kind of operations run through int erface dispatch (a form where LTO has diffic ulty resolving the callee directly)

Both are self-contained within `libil2cpp.so`. For the binary generated in each configuration, I verified the md5 of `libil2cpp.so` extracted from the APK every time. The iteration count is adjusted at run time so that one sample takes about 100ms. Because the count differs per run, values are normalized to the time per pass for comparison.

In iface, four classes implementing `IGenOp` are generated for each assembly (64 in total), and the benchmark calls them in turn through the interface (List 12.9). The method body calls the same `Core.Mix` / `Core.Idx` as direct, but which class's `Apply` is called is determined at runtime.

▼List 12.9 Classes and call loop generated for the iface workload

```
sealed class Op0 : IGenOp
{
    int m_State = 20612;
    public int Apply(int x, int[] buf)
    {
        x = Core.Mix(x, 347275849);             // Cross-boundary call to the shared
    assembly
        x += buf[Core.Idx(x ^ m_State, 7)];   // Array access of the same kind as
    direct
        m_State = (m_State + x) | 1;
        return x;
    }
}

// Benchmark side: iterate over the 64 implementations through the interface
for (int i = 0; i < ops.Count; i++)
    a += ops[i].Apply(a ^ p, buf);       // The callee is determined at runtime
```

12.5.4 CPU Measurements (Pixel 8a)

Table 12.7: CPU measurement results of IL2CPP ThinLTO on a Pixel 8a

Configuration	direct (us/pass)	iface (us/pass)
off(Release, -Os)	19.88	0.404
thin (manual ThinLTO)	15.54 (-21.8%)	0.375 (-7.3%)

Manual ThinLTO reduced direct by 21.8% and iface by 7.3%.

Function calls and instruction counts in direct

For the same call structure as in §5.3.1, I compared the disassembly with and without LTO.

Table 12.8: Disassembly comparison of the direct workload

`C000_M2`	No LTO	ThinLTO
Instruction count	15	11
Function calls (`bl`)	2	0
Stack frame	Yes	No
Saving of callee-saved regist ers	x19, x20, x29, x30	None

Table 12.8: Disassembly comparison of the direct workload

C000_M2	No LTO	ThinLTO
Memory accesses	5	1

The bodies of `Core.Mix` and `Core.Idx` are expanded into the caller, which removes two `bl` instructions and the stack and register save processing. Expressions spanning both functions are also optimized together. The 21.8% reduction in direct is the result of this change accumulating over 3,072 methods.

Why the improvement in iface is small

An interface call generated by IL2CPP has, in outline, the form List 12.10.

▼List 12.10 Outline of the interface call code generated by IL2CPP

```
const VirtualInvokeData& invokeData =
    il2cpp_codegen_get_interface_invoke_data(slot, obj, declaringInterface);
return ((Func)invokeData.methodPtr)(obj, p1, p2, invokeData.method);
```

The callee is a `invokeData.methodPtr` obtained at run time, so it cannot be resolved to a direct call like a normal C++ vtable call. However, the part where the method body calls a helper in another TU can be inlined, and iface also improved by 7.3%.

If interface / virtual calls are taking a long time, you can also consider `sealed`, receiving concrete types, and static dispatch via generics. What ThinLTO improves readily is the part that statically calls small functions in other TUs.

12.5.5 Size and Build Time

Table 12.9: Size and build time of IL2CPP ThinLTO

Configuration	libil2cpp.so (MB)	text of the same	Cold build
off	37.36	baseline	62s
thin	38.16 (+2.2%)	+2.3%	71s (+15%)

In this configuration, `libil2cpp.so` increased by 2.2% due to inline expansion. LTO can also remove dead code, so the direction of the size change depends on the codebase and the optimizations applied. The cold build increased 15%, from

62s to 71s. With ThinLTO, cross-TU optimization and machine code generation are performed at link time, so the link time increases accordingly.

12.6 Optimizations Enabled by the Master Configuration

In the Master configuration, the optimization level also changes at the same time as LTO (Table 12.5). To find out which setting is responsible for the CPU time reduction, we measured each separately. To examine the effects of the optimization level (`-Os` → `-O2`) and of LTO separately, I measured a 2×2 matrix of {-Os, -O2} × {no LTO, ThinLTO} (Android 6000.5 / arm64).

Table 12.10: CPU measurement comparison of optimization levels and ThinLTO

direct (us/pass)	No LTO	ThinLTO
-Os (Release)	19.64 (baseline)	15.54 (-20.9%)
-O2(Master)	18.47 (-5.9%)	15.54 (-20.9%)

Table 12.11: libil2cpp.so size comparison of optimization levels and ThinLTO

libil2cpp.so	No LTO	ThinLTO
-Os	37.36MB	38.16MB (+2.2%)
-O2	39.14MB	44.14MB (+12.8%)

The CPU time reduction was 5.9% with `-O2` alone and 20.9% in the configuration including ThinLTO. In this benchmark, most of the Master configuration's improvement was obtained from ThinLTO. Release + manual ThinLTO and Master had the same CPU time of 15.54us, but the final `libil2cpp.so` sizes were 38.16MB and 44.14MB, and Release + manual ThinLTO was about 13.5% smaller.

Table 12.12: Cold build time of Release + ThinLTO and Master

Configuration	Cold build
Release (`-Os`, no LTO)	62s
Release + ThinLTO	71s (+15%)
Master (`-O2` + ThinLTO)	86s (+39%)

The cold builds took 71s and 86s, so the configuration that adds only ThinLTO to Release was 15 seconds shorter. In Unity's published measurements as well, the

Master configuration (on the `libil2cpp.so` side) showed CPU -2.3%, time to first display -5.7%, `libil2cpp.so` size +18.7%, and build time +30 to 35%. The CPU improvement was -2.3% in Unity's results, versus -20.9% in this chapter's benchmark. However, Unity measured the entire main thread of real projects, whereas this chapter measured only the path that repeatedly calls small functions so the numbers cannot be compared directly. On the other hand, size and build time in this chapter's Master are +18.2% and +38.7%, close to Unity's published values.

12.7 Verification on real titles

We also tried ThinLTO on two real titles that are larger than the benchmark. Table 12.13 shows the Unity version, number of assemblies, amount of source code, and binary size.

Table 12.13: Unity version and code scale of the real titles

Item	Title A	Title B
Unity version	6000.5.3f1	6000.3.15f1
Number of assemblies in the player	158	285
Compiled `.cs`	6,969 files	16,700 files
Lines of the same	About 1.38 million lines	About 3.19 million lines
Of which in-house code	1,943 files / about 350,000 lines	8,606 files / about 1.54 million lines
`libil2cpp.so` (uncompressed)	189.0MB	294.3MB

12.7.1 Size and application check for Release + ThinLTO

Table 12.14: Size and build time of Title A

Item	baseline	ThinLTO	Difference
libil2cpp.so (uncompressed)	189.01MB	194.70MB	+5.69MB (+3.0%)
Same `.text`	8.54MB	8.98MB	+0.44MB (+5.1%)
APK	165.85MB	166.94MB	+1.08MB (+0.65%)
Player build portion	about 396s	about 433s	about +9%

`.text` +5.1% is larger than the benchmark (+2.3%), consistent with the expectation that the more finely asmdefs are split, the more cross-module inlining occurs. Even with a 189MB libil2cpp.so, the ThinLTO link completed on a local machine.

For Title B, we compared the size of each section of `libil2cpp.so` using `llvm-readelf -S` (Table 12.15).

Table 12.15: Section differences of libil2cpp.so in Title B

Item	baseline	ThinLTO	Difference
libil2cpp.so (whole file)	294.30MB	304.12MB	+3.34%
`il2cpp`	145.85MB	162.22MB	+11.23%
`.text`	10.69MB	11.14MB	+4.18%
`.rela.dyn`	50.28MB	45.01MB	-10.48%
`.got`	1.76MB	1.4KB	-99.92%
`.gcc_except_table`	3.70MB	3.62MB	-2.32%

The `.text` increase of 4.18% falls between Title A (5.1%) and the benchmark (2.3%). `.got` decreased from 1.76MB to 1.4KB. This indicates that LTO resolved cross-module references and converted indirect references through the GOT into direct references. The 10.5% reduction in `.rela.dyn`, which includes dynamic relocation entries, corresponds to this change as well.

12.7.2 Master and full LTO on a real title (Title A)

We tried the comparison of the Master configuration and full LTO from §6 at real-title scale.

Table 12.16: Build configuration comparison of Title A

Title A	baseline	Release + thin	Master	full LTO
libil2cpp.so	189.0MB	194.7MB (+3.0%)	222.2MB (+17.6%)	–
Build	453s	465s	537s (9 min)	failed after 2,903s (48 min)
Peak RSS on the linker path	–	–	12.9GB	20GB → `Killed : 9` (OOM)

With full LTO, after 48 minutes of building, memory usage during linking reached 20 GB and the OS forcibly terminated the process. In the benchmark in §5.3 as well, full LTO kept CPU time on par with ThinLTO while the cold build became about 3.5 times longer, from 71s to 247s. The build could not be completed for this title, and ThinLTO was more practical.

The size of Master's `libil2cpp.so` increased by about 33MB from the baseline, but CPU performance was on par with Release + thin. With Release + thin, the size increase stays at about 5.7MB, so for this title as well, performance equivalent to Master was obtained with a smaller binary.

12.7.3 CPU cycles on a real title (Title B)

On a steady-state out-of-game screen, we measured the sum of the CPU cycles of all threads on a Pixel 8a. ThinLTO and the baseline were measured alternately for three sessions each, and the value of each session is the average of six 20-second measurements.

Table 12.17: CPU cycles by session for Title B

Session	Total cycles (Gcyc)	UnityMain (Gcyc)
ThinLTO #1	44.88	27.48
ThinLTO #2	45.18	27.36
ThinLTO #3	45.54	27.60
baseline #1	46.06	29.05
baseline #2	45.71	27.89
baseline #3	46.15	28.23
Average (ThinLTO vs baselin e)	45.20 vs 45.97 → -1.68%	27.48 vs 28.39 → -3.21%

12.7.4 Comparison with Unity's published values

I compare the values Unity published for the Master configuration (on the `libil 2cpp.so` side) with the results of Title B. The benchmark results from §6 are also listed.

Table 12.18: Comparison of Unity's published values and this chapter's measurement results

Metric	Unity published (6 pr ojects, Master)	Benchmark	Title B
CPU	Main thread -2.3%	`direct` -21.8%	UnityMain -3.21% / total cycles -1.68%
`libil2cpp.so`	+18.7%	+2.2% (Release+thin)/ +18.2%(Master)	+3.0%

Table 12.18: Comparison of Unity's published values and this chapter's measurement results

Metric	Unity published (6 projects, Master)	Benchmark	Title B
Build time	+30 to 35%	+15% (Release+thin) / +38.7% (Master)	+9 to 15%

Unity's CPU time reduction of -2.3% and Title B's UnityMain -3.21% were both improvements of a few percent. The gap from the benchmark's -21.8% can be explained by the fact that measurements of real titles also include processing outside LTO's scope. A similar tendency is seen in the six projects Unity measured.

Size and build time differed by configuration. Unity's published values for the Master configuration are +18.7% and +30 to 35%, respectively. They were close to the Master configuration measured in this chapter: +18.2% (44.14MB / 37.36MB) and +38.7% (86s / 62s). The baseline `libil2cpp.so` sizes are also close, at 37.26MB / 37.36MB.

Adding only ThinLTO to Release gave equivalent CPU performance with +2 to 3% size and +9 to 15% build time. From the 2×2 comparison in §6, the increases in size and build time in the Master configuration are considered to come mainly from the change to `-O2`.

12.7.5 Optimization scope and improvement on real titles

In Title B, the improvement is concentrated on the main thread. While UnityMain decreased by -3.21%, all non-UnityMain threads together changed by +0.8%, which is within the 4% drift between sessions.

In the binaries as well, `.so` other than `libil2cpp.so` is almost unchanged. Two native plugins have identical md5, and the difference in `.text` of the remaining one is +0.008%, within the range of build nondeterminism. The optimization by `--compiler-flags` this time did not reach these plugins.

UnityMain's -3.2% is smaller than the benchmark's -22%. On this screen, facial expressions, blinking, hair and clothing sway, IK, and so on run as jobs. These are contained in native binaries outside `libil2cpp.so` and are outside the scope of the

optimization by this flag.

The improvement for IL2CPP generated code alone, excluding the out-of-scope processing, is considered larger than the -3.21% of UnityMain as a whole. However, determining the exact value requires separating the in-scope and out-of-scope processing time with the Profiler. Jobs are also executed on the main thread by scheduling, so UnityMain's measurement also includes out-of-scope processing.

12.8 Applying ThinLTO on iOS

I also applied ThinLTO to the iOS version of Title B. The effect was confirmed in the validation benchmark, but the real title did not gain an improvement that justified the increase in build time.

12.8.1 LTO structure on iOS

Table 12.19: Comparison of the LTO structure on Android and iOS

Aspect	Android	iOS
il2cpp output	`libil2cpp.so`	`libGameAssembly.a`
Final link	LLD called by il2cpp	ld64 when Xcode creates `UnityFramework`
Arguments passed	`--compiler-flags=-flto=thin --linker-flags=-flto=thin`	`--compiler-flags=-flto=thin` only
Where build time increases	Unity build step	`Build ipa` (xcodebuild) step

On iOS, il2cpp only creates an archive, so `--linker-flags` is not passed. LTO code generation happens at link time on the Xcode side, which reads the archive containing this bitcode.

Because il2cpp is responsible for compiling the generated code, the flag must be specified in `AdditionalIl2CppArgs`. Changing `LLVM_LTO = YES_THIN` in Xcode does not apply to the generated code.

Engine code is provided as a prebuilt `libiPhone-lib.a` and has no bitcode, so it is not subject to LTO. There was also no setting to swap in an LTO-applied engine, like the Android engine code LTO toggle (§3).

12.8.2 Verification benchmark on iPhone 13

First, we ported the same verification benchmark as in §5.3 to iOS and checked whether ThinLTO on generated code also works on iOS. `direct` consists of 3,072 small methods divided into 16 asmdefs a configuration in which ThinLTO is most likely to show an effect. Therefore, the improvement rate here is not a value obtainable as is in a typical title, and should be treated as close to the upper bound when conditions are favorable.

The device is an iPhone 13 (`iPhone14,5` / iOS 18.7.8). I used Unity 6000.5.3f1, IL2CPP Release, and OptimizeSpeed. The only difference between baseline and ThinLTO is the presence of `--compiler-flags=-flto=thin`. Xcode is `ReleaseForRunning`, development signing is `get-task-allow=true`, and Unity's Development Build is disabled.

For each session, I uninstalled, reinstalled, and launched the app, and recorded 30 samples after five warmups. The measurement order was baseline / ThinLTO / ThinLTO / baseline for three blocks, for a total of 12 sessions. The thermal state was Nominal in all sessions. The iteration count is adjusted automatically per session, so `direct` and `iface` are normalized per pass, and Physics per step.

Table 12.20: iPhone 13 validation benchmark results

Workload	baseline	ThinLTO	Difference
`direct`	21,231.31ns/pass	16,184.31ns/pass	-23.77%
`iface`	366.96ns/pass	370.39ns/pass	+0.93%
Physics (out-of-scope control)	147.82us/step	147.36us/step	-0.31% (distributions overlap)

`direct` was -23.6 to -23.9% in all three blocks, and all baseline and ThinLTO sessions were completely separated. It is also close to Android's -21.8% (§5.4), which confirms that ThinLTO on generated code actually works on iOS as well. On the other hand, `iface` worsened slightly, and the out-of-scope Physics showed no difference. ThinLTO's effect is concentrated on paths where static small function calls cross TUs. The binary sizes of the same benchmark are shown in Table 12.21.

Table 12.21: Binary size of the iPhone 13 validation benchmark

Item	baseline	ThinLTO	Difference
UnityFramework	98.76MB	107.52MB	+8.76MB(+8.87%)
`__text`	20.09MB	20.46MB	+0.37MB(+1.83%)
`il2cpp`	22.49MB	30.78MB	+8.30MB(+36.89%)

12.8.3 Results for Title B and the reason adoption was skipped on iOS

Next, we applied ThinLTO to the iOS version of Title B. Table 12.22 shows the size of UnityFramework and Table 12.23 shows the build time.

Table 12.22: Section differences of iOS UnityFramework

Section	baseline	ThinLTO	Difference
UnityFramework (whole file)	316.53MB	393.47MB	+24.31%
`__TEXT` (segment)	261.83MB	338.53MB	+29.29%
`il2cpp`	167.02MB	240.26MB	+43.85%
`__text`	57.99MB	61.70MB	+6.41%
`__gcc_except_tab`	6.85MB	6.63MB	-3.16%
`__unwind_info`	3.44MB	3.38MB	-1.75%
`__got`	18.8KB	18.8KB	±0
ipa (whole archive)	233.83MB	246.93MB	+5.60%

Table 12.23: iOS build time

Step	baseline	ThinLTO
`Build ipa`(xcodebuild)	357s(5m57s)	3,094s(51m34s)

UnityFramework increased by +24.31% and the ipa by +5.60%. The build time increased about 9 times, by 46 minutes. Most of the increase came from linking on the Xcode side and `dsymutil`, and it is substantially longer than the +9 to 15% on Android. A configuration with `GCC_OPTIMIZATION_LEVEL` lowered to `-Os` (`-Os` + thin) still took 53m51s and did not shorten it.

On Title B's steady-state screen, the simple average of `cpu/wall` worsened slightly with ThinLTO by +1.08%, and even among pairs with matching thermal states, the im-

provement was only -1.45%. The distributions of baseline and ThinLTO overlapped, and a large CPU improvement could not be isolated. The memory difference of +0.40% is also below the within-session variation. The validation benchmark's -23.77% is a value obtained by concentrating calls favorable to ThinLTO, and did not apply to the real title as a whole.

We could not confirm a clear improvement on a real title, and an additional time of about 46 minutes would be required for every cold build or CI run, so in this verification we decided not to adopt manual ThinLTO on iOS.

12.9 Adopting it in your own title

On Android, we recommend first adding `-flto=thin` to the Release configuration. In this benchmark, it gave CPU performance equivalent to Master with a smaller binary and a shorter build time. On the steady-state screen of a real title as well, total cycles decreased by 1.68% and UnityMain by 3.21% (§7.3).

We do not recommend the engine code LTO toggle (§3) in this configuration, because disabling Strip Engine Code causes a large increase in size and memory.

Manual ThinLTO on iOS did not provide an improvement commensurate with the increase in build time, so we decided not to adopt it.

Before adoption, examine the scripting time in the PlayerLoop with a Profiler capture of a Release build. The target of the manual flag is within `libil2cpp.so`; the render thread, GPU, engine internals, and GC are not targets. Jobs compiled by a separate AOT pipeline are also not included (§7.5). For titles that use DOTS/Jobs heavily, also check their processing time.

For example, if scripting takes 8 ms of a 33 ms frame, even a reduction of about 20% in that part shortens the whole frame by only about 1.6 ms. Consider adopting it only if scripting accounts for a large share of the whole frame and there is room for improvement, or if a high frame rate must be maintained.

If the increase in build time hampers daily development work, you can apply it only

to builds from staging onward and not to everyday development builds. Because the flag is added in the build preprocessing (List 12.6), it can be switched for each type of build.

After configuring, confirm the application using the procedures in §3.1 and §5.2. The places to check are inputdata, the binary md5, `-flto` in the compile command, and the bitcode of object files. A build may succeed even when the LTO conditions are not met, so the build result alone cannot tell you whether it was applied.

Chapter 13

iOS Memory Limits and Virtual Address Space

Minhyuk Kim

CyberAgent, Inc. SGE Core Technology Division

Name：Minhyuk Kim Affiliation：CyberAgent, Inc. SGE Core Technology Division

The memory available to a game on a device naturally has an upper limit. This limit becomes a problem when assets overlap during a scene transition or when temporary rendering buffers increase.

In addition to reducing the memory peak within the game, there is also room to adjust the memory limit itself. Xcode's "Increased Memory Limit" is a setting that raises that limit on supported devices. A different setting, "Extended Virtual Addressing", expands the virtual address space available to the process. The names are similar, but they address different areas.

This chapter introduces how to choose between the two settings, their effects on physical devices, and experience operating them on the App Store. The numbers come from the author's measurements on physical devices and are not per-device limits guaranteed by Apple.

13.1 Memory Limit and Virtual Address Space Are Separate Constraints

When an app runs out of memory (OOM), you need to distinguish at least the following two cases.

Table 13.1: Two types of memory constraints to distinguish in iOS apps

Constraint	Main values to check	Typical manifestation
Physical memory	`phys_footprint`	The system terminates the app for high memory use (jetsam)
Virtual address space	VM Region Summary, Virtual Size	Memory allocation fails, follo wed by a crash

Physical memory relates to the pages that must actually be held in RAM. On iOS, the amount of physical memory an app can use is not the installed RAM itself. It also depends on the system state at that moment and the limit applied to the app.

Apple's `os_proc_available_memory()` returns the amount that can be allocated before the current app reaches its memory limit. According to Apple's description, this return value corresponds to the "current memory limit minus the app's memory footprint" and is not necessarily the same as the device's free physical memory. The limit can also change during the app lifecycle, so do not cache the value and treat it as reference information [*1].

The virtual address space is the range that manages which addresses memory and files are mapped to. For example, even if you `mmap` a large file, not all of its pages are necessarily loaded into RAM immediately. However, virtual addresses are still needed to hold that range. Conversely, even when the physical memory footprint is still low and `os_proc_available_memory()` has headroom, a new address range may fail to be reserved, causing `malloc` and similar calls to fail.

[*1] Apple Developer Documentation, `os_proc_available_memory` `https://developer.apple.com/documentation/os/os_proc_available_memory`

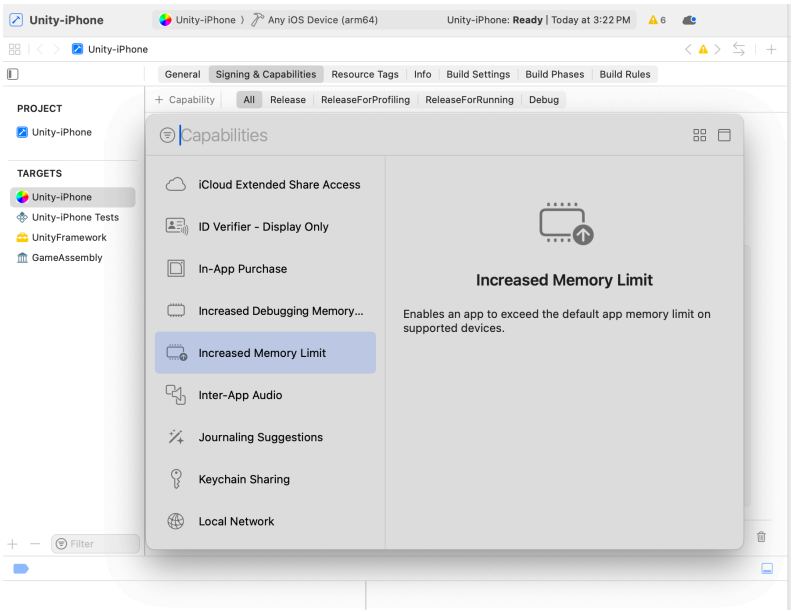
Therefore, to distinguish the two kinds of constraints in Table 13.1, you need to check whether a jetsam report remains or whether an allocation failure is the starting point in an ordinary crash report. Even if `os_proc_available_memory()` returns a large value, the virtual address space does not necessarily have the same amount of headroom.

13.2 Roles of the Two Entitlements

13.2.1 Increased Memory Limit

The key for Increased Memory Limit is `com.apple.developer.kernel.increased-memory-limit`. Apple describes it as an entitlement that tells the system that exceeding the default app memory limit on supported devices may improve the app's core functionality [*2]. The Xcode Capability list also shows a conditional description such as “on supported devices”, as in Figure 13.1.

[*2] Apple Developer Documentation, `com.apple.developer.kernel.increased-memory-limit` `https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.increased-memory-limit`



▲Figure 13.1 Increased Memory Limit shown in the Xcode Capability list

In other words, specifying it does not add a fixed amount on every device. Apple's documentation also states that only some devices can use the increased limit, and asks that apps work correctly even when the additional memory is unavailable.

Measuring Increased Memory Limit on physical devices

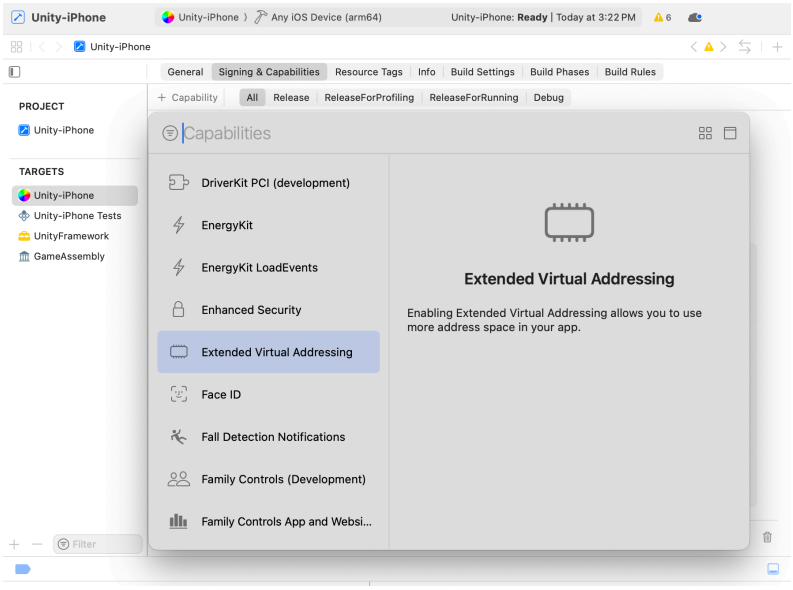
Table 13.2: Memory measured at OOM with and without Increased Memory Limit (iPhone)

Installed RAM	Disabled	Enabled
3GB	About 1,850MB	About 1,850MB
4 GB	About 2,050 MB	About 2,290 MB
6 GB	About 3 GB	About 4 GB

Memory increased on the 4GB and 6GB iPhones, whereas on the 3GB iPhone there was no difference between enabled and disabled. Also, the memory at OOM on iPad differs from that on iPhone to begin with, so the result when enabled is likely to differ as well. If 3GB iPhones account for a large share of your target users' devices, you should not expect that simply adding Increased Memory Limit will rescue them.

13.2.2 Extended Virtual Addressing

The key for Extended Virtual Addressing is `com.apple.developer.kernel.extended-virtual-addressing`. Apple describes it as a Boolean value indicating access to a larger address space, and gives games as an example [*3].



▲Figure 13.2 Extended Virtual Addressing shown in the Xcode Capability list

This setting does not add RAM. For an app that is jetsammed as it approaches the physical memory limit, adding only Extended Virtual Addressing does not guarantee that it can keep using the same amount of RAM. It is expected to help when the virtual address space becomes constrained before physical memory is exhausted, because of large memory maps, fragmented address regions, or large fixed reservations.

Enabling it has almost no cost. Page tables are created only for the range actually

[*3] Apple Developer Documentation, Extended Virtual Addressing Entitlement `https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.extended-virtual-addressing`

mapped, so they do not grow when the limit is widened [*4]. What to consider is whether there are devices on which it has an effect. Adding Increased Memory Limit also widens the address space, so there are devices on which `max_address` does not change even when both are added.

On the other hand, on devices where Increased Memory Limit has no effect, such as the 3GB device in Table 13.2, Extended Virtual Addressing is the only means of widening the address space. Increased Memory Limit is ineffective if the device does not support it, even if the signature is correct, but Extended Virtual Addressing has no such condition.

Virtual Address Space Limits

What Extended Virtual Addressing widens is the address space limit, which is separate from the footprint limit. This limit is determined by one of the Table 13.3 values that `pmap_max_64bit_offset()` returns according to the installed memory [*5].

Table 13.3: Address space limits for arm64 user processes

Condition	Constant	Limit
`max_mem` > 3GB	`ARM64_MAX_OFFSET_DEVICE_LARGE`	17.375GB
`max_mem` above 1 GB up to 3 GB	`ARM64_MAX_OFFSET_DEVICE_SMALL`	13.375 GB
`max_mem` 1 GB or less	`ARM64_MIN_MAX_ADDRESS`	12.5 GB
Extended by any means, including entitlements	`MACH_VM_JUMBO_ADDRESS`	63 GB

`max_mem` is the physical memory recognized by the kernel and is smaller than the RAM capacity in the catalog specifications. All of these are kernel compile-time constants, so they may change between OS versions.

[*4] Apple Open Source, xnu-12377.121.6, `osfmk/arm/pmap/pmap.c` (`pmap_max_64bit_offset`) https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/osfmk/arm/pmap/pmap.c

[*5] Apple Open Source, xnu-12377.121.6, `osfmk/arm/pmap/pmap.c` (`pmap_max_64bit_offset`) https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/osfmk/arm/pmap/pmap.c

13.2.3 Track Record on the App Store

A concern when adopting these Entitlements is whether they affect App Store review. Among several titles in operation that the author knows of, there are ones that enabled either Increased Memory Limit or Extended Virtual Addressing, passed App Store review, and are being distributed.

Apple itself, in "Bring your high-end game to iPhone 15 Pro" [*6] for game developers, advises enabling Increased Memory Limit to increase the available memory. These two are public features that you can consider adopting in a product. This does not guarantee the review result of an individual app as a whole, but there is no need to rule them out for fear that, because they concern the memory limit, review might ask for an explanation or reject the app.

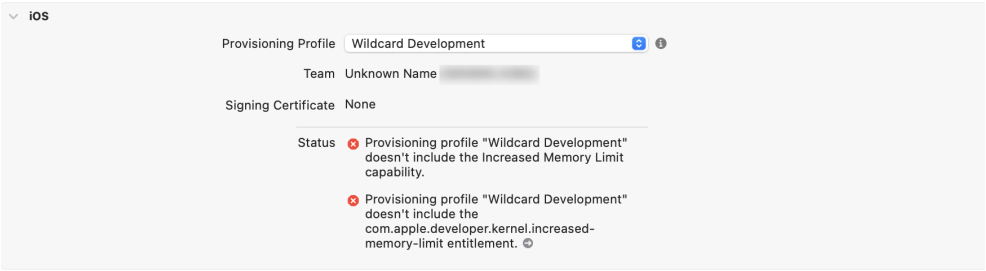
13.3 Check the App ID and Developer Account

13.3.1 Wildcard Profiles Cannot Be Used

These two Entitlements cannot be enabled with a wildcard App ID. If you sign multiple apps together, as in `com.example.*`, you need to switch to an Explicit App ID that matches the game's Bundle ID.

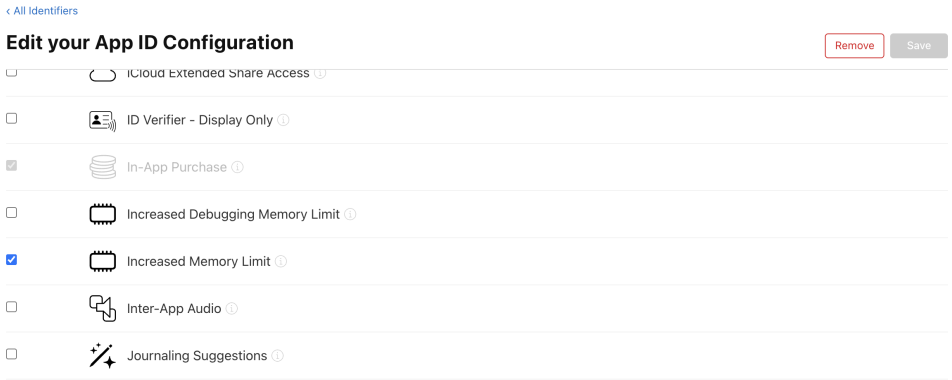
If you try to sign with it as is, Xcode's Signing & Capabilities shows the error Figure 13.3. Note that the capability name and the entitlement key are reported separately. Both are pointed out: the profile does not include the capability, and there is no permission corresponding to the key.

[*6] Apple Developer Tech Talks, Bring your high-end game to iPhone 15 Pro `https://developer.apple.com/videos/play/tech-talks/111372/`



▲Figure 13.3　Error when trying to sign with a wildcard Provisioning Profile

In Certificates, Identifiers & Profiles on Apple Developer, select the target App ID and enable the required capability, as in Figure 13.4. Apple also explains that the capabilities set on an App ID serve as an allow list of features the app can use, and that features requiring an Explicit App ID cannot be selected with a wildcard [*7].



▲Figure 13.4　Enabling Increased Memory Limit in the App ID settings

With manual signing, regenerate the Provisioning Profile after changing capabilities and update the one used by the build. With automatic signing, confirm that the updated profile is selected for the correct Team and Bundle ID. If you only add keys to `.entitlements` while keeping the old profile, the permissions the app requests do not match the permissions on the signing side. Align these first, before investigating

[*7] Apple Developer, Register an App ID `https://developer.apple.com/help/account/identifiers/register-an-app-id/`

the Unity-side settings.

13.3.2 Extended Virtual Addressing Requires a Paid Membership

Extended Virtual Addressing cannot be enabled in a configuration that uses a free Apple Account as the Personal Team in Xcode. Apple's capability support table lists as eligible memberships in the Apple Developer Program and the Apple Developer Enterprise Program, and does not include free Apple Accounts [*8].

On the other hand, Increased Memory Limit can be enabled even with a Personal Team.

13.4 Configure Automatically in the Unity Post-Build Process

The minimum content of the Entitlements file is List 13.1. This is an example that enables both at the same time.

▼List 13.1 Minimal memory-related entitlements

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
    PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.developer.kernel.increased-memory-limit</key>
    <true/>
    <key>com.apple.developer.kernel.extended-virtual-addressing</key>
    <true/>
</dict>
</plist>
```

You can also add them manually from Xcode's Signing & Capabilities, but you would have to re-add them at every export, which makes it easy for builds with missing settings to slip in. In Unity, a post-build process can update the Entitlements file and the `CODE_SIGN_ENTITLEMENTS` that references it together. The process runs at

[*8] Apple Developer, Supported capabilities (iOS) `https://developer.apple.com/help/account/reference/supported-capabilities-ios/`

build time, before signing, and does not obtain the permission from C# after the app launches.

13.4.1 Use the Dedicated API in Unity 6000.0.11f1 and Later

In Unity 6000.0.11f1, the following methods were added to `ProjectCapabilityManager` [*9].

- `AddIncreasedMemoryLimit()`
- `AddExtendedVirtualAddressing()`

With these, you can add Boolean values to the entitlements and reflect them in the Xcode project in a single step.

A corresponding definition of `PBXCapabilityType` is also provided. However, this is not a C# enum but a class that represents the requirements of a Capability. Passing this value to `PBXProject.AddCapability()` is not the same as adding the necessary keys to the Entitlements. Therefore, you basically use `ProjectCapabilityManager`, which handles both.

The following is a sample that adds a Capability using `ProjectCapabilityManager`.

▼List 13.2 Post-build process that adds memory capabilities in Unity 6000.0.11f1 and later

```
#if UNITY_IOS
using UnityEditor;
using UnityEditor.Callbacks;
using UnityEditor.iOS.Xcode;

static class IosMemoryCapabilities
{
    [PostProcessBuild]
    static void OnPostprocessBuild(BuildTarget target, string buildPath)
    {
        if (target != BuildTarget.iOS) return;

        string pbxPath = PBXProject.GetPBXProjectPath(buildPath);
```

[*9] Unity 6.0 Scripting API, ProjectCapabilityManager https://docs.unity3d.com/6000.0/Documentation/ScriptReference/iOS.Xcode.ProjectCapabilityManager.html

```
        var pbx = new PBXProject();
        pbx.ReadFromFile(pbxPath);
        string mainTarget = pbx.GetUnityMainTargetGuid();

        string entitlements = pbx.GetBuildPropertyForAnyConfig(
            mainTarget, "CODE_SIGN_ENTITLEMENTS");
        if (string.IsNullOrEmpty(entitlements))
            entitlements = "Unity-iPhone.entitlements";

        var capabilities = new ProjectCapabilityManager(
            pbxPath, entitlements, targetGuid: mainTarget);
        capabilities.AddIncreasedMemoryLimit();
        capabilities.AddExtendedVirtualAddressing();
        capabilities.WriteToFile();
    }
}
#endif
```

`UnityEditor.iOS.Xcode` is included in iOS Build Support, so enclose it with `#if UNITY_IOS`. Otherwise, compilation of the entire Editor fails in an environment without this module, so be careful.

This sample targets a configuration that uses a common Entitlements file for all build configurations and specifies its path relative to the Xcode project. If you use separate files for Debug and Release, or include build variables in the path, adapt it to the processing that applies the change to the actual file of each configuration.

The final save is done with `capabilities.WriteToFile()`. If other post-build processes edit the Xcode project, re-saving `pbx` may cause the Capability Manager to overwrite the saved changes, so be careful to reload the updated file before modifying it.

13.4.2 Update the plist Directly in Versions Without the Dedicated API

To perform the same processing in an Editor without the dedicated API, replace the five lines from generating `ProjectCapabilityManager` of List 13.2 to saving it with List 13.3.

▼List 13.3 Alternative process for Unity without the dedicated Capability API

```
string fullPath = System.IO.Path.Combine(buildPath, entitlements);
System.IO.Directory.CreateDirectory(
    System.IO.Path.GetDirectoryName(fullPath));
var plist = new PlistDocument();
if (System.IO.File.Exists(fullPath))
    plist.ReadFromFile(fullPath);

plist.root.SetBoolean(
    "com.apple.developer.kernel.increased-memory-limit", true);
plist.root.SetBoolean(
    "com.apple.developer.kernel.extended-virtual-addressing", true);
plist.WriteToFile(fullPath);

if (string.IsNullOrEmpty(pbx.FindFileGuidByProjectPath(entitlements)))
    pbx.AddFile(entitlements, entitlements);
pbx.SetBuildProperty(mainTarget, "CODE_SIGN_ENTITLEMENTS", entitlements);
pbx.WriteToFile(pbxPath);
```

By loading the existing file and then adding the two keys, it updates without deleting Entitlements such as Push notifications. What is modified is `.entitlements`, not `Info.plist`. Also, the file is used for signing only when it is referenced from `CODE_SIGN_ENTITLEMENTS`.

Adding C# code alone does not enable the permission on the account side. With either method, the App ID and Provisioning Profile on the Apple Developer side must be set up separately.

13.5 Verify the application in two steps

Verify whether the settings are reflected in two steps: whether the keys are in the signature, and whether the kernel applied them on a physical device.

13.5.1 Verify that they are included in the signature

Whether done manually or in post-build processing, the mere existence of a List 13.1 file in the project is not evidence that it is reflected in the app installed on the device. Entitlements are ultimately embedded in the code signature. The exported `.ipa` is in zip format, so extract it and run List 13.4 on the app binary in `Payload`.

▼List 13.4 Checking the entitlements embedded in the signed app

```
unzip -q YourApp.ipa -d ipa
codesign --display --entitlements :- ipa/Payload/YourApp.app
```

In the output plist, check whether the two keys of List 13.1 are present. Apple also advises post-signing verification with this command [*10].

13.5.2 Verify that the limit changed on a physical device

Even if the keys are visible in List 13.4, it does not mean the kernel actually applied them. If the device does not support Increased Memory Limit, it is treated as ineffective even when signing is correct.

Whether it is applied on a physical device is determined by directly reading the values the kernel holds for its own process. `task_info()` retrieves them; it is a Mach API included as standard in the iOS SDK (`<mach/mach.h>`). It cannot be called from C#, so it is read from the `.mm` placed in `Assets/Plugins/iOS/` and returned to C#.

▼List 13.5 Native plugin that reads the application status

```
#import <mach/mach.h>

extern "C" void UPT_ReadLimits(uint64_t *memlimit, uint64_t *ceiling)
{
    task_vm_info_data_t i;
    mach_msg_type_number_t c = TASK_VM_INFO_COUNT;
    if (task_info(mach_task_self(), TASK_VM_INFO, (task_info_t)&i, &c) != KERN_SUCCESS)
     {
        return;
    }

    // Footprint limit (Increased Memory Limit)
    *memlimit = i.limit_bytes_remaining + i.phys_footprint;
    // Address space limit: extended if it has grown to 0xF_C000_0000
    *ceiling  = i.max_address;
}
```

On the C# side, declare `UPT_ReadLimits` with `DllImport("__Internal")` and call it.

[*10] Apple Developer Documentation, Diagnosing Issues with Entitlements `https://developer.apple.com/documentation/bundleresources/diagnosing-issues-with-entitlements`

The value to look at is the limit. The remaining amount returned by `os_proc_available_memory()` decreases as the footprint grows, so looking at the value alone cannot distinguish whether the limit has not risen or the app has simply used more.

`limit_bytes_remaining` is obtained from the same kernel function as that `os_proc_available_memory()` [*11], so adding the current footprint restores the limit in effect. The address-space-side `max_address` can also be obtained from the same structure, so a single call can verify both entitlements. `max_address` is filled only when you pass `TASK_VM_INFO_COUNT`.

However, the effect of Extended Virtual Addressing alone cannot be isolated from `max_address`. The same limit appears in a build with Increased Memory Limit and in a build with Xcode's Enhanced Security enabled. What can be confirmed is only whether the limit has expanded to 63GB.

13.6 Virtual Address Reservation That Became Apparent in iOS 26.4

13.6.1 `JS VM Gigacage (reserved)` Is Not 1 GB of RAM Usage

In verification on a 3GB iPhone running iOS 26.4, a region of about 1GB appeared as `JS VM Gigacage (reserved)` in the VM Region Summary of the crash report, and enabling Extended Virtual Addressing prevented the drop in memory at OOM. This region is not the actual Gigacage but a JavaScriptCore Structure Heap reservation displayed under a reused existing VM tag name [*12].

[*11] Apple Open Source, xnu-12377.121.6, `bsd/kern/kern_memorystatus.c` (`memorystatus_available_memory_internal`) `https://github.com/apple-oss-distributions/xnu/blob/xnu-12377.121.6/bsd/kern/kern_memorystatus.c`

[*12] Apple Developer Forums, iPadOS 26.4+ significantly reduced per-app memory limit from 6GB to 3GB on 8GB iPad, breaking memory-intensive apps `https://developer.apple.com/forums/thread/826325`

13.6.2 The WebKit Fix Reduces the Reservation According to Device RAM

The public JavaScriptCore bug 315240 records a policy of reducing the reservation according to the amount of physical memory, because on low-memory devices the Structure Heap reservation occupies a large share of the virtual address space [*13].

In the WebKit commit that addresses this issue, for iOS-family platforms, the smaller of the base reservation size and 20% of physical memory is taken, and then rounding it down to a power of two. The public commit description gives an example of 512 MB for a 3 GB device [*14].

When measured on an iPhone SE (2nd generation, 3GB RAM) with iOS 26.6.1, which includes this fix, the displayed reservation shrank from about 1GB to 512MB, and the drop in memory at OOM seen in 26.4 was resolved even without enabling Extended Virtual Addressing.

13.7 Summary

Increased Memory Limit and Extended Virtual Addressing are settings that lose almost nothing when enabled. Page tables are created only for the range actually mapped, so widening the address space does not increase the cost. There is also a track record of passing App Store review and being distributed. As long as you can prepare the Explicit App ID and profile, we think it is generally good to enable both.

However, the effect depends on the device. In the author's measurements on i Phones, the headroom gained from Increased Memory Limit was about 240MB on 4GB devices and about 1GB on 6GB devices, but there was no difference on 3GB devices. iPad requires separate measurement. Extended Virtual Addressing does not increase physical memory and is effective only when the virtual address space

[*13] WebKit Bugzilla 315240, Scale down StructureAlignedMemoryAllocator reservation in proportion to phys-mem size https://bugs.webkit.org/show_bug.cgi?id=315240

[*14] WebKit commit cffeaa6ccbf42e8d5ff626cc9806f7eac3c66e6b, Scale down structure-heap reservation size with physical memory https://github.com/WebKit/WebKit/commit/cffeaa6ccbf42e8d5ff626cc9806f7eac3c66e6b

runs out first. The Structure Heap reservation issue seen in iOS 26.4 was mitigated in the iOS 26.6 series, but it can still be expected to help when supporting older OS versions or when the app's own memory map uses a lot of address space.

Neither reduces memory usage itself. To keep running on devices without the additional limit, you still need separate work to reduce the peak of textures and temporary RenderTextures, the residency period of assets, large allocations of native plugins, and the lifetime of file mappings. Use the two Entitlements as settings that widen the headroom remaining after that.

Chapter 14

Automating Physical-Device Profiling of Daily Builds

Minhyuk Kim

CyberAgent, Inc. SGE Core Technology Division

Name：Minhyuk Kim Affiliation：CyberAgent, Inc. SGE Core Technology Division

In most cases, profiles on physical devices are taken only when someone thinks they are needed. Before a release, someone connects the Profiler, finds a spike or a memory increase, and searches for the commit that caused it. In this flow, the range in which to look for when things got worse tends to be several weeks of diffs. This chapter covers a setup that automatically runs measurement on physical devices with every daily build.

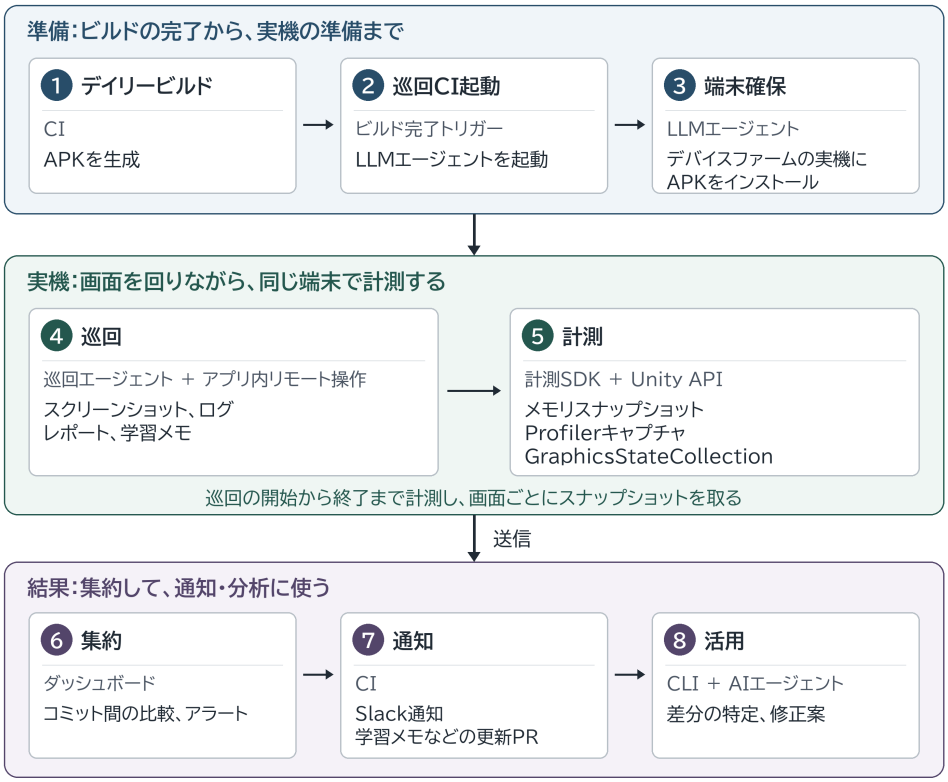
The setup consists of three components: a device farm that lends out physical devices, a remote control server embedded in the app that accepts operations from outside, and a measurement SDK that sends measurement data to a server. The job borrows a physical device remotely from the device farm, navigates through the game's screens in turn with the remote control, and sends memory snapshots and Profiler captures with the measurement SDK. If you add this to CI as a job that follows the daily build, per-commit measurement data accumulates with every build. The accumulated data can be retrieved from the command line, so it also serves as an entry point for having an AI agent read the differences and proceed as far as a fix proposal.

This chapter explains the daily patrol workflow operated in an internal project and

the improvements made along the way. At the time of writing, only Android devices are targeted; iOS support is planned.

14.1 Connect a measurement job after the build

The pipeline proceeds in the order Figure 14.1. Securing a device, operating the app, and sending measurement data are each handled by a separate tool. This division of roles still works even if you replace any of them with another tool.



▲Figure 14.1　Stages of the daily measurement pipeline

The daily build itself is left untouched. The daily build needs neither an embedded patrol scenario nor preconfigured measurement code. By not placing code that anticipates such measurement, the patrol does not break even during development,

when the internal structure changes drastically, and we could maintain stability and flexibility. The method is explained in detail below.

14.2 Receive the build completion in CI and start the patrol

When the daily build completes, another workflow subscribes to the build completion trigger and starts the patrol only on success. What is passed to the subsequent workflow at this point is the ID of the run that built the APK.

The started workflow identifies the build artifact from the run ID and downloads it. After that, it launches the LLM agent responsible for the patrol. At the time of writing, the model is Claude Opus 5.5, and effort, which determines the depth of reasoning, is set to medium.

14.3 Unity measurement is performed from inside the app

What this chapter automates is measurement on the Unity side. The purpose is to obtain, in the same form every day, the breakdown inside Unity, such as which assets occupy memory and which Profiler markers are heavy, and to compare them.

There are of course many useful measurement methods that work from outside the device. On iOS, Time Profiler in Instruments can trace bottlenecks at the call stack level. On Android as well, simpleperf and the Android Studio CPU Profiler that uses it can sample down to the level of native functions generated by IL2CPP. Using such tools for automation is quite conceivable, but it is out of scope here.

The Unity breakdown is usually obtained by connecting the Unity Editor's Profiler or Memory Profiler to a physical device. However, putting the Editor into a CI patrol suddenly increases what you have to think about. Every time the project is opened on a runner, you have to wait for the Library folder to import, and each runner needs a Unity license. In addition to the connection between the Editor and the device, you also have to prepare a path such as Unity MCP for operating the Editor from the

agent, which complicates the setup.

For this reason, we decided not to use the Editor on CI, use only runtime APIs, and complete the measurement within the application.

14.4 Borrow devices from the device farm

The device farm is STF, a platform that lets you operate physical devices from a browser [*1]. However, this chapter does not use operation of the device farm through the screen but only securing a device through the API: when you reserve a device, a remote adb endpoint is returned, and the CI runner can handle it in the same way as a USB connection. From reservation to APK installation is combined into one script, which CI calls in a single step.

If the patrol is the only purpose, one free device running Android 10 or later is enough, but if you want to compare measurements from day to day, the comparison is meaningless if the device changes every day. It is better to fix the model for measurement and skip the day if none is free, so that you do not accumulate data that cannot be compared.

14.5 Navigate the screens with in-app remote control

To operate the app, we use a remote control server embedded in the app. When the app starts, it waits for commands only inside the device, and the CI runner connects via adb and sends commands. In addition to fixed commands such as checking whether the app is running, obtaining the object structure on the scene, taking screenshots, and obtaining logs, it can execute arbitrary processing written in Java Script inside the app.

This JavaScript is interpreted by Jint [*2] and can directly touch C# types. Finding a UI button by name and pressing it, reading the current node of the navigation, and starting the measurement SDK can all be done through the same path. It is close to an environment where you can write and try C# code on the spot on a physical

[*1] DeviceFarmer/stf (successor repository to OpenSTF) `https://github.com/DeviceFarmer/stf`
[*2] sebastienros/jint, JavaScript Interpreter for .NET `https://github.com/sebastienros/jint`

device, and you can call the measurement SDK API from outside without rebuilding. You do not need to rebuild the APK each time you change a threshold or the target screen.

To compare measurements every day, you need to go through the same screens in the same order every day. Taps at fixed coordinates shift when the device resolution or layout changes, so we obtain runtime UI objects by name and fire the UI framework's events directly. Which screen the app is currently on is determined by reading the navigation state. If the project has a screen transition graph definition, it can be used as is as input to the patrol plan.

14.6 Upload two kinds: snapshots and captures

For sending measurement data, we use the measurement platform developed in the author's organization. The SDK embedded in the app sends measurement data to a server, which accumulates it and provides day-to-day comparison and alerts on regressions through a dashboard.

The SDK can send more than Unity memory snapshots and Profiler captures. It can also send rendering load such as the CPU time breakdown and the number of draw calls, the processing time of arbitrary sections in code, app startup time, and the transition of memory usage sampled at fixed intervals, so these can also become targets of regular observation if built into the patrol. This chapter focuses on two of them, Table 14.1.

Table 14.1: Measurement data sent by the daily patrol

Kind	When captured	Prerequisite	How much to send
Memory snapshot	After reaching each screen and it stabiliz es	Development Build	One per screen, send all
Profiler capture	One per patrol, from start to end	Build with Profiler enabled	Send only frames ex ceeding thresholds

14.6.1 Take snapshots for each screen

Memory snapshots use the same format as Unity's Memory Profiler and, by Unity's specification, can be taken only in a Development Build[*3]. The daily build's patrol APK must be a Development Build, and a build with release settings cannot be reused for measurement.

A snapshot is taken once when the patrol reaches a screen and loading has settled. The name includes the screen name and the commit hash, so that data for the same screen from other days can be lined up later.

14.6.2 Take one capture per patrol and filter it on the device

For a Profiler capture, the SDK reads through the raw Profiler log that Unity writes out, on the device, writes only the frames that match the conditions to a separate file, and uploads it. We do not keep all frames as they are because the Profiler is not suited to long recordings. The Editor's Profiler window can hold at most 2,000 frames, which is a little over 30 seconds at 60 fps. Old frames are discarded first, so even if you connect the Editor and record a patrol of several tens of minutes, only the last 30 seconds or so remain. Writing to a file each time removes the limit on the number of frames, but then the file grows to tens of GB for one patrol. That is too large to keep as a CI artifact or to accumulate every day.

The capture starts at the beginning of the patrol, and stopping and uploading are done together at the end. What is decided at the start is the threshold for which frames to keep, specified by two values: frame time and GC allocation amount. Stopping completes after the unread part of the raw log is consumed. The filtering proceeds in parallel during measurement, so even when writing is fast, as with Deep Profiling, almost no unread data accumulates. Even so, to avoid returning the device before stopping finishes, stopping starts with some margin before the patrol deadline.

[*3] Unity Manual, Memory Profiler package `https://docs.unity3d.com/Packages/com.unity.memoryprofiler@1.1/manual/index.html`

14.6.3 Read only the delimiters and match GC by timestamp

The raw log is made by appending per-thread buffers in block units in sequence. To know which block belongs to which frame, you need to find the frame delimiters. The clue is how the Profiler writes. The Profiler switches the thread buffer immediately before and after writing a frame delimiter, so the delimiter arrives as a standalone block of only a dozen or so bytes. The filtering thread therefore opens only blocks no longer than this length and checks whether each is a delimiter. It does not read the contents of longer blocks and treats them as belonging to the current frame of that thread. When the next frame delimiter arrives on the main thread, the frame length is determined, so it decides there whether to keep the frame, and writes out only the blocks of frames to keep, in their original order. Definition blocks that belong to no frame, such as sampler names and thread names, are all written out, so the output can be opened as is in Unity's Profiler window.

The GC allocation amount cannot be obtained from the raw log. The frame statistics in the raw log also have a GC field, but it was 0 in every capture we checked, from Unity 2022.3 to 6000.5. So during measurement, the main thread reads the `GC Allocated In Frame` counter with ProfilerRecorder every frame. The Profiler Recorder value carries no timestamp, so within that frame we read `ProfilerUnsafeUtility.Timestamp` and attach it to the value. This time uses the same clock as the time recorded at the frame delimiters in the raw log. The filtering thread matches the values recorded between the start of one frame and the start of the next as that frame's GC allocation amount. The value for a frame becomes available during the next frame, so the decision waits several frames until the value arrives.

You may wonder whether filtering on the device makes the measurement itself heavier. The filtering runs on a dedicated thread, and the main thread only checks how the data grows and reads the GC counter. As described above, the contents of most blocks are not read, so the processing for filtering is limited to a small part of the data. The filtering thread and the Dispatcher thread that writes out Profiler data are identified by thread name and excluded from the output. The filtering processing is also excluded from Deep Profiling. If it were included, every method call in the filtering thread would become a sample and be written into the very raw log being

read. However, the GC counter is a process-wide value, so it includes the several KB per frame that the filtering thread allocates. This is sufficiently small compared with the thresholds described below, but including the load of enabling the Profiler itself, we cannot say the measurement adds no overhead at all.

14.6.4 Decide thresholds by the problem to see and the data volume

In the author's patrol, the thresholds are currently 300 ms and 2 MB. For a 30 fps title, this is about 10 times the usual frame time, and the setting is meant to check whether large stalls during screen transitions and loading have increased since yesterday, not minor hitches during play. With this setting, only a small part of the whole patrol remains as frames, within an amount that can be sent every day. Lowering the thresholds picks up minor hitches too, but more frames remain, the file becomes larger, and it reaches the output limit described below sooner. Adjust the thresholds while watching the data volume, according to the target title's frame rate and the problem you want to solve with measurement.

14.6.5 Fit a long patrol within the device's storage

As the patrol gets longer, where to put the raw log also becomes a problem. The SDK switches the raw log output destination at fixed sizes and splits it into sequentially numbered files, and the filtering thread deletes files it has finished reading. As a result, the device's storage usage plateaus at a few files. If filtering falls behind and too many unprocessed files accumulate, recording is stopped. When switching files, Unity rewrites all definitions of sampler names and object names that have appeared so far at the beginning of the new file. The output is concatenated into one file, so most definitions from the second file onward are duplicates. Definitions that are byte-for-byte identical to the one written immediately before are therefore omitted from the output. The output file has an upper limit of 1.5 GB, and recording stops when the next frame to keep would exceed the limit. Uploads are limited to 2 GB, so stopping before that ensures that the frames kept up to the limit are reliably sent.

Captures also embed a screenshot of each frame. This is because, with Profiler numbers alone, it is hard to tell which screen was displayed and what was shown

when a spike occurred. The screen is obtained with asynchronous GPU readback, so the image becomes available several frames later. We therefore re-embed the latest available image at the end of every frame, so that an image is attached whichever frame is kept. From Unity 6.6, the Profiler's built-in feature can also record screenshots continuously, and the feature to take screenshots is enabled by default. However, capture is fixed at once every 15 frames, and the images recorded on the author's Android device were only 256×112 pixels, so UI text was unreadable. Also, on devices where AsyncGPUReadback is not available there is no option, and screenshots were silently disabled. In one day's capture filtered at 300 ms and 2 MB, of the 71 frames that remained, only 6 had a built-in screenshot attached. This is a poor match for filtering that leaves scattered frames, so we continue to use the SDK's own capture.

14.6.6 Attach Metadata to All Data

Besides the metadata attached when uploading a capture, the SDK has a mechanism for attaching information common to all measurement data. If you register the commit hash embedded at build time, the build's run ID, and the device model right after initialization, the same information is attached to both snapshots and captures, and the operating side no longer needs to pass it each time. Measurement results can be ordered by commit because of this registration.

14.7 Also collect shader warmup data through the patrol

The patrol goes through the game's main screens every day, so it also serves as a place to collect data for shader warmup. GraphicsStateCollection is an asset that records the combinations of shader variants and rendering states used during execution so that they can be warmed up in advance from the next time on.

The collection does not use the measurement SDK but calls Unity APIs directly from the remote control. Recording starts at the beginning of the patrol, the patrol goes through the screens as usual, and at the end recording stops and the result is written as a file to the app's storage area on the device. The written file is pulled to the CI runner with adb. It uses neither the Editor nor a Player connection, so,

as with measurement, it is completed entirely within CI. However, in IL2CPP builds, code stripping may remove members of unused types, making them uncallable from the remote control. In the build for the patrol, you need to keep the GraphicsState Collection types with link.xml.

The pulled file is not put into the project as is. Only on days when the patrol finished without stopping on an error and the file was saved, it is merged into the existing asset with the same platform and quality settings, variants that were reported as not found during warmup are removed, and a PR to update the asset is created automatically and merged automatically. In this way, the records of the screens passed through in daily patrols accumulate little by little.

That said, this method alone cannot cover all necessary variants. What is recorded is only the screens the patrol actually went through and what was rendered at that time. Screens the patrol does not pass through, and effects that appear only under specific conditions, are missing from the records. For such omissions, and for pitfalls such as stutters remaining even after warmup, see Part 1, Chapter 6. If you have read this book from the beginning, you may already have read it.

14.8 Delegate everything from regression detection to the fix PR to an agent

Even if data accumulates every day, if nobody opens the dashboard every morning, it ends up merely accumulating. This measurement platform also has a CLI that retrieves data from the command line, and in this chapter's pipeline, everything from confirming a regression to creating a fix PR is delegated to an agent that uses this CLI. Rather than a person writing a request each time, it runs daily as a step following measurement.

When the patrol's measurement finishes, a comparison agent separate from the patrol starts. This agent retrieves today's and the previous day's measurement data with the CLI, compares the spikes remaining in the snapshots and captures of the same screen, and checks whether anything has worsened. If a regression is found, it summarizes which value on which screen changed by how much, the data used

for the comparison, and the range of commits made in between, and registers it as an issue in the backlog. If there is no regression, nothing is registered.

The registered issue is picked up by yet another agent, in charge of fixes. It reviews the issue, narrows down the cause from the commit history, applies a fix, and creates a PR. People get involved from the review of that PR.

14.8.1 Verify the fix on a physical device with the same mechanism

Verification of a fix is done separately from the daily patrol. The daily patrol goes through fixed screens in a fixed order and is not suited to matching the screens and conditions that need to be checked for each issue. So a fix verification agent starts when an issue in the backlog moves to the fix completed (awaiting verification) state. This agent uses the same mechanism as the daily patrol, namely a device from the device farm, in-app remote control, and the measurement SDK, runs the build that includes the fix on a physical device, and compares it with the baseline data left in the issue. Only after it confirms on a physical device that the regression is resolved is the ticket handed to QA. Developers do not need to touch a physical device to verify that the fix worked.

This verification mechanism can be used not only for performance regressions but also for everyday bug fixes. As described above, with remote control, processing that can be touched directly, effectively as with reflection, can be executed on the spot, so the agent can operate and read almost everything that happens in the game. If an issue describes reproduction steps and expected results, the agent can follow those steps on a physical device and verify whether the fix works, including the game's internal state.

14.9 Operations to keep it running every day

14.9.1 Division of Roles Between the Patrol Agent and the Measurement Procedure

In this chapter's patrol, a coding agent is given the screen transition graph and how to use the remote control, and performs the patrol. It can work around unreachable screens and rebuild the plan when new screens are added, but the downside is that one patrol takes around 90 turns and costs around 5 dollars. Parts that must be the same every time, such as starting and stopping measurement, are not left to the agent's judgment but fixed as reusable procedures, and the agent only executes them at the specified timing. What the patrol learns is also kept in the repository as a PR for use as input next time.

14.9.2 Handing Over Results

The patrol results are kept as a report, screenshots, logs, and a machine-readable result file, and CI reads the result file and notifies Slack. The measurement results themselves are on the dashboard, so the notification only needs the success or failure of the upload and a link to the dashboard. On days when the comparison finds a regression, the notification to the same channel includes a link to the registered issue. Conversely, on days with no upload, the notification also states that nothing was measured, because measurement silently stopping is the hardest state to notice.

14.10 Future challenges

Because snapshots and captures are taken in the same patrol, the two kinds of data contaminate each other. While a snapshot is being taken, frames stop for several seconds, so it is recorded as a spike in the capture running at the same time. Conversely, during capture, the memory of the Profiler buffer is included in the snapshot, so care is needed when reading the data. If you want to view the data accurately, separating the patrols for snapshots and for captures is also an option, but it roughly doubles the time the device is borrowed, so a compromise such as lowering the frequency is necessary.

The granularity of Profiler captures is also a challenge. Daily measurement turns off Deep Profiling, so game-side scripts are visible only in units such as Mono Behaviour's `Update`. For places where you want finer detail, you need to add Profiler Marker manually or enable Deep Profiling. We are therefore exploring methods such as automatically inserting ProfilerMarker into methods of specified assemblies in build-time post-processing. This can be used like a lightweight, targeted Deep Profiling, so we expect to obtain a finer breakdown while keeping the data volume and overhead low.

14.11 Summary

After the daily build, we connected a job that borrows a device from the device farm, runs the game with in-app remote control, and sends memory snapshots and filtered Profiler captures to the measurement SDK. By attaching the commit hash as metadata, measurement results become a per-commit history, and by retrieving them with the CLI, everything from regression detection to issue registration and fix PR creation can be delegated to an AI agent.

The setup in this chapter may look large-scale, but you do not need to prepare everything at once. If you are planning to automate measurement, even just connecting one physical device to a build machine via USB and taking memory snapshots from an interpreter that works through reflection over adb may already serve as a solid regular observation of memory. We hope this chapter helps you take the first step toward measurement automation.

Chapter 15

Simulating a Long Duration in ParticleSystem Causes a Huge Spike

KATO YUYA

Colorful Palette

Name：KATO YUYA Affiliation：Colorful Palette

15.1 Introduction

Unity's ParticleSystem provides ParticleSystem.Simulate, which lets you control the particle simulation time to an arbitrary value. However, if you specify a large number of seconds for ParticleSystem.Simulate, a large spike can occur when the simulation starts. This article describes a concrete example of this and points to note.

15.2 What Is ParticleSystem.Simulate?

Official documentation for ParticleSystem.Simulate [*1] With ParticleSystem.Simulate, you can simulate the particle effect up to the time specified by the first argument and then play the particles again from that time.

▼List 15.1 Replay from the 0-second point

```
ParticleSystem.Simulate(0f, false, true);
```

[*1] https://docs.unity3d.com/ja/2022.3/ScriptReference/ParticleSystem.Simulate.html

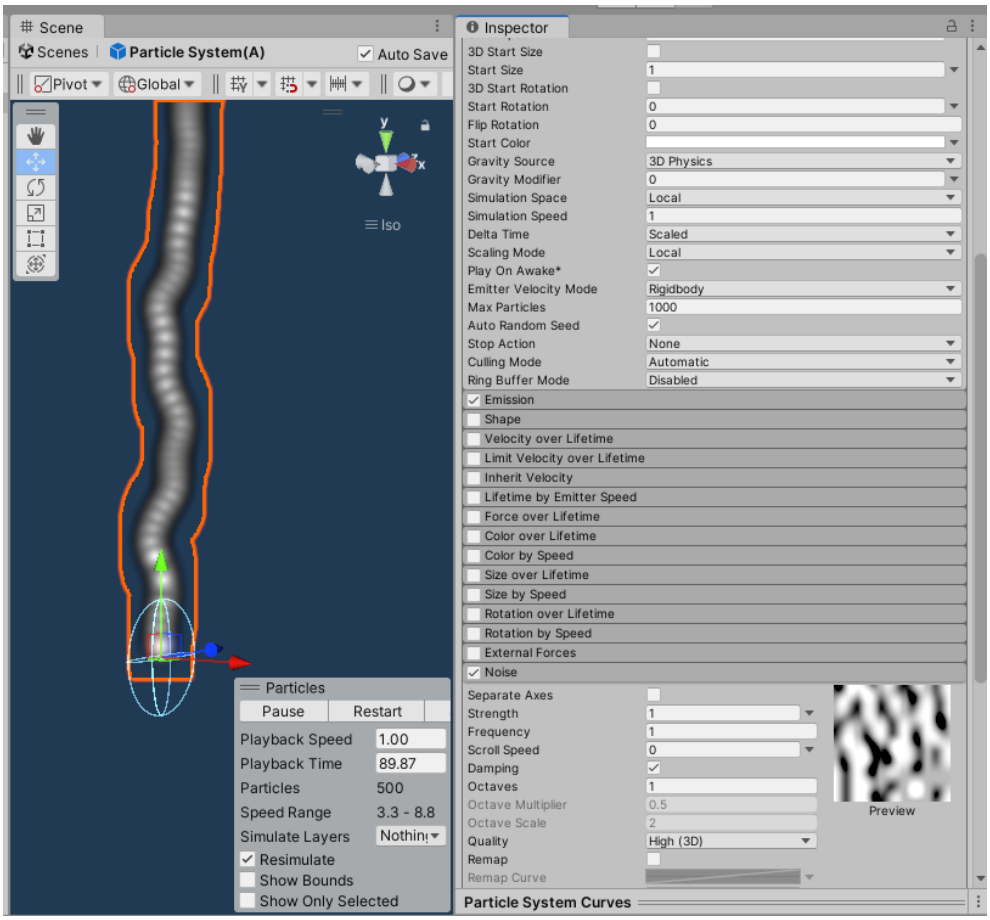
Specifying 0 seconds, as in List 15.1, lets you build a mechanism that replays particles that were partway through playback from the beginning.

▼List 15.2　Replay from the 5-second point

```
ParticleSystem.Simulate(5f, false, true);
```

Specifying 5 seconds, as in List 15.2, lets you build a mechanism that replays the playing particles from the 5-second point.

15.3 Problematic Usage



▲Figure 15.1 Problematic particles

The Noise module is additionally turned on for a particle created with Unity's defaults. As for the other adjusted parameters, to make the particles easier to see, the emission is adjusted in the Emission module so that the particles are emitted in a straight line.

▼List 15.3 Replay from the 20-second point

```
ParticleSystem.Simulate(20f, false, false);
```

If particles that use the Noise module are simulated with an excessively large number of seconds specified in the first argument, as in List 15.3, a large spike occurs during processing.

▼List 15.4 Measuring the simulation load

```
void OnClickA()
{
    // Destroy the playing particle if there is one
    if (_activeParticle != null)
    {
        Destroy(_activeParticle);
        _activeParticle = null;
    }

    // Create a new one and play it
    _activeParticle = Instantiate(particlePrefab, Vector3.zero, Quaternion.identity);
    var ps = _activeParticle.GetComponent<ParticleSystem>();
    if (ps != null)
    {
        ps.Play();
    }
}

void OnClickB()
{
    if (_activeParticle == null) return;

    var ps = _activeParticle.GetComponent<ParticleSystem>();
    if (ps != null)
    {
        Profiler.BeginSample("再シミュレーション");
        // Re-simulate
        ps.Simulate(20f, false, false);
        Profiler.EndSample();

        ps.Play();
    }
}
```

We check how much load is applied by using the Profiler. This is code in which ButtonA creates a particle and ButtonB simulates it. To make it easier to check

in Unity's Profiler whether ParticleSystem.Simulate is the problem, we wrap it with Profiler.BeginSample and Profiler.EndSample to make the investigation easier.



▲Figure 15.2　The spike occurring

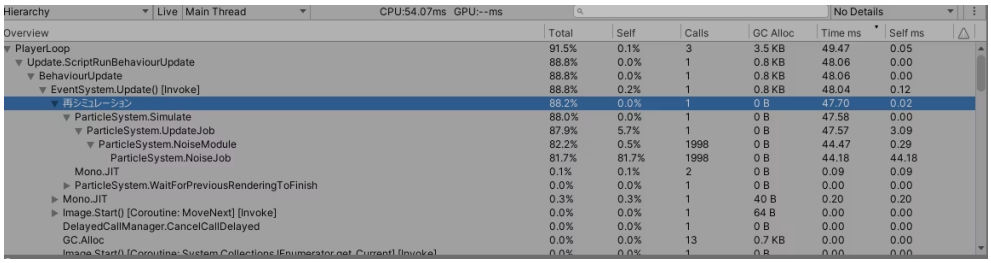


Figure: Processing time As shown, during ParticleSystem.Simulate, NoiseJob is called about 2000 times by the Noise module of the ParticleSystem, which causes a spike of about 50 ms. This number of calls can be reduced somewhat by lowering the ParticleSystem's Max Particles.

15.4 Conclusion

If you need to build a mechanism that emits particles from an arbitrary number of seconds by using ParticleSystem.Simulate,

- Is there a case where a long duration is specified and particles are emitted from midway?
- Is the Noise module used?

Check these two points, and avoid using ParticleSystem.Simulate, especially when using particles with the Noise module enabled.

Chapter 16

Reducing Bone Animation Load with VAT

Tatami Yuuki

CyberAgent, Inc. SGE Core Technology Division

X @tatamin0207

16.1 Introduction

In typical bone animation in Unity, the CPU performs playback calculations such as bone updates every frame. As a result, CPU load accumulates as the number of animated objects increases, which causes the frame rate to drop. This chapter introduces VAT, a technique that reduces this load by replacing these runtime calculations entirely with precomputation.

16.2 What Is VAT

VAT stands for Vertex Animation Texture. It is a technique that bakes the vertex deformation data of an animation into a texture in advance and plays it back by reading the texture in the vertex shader. Each pixel of the texture corresponds to "the position of a certain vertex at a certain frame," and shifting the read location according to the playback time makes the mesh appear to move (Figure 16.1). To preserve precision, positions are stored as values normalized from 0 to 1 and are restored to actual coordinates in the vertex shader.



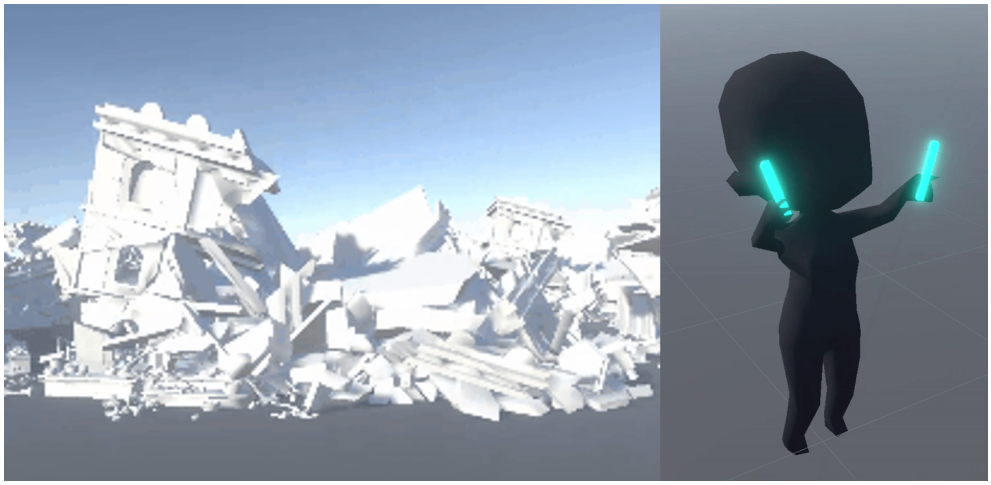
▲Figure 16.1 How a VAT texture works. The horizontal axis corresponds to vertices and the vertical axis to frames

With VAT, runtime playback calculations such as bone updates can be replaced entirely with precomputation. This is an efficiency gain that even GPU Skinning cannot achieve.

On the other hand, VAT has the following drawbacks.

- Dedicated assets for VAT must be prepared
- The texture size grows with the number of vertices and frames
- Because exact values are required, VAT does not work well with texture compression and is effectively handled uncompressed
- Because the motion is fixed once baked into the texture, some control is possible with ingenuity, but VAT is not suited to fine-grained control

Given these characteristics, VAT is particularly effective when a large number of objects play a predetermined animation. It suits effects that do not require individual control, such as building destruction, crowds, and background decorations (Figure 16.2).



▲Figure 16.2　Animation with VAT (left: rigid body, right: soft body)

16.3 Performance Verification

This section presents the results of comparing the performance of VAT with other animation methods on a physical device.

16.3.1 Measurement Environment and Comparison Conditions

The measurement environment is shown in Table 16.1.

Table 16.1: Measurement environment

Item	Details
Physical device	Pixel 8a (GPU: Mali-G715)
Graphics API	Vulkan
Unity version	6000.3.2f1
Measurement APIs	FrameTimingManager, ProfilerRecorder
Model	Building collapse animation (85,209 vertices, 1,315 fragments)
Animation	300 frames
Measurement method	Object count varied from 0 to 16; median of 3 runs of 300 frames per condition

The three methods compared are shown in Table 16.2.

Table 16.2: Comparison conditions

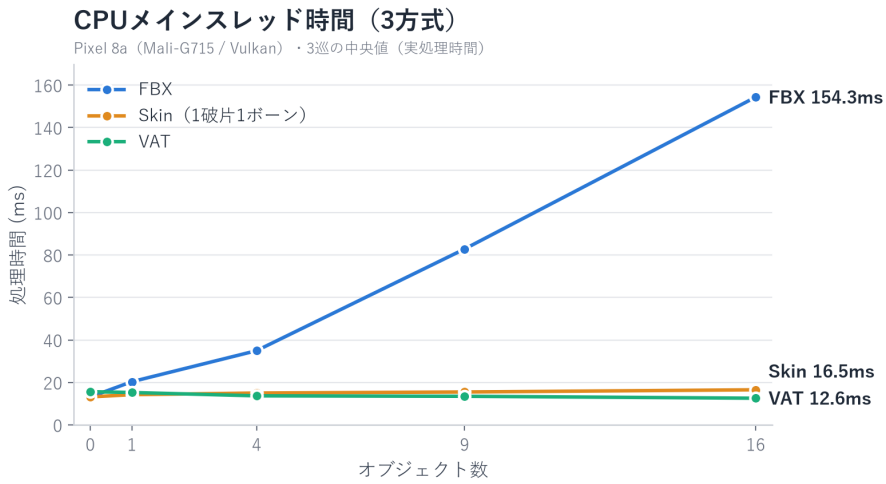
Method	Configuration	Optimization enabled
FBX method	Each fragment is placed as an individual Mesh Renderer and played back with an Ani mator	SRP Batcher
Skin method	One bone per fragment, merged into a single Skinned MeshRenderer and played back with an Animator	SRP Batcher
VAT method	Motion of all fragments is baked into a texture and rend ered as a single mesh (Rigid-Body Dynamics mode)	GPU Instancing

The FBX method has each fragment as an individual Mesh Renderer and plays it back with an Animator (the fragments are rigid bodies moved by Transform, and skinning is not used). The Skin method assigns one bone to each fragment and combines everything into a single Skinned Mesh. The 1,315 fragments become 1,315 bones as they are, and there is one Renderer. Animation playback is handled by the Animator on the CPU, and vertex deformation is performed by GPU Skinning. The VAT method bakes the motion of all fragments into a texture and renders them as a single mesh. The appearance is identical across all three methods.

For the draw optimization settings, each method is compared with the setting that favors it enabled. Because SkinnedMeshRenderer does not support GPU Instancing, the best option available for the Skin method is the SRP Batcher. Note that converting to bones for the Skin method also requires a dedicated conversion pipeline, which is the same condition as for VAT.

16.3.2 CPU Time

The CPU time (main thread) measurement results are shown in Figure 16.3. With 16 objects displayed, the FBX method grows to 154.3 ms, while the Skin method (16.5 ms) and the VAT method (12.6 ms) remain nearly flat.



▲Figure 16.3　CPU main thread time by object count

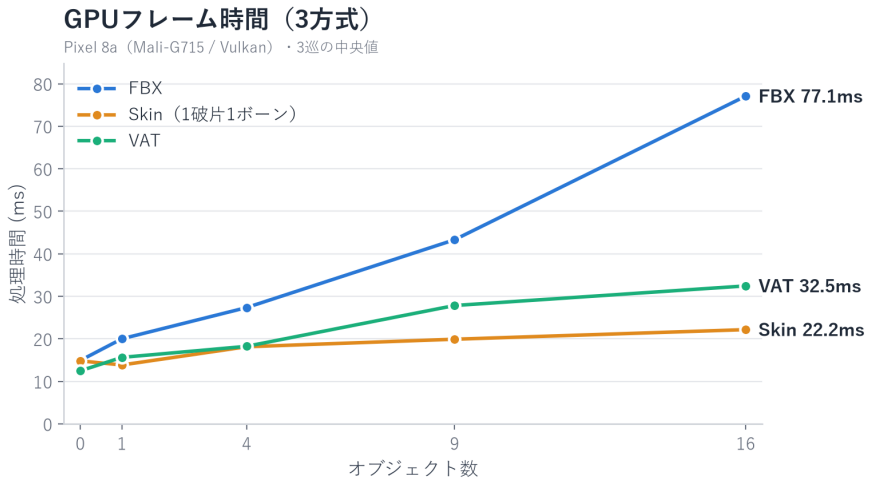
The breakdown is shown in Table 16.3. The largest factor behind the difference between the FBX method and the other two is the draw call submission process. In the FBX method, 1,315 fragments per model become draw targets, whereas in the two methods that combine them into a single mesh, this processing nearly disappears. The same applies to per-fragment bounding box calculation and culling. On the other hand, the difference between the Skin method and the VAT method is the Animator update. In the Skin method, the CPU handles playback, so 4.5 ms remains at 16 objects, whereas in the VAT method the playback itself moves to the GPU, so it disappears entirely. This difference widens as the number of objects increases.

Table 16.3: Breakdown of CPU time with 16 objects displayed (ms)

Process	FBX method	Skin method	VAT method
Main thread total	154.30	16.53	12.56
of which draw call submission (SRPBatcher.Flush)	76.17	0.03	0.02
of which Animator update	5.30	4.54	0.02

16.3.3 GPU Time

The GPU time measurement results are shown in Figure 16.4. With 16 objects, the FBX method takes 77.1 ms, while the Skin method is the fastest at 22.2 ms and the VAT method takes 32.5 ms.



▲Figure 16.4　GPU time by object count

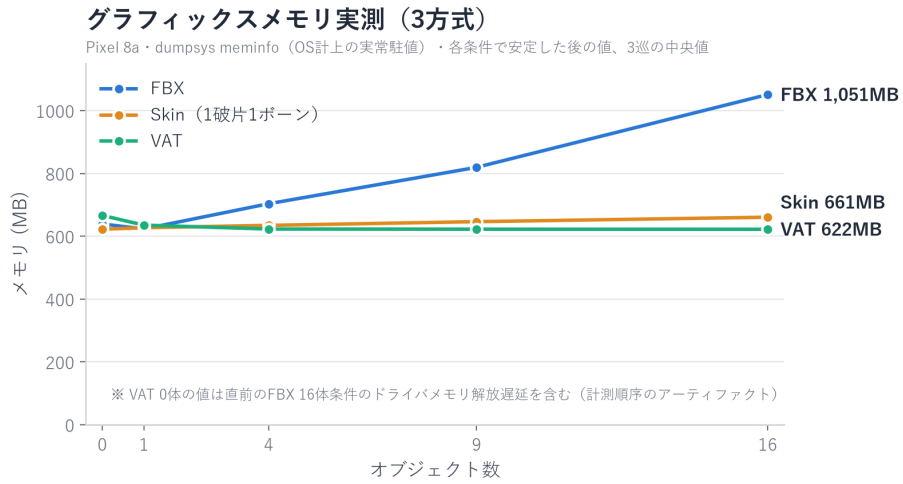
The main cause of the difference between the FBX method and the other two is the number of split draw commands. In the FBX method, each fragment issues its own draw commands in every render pass, reaching about 210,000 per frame, whereas the VAT method consolidates them into instanced draws, resulting in 99. The Skin method also stays at a similar level because there is one Renderer per object. Each

draw command carries a fixed cost, so the difference in the number of splits appears directly as a difference in processing time.

The difference between the Skin method and the VAT method comes from the number of deformation calculations. GPU Skinning calculates the deformation only once per frame, and all render passes such as shadow and depth reuse the result. VAT, in contrast, redoes the deformation from the texture in the vertex shader for every pass. The environment in this test has a scene configuration with many passes, using four shadow cascades, so the gap with the Skin method widened accordingly (shadow map rendering: 44.3 ms for the FBX method, 7.4 ms for the Skin method, 16.0 ms for the VAT method). In configurations with fewer passes that perform vertex calculations, this gap narrows.

16.3.4 GPU Memory Allocation

The GPU memory measurement results are shown in Figure 16.5. The values are measured figures for the graphics driver as accounted by the OS (GL mtrack). The increase from 0 to 16 objects is +411.8 MB for the FBX method (about 26 MB per object), +38.2 MB for the Skin method (about 2.4 MB per object), and almost ± 0 for the VAT method.



▲Figure 16.5　GPU memory allocation by object count

This difference stems from the structural difference in what increases when one object is added. In the FBX method, rendering data such as matrices is allocated in GPU memory for every fragment. In the Skin method, what increases is only one vertex buffer per object, which receives the skinning deformation results. The VAT method computes the deformation on the fly and holds no new buffers, so nothing increases. Note that the VAT texture is about 6.6 MB uncompressed, which is sufficiently small compared with the differences in the increases.

16.3.5　Summary of Verification Results

The measured values with 16 objects displayed are summarized in Table 16.4.

Table 16.4: Summary of measured values with 16 objects displayed

Metric	FBX method	Skin method	VAT method
CPU main thread (ms)	154.3	16.5	12.6
GPU frame (ms)	77.1	22.2	32.5
Draw commands (per frame)	about 210,000	Same level as the VAT method	99

Table 16.4: Summary of measured values with 16 objects displayed

Metric	FBX method	Skin method	VAT method
GPU memory increase (0 to 16 objects, MB)	+411.8	+38.2	almost ±0

Compared with the FBX method, both the Skin method and the VAT method are significantly better on all metrics. The reason comes down to "being handled as a single mesh."

The comparison between the Skin method and the VAT method shows trade-offs: the Skin method is better in GPU time, and the VAT method is better in CPU main thread time and memory. The GPU time of the VAT method depends heavily on whether the pass configuration involves a lot of vertex deformation processing. Therefore, VAT is the first candidate for configurations with few passes that perform vertex calculations, or when the goal is to reduce CPU load.

Note that this comparison used rigid-body destruction, a subject that is easy to express with bones. For models that would require many bones or fall back to approximation, such as cloth or human bodies, and for cases with many objects, the CPU load becomes pronounced, so VAT is considered to have an even greater advantage.

16.4 Creating VAT Assets

An easy way to create VAT assets is to use the Labs Vertex Animation Textures ROP (hereafter, VAT ROP) that comes with Houdini[*1]. Houdini is a DCC tool from SideFX that excels at procedural generation. VAT ROP provides presets and compatible shaders for Unity, and it accepts not only simulation results inside Houdini but also FBX animations created in other DCC tools as input.

VAT ROP has the four modes shown in Table 16.5.

[*1] A tool included in the SideFX Labs package. Version 3.0 was used for the verification in this chapter.

Table 16.5: VAT ROP modes

Mode	Use
Soft-Body Deformation	Basic vertex deformation. Requires that the t opology does not change
Rigid-Body Dynamics	Optimized for rigid-body destruction. Because motion is recorded per fragment, the texture size can be kept small
Dynamic Remeshing	Supports things whose topology changes, such as liquids and tearing cloth
Particle Sprites	For particle simulation

The procedure is simply to specify the node of the animation to convert to VAT in Input Geometry, the mode to use in Mode, and Unity in Target Engine, and then press the Render All button. The assets in Table 16.6 are output at once.

Table 16.6: Assets required for VAT playback

Asset	Role
Position texture	Position of each vertex at each frame (norm alized to 0 to 1)
Rotation texture	Orientation of the deformed faces. Normals and tangents are reconstructed from it to pre serve shading
Vertex color texture	Destination for baking Houdini-side attribute s. A free slot that can be used, for example, to improve interpolation quality
FBX	Mesh with identification numbers embedded in the UVs that indicate the texture reference positions
Material	Auxiliary data such as frame count, FPS, and bounding box already set
Shader	The main body of the playback calculation for each mode (installed separately)

The shader is not included in the output files, because the calculation is fixed regardless of the animation. Install the SideFX Vertex Animation Textures package separately from Unity's Package Manager (its location can be checked in the Real-Time Shaders tab of VAT ROP). In addition to Shader Graphs supporting the four modes, the package includes documentation and Presets for import settings.

16.5 Notes on Adoption

With VAT, the animation breaks if precision is insufficient, so a format suitable for the texture must be specified. Likewise, there are recommended import settings for the FBX side. Follow the "VAT Import Settings Guide" included in the package and apply the dedicated import settings, including disabling texture compression.

Also, the official Shader Graph is positioned strictly as a template, and customization is necessary in practice. Rather than using the template directly, copy it for your project before use. However, the copied Shader Graph still references the Sub Shader Graph on the package side, so be sure to switch the references to the copied side.

Note that the import settings of the output files and the reference settings, such as assigning textures and shaders to materials, must be done manually and are prone to trouble caused by missed settings. In projects that mass-produce assets, there is room to automate import settings and reference resolution with a Scripted Importer or similar. For improvements in this area, see the materials uploaded to CEDiL[*2].

16.6 Summary

VAT is a technique that bakes animation into a texture and plays it back on the GPU. Its defining feature is that it can greatly reduce draw calls, and compared with a configuration that has a Renderer per fragment, it was substantially more efficient, along with the Skin method. The two involve trade-offs: the Skin method is better in GPU time, and the VAT method is better in CPU time and memory. VAT is the first candidate for configurations with few passes that perform vertex calculations, or when the goal is to reduce CPU load. It has constraints such as the need to prepare dedicated assets and fixed motion, but it is a powerful option when a large number of objects play a predetermined animation.

[*2] https://cedil.cesa.or.jp/cedil_sessions/view/3263

Chapter 17

Manuscript

Yamazaki Masahiro

Applibot, Inc.

mao-test-h X @TEST_H_

17.1 iOS Built-in Performance Measurement Features for Unity

- Name: Yamazaki Masahiro
- Affiliation: Applibot, Inc.
- X: `https://x.com/TEST_H_`
- GitHub: `https://github.com/mao-test-h`

17.2 Introduction

Unity provides measurement features such as the Unity Profiler, but for iOS apps you can also use the performance measurement features built into the OS.

These Apple-provided features can be used without depending on Unity's Development Build flag. For example, you can get a rough view of the load before measurement code is integrated, or in a configuration close to a release build with debug features disabled, and you can receive only the trace results from a tester's device. After this rough assessment narrows down the bottleneck candidates to some extent, you can switch to a build with Development Build enabled and investigate in detail with the Unity Profiler or Instruments.

This article introduces the following two features from those that iOS provides built in, and also explains the new features added in macOS 27 / iOS 27.

- Metal Performance HUD
- Performance Trace

We have also published "MetalPerfKit-Unity" [*1], an OSS sample repository that makes these features usable from Unity. See it as well for implementation details.

Note: In the following explanation, the term iOS is used throughout, and it includes iPadOS unless otherwise stated.

17.2.1 Test Environment

The behavior described in this article was verified in the following environment.

- Unity 6000.0.81f1
- Xcode 26.6 / 27.0 beta
- iOS 26.6 / 27.0 beta
- macOS 27.0 beta

Note: At the time of writing, iOS 27, macOS 27, Xcode 27, and the `StateReporter` API described later are in beta. This article reflects information as of August 2026, and APIs, commands, settings screens, and behavior may change before the official release.

17.3 Metal Performance HUD

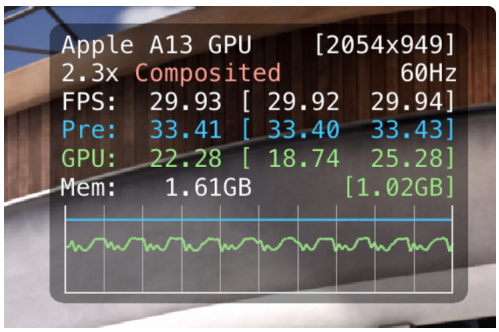
The Metal Performance HUD is a tool that overlays performance information of an application that uses Metal on the screen in real time, or outputs it as a log. Apple's documentation [*2] describes it as a feature for quickly grasping the current state and finding the range to capture in detail with Xcode or Instruments.

This feature itself existed before Metal 4 (the iOS 16 generation), but it was signif-

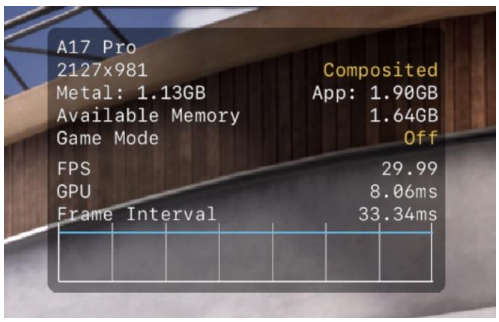
[*1] MetalPerfKit-Unity `https://github.com/mao-test-h/MetalPerfKit-Unity`

[*2] Monitoring your Metal app' s graphics performance `https://developer.apple.com/documentation/xcode/monitoring-your-metal-apps-graphics-performance`

icantly enhanced along with Metal 4 (the iOS 26 generation) announced at WWDC 2025. Figure 17.1 and Figure 17.2 show the difference in display before and after Metal 4.



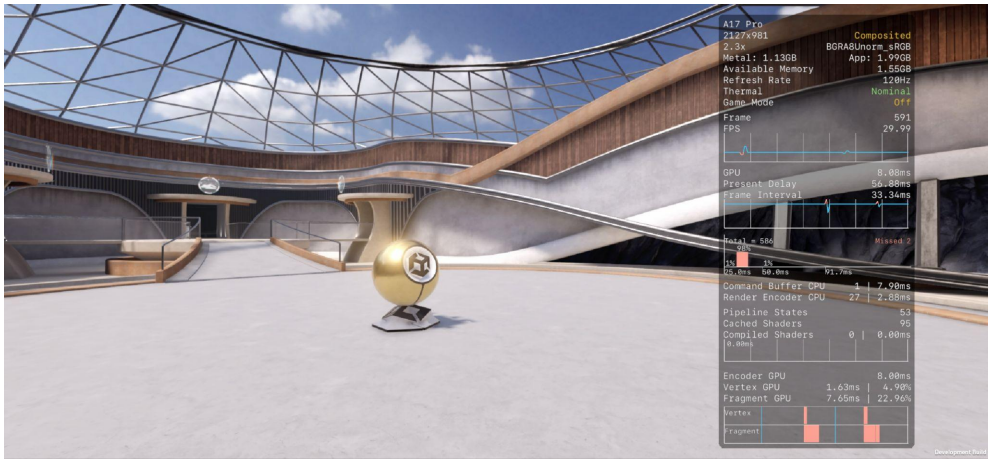
▲Figure 17.1 Metal Performance HUD before Metal 4



▲Figure 17.2 Metal Performance HUD in Metal 4 and later

The main enhancements since Metal 4 include more measurement items, customization of display position, opacity, and scale, and more items that can be controlled dynamically from the API.

Figure 17.3 shows an example with the main metrics displayed.



▲Figure 17.3　Example with the main metrics displayed

For details on each metric, see the official documentation "Understanding the Metal Performance HUD metrics" [*3].

The author has also published an article [*4] about the Metal Performance HUD and the technical doujinshi "UniTips Vol.12" [*5]. They cover everything from basic usage to techniques for using it with Unity, so refer to them for details.

The following sections are based on Metal 4 (the iOS 26 generation).

17.3.1　Opening the HUD

There are several ways to display the HUD. This section explains two: "opening it from the Settings app" and "controlling its display through the API."

[*3] Understanding the Metal Performance HUD metrics `https://developer.apple.com/documentation/xcode/understanding-metal-performance-hud-metrics`

[*4] 【Unity】Measuring GPU load on a mobile app `https://qiita.com/mao_/items/3b531d17cacb86036b7f`

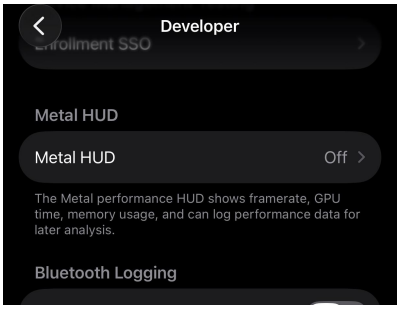
[*5] UniTips Vol.12 (explained in "Chapter 2: Using the Metal Performance HUD Enhanced Since Metal 4 with Unity") `https://techbookfest.org/product/8mHpXz8LJCKAsLxENJKwaz?productVariantID=ewK5GtNTjkz4k8heM4BrxU`

Opening from the Settings App

You can enable it from the Settings app on the iOS device. The steps are as follows.

Note: As a prerequisite, the `Developer` menu must be displayed. Depending on the OS version, the menu labels may differ.

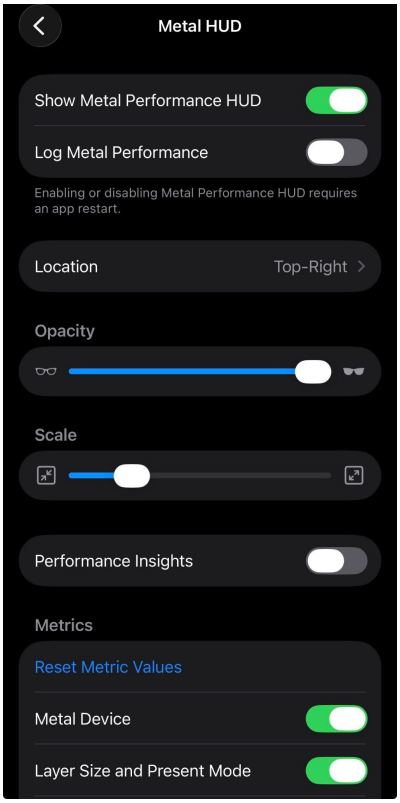
1. Open the iOS Settings app
2. Open the `Developer` menu and open `Metal HUD` within it (see Figure 17.4)



▲Figure 17.4 Opening Metal HUD from the Developer menu

When you open `Metal HUD`, the Figure 17.5 screen appears. Enable `Show Metal Performance HUD` there and restart the target app to display the HUD.

On this screen you can also switch the displayed metrics and change the opacity, scale, and display position. If you enable `Log Metal Performance` in the same section, performance information is output as a log at regular intervals. Details are explained in "About the Log Feature" later.



▲Figure 17.5　Metal HUD settings screen

Controlling Display with the API

To control the HUD display dynamically while the app is running, pass a setting value to the `developerHUDProperties` property of the `CAMetalLayer` class. List 17.1 enables the HUD and log output and places the HUD at the top left.

▼List 17.1　Controlling HUD display with developerHUDProperties

```
myMetalLayer.developerHUDProperties = [
   "mode": "default",      // HUD ON/OFF
   "logging": "default",   // Log feature ON/OFF
   "positionX": 0,         // Display position (X coordinate in pixels)
   "positionY": 0,         // Display position (Y coordinate in pixels)
]
```

To hide the HUD, set `disabled` to `mode`. Previously, this property could only control the HUD and log feature ON/OFF and could not adjust the position, but since Metal 4, you can specify the position in pixels with `positionX` / `positionY`.

You can also pass environment variable keys supported by the HUD in this Dictionary. For example, passing `MTL_HUD_OPACITY` and `MTL_HUD_SCALE` adjusts the opacity and scale in the range 0.0 to 1.0 (List 17.2).

▼List 17.2 Adjusting opacity and scale with environment variable keys

```
myMetalLayer.developerHUDProperties = [
   ...,
   "MTL_HUD_OPACITY": opacity,     // Opacity (0.0 to 1.0)
   "MTL_HUD_SCALE": scale,         // Scale (0.0 to 1.0)
]
```

Many other items can also be configured. For details, see the official documentation "Customizing the Metal Performance HUD" [*6].

To call this from Unity, obtain the `CAMetalLayer` used for rendering in an iOS native plugin and call it from C# through P/Invoke. `CAMetalLayer` can be obtained via `UnityFramework` as in List 17.3.

▼List 17.3 Obtaining CAMetalLayer via UnityFramework

```
private func getMetalLayer() -> CAMetalLayer? {
   guard let instance = UnityFramework.getInstance(),
         let rootView = instance.appController().rootView,
         let metalLayer = rootView.layer as? CAMetalLayer else {
       return nil
   }
   return metalLayer
}
```

MetalPerfKit-Unity exposes these implementations as the `PerformanceHUDSwitc`

[*6] Customizing the Metal Performance HUD `https://developer.apple.com/documentation/xcode/customizing-metal-performance-hud`

her class, which can be called from C# as in List 17.4. Whereas the native API specifies the position in pixels, PerformanceHUDSwitcher accepts coordinates as relative positions from 0.0 to 1.0 so that it works independently of resolution, and internally converts them into pixel coordinates according to the size of the drawing area.

▼List 17.4 Example call to PerformanceHUDSwitcher

```
// Display the HUD at the top right of the screen (x=1.0, y=0.0)
// (Coordinates are specified as relative positions from 0.0 to 1.0)
PerformanceHUDSwitcher.SetPerformanceHUDVisible(true, 1.0f, 0.0f);
```

For a complete implementation example, see the MetalPerfKit-Unity source code.

17.3.2 About the Log Feature

If you enable the Log Metal Performance described earlier or the logging of the API, performance information is output as a log at 1-second intervals while the app is running. The log is output in the List 17.5 format starting with metal-HUD:, and you can view it in the Xcode console or in macOS Console.app.

▼List 17.5 Output format of the metal-HUD log

```
metal-HUD: <first-frame-number-integer>,<graphics-memory-usage-float>,<process-memory-
    usage-float>,<first-frame-present-interval-float>,<first-frame-gpu-time-float>,...<
    last-frame-present-interval-float>,<last-frame-gpu-time-float>
```

From the beginning, the log records the frame number at which sampling started, the graphics memory usage, and the memory usage of the whole process. After that, pairs of Frame Present Interval and GPU Time for each sampled frame are recorded repeatedly. Figure 17.6 shows an actual output example.



▲Figure 17.6　Log output example of the Metal Performance HUD

If you aggregate the data as comma-separated values, you can later analyze irregular frame intervals, GPU Time, and changes in memory usage. As long as these logs remain in Unified Logging, you can also implement something like "retrieve the logs for the past N seconds and save them to a file" using the `OSLogStore` API. MetalPerfKit-Unity provides this feature as the `PerformanceLogger` class (List 17.6).

▼List 17.6　Retrieving and saving logs with PerformanceLogger

```
// Retrieve the logs for the past 60 seconds and save them to a file
PerformanceLogger.FetchPerformanceLogs(60, FileUtility.GenerateFetchLoggingFilePath());
```

17.4　Performance Trace

Performance Trace is a feature added in iOS 26 that can collect performance data on an iOS device alone, without connecting to a Mac. The collected trace files can be transferred to a Mac and analyzed with Instruments or the `metalperftrace` command described later.

Because everything is completed on the device itself, you can record situations that are hard to reproduce while connected to a Mac, such as long play sessions. You can also have someone else, such as a tester, share a trace file collected on their device and analyze it locally.

Among these, this chapter mainly introduces Lookback Collection, which was ex-

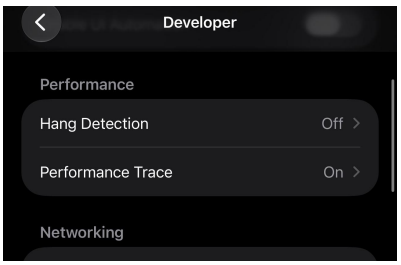
plained along with its enhancements at WWDC 2026.

17.4.1 Usage

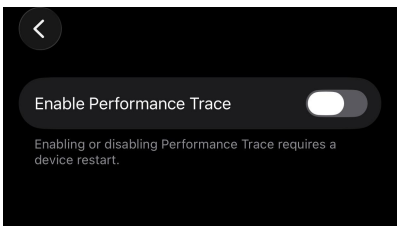
Performance Trace must be enabled from the `Developer` menu. The steps are as follows.

1. Open the iOS Settings app
2. Open the `Developer` menu and open `Performance Trace` within it
3. Enable `Enable Performance Trace` (switching between enabled and disabled requires restarting the device)
4. Select any `Tracing Mode` (details later)

Figure 17.7 and Figure 17.8 show `Performance Trace` in the `Developer` menu and its settings screen, respectively.



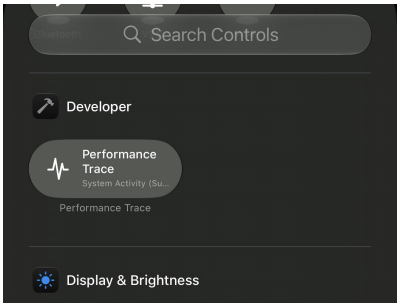
▲Figure 17.7 Performance Trace in the Developer menu



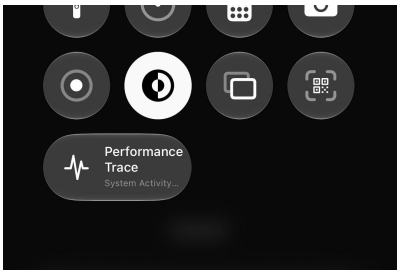
▲Figure 17.8 Enable Performance Trace settings screen

Once it is enabled, add `Performance Trace` to Control Center so that you can start and stop traces and collect trace data. Open Control Center and add it from the `D`

eveloper section. Figure 17.9 and Figure 17.10 show the steps and the state after adding it.



▲Figure 17.9 Adding Performance Trace to Control Center



▲Figure 17.10 State after adding the Performance Trace control

17.4.2 About Tracing Mode

Tracing Mode has the four kinds shown in Table 17.1 (System Activity has two kinds, Summary and Detailed), and you can also switch between them by long-pressing the Performance Trace control in Control Center.

Table 17.1: Kinds of Tracing Mode

Mode	Overview
Power Profiler	Records the impact on power consumption
System Activity (Summary / Detailed)	Records system activity

Table 17.1: Kinds of Tracing Mode

Mode	Overview
Lookback Collection	Collects, retroactively, the data that the system continuously records

For Power Profiler and System Activity, tap the `Performance Trace` control in Control Center to start recording and tap it again to stop. Lookback Collection, on the other hand, requires no recording operation and collects data going back in time from the moment you tap. Details are described later.

Collected trace files are listed in `Available Trace Files` on the Performance Trace screen, and you can transfer them to a Mac or other developers with AirDrop or similar via the share button. Extracting a shared archive gives you a `.atrc` file that can be opened in Instruments. Trace files are deleted automatically when the device's remaining storage becomes low, and can also be deleted manually by swiping.

Additional notes on each mode are as follows.

Power Profiler

Power Profiler is a mode that measures the impact (Power Impact) that the app's use of subsystems such as CPU, GPU, display, and network has on the device's power consumption. This article omits the details, but they are explained in the official documentation "Measuring your app's power use with Power Profiler" [*7] and the WWDC 2025 session "Profile and optimize power usage in your app" [*8].

System Activity

System Activity is a trace mode that comes in two kinds, "Summary" and "Detailed." As far as the author could confirm, at the time of writing (August 2026), Apple's documentation and WWDC sessions contain no explanation of the detailed specifications of System Activity or of the difference between Summary and Detailed.

[*7] Measuring your app's power use with Power Profiler `https://developer.apple.com/documentation/xcode/measuring-your-app-s-power-use-with-power-profiler`

[*8] WWDC 2025 - Profile and optimize power usage in your app `https://developer.apple.com/videos/play/wwdc2025/226/`

Therefore, this article does not go into the details of each mode and introduces only the information confirmed in actual traces.

For reference, List 17.7 shows the result of recording a System Activity (Summary) trace on a device running iOS 27 and analyzing it with the `metalperftrace` command described later.

▼List 17.7 Analysis result of a System Activity (Summary) trace

```
$ metalperftrace overview system_activity_summary_ios27.atrc
[MetalPerfKit-Unity pid:1974]
   Mem: 799.0 MiB (799.0 Peak, 616.8 Metal, 2577.0 Headroom)
   Total CPU Time: 12.679s (20.47% Sys, 79.53% User)
   Instructions: 43.33B (60.88% P, 39.12% E)
   Cycles: 21.04B (38.97% P, 61.03% E)
   Disk Reads / Writes: 9.13 / 0.08 MiB (Logical Write 0.07)
   [States]
     com.DefaultCompany.MetalPerfKit-Unity.graphics: 3 transitions, last known state:
    URP-HighFidelity
     com.DefaultCompany.MetalPerfKit-Unity.screen: 11 transitions, last known state:
    Title
 UnityView (UnityDefaultViewController) (1290x2796)  Interval 30.037s  Active 30.037s
   44.5 FPS  1202 Frames  1202 Detached  11 Skipped
   Frame Time           avg:  22.47ms  min:  16.67  max: 420.93  stddev:  30.68  p95:
    50.01  p99: 133.36
   CPU Begin-to-Present avg:  17.24ms  min:   1.14  max: 794.63  stddev:  31.76  p95:
    31.28  p99: 111.73
   On-GPU Time          avg:  15.50ms  min:   1.81  max: 424.87  stddev:  37.24  p95:
    51.17  p99: 134.48
   Next Drawable Wait   avg:  14.53ms  min:   0.00  max: 401.43  stddev:  32.11
```

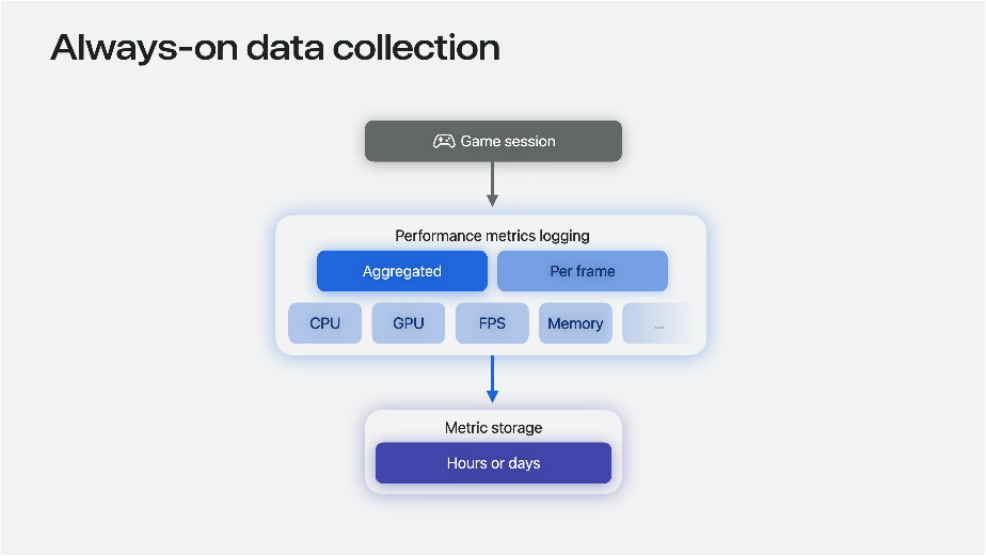
In this trace, in addition to per-process memory usage, CPU time, and disk I/O, we could see statistics such as FPS, Frame Time, CPU Begin-to-Present, and On-GPU Time per `CAMetalLayer`. The `[States]` item records the app's state transitions, but these come from the `StateReporter` API explained in the next chapter, so details are described later.

Lookback Collection

Lookback Collection is a mode that retroactively collects the performance data that the system continuously records. As far as the author could confirm, Lookback Collection already existed as an option of `Tracing Mode` as of iOS 26, but it was not

explained at WWDC 2025 or in the official documentation. Its capabilities were then enhanced at WWDC 2026, and the session video "Find and fix performance issues in your Metal games" [*9], which explains the enhancements, was published.

According to the talk, while a game is running, the system continuously records performance metrics (CPU, GPU, FPS, memory, and so on) and keeps that data for several days. Because Lookback Collection uses this mechanism, it can retrieve data retroactively even for a long play session by a tester. Figure 17.11 shows an overview of this mechanism.

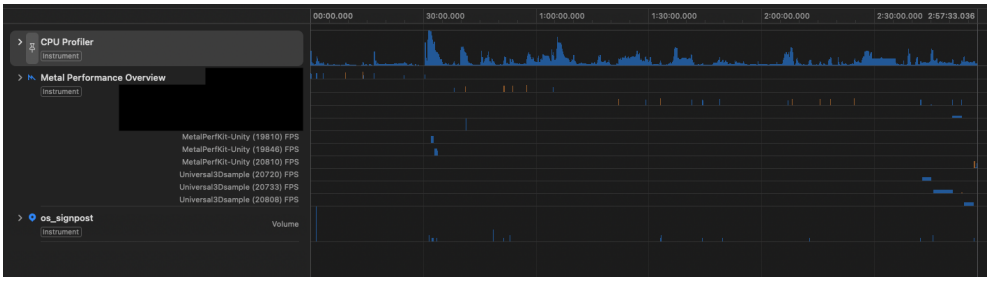


▲Figure 17.11 Continuous recording of performance metrics by the system (from the WWDC 2026 session)

To use it, play-test as usual and then simply tap the `Performance Trace` control in Control Center. Collection and processing of a trace going back the configured amount of time from the moment of the tap begins, and when it completes, a notification arrives and the trace is added to `Available Trace Files`.

[*9] WWDC 2026 - Find and fix performance issues in your Metal games `https://developer.apple.com/videos/play/wwdc2026/388/`

Figure 17.12 shows an example in which a Lookback Collection trace covering about 3 hours was collected on a device running iOS 26 and opened in Instruments. The multiple apps (processes) that were running during the period covered are listed, and you can get an overview of the changes in FPS and other metrics for each.



▲Figure 17.12　Example of a Lookback Collection trace opened in Instruments

Expanding a process further, as in Figure 17.13, you can see how frame metrics such as FPS, GPU Usage %, Frame Interval, and CPU / GPU time change per `CAMetalLayer`.



▲Figure 17.13　Changes in frame metrics per CAMetalLayer

Note: In my iOS 27 environment, `Error processing Performance Trace` was displayed during aggregation and it did not work correctly. Regarding this issue, even at the official release of iOS / iPadOS 27, the release notes in `Control Center -> Known Issues` list a known issue (184729556): "Collecting Power Profiler or Lookback traces may not generate a trace." [*10]

17.5 New Features Added in macOS 27 / iOS 27

As introduced in the previous chapter, the system continuously records and stores Metal performance metrics while a game is running. Built around this mechanism, WWDC 2026 announced the `StateReporter` API, which attaches app-specific information to traces, and the `metalperftrace` command, which analyzes traces from the CLI.

17.5.1 StateReporter API

Even if you can identify the time at which the frame rate dropped in the trace data, Metal performance metrics alone do not tell you what the game was doing at that moment. This is because app-specific information, such as which scene of the game is open, how many objects there are, and what the graphics settings are, is not automatically included in Metal performance metrics.

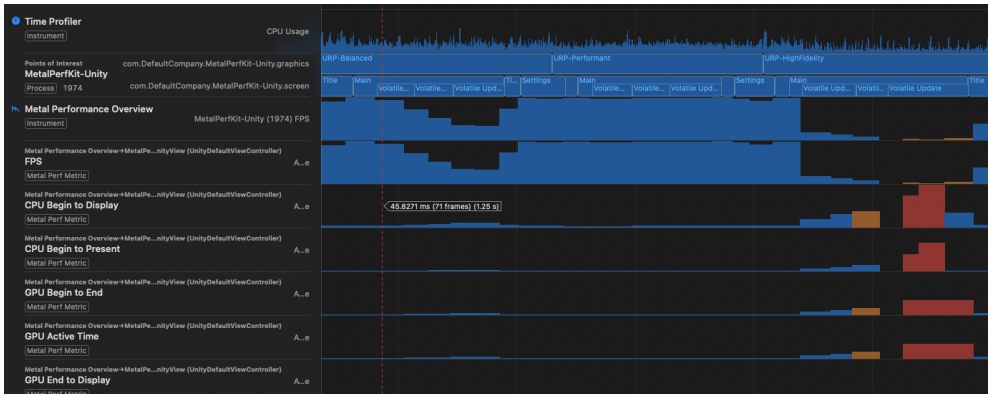
Using the `StateReporter` API [*11] of the `StateReporting` framework [*12] added in iOS 27, you can report the app's state to the system and attach app-specific context to the trace data.

The reported state is used for per-state aggregation in `metalperftrace` and for displaying Points of Interest in Instruments. Figure 17.14 shows the System Activity trace from the previous chapter opened in Instruments. The reported states appear as a Points of Interest track.

[*10] iOS & iPadOS 27 Release Notes `https://developer.apple.com/documentation/ios-ipados-release-notes/ios-ipados-27-release-notes`

[*11] StateReporter `https://developer.apple.com/documentation/statereporting/statereporter`

[*12] Getting started with StateReporting `https://developer.apple.com/documentation/StateReporting/getting-started-with-statereporting`



▲Figure 17.14 Example of reported states displayed as Points of Interest

`StateReporter` consists of the four elements shown in Table 17.2.

Table 17.2: The four elements of StateReporter

Element	Overview	Example
Domain	The functional area to report. A reverse-DNS string; only one state can be active at a time per domain	`com.mygame.level`
Label	A short string indicating the current state	`Level 1`
Stable metadata	Metadata that does not change for the lifetime of the state. Combined with the Label, it uniquely identifies the state	`{id: 1001}`
Volatile metadata	Volatile metadata that can change within a state. Updati ng it does not cause a state t ransition	`{health: 20}`

You can define multiple domains. For example, you can report "level progress," "graphics settings," and "network state" as independent domains and analyze them in combination.

In Swift, you use it as in List 17.8.

▼List 17.8　Example of using StateReporter from Swift

```
import StateReporting
// Get the Reporter for the domain
let reporter = StateReporter.reporter(for: "com.mygame.gameplay")
// Report a state transition
reporter.reportTransition(to: "Playing")
// Pass nil to end the state
reporter.reportTransition(to: nil)
```

Using It from Unity

Because `StateReporting` is an API for Swift / Objective-C, a native plugin is required to use it from Unity. MetalPerfKit-Unity provides a class `StateReporter` that wraps it as a preview feature, which can be used from C# as in List 17.9.

▼List 17.9　Example of using StateReporter from C#

```
using MetalPerfKit;
// Get the Reporter for the domain
var reporter = StateReporter.ReporterForDomain($"{Application.identifier}.screen");
// Report a state transition (with volatile metadata)
reporter.ReportTransitionToStateLabel(
   "Main",
   volatileMetadata: new Dictionary<string, StateReporterMetadataValue>
   {
       ["CubeCount"] = cubeCount,
   });
// Keep the state and update only the volatile metadata
reporter.ReportVolatileMetadataUpdate(
   new Dictionary<string, StateReporterMetadataValue>
   {
       ["CubeCount"] = cubeCount,
   });
```

The repository's `Assets/_Example/2_MetalPerformanceMetrics` also includes a sample that reports screen transitions (Title / Main / Settings) and graphics quality (three URP levels) as independent domains. The `[States]` in the System Activity trace shown in the previous chapter was recorded with this sample.

17.5.2 metalperftrace

`metalperftrace` is a CLI tool added in macOS 27 that can analyze trace files and collect performance metrics of macOS apps. The collection feature targets apps running on macOS and is not covered in this article, but you can use this tool to analyze trace files collected on iOS devices.

Analyzing Trace Files

Use the `overview` subcommand to analyze trace files (List 17.10).

▼List 17.10 Analyzing a trace file with the overview subcommand

```
# Analyze a collected trace file
metalperftrace overview MyGameTrace.atrc
```

The output is as introduced in the previous chapter. If you also pass `--include-state-transitions`, you can see the details of the state transitions recorded with `StateReporter`. List 17.11 is an example of analyzing the trace of the sample app described earlier, and it outputs the history of changes to each graphics quality together with the time and percentage spent in each state.

▼List 17.11 Analysis result including state transition details

```
$ metalperftrace overview --include-state-transitions system_activity_summary_ios27.
    atrc
[MetalPerfKit-Unity pid:1974]
   ...
   [States]
     com.DefaultCompany.MetalPerfKit-Unity.graphics
       URP-Balanced  (32.75%, 9.904s)
         (2026-08-16T14:22:33.025Z → 2026-08-16T14:22:42.930Z)
       URP-Performant  (29.43%, 8.900s)
         ...
       URP-HighFidelity  (37.82%, 11.437s)
         ...
     com.DefaultCompany.MetalPerfKit-Unity.screen
       Title  (5.07%, 1.535s)
         ...
       Main  (21.09%, 6.379s)
         ...
```

```
      Title  (2.22%, 0.671s)
        ...
      Settings  (6.28%, 1.899s)
        ...
```

Note: The output is partially excerpted and formatted. In practice, the same state is also output repeatedly in chronological order, each with start and end timestamps.

Passing `--aggregate` also lets you aggregate metrics per state. Specifying `--domain com.mygame.graphics` narrows the result to a specific domain, and combining it with `--state-label "High"` narrows it down to a specific state. You can check the correlation between app state and performance, such as "the average FPS drops only when the graphics setting is High," with a single command.

These outputs can also be obtained in JSON format by passing `--json`. The talk also stated that "structured output can be used for regression tests with scripts and for automatic triage of performance issues by AI agents." Apple's "Metal Developer Tools" page [*13] also introduces a workflow combined with agents under the title "Hello, agentic debugging."

[*13] Metal Developer Tools `https://developer.apple.com/metal/tools/`

Chapter 18

Switching Rendering Settings Per Device

NAOKI SAKUMA

CyberAgent, Inc. SGE Core Technology Division

Mobile games are expected to be played on a wide range of devices. Applying the same rendering settings uniformly can cause crashes, dropped frames, and heat generation on low-spec devices. Conversely, high-spec devices may be able to deliver better quality by making use of their performance. This section introduces a mechanism for switching to rendering settings appropriate for each device.

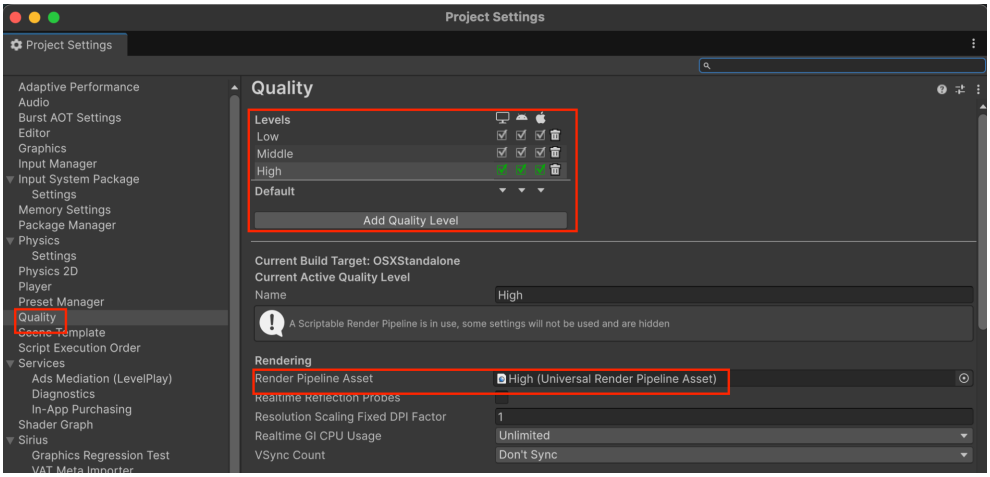
18.1 Switching Methods

Two approaches are common in games: a preset format such as low, medium, and high, and a format in which the user directly adjusts values.

18.1.1 Preset Format

This method is for switching in units of presets. It is used both when letting the user choose a preset and when determining one automatically from the device.

In ProjectSettings > Quality, click Add Quality Level and add as many levels as there are presets. Then attach the RenderPipelineAsset corresponding to each preset. If needed, you can also configure the other Quality items (Rendering, Textures, and so on).



In code, calling QualitySettings.SetQualityLevel switches to any QualityLevel.

18.1.2 Direct Value Manipulation Format

Use this method when letting the user adjust parameters. Parameters can be manipulated directly with code such as the following.

```
var urpAsset = (UniversalRenderPipelineAsset)GraphicsSettings.currentRenderPipeline;
// Changing the resolution
urpAsset.renderScale = value;
// Changing the number of shadow cascades
urpAsset.shadowCascadeCount = value;
// Others include the shadow resolution and the number of additional lights
```

18.2 Switching Per Device

There may be cases where you want to recommend or automatically set a preset depending on the device. In that case, the SystemInfo type is useful.

In particular, deviceModel, graphicsDeviceName, and systemMemorySize return values that are easy to use for identification. Examples of the returned values are shown below.

Table 18.1: Examples of values returned by SystemInfo for each device

Model	deviceModel	graphicsDeviceName	systemMemorySize
iPhone 13	iPhone14,5	Apple A15 GPU	about 3800
iPad Pro 11-inch (3rd generation)	iPad13,4	Apple M1 GPU	about 7800
Galaxy S23	samsung SM-S911B	Adreno (TM) 740	about 7500
Pixel 8	Google Pixel 8	Mali-G715	about 7400

On iOS, the naming conventions of deviceModel and graphicsDeviceName are consistent, and within the same series, a larger number after A or M generally means higher specs. Combining them therefore allows devices to be classified to a certain degree.

On Android, the returned values vary by model, so it is difficult to compare them uniformly as on iOS. Therefore, it is good to classify devices by approaches such as the following:

- Decide by RAM alone
- Move the data to a master table and compare against it
- Run a benchmark tool at startup

and so on.

18.3 Custom RendererFeature

When you create a custom RendererFeature or similar, it is also useful to provide an API for switching rendering settings. Examples include:

- Lowering the format or resolution of temporary textures
- Reducing the number of Blits
- Replacing with a simpler shader

and similar measures.

Chapter 19

UniTask and Lifetime Management of Cancellation

Inada Taketo

CyberAgent, Inc. SGE Core Technology Division

Name：Inada Taketo

Affiliation：CyberAgent, Inc. SGE Core Technology Division

X account / Website / GitHub account (optional):

In asynchronous processing with UniTask, if termination conditions and cancellation are not managed properly, operations that never complete and unnecessary references can remain.

The previous book, "Unity Performance Tuning Bible", introduced a case in which UniTask keeps waiting even after the MonoBehaviour is destroyed. This section introduces a case in which unnecessary objects keep being referenced because a CancellationTokenSource is not disposed.

Note that CancellationTokenSource and CreateLinkedTokenSource are not Uni Task-specific features but APIs that make up cancellation handling in .NET.

Among the code covered in this section, the using declaration (using var) is C# 8 syntax and is available in Unity 2020.2 and later. In addition, the internal implementation of CreateLinkedTokenSource described later is based on the Mono imple-

mentation for Unity 2022.3 published by Unity Technologies (the unity-2022.3-mbe branch) and may vary depending on the Unity version.

19.1 Disposing a CancellationTokenSource

CancellationToken is a value type for conveying a cancellation request to an operation. CancellationToken itself does not implement IDisposable, so there is no need to call Dispose.

However, when a callback is registered with CancellationToken.Register, the returned CancellationTokenRegistration is IDisposable[*1]. A callback registered before cancellation is held by the originating CancellationTokenSource, so unless you Dispose the registration to unregister it when it is no longer needed, the objects referenced by the callback may not be released.

On the other hand, CancellationTokenSource implements IDisposable. When a CancellationTokenSource is no longer in use, its owner calls Dispose.

▼List 19.1 Example of disposing a CancellationTokenSource

```
private async UniTask LoadCancelableAsync()
{
    using var cts = new CancellationTokenSource();

    // The code that issues the cancellation request is omitted
    await LoadAsync(cts.Token);
}
```

In List 19.1, the CancellationTokenSource is managed with a using declaration. cts.Dispose() is called automatically when the method exits, not only when the asynchronous operation completes normally but also when it ends through cancellation

*1

or an exception.

Cancel and Dispose have different roles. Cancel requests cancellation of the operation, while Dispose unregisters and releases the timers, callback registrations, and so on held by the CancellationTokenSource. Calling Dispose alone does not notify the operation of a cancellation request.

Note that even if you forget to call Dispose, the CancellationTokenSource and its related objects become eligible for garbage collection once none of them is referenced from outside[*2]. However, references can remain, for example through callback registrations on a long-lived CancellationTokenSource, so as a rule, Dispose it as soon as you finish using it.

19.2 CancellationTokenSource.CreateLinkedTokenSource

CancellationTokenSource.CreateLinkedTokenSource creates a new CancellationTokenSource that monitors multiple source tokens. When any of the source tokens is canceled, the Token of the created CancellationTokenSource enters the canceled state.

The source tokens are not canceled by one another; cancellation propagates from the source tokens to the linked source.

In the following, let A and B be the CancellationTokenSources that issue tokenA and tokenB, respectively.

In the Mono implementation for Unity 2022.3 published by Unity Technologies on GitHub[*3], CreateLinkedTokenSource registers a cancellation callback on the source tokens. Simplifying the parts relevant to the reference relationships gives List 19.2.

▼List 19.2 Key points of the internal implementation of CreateLinkedTokenSource

*2

*3

```
linkedCts.m_linkingRegistrations[0] =
    tokenA.InternalRegisterWithoutEC(
        s_LinkedTokenCancelDelegate,
        linkedCts);

linkedCts.m_linkingRegistrations[1] =
    tokenB.InternalRegisterWithoutEC(
        s_LinkedTokenCancelDelegate,
        linkedCts);
```

linkedCts is referenced by the callbacks registered on the source tokens, and linked Cts also holds the registration information for A and B.

Therefore, if B is a long-lived CancellationTokenSource used throughout the application and is not canceled for a long time, the following references can remain unless linkedCts is disposed.

Long-lived B → callback registration → linkedCts → registration information for A → short-lived A

This reference relationship alone does not necessarily cause a memory leak. Once none of the objects is referenced from outside, they all become eligible for garbage collection together. However, while B is alive and the linking callback has not been unregistered, linkedCts continues to be referenced from B, and the short-lived A remains reachable through it.

In addition, if the callbacks registered on linkedCts.Token reference other objects, those objects may also be retained.

When linkedCts is disposed[*4], the CancellationTokenRegistrations for A and B are disposed internally, and the callbacks registered on the source tokens are unregistered.

It is good to manage the return value of CreateLinkedTokenSource with a using declaration or similar, within the scope of the operation that uses the linked token.

▼List 19.3　Example of disposing a linked CancellationTokenSource

```
private async UniTask LoadAsync(
    CancellationToken tokenA,
    CancellationToken tokenB)
{
    using var linkedCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            tokenA,
            tokenB);

    await LoadAssetAsync(linkedCts.Token);
}
```

In List 19.3, linkedCts.Dispose() is called after LoadAssetAsync ends through normal completion, cancellation, or an exception, and the registrations on the source tokens are removed.

Even if a linked CancellationTokenSource is already in the canceled state, registrations on other source tokens may remain. Always Dispose the return value of CreateLinkedTokenSource as soon as it is no longer needed.

*4

19.3 Preventing Missed Dispose Calls on IDisposable Objects as a Team

So far, we have looked at missed disposal of IDisposable objects using CancellationTokenSource as an example. The countermeasures assume that the owner of an object calls Dispose at the appropriate time. However, as a project grows, it becomes difficult to keep checking every creation site through code review alone. In particular, CreateLinkedTokenSource can create an object with very little code, which makes it easy to overlook that the return value is IDisposable.

To prevent recurrence as a team, in addition to individual attention, it is good to combine steering objects that require disposal toward easy-to-use APIs, detecting problems early with an Analyzer, and, where necessary, detecting new violations in CI. Rather than concentrating all responsibility on the Analyzer, it is effective to support the same policy from design, static analysis, and operations.

The disposal with using and the detection with an Analyzer introduced here are not limited to CancellationTokenSource and can also be applied to other resources that implement IDisposable.

Among such mechanisms, this section introduces a method that uses a Roslyn Analyzer, which can automatically inspect C# code.

19.4 What Is a Roslyn Analyzer

A Roslyn Analyzer is a mechanism that statically analyzes C# code at compile time or in the IDE and displays warnings or suggestions at locations that may be problematic. Based on syntax, type information, execution paths, and so on, it can detect signs of bugs and violations of coding conventions, and if a Code Fix is provided, it can also suggest fixes.

What an Analyzer detects as a problem is defined as a diagnostic rule, and each rule is assigned an identifying diagnostic ID such as CA2000. CA2000 is an existing rule provided by .NET, while UTK001 and UTK002, described later, are proposed

rules to be added by the project itself.

In a Unity project, an Analyzer is introduced by placing its DLL under Assets and assigning the RoslynAnalyzer label[*5]. You can also use an asmdef to limit the assemblies to which it applies.

19.5 Using an Existing Analyzer as the First Line of Defense

The first thing to consider is CA2000, included in the .NET code quality rules[*6]. CA2000 detects paths in which a locally created IDisposable object goes out of scope without being disposed. Because CancellationTokenSource is also IDisposable, it can serve as the first line of defense for detecting a missed disposal of new CancellationTokenSource().

However, depending on the Analyzer version and analysis settings, the extent to which it can track the return value of a method that creates and returns an object, such as CreateLinkedTokenSource, and the transfer of ownership between methods varies. When adopting it, it is good to check, in a small verification project, both detected and undetected examples for new CancellationTokenSource() and Create LinkedTokenSource. Patterns that existing rules cannot detect sufficiently can be supplemented with the custom rules described later, as needed.

19.6 Example Custom Rule Design for Asynchronous Processing

If an existing Analyzer cannot sufficiently track the return value of CreateLinked TokenSource, one option is to supplement it with a project-specific Roslyn Analyzer. The following is an example of rule design when implementing a custom Analyzer. For illustration, UTK001 and UTK002 are used as provisional diagnostic IDs.

UTK001 is assumed to check whether a local variable created with new

[*5]
[*6]

CancellationTokenSource() or CancellationTokenSource.CreateLinkedToken Source() is protected by a using statement, a using declaration, or a Dispose in a finally clause. The analysis covers not only all normal completion paths but also paths that exit the method through return or an exception.

▼List 19.4 Example detected by UTK001

```
private async UniTask LoadAsync(
    CancellationToken tokenA,
    CancellationToken tokenB)
{
    var linkedCts =
        CancellationTokenSource.CreateLinkedTokenSource(
            tokenA,
            tokenB);

    await LoadAssetAsync(linkedCts.Token);
}
```

In List 19.4, the method exits without linkedCts being disposed. This rule is assumed to issue a warning at the CreateLinkedTokenSource call site and guide the developer to guarantee disposal with a using declaration or try-finally.

UTK002 is assumed to check whether the CancellationTokenRegistration returned by CancellationToken.Register is left undisposed. Code that discards the return value of Register leaves the responsibility for unregistering unclear. If the callback should be valid only during the asynchronous operation, it is good to manage it with a using declaration. If it is held in a field, Dispose it in the owner's termination process.

▼List 19.5 Example of managing a CancellationTokenRegistration

```
private async UniTask WaitAsync(CancellationToken token)

{

    using var registration = token.Register(OnCanceled);



    await WaitForSignalAsync(token);

}
```

In List 19.5, registration is disposed when WaitForSignalAsync finishes. On the other hand, some designs match the lifetime of the registration to that of the Token, such as permanently registering on a Token shared across the whole application. Warning on such cases uniformly increases false positives, so it is good to try UTK002 at the information level first and make it possible to exclude safe patterns that appear frequently in the project.

A CancellationTokenSource held in a field makes ownership harder to determine than a local variable. As a second-stage rule, you can check whether the old value is disposed before a new CancellationTokenSource is assigned to the field, and whether it is disposed in the owner's termination process, such as OnDestroy of a MonoBehaviour. However, ownership may be transferred to another object, so it is good to introduce this separately from the rule for local variables.

19.7 Standardizing Fixes with Code Fix

For a Roslyn Analyzer, you can also provide a Code Fix that automatically fixes detected problems in the IDE. For example, for the simple case of UTK001 in which ownership is entirely contained within the method, providing a Code Fix that converts var to using var makes it easier to standardize within the team how patterns that can be safely converted mechanically are fixed. For cases in which the disposal timing cannot be determined uniquely, such as assignment to a field or transfer of ownership through a return value, it is good not to offer an automatic fix and to keep to a diagnostic that prompts a design review.

In code review, it is good to check not only the Analyzer results but also whether

the responsibilities for creation, Cancel, and Dispose can be read from the code. Note also that the absence of Analyzer warnings alone does not prove safety.

19.8 Problems an Analyzer Alone Cannot Detect

Static analysis has limits. When a CancellationTokenSource is stored in a collection, or when ownership is transferred by passing it to another object, the final disposal location may not be determined accurately. If the analysis is weakened to avoid false positives, missed detections increase instead.

This area can be supplemented by runtime verification in a Development Build or a test environment. If CancellationTokenSources are created through a project-wide creation mechanism or a type that manages lifetime, and the numbers created and disposed are recorded, you can check at scene transitions or at the end of tests whether any undisposed scopes remain. If stack traces at creation are also recorded, it is good to enable this only in development environments, considering the load.

As needed, it is good to combine an investigation that checks memory growth with the Profiler, runtime detection of undisposed objects, and edit-time warnings from a Roslyn Analyzer. Rather than relying only on a flow in which a person investigates after a problem occurs, a flow that can suggest even the fix at the time the code is written helps prevent recurrence across the team.

19.9 References

Unity Manual "Roslyn analyzers and source generators"

Microsoft Learn "CA2000: Dispose objects before losing scope"

Unity Technologies "Mono CancellationTokenSource.cs (unity-2022.3-mbe branch)"

Microsoft Learn『CancellationTokenSource.Dispose Method』

Microsoft Learn『CancellationTokenRegistration.Dispose Method』

Unity Manual "Managed memory"

https://docs.unity3d.com/ja/2022.3/Manual/roslyn-analyzers.html (Accessed: September 8, 2026)

https://learn.microsoft.com/ja-jp/dotnet/fundamentals/code-analysis/quality-rules/ca2000 (Accessed: September 8, 2026)

https://github.com/Unity-Technologies/mono/blob/unity-2022.3-mbe/mcs/class/referencesource/mscorlib/system/threading/CancellationTokenSource.cs (Accessed: September 8, 2026)

https://learn.microsoft.com/ja-jp/dotnet/api/system.threading.cancellationtokensource.dispose (Accessed: September 8, 2026)

https://learn.microsoft.com/ja-jp/dotnet/api/system.threading.cancellationtokenregistration.dispose (Accessed: September 8, 2026)

https://docs.unity3d.com/ja/2022.3/Manual/performance-managed-memory.html (Accessed: September 8, 2026)

Chapter 20

Lifetime Management of AssetBundles

Inada Taketo

CyberAgent, Inc. SGE Core Technology Division

Name：Inada Taketo Affiliation：CyberAgent, Inc. SGE Core Technology Division X account / Website / GitHub account (optional):

Games load many assets at runtime, such as textures, meshes, and audio. If each implementer manages them individually, missed releases and unnecessary assets staying resident are likely to occur.

Therefore, as a rule, consolidate the entry points for loading and releasing assets that are loaded dynamically from AssetBundles, Addressables, and similar sources into the system foundation. Determine the asset's lifetime at load time, and build a mechanism in which the foundation releases the assets together when that lifetime ends.

In this section, "release" means releasing the references and release handles managed by the loading foundation. Whether an asset is actually unloaded from memory depends on the implementation of the foundation, the reference counts, and the state of dependencies.

Note that the classifications and implementation methods introduced below are examples. Choose the approach that fits the game structure of each project.

20.1 Managing Asset Lifetimes Separately

First, organize the periods during which assets are needed in the game. For example, they can be classified as follows.

Table 20.1: Example classification of asset lifetimes

Lifetime	Example retention period
Common	Until common data is reset, such as at logout, account switching, or return to the title screen
Scene	Until the target scene ends
View	Until the target screen or dialog is closed
Custom	Until a period defined independently, such as a battle or an event, ends

What matters is not names such as Common, Scene, and View, but making clear when each lifetime starts and when it ends.

If the classification is too coarse, unnecessary assets remain for a long time; if it is too fine, the number of loads and releases and the management cost increase. Also, when assets with different lifetimes are included in the same AssetBundle, unloading per AssetBundle cannot select and unload only some of the assets. Design the lifetime classification together with the AssetBundle granularity (Chapter 5, "Tuning Practice - AssetBundle", of the previous book, "Unity Performance Tuning Bible").

20.2 Determining the Lifetime at Load Time

Once the lifetimes are classified, decide the registration destination when loading an asset.

One implementation example is to specify the lifetime as an argument of the load function.

▼List 20.1 Example of loading with a specified lifetime

```
var background =
    await AssetLoader.LoadAsync<GameObject>(
```

```
        address,
        AssetLifetime.Scene);
```

In List 20.1, the loaded asset is registered to the Scene lifetime. When the scene ends, the foundation releases the registered assets together.

As another implementation example, there is also a method of separating the load functions by lifetime.

▼List 20.2 Example of separating load functions by lifetime

```
var commonIcon =
    await AssetLoader.LoadCommonAsync<Sprite>(
        commonAddress);

var stagePrefab =
    await AssetLoader.LoadSceneAsync<GameObject>(
        stageAddress);

var dialogPrefab =
    await AssetLoader.LoadViewAsync<GameObject>(
        dialogAddress,
        viewId);
```

Separating the functions, as in LoadCommonAsync, LoadSceneAsync, and Load ViewAsync, lets you tell the lifetime from the call site and makes it easier to prevent mistakes in specifying the registration destination. On the other hand, specifying it by argument makes it easier to add custom lifetimes.

With either method, what matters is that the lifetime is always determined at load time and that the foundation can release the asset when that lifetime ends.

To handle the case in which the lifetime ends before an asynchronous load completes, track the operation from the start of the load, and either cancel it if the lifetime ends midway or make it possible to release the result after completion. Also, when the same asset is used from multiple lifetimes, manage a reference count and release the asset when the last use ends.

20.3 Tying Lifetimes to a GameObject

As another management method, a GameObject can be made the owner of a lifetime.

Add a dedicated Component to the root GameObject of a screen or dialog and register the release handles of the loaded assets. When the GameObject is destroyed, release the registered handles from OnDestroy.

▼List 20.3 Example of tying a lifetime to a GameObject

```
public sealed class AssetLifetimeScope : MonoBehaviour
{
    private readonly List<IDisposable> _handles =
        new List<IDisposable>();

    private bool _released;

    public void Add(IDisposable handle)
    {
        if (_released)
        {
            handle.Dispose();
            return;
        }

        _handles.Add(handle);
    }

    private void OnDestroy()
    {
        _released = true;

        for (var i = _handles.Count - 1; i >= 0; i--)
        {
            _handles[i].Dispose();
        }

        _handles.Clear();
    }
}
```

In List 20.3, even if an asynchronous load completes and Add is called after the

GameObject has been destroyed, the added handle is released immediately.

However, merely destroying the GameObject does not automatically release dynamically loaded assets. The dedicated Component must call the release process corresponding to the loading foundation.

Do not create a management GameObject for each asset; consolidate them into one per lifetime unit, such as one screen or one dialog. This method is effective when the lifetimes of the GameObject and the assets coincide, but it may not suit assets used across multiple scenes. Use it together with per-lifetime management tables as needed.

20.4 Checking for Missed Releases

Take Memory Profiler snapshots before and after a screen or scene transition and compare them to see whether assets of the ended lifetime remain. It is also effective to repeat the same transition several times and check whether assets or load handles keep increasing.

For measurement, compare states against the same baseline after the necessary release processing has completed, and ultimately verify in a player build on a physical device. However, some assets are referenced by other features or intentionally held as a cache, so do not judge a missed release merely because an asset remains; check the actual referrers.

Changing asset management late in development affects many load processes. Organize the necessary lifetimes early in development, and design a mechanism in which a lifetime suited to the project is applied without implementers having to think about individual release timing.

Chapter 21

Shader Warmup with GraphicsStateCollection

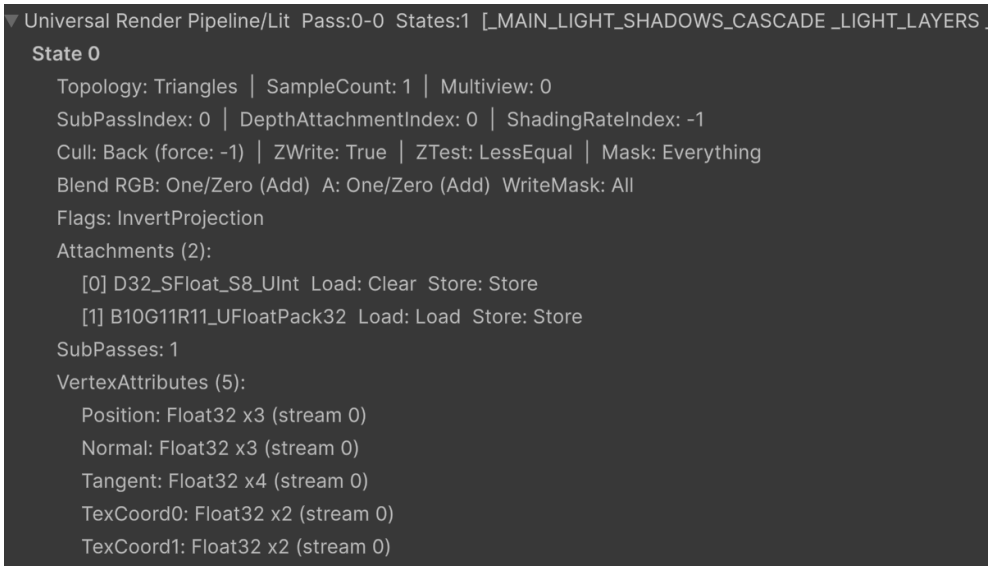
Amano Takuro

CyberAgent, Inc. SGE Core Technology Division

When a shader is used for rendering for the first time, compilation runs at runtime and can cause a hitch known as shader compilation stutter. This chapter covers shader warmup with GraphicsStateCollection on mobile platforms (Metal and Vulkan), from basic usage to real-world cases where stutter remains after adoption and how to address them. The content of this chapter targets Unity 6.3 LTS (6000.3).

21.1 GraphicsStateCollection Overview

GraphicsStateCollection (GSC) is an API available since Unity 6 （出典ID: gsc_docs）. It records the combinations of shader variants and graphics states (rendering settings used at draw time) that are used during gameplay, and precompiles them together, that is, warms them up. It is positioned as a replacement for ShaderVariantCollection on modern graphics APIs such as Metal, Vulkan, and D3D12. Note that as of Unity 6000.3, it is an Experimental API in the `UnityEngine.Experimental.Rendering` namespace, and its specification may change in future versions.



▲Figure 21.1　Example of information recorded in a GSC

The figure shows an example in which the information recorded in a GSC is retrieved by a script and displayed in an EditorWindow. How to retrieve the recorded contents is described at the end of this chapter (List 21.11).

A shader cannot run on the GPU as is; it must be compiled into GPU machine instructions. However, machine instructions differ by GPU model and driver, so shaders cannot be distributed precompiled for all devices. Therefore, a shader goes through the following flow before it runs on the GPU.

1. The developer writes a shader (asset) in a shader language such as HLSL
2. At build time, Unity compiles the shader into an intermediate representation for each graphics API and stores it in the app or AssetBundle. This intermediate representation is SPIR-V for Vulkan and Metal Shading Language (MSL) for Metal （出典ID: gsc_spirv_msl）
3. While the app is running, when the shader is used for rendering for the first time, a shader program is generated from the stored intermediate representation
4. Then, the GPU driver combines the shader program with the rendering set-

tings and compiles it into GPU machine instructions

The basic idea of shader warmup is to complete the runtime compilation in steps 3 and 4 in advance, before the first draw.

A single shader generates multiple executable programs, one for each pass (rendering stage) and for each combination of conditions such as light influence and fog (shader keywords). This unit of generation is called a shader variant.

21.1.1 Pipeline State Object

In modern graphics APIs, the unit compiled at draw time is called a Pipeline State Object (PSO). A PSO is an object in which the GPU driver compiles a shader variant together with rendering settings (vertex layout, render pass attachments such as color format and MSAA, blend state, depth test, and stencil state) into GPU machine instructions. Generated PSOs are cached on the device and reused for the same rendering settings. In other words, even for the same shader variant, different rendering settings are compiled as separate PSOs.

21.1.2 Causes of Shader Compilation Stutter

Stutter is caused by a two-stage compilation process that runs at runtime. When the GPU draws a shader variant for the first time, shader program generation on the Main Thread (step 3) and PSO generation by the GPU driver on the Render Thread (step 4, compilation into machine instructions) occur.

In the Unity Profiler, the former can be observed as `Shader.CreateGPUProgram` on the Main Thread, and the latter as `GpuProgramMetal.GetCachedPipeline` (iOS) or `CreateGraphicsPipelineImpl` (Android) on the Render Thread. PSO generation is especially costly on Metal (iOS) and Vulkan (Android), and is the main cause of stutter at first draw. Warmup using a GSC is a means of completing this PSO generation at any chosen time, such as right after the game starts.

The conventional ShaderVariantCollection can manage which shader variants to compile, but it does not hold rendering settings. Therefore, warmup with Shader VariantCollection stops at shader program generation in step 3, and the GPU driver

generates the PSO again at actual draw time, so stutter remains.



▲Figure 21.2 Example of compilation on the Main Thread and Render Thread

The figure shows a frame in which compilation occurred on Vulkan (Android), displayed in the Profiler Timeline view. The upper part shows `Shader.CreateGPUProgram` selected on the Main Thread, where the target shader name, pass name, and keywords can be checked. The lower part shows `CreateGraphicsPipelineImpl` selected on the Render Thread. In this example, a single PSO generation takes tens of milliseconds, which shows that it is the main cause of stutter.

21.2 Warmup Flow Using a GSC

Warmup using a GSC is divided into two phases: collecting rendering settings (tracing) and warmup (precompilation).

21.2.1 Collecting Rendering Settings

List 21.1 shows the basic form of tracing. The rendering settings of objects drawn during tracing are recorded in the GSC as `GraphicsStateCollection.GraphicsState`. The listing puts everything into a single method, but in practice `BeginTrace()` and `EndTrace()` are called at separate times, and draws are collected over multiple frames.

▼List 21.1 Tracing rendering settings

```
private GraphicsStateCollection _gsc;

void Trace() {
    // Start collection
    _gsc = new GraphicsStateCollection();
    _gsc.BeginTrace();

    // Render the objects to be warmed up over multiple frames here
    // The rendered graphics settings are recorded as GraphicsState

    // End collection and save
    _gsc.EndTrace();
    _gsc.SaveToFile(path);
}
```

The generated `GraphicsStateCollection.GraphicsState` differs for each graphics API. Perform tracing on the device that actually runs the built app.

21.2.2 Warmup (Precompilation)

Load the collected GSC and run warmup (List 21.2). In this example, the GSC imported into the project is loaded as an Addressables asset.

▼List 21.2 Distributed warmup with WarmUpProgressively

```
async Task WarmUpAsync() {
```

```
    // Run after loading the assets that contain the shaders
    _gsc = await Addressables.LoadAssetAsync<GraphicsStateCollection>(address).Task;

    // Incremental WarmUp (Jobs-based; compiles n items per frame)
    StartCoroutine(WarmUpCoroutine());
}

IEnumerator WarmUpCoroutine() {
    JobHandle handle = default;
    int requestedCount = 0;
    // isWarmedUp may not become true on error, so completion is determined by the
    number of requested items
    while (requestedCount < _gsc.totalGraphicsStateCount) {
        handle = _gsc.WarmUpProgressively(4, handle);
        handle.Complete();
        requestedCount += 4;
        yield return null;
    }
}
```

The key points when implementing warmup are as follows.

- Warmup can also be run with `WarmUp(dependency)`, but it compiles everything at once and causes a spike. Specify the number of compilations per call with `WarmUpProgressively(count, dependency)` to distribute them over multiple frames
- Run it after the shader assets are loaded. If a shader instance does not exist, warmup of that variant is skipped
- Completion can be checked with `isWarmedUp` or `completedWarmupCount == totalGraphicsStateCount`
- However, if a warmup error occurs, `isWarmedUp` may never become true (confirmed in Unity 6000.3). For this reason, the listing determines completion by comparing the total number of requested compilations with `totalGraphicsStateCount`, which avoids an infinite loop

21.3 Implementation Example of a Collection Scenario

Another approach is to call `BeginTrace()` at app startup and collect rendering settings by playing manually. However, depending on the scale of the app, it is difficult

to cover everything manually with every version update. This section introduces a method that collects settings by mechanically running a scenario that draws assets sequentially in a dedicated collection scene. The implementation example uses Addressables and UniTask （出典ID: gsc_unitask）. Even if you do not use them, you can implement the same flow by replacing asynchronous asset loading and waiting for one frame of rendering with the mechanisms of your own project. The overall flow is as follows.

1. Download the required assets (not needed if everything is covered by built-in assets)
2. Get the Addresses of the assets needed for collection from the Addressables content catalog (`IResourceLocator`)
3. Switch Quality Settings (not needed if shaders do not change with Quality Settings)
4. Start collection (`BeginTrace()`)
5. Load the scene additively, then sequentially load and draw the assets to collect (model rendering, `ParticleSystem` playback, and Timeline playback)
6. End collection (`EndTrace()`)
7. Remove unneeded shaders from the GSC
8. Save the GSC (`SaveToFile(path)`)
9. Unload the scene and repeat steps 3 to 8 while switching Quality Settings

From the content catalog, obtain the assets that affect rendering settings (scenes), model assets such as characters and props, effect assets, and Timeline assets, and draw them sequentially while switching rendering settings to collect comprehensively. Collection is performed for each quality level because the rendering settings that affect PSOs, such as MSAA and HDR, change depending on the Quality Level.

List 21.3 shows the skeleton of the process. Tracing is repeated for each quality level, and the result is saved to a separate file named after the Quality Level. At runtime, the file corresponding to the current Quality Level is selected, loaded, and warmed up.

▼List 21.3　Skeleton of the collection scenario

```
private GraphicsStateCollection _gsc = new GraphicsStateCollection();
```

```
private async UniTaskVoid StartAsync() {
    // Fetch the content catalog and download the required assets
    // Fetch the lists of Addresses by Prefix in advance
    await AddressableSetupAsync();

    var qualitySettingNames = QualitySettings.names;
    for (var i = 0; i < qualitySettingNames.Length; i++) {
        QualitySettings.SetQualityLevel(i);
        await TraceAsync();
    }
}

private async UniTask TraceAsync() {
    // Start GSC collection
    _gsc.ClearVariants();
    _gsc.BeginTrace();

    // Run the collection scenario
    await ExecuteScenarioAsync();

    // End GSC collection
    _gsc.EndTrace();

    // Remove shaders not targeted for collection from the GSC
    RemoveUnnecessaryVariants();

    var qualityLevelName = QualitySettings.names[QualitySettings.GetQualityLevel()];
    _gsc.SaveToFile(_gscSavePathFormat + $"_{qualityLevelName}");
}
```

In the scenario body, models, effects, and Timelines are drawn sequentially for each scene. List 21.4 is an excerpt of the part that creates and draws character models in batches of a few at a time. Waiting for one frame of rendering records that frame's rendering settings in the GSC.

▼List 21.4 Sequential drawing of model assets (excerpt)

```
// Collect character models
for (var i = 0; i < _charaAddressArray.Length; i += _characterNum) {
    // Spawn a few character models at a time
    var batchCount = Mathf.Min(_characterNum, _charaAddressArray.Length - i);
    for (var j = 0; j < batchCount; j++) {
        var charaHandle = Addressables.LoadAssetAsync<GameObject>(_charaAddressArray[i
    + j]);
        var charaPrefab = await charaHandle.ToUniTask();
        // Place them evenly at positions within the camera view
```

```
        var pos = new Vector3((j - batchCount * 0.5f) * 2f, 0f, 5f);
        _charaObjectArray[j] = Object.Instantiate(charaPrefab, pos, Quaternion.identity
    );
        Addressables.Release(charaHandle);
    }

    // Wait one frame for rendering
    await UniTask.NextFrame();

    for (var j = 0; j < batchCount; j++) {
        DestroyImmediate(_charaObjectArray[j]);
        _charaObjectArray[j] = null;
    }
}
```

`ParticleSystem` is also created in batches of a few at a time, but differs in that the mid-playback state (0.5 seconds in this example) is reproduced synchronously with `Simulate()` before drawing (List 21.5). Because there is no need to wait for real-time playback, the collection time can be shortened.

▼List 21.5 Collecting ParticleSystem (excerpt)

```
// Collect ParticleSystems
for (var i = 0; i < _particlesAddressArray.Length; i += _effectNum) {
    var batchCount = Mathf.Min(_effectNum, _particlesAddressArray.Length - i);
    for (var j = 0; j < batchCount; j++) {
        var particleHandle = Addressables.LoadAssetAsync<GameObject>(
    _particlesAddressArray[i + j]);
        var particlePrefab = await particleHandle.ToUniTask();
        var pos = new Vector3((j - batchCount * 0.5f) * 3f, 0f, 5f);
        _particleObjectArray[j] = Object.Instantiate(particlePrefab, pos, Quaternion.
    identity);
        Addressables.Release(particleHandle);

        // Reproduce the state after _effectSimulateTime seconds synchronously (Play is
     not required)
        var ps = _particleObjectArray[j].GetComponentInChildren<ParticleSystem>();
        if (ps != null) {
            ps.Simulate(_effectSimulateTime, withChildren: true, restart: true);
        }
    }

    // Wait one frame for rendering
    await UniTask.NextFrame();

    for (var j = 0; j < batchCount; j++) {
```

```
        DestroyImmediate(_particleObjectArray[j]);
        _particleObjectArray[j] = null;
    }
}
```

Timeline-based sequences are collected by actually playing them with `PlayableDirector` (List 21.6). Because it waits for the playback duration, it is better to limit the target Timeline assets to those needed for collection.

▼List 21.6　Collecting Timeline (excerpt)

```
// Collect Timelines
foreach (var timelineAddress in _timelineAddressArray) {
    var timelineHandle = Addressables.LoadAssetAsync<GameObject>(timelineAddress);
    var timelinePrefab = await timelineHandle.ToUniTask();
    var timelineObj = Object.Instantiate(timelinePrefab, Vector3.zero, Quaternion.
    identity);
    Addressables.Release(timelineHandle);
    var director = timelineObj.GetComponent<PlayableDirector>();
    if (director != null) {
        director.Play();
        await UniTask.Delay((int)(director.duration * 1000));
    }
    DestroyImmediate(timelineObj);
}
```

After collection, keep only the shaders you want to warm up (List 21.7). The shaders to keep are specified with an allowlist (`_allowShaderNames`), and shaders outside it and variants with broken references (null) are removed from the GSC.

▼List 21.7　Removing variants outside the collection target

```
private void RemoveUnnecessaryVariants() {
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    _gsc.GetVariants(variants);

    foreach (var variant in variants) {
        if (variant.shader == null) {
            _gsc.RemoveVariant(null, variant.passId, variant.keywords);
            continue;
        }

        if (_allowShaderNames.Contains(variant.shader.name)) {
            continue;
        }
```

```
        // Remove unnecessary shaders from the GSC
        _gsc.RemoveVariant(variant.shader, variant.passId, variant.keywords);
    }
}
```

Scenario and asset management differ by project, so adjust the scene structure and collection targets to fit your own project.

21.4 Real-World Cases Where Stutter Remains and Their Remedies

Stutter can remain even after adopting warmup with a GSC. This section introduces real-world cases and their remedies.

21.4.1 Missing Variants Due to Shader Stripping

In this case, a `variant not found` error occurs in the warmup log, and compilation of the corresponding variant (such as `Shader.CreateGPUProgram` in the Profiler) is also not observed at runtime.

The cause is that the shader variant contained in the GSC is not included in the app or AssetBundle. Depending on the settings in **Project Settings > Graphics > Shader Stripping**, variants that are not used are excluded at build time. This problem tends to occur when the required variants change because of changes to shaders or assets.

Check the Shader Stripping settings, and collect the GSC again whenever you make changes that affect variants.

Note that the same `variant not found` error also occurs when a variant is recorded in the GSC but `GraphicsState` is 0. In this case, compilation of that variant occurs at runtime, so you can distinguish the two cases by whether compilation occurs at runtime.

21.4.2 Mismatch Warning Due to Metadata Mismatch

In this case, the following warning appears in the console when warmup runs.

```
WarmUp may be inaccurate due to GraphicsDeviceType mismatch.
WarmUp may be inaccurate due to RuntimePlatform mismatch.
WarmUp may be inaccurate due to Quality Level mismatch.
```

The cause is that the `graphicsDeviceType` / `runtimePlatform` / `qualityLevelName` metadata stored in the GSC file does not match the execution environment. The main cases are the following two.

- A GSC collected on a different platform is reused (for example, a GSC collected on an iOS device is used in an Android build)
- When the GSC was edited and saved by an Editor script, the metadata was overwritten with the Editor's Platform setting

The remedy depends on the case. For a GSC collected on a different platform, the `GraphicsState` inside does not match the execution environment even if the metadata is rewritten, so collect it again on the target platform. If the metadata was overwritten by the Editor's Platform setting, correct the metadata to the values of the actual device. Note that a `qualityLevelName` mismatch can be ignored only when the rendering settings that affect PSOs (such as MSAA and HDR) and the shaders do not change by Quality Level. In that case, the warning can be avoided by rewriting the value just before warmup, as in List 21.8.

▼List 21.8 Avoiding the warning by rewriting qualityLevelName

```
// Load the GSC
var gsc = LoadGsc();
// Change qualityLevelName to match the runtime environment
gsc.qualityLevelName = QualitySettings.names[QualitySettings.GetQualityLevel()];
// Run warmup
```

21.4.3 Double Instantiation Due to Duplicate AssetBundle Packing on iOS (Metal)

This case occurs only on iOS (Metal). There are no errors in the warmup log, and `Shader.CreateGPUProgram` does not occur on the Main Thread in the Profiler, yet `GpuProgramMetal.GetCachedPipeline` occurs on the Render Thread.

The cause is double instantiation of a shader due to duplicate packing in Asset Bundles. When the same shader is packed redundantly into multiple AssetBundles, Unity creates separate shader instances in memory. Warmup creates PSOs only for the shader instances referenced by the GSC, so a shader that is a separate instance is not warmed up even if it is the same shader. This problem does not occur on Vulkan. This is probably because PSO cache management differs between Metal and Vulkan.

An example of a configuration in which this occurs is as follows. The same shader X is packed into both bundles because of dependencies.

```
- Asset
  - Shader X
  - Material A (uses Shader X)
  - Material B (uses Shader X)
  - GraphicsStateCollection

- AssetBundle configuration
  - Bundle A
    - GraphicsStateCollection
    - Material A (uses Shader X)
    - Shader X <- packed due to dependency
  - Bundle B
    - Material B (uses Shader X)
    - Shader X <- packed due to dependency
(Shader X has no address assigned)
```

When bundle A and bundle B are loaded in this configuration, stutter occurs when drawing material B, as follows.

```
- Runtime flow
  1. Load Bundle A and Bundle B
     - Shader X in Bundle A (instanceId=100)
```

```
    - Shader X in Bundle B (instanceId=200)
  2. Run GraphicsStateCollection warmup
    - Warmup is performed with Shader X (instanceId=100), which the GSC references
  3. Render Material A
    - No stutter, because the PSO cache for Shader X (instanceId=100) has already been
    created
  4. Render Material B
    - There is no PSO cache for Shader X (instanceId=200),
      Render so PSO creation runs on a thread and a stutter occurs
```

The remedy is to put the shader into a single bundle (a shader-only group) and eliminate implicit packing into content bundles. If consolidating into a single bundle is difficult, prepare a GSC for each bundle. Duplicated shaders can be checked with code like List 21.9. If you use Addressables, you can also check shader duplication in the **Addressables Report** window.

▼List 21.9 Checking for duplicate shader instances

```
using System.Linq;

// Run after loading the AssetBundles to check for duplicate instances
var all = Resources.FindObjectsOfTypeAll<Shader>();
var duplicates = all.GroupBy(s => s.name).Where(g => g.Count() > 1);
foreach (var g in duplicates)
    foreach (var s in g)
        Debug.Log($"Duplicate: {s.name} | InstanceID: {s.GetInstanceID()}");
```

21.4.4 Warmup Failure for Shaders Using SubPass on iOS (Metal)

The problems so far were caused by mistakes in project settings or bundle design. Next, we introduce a problem that arises from the GSC specification.

A mechanism that splits one render pass into multiple drawing steps and lets later steps reference the results of earlier steps without writing them back to main memory is called a SubPass. On mobile GPUs, this series of steps can be completed in fast on-chip memory (tile memory), which saves memory bandwidth. On Metal and Vulkan, SubPass is executed as a Native Render Pass. Only on iOS (Metal), there are cases where PSO compilation occurs during gameplay even though warmup was performed and no errors appear in the log. The cause is that the warmup process

on iOS (Metal) does not work as expected for shaders drawn inside this SubPass (Native Render Pass). It occurs regardless of whether input attachments, which read the output of the previous SubPass, are used. Since the internal warmup process is a non-public API, this section only describes the conditions under which it occurs.

The affected conditions are as follows.

- The target is iOS / Metal (a device with an Apple A/M-series chip)
- `RenderGraph.nativeRenderPassesEnabled`, a setting of URP's rendering infrastructure (RenderGraph), is enabled (in Unity 6000.5 and later this setting is removed and always enabled)
- Shaders that draw inside a SubPass (Native Render Pass) are used

It occurs in the same way for write-only SubPasses (such as the GBuffer pass of URP Deferred) and for SubPasses with input attachments (such as the Deferred Lighting pass). List 21.10 shows a case that sets input attachments as an example of an affected SubPass.

▼List 21.10 Case of setting input attachments on a SubPass

```
// Started with ScriptableRenderContext.BeginRenderPass,
// Example of setting a SubPass inside the render pass
var inputAttachments = new NativeArray<int>(new[] { 0, 1 }, Allocator.Temp);
context.BeginSubPass(
    colors,
    inputAttachments,   // Bind the output of the previous SubPass
                        // Converted to framebuffer fetch on Metal, but
                        // this is not reflected in PSO creation during GSC warmup
    false
);
```

In Unity 6000.3, a fundamental fix is difficult, so either review the use of SubPass or avoid stutter during play with pre-rendering. Pre-rendering is a technique that actually draws, once, objects that use the target shaders, for example during a loading screen, to build the PSO cache. It is like performing, inside the game, the same sequential drawing as this chapter's collection scenario in place of warmup.

21.4.5 Diagnostic Points When Stutter Remains

As seen so far, shader warmup using a GSC is closely tied to the project's asset management design (Addressables configuration, shader keyword definitions, and bundle design), so debugging after adoption is often necessary. Check the warmup log and take repeated measurements with the Profiler to confirm that no warmup is missed. In particular, keeping the following points in mind speeds up diagnosis.

- Whether warmup errors or `variant not found` appear in the warmup log
- Whether `Shader.CreateGPUProgram` occurs on the Main Thread and `GpuProgramMetal.GetCachedPipeline` (iOS) / `CreateGraphicsPipelineImpl` (Android) occurs on the Render Thread in the Profiler
- Whether the Shader Stripping settings are appropriate
- Whether AssetBundle packing is as intended

To check the variants recorded in a GSC, you can use the variant list output (List 21.11) introduced in the next section.

21.5 Examples of GSC File Operations

Finally, we introduce two examples of GSC file operations. Both are intended to be run from Editor scripts, and can be used to inspect the contents of a collected GSC or to merge the results of multiple collections.

21.5.1 Checking the Variant List and State Count

List 21.11 is an example that loads a GSC file and outputs its metadata and the list of recorded variants.

▼List 21.11 Checking the variant list and State count

```
void PrintVariantInfo() {
    // Load the GSC
    var gsc = new GraphicsStateCollection();
    if (!gsc.LoadFromFile(path))
        return;

    // Get the variants
```

```
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    gsc.GetVariants(variants);

    // Output the GSC metadata
    Debug.Log($"[GSC] Variants={variants.Count} TotalStates={gsc.
    totalGraphicsStateCount} "
            + $"Platform={gsc.runtimePlatform} Device={gsc.graphicsDeviceType} Quality
    ={gsc.qualityLevelName}");

    // Output the variant details
    foreach (var v in variants) {
        int stateCount = gsc.GetGraphicsStateCountForVariant(v.shader, v.passId, v.
    keywords);
        string kws = v.keywords.Length > 0
            ? string.Join(" ", v.keywords.Select(k => k.name))
            : "<no keywords>";
        Debug.Log($"  {v.shader?.name} | states={stateCount} | {kws}");
    }
}
```

21.5.2 Merging GSCs

List 21.12 is an example that merges two GSCs. It can combine results collected over multiple runs into one.

▼List 21.12 Merging two GSCs

```
void MergeGsc() {
    // Load the two GSCs
    var src = new GraphicsStateCollection();
    var dst = new GraphicsStateCollection();
    if (!src.LoadFromFile(srcPath) || !dst.LoadFromFile(dstPath))
        return;

    // Get the variants
    var variants = new List<GraphicsStateCollection.ShaderVariant>();
    var states = new List<GraphicsStateCollection.GraphicsState>();
    src.GetVariants(variants);

    // Add from the src GSC to the dst GSC
    foreach (var v in variants) {
        // Get the GraphicsStates contained in the variant
        src.GetGraphicsStatesForVariant(v.shader, v.passId, v.keywords, states);
        foreach (var s in states) {
            // Add the State
            dst.AddGraphicsStateForVariant(v.shader, v.passId, v.keywords, s);
```

```
        }
    }

    dst.SaveToFile(dstPath);
}
```

Depending on the AssetBundle configuration, you may need to move or merge the collected `GraphicsState` into another GSC. The typical case is preparing a GSC for each bundle, as described in the remedy for double instantiation. In that case, combine the operation examples in this section.